

AI in signal and audio processing

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Class details

- 24h: 3h*8
- Grade: 50% Labs
50% Final Project
- All the material will be online: https://github.com/tdejaeger/Epita_lectures

Class organisation

- Lectures (30min-1h each class)
- Labs during class
- Project

Final Project Overview — COVID19 detection

- **Project Goal:** Build a complete AI pipeline that predict **COVID19** using audio processing and deep learning techniques.
- You can work alone or in group (max 2p).
- Output: Presentation (10-15min)+ jupyter notebook/google colab

Data: [Coswara: A respiratory sounds and symptoms dataset for remote screening of SARS-CoV-2 infection | Scientific Data](#)

Grade:

- /5 Work, results, analysis
- /5 Presentation, oral expression, slides
- /5 Critical thinking
- /3 How you answer the questions
- /2 the questions you will ask during other presentations

Syllabus

- **Session 1:** Introduction & Fundamentals of Signal/Audio
- **Session 2:** Audio Features & Librosa
- **Session 3:** Audio Processing with Torchaudio
- **Session 4:** Sound Detection and Classification
- **Session 5:** Sequence Modeling with RNNs and LSTMs
- **Session 6:** Automatic Speech Recognition (ASR)
- **Session 7:** Generative IA audio and ICA
- **Session 8:** Project presentation

Environment & Requirements

Tools & Libraries to Install

Session	Topic	Tools / Libraries
1	Fundamentals of Signal & Audio Processing	numpy, scipy, matplotlib
2	Audio Features & Librosa	librosa, scikit-learn, matplotlib
3	Audio Processing with Torchaudio	torchaudio, PyTorch
4	Sound Classification	librosa, scikit-learn, PyTorch, matplotlib
5	Sequence Modeling (RNN, LSTM)	PyTorch, torchaudio
6	Automatic Speech Recognition (ASR)	torchaudio, transformers, Wav2Vec2, Whisper
7	Speaker & Emotion Recognition	librosa, PyTorch, scikit-learn

You can run this course on **Google Colab** or **locally**.

Google Colab

What is Google Colab? Google Colab is a free cloud service that supports Jupyter notebooks and provides free access to computing resources including GPUs.

To use it:

- Log in with a Google account
- Open a .ipynb notebook from Drive or GitHub, or create a new one
- To enable GPU: Runtime > Change runtime type > GPU

Why we use it:

- Free and powerful
- No local install needed
- Familiar interface for Jupyter users

Upload files as a drive from your local machine

```
from google.colab import files  
input_file = files.upload()
```

Fundamentals of Signal and Audio Processing

Session 1

Fundamentals of Signal and Audio Processing

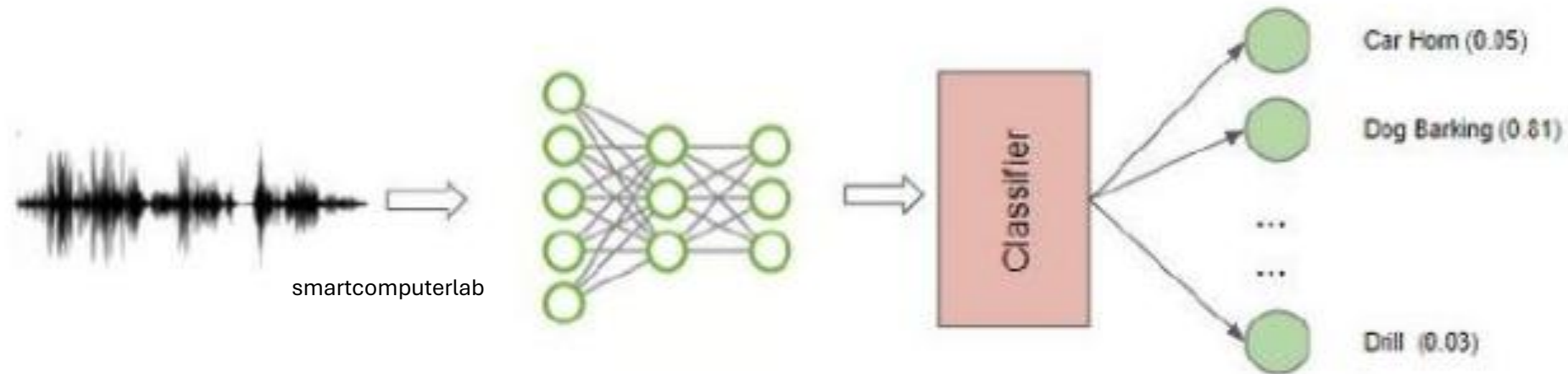
Session 1

Objectives:

- Understand what a signal is and its mathematical properties
- Explain key differences between continuous and discrete signals
- Describe sound as a physical and perceptual phenomenon
- Understand the concept of sampling and digitization
- Visualize time-domain vs. frequency-domain representations

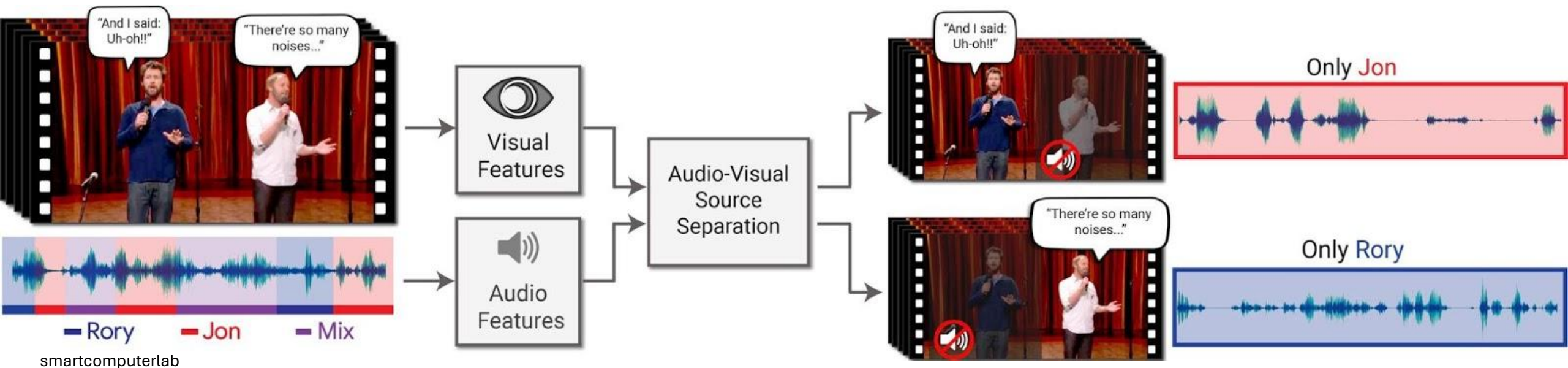
Why audio IA is important/useful for?

- **Classification**



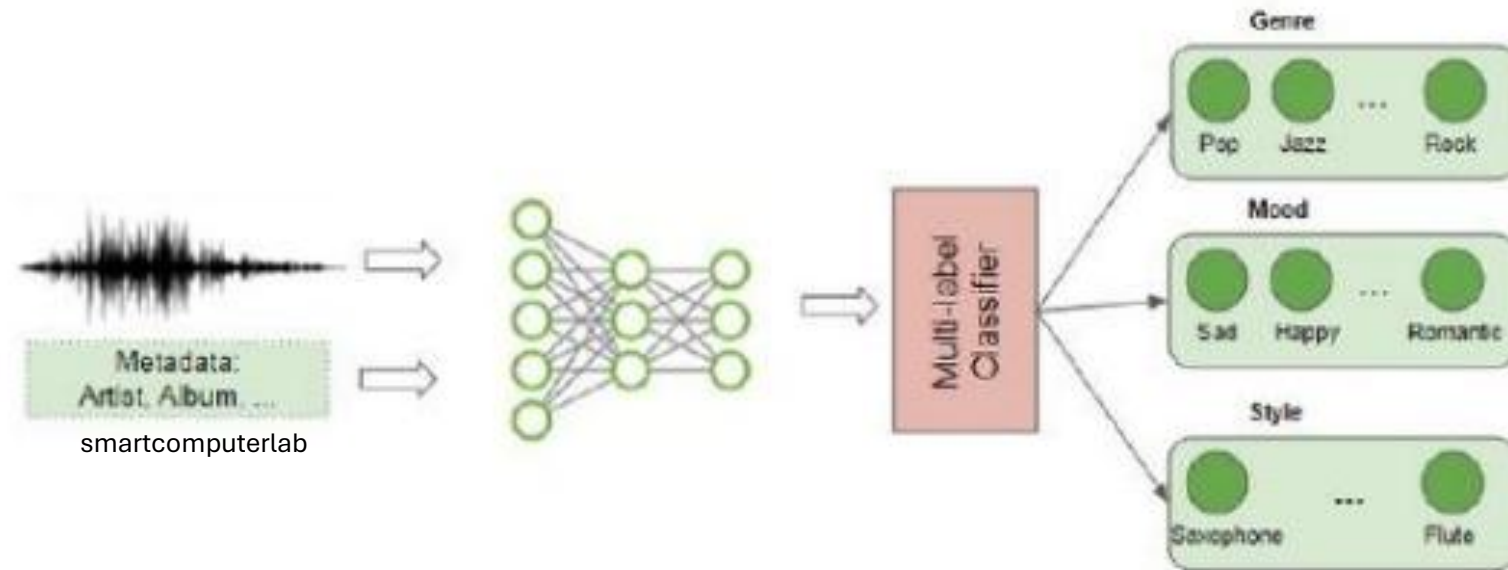
Why audio IA is important/useful for?

- Separation and segmentation



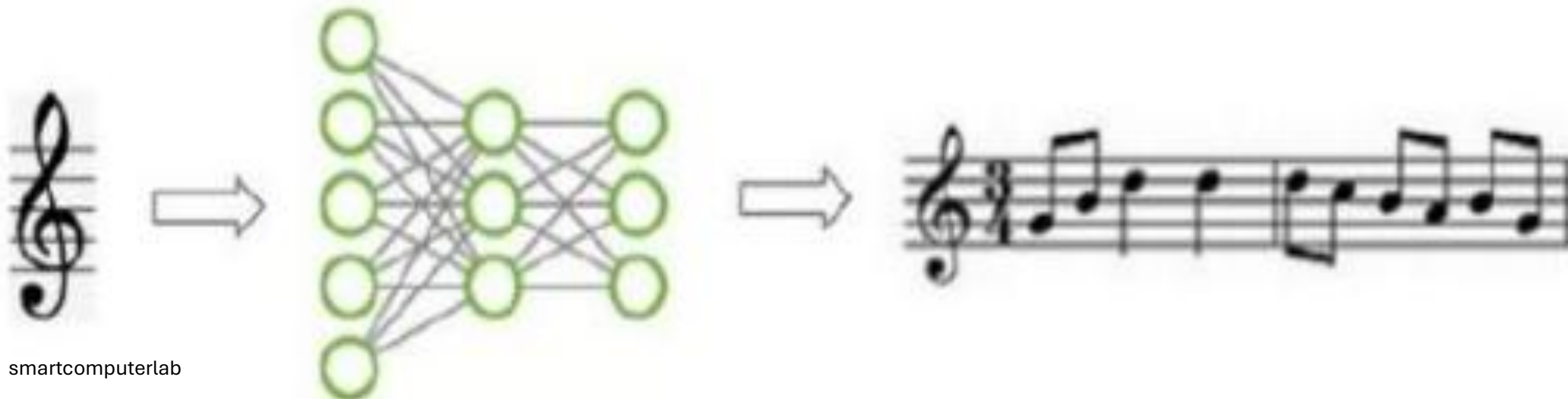
Why audio IA is important/useful for?

- Music genre classification



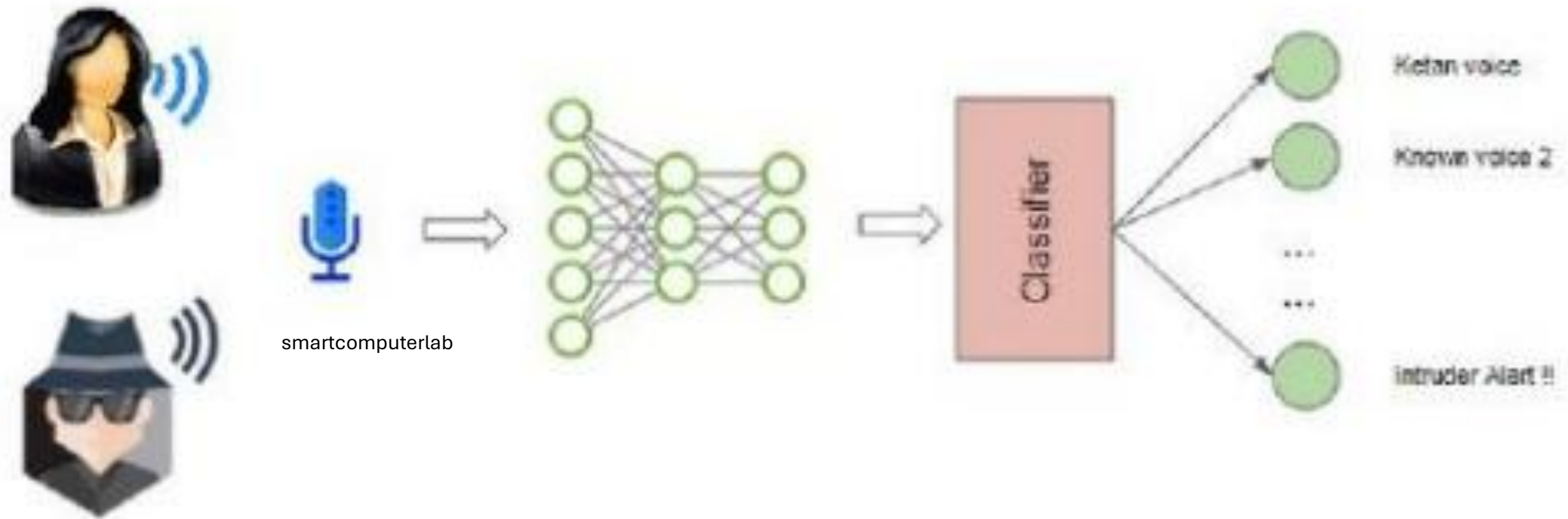
Why audio IA is important/useful for?

- **Music generation**



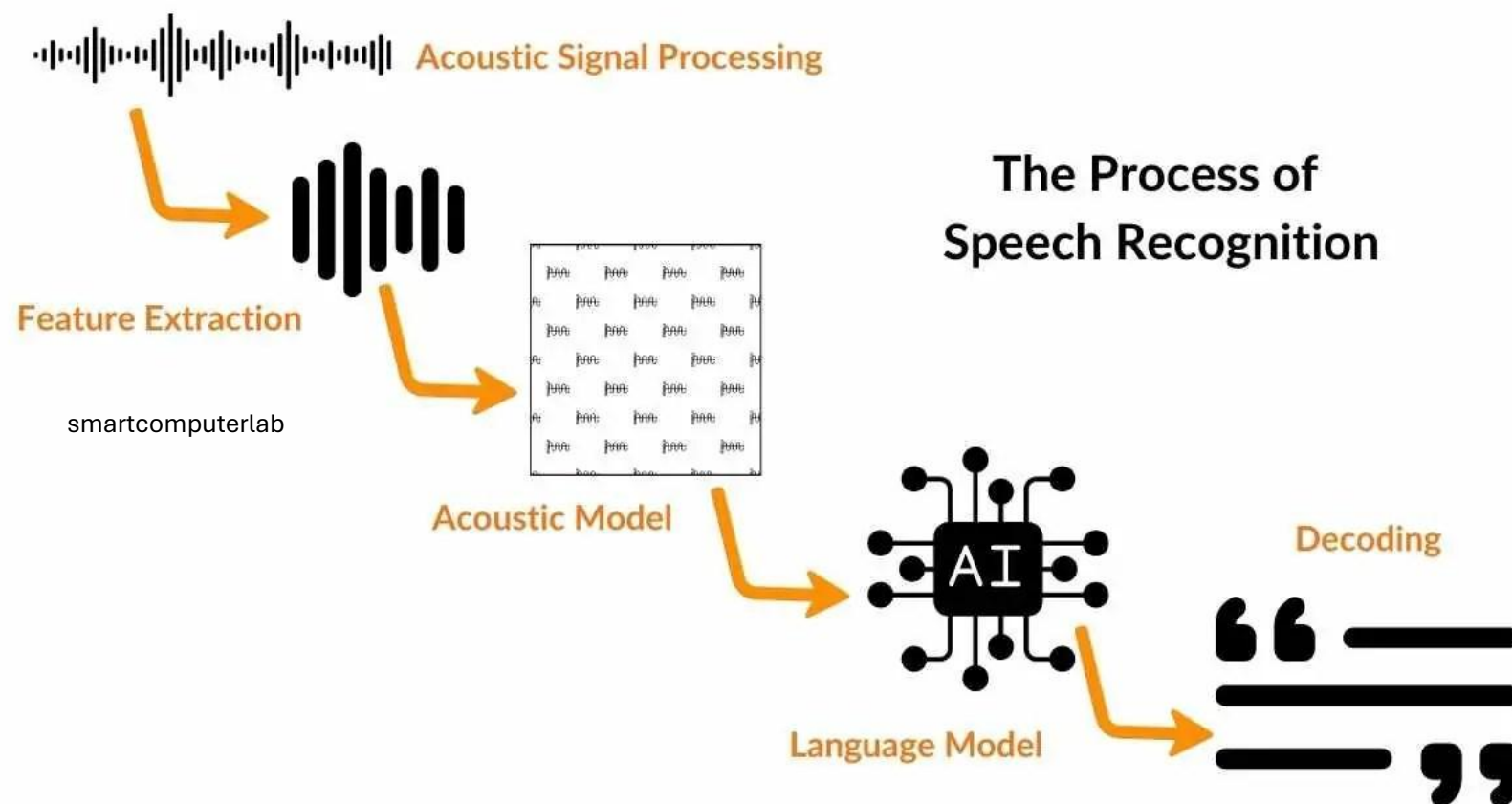
Why audio IA is important/useful for?

- **Voice recognition**



Why audio IA is important/useful for?

- Speech to text



What is Signal and Audio Processing

Definition:

Signal processing is the science of analyzing and manipulating signals: any measurable time-dependent data. Transforming, examining, and interpreting audio signals to extract meaningful information.

Audio signal processing focuses on sound-related signals like speech, music, or noise

Why using IA ?

- Audio data is complex: noisy, variable, context-dependent : ideal for machine learning.
- You can:
 - Analyze thousands of sounds rapidly
 - Detect patterns invisible to humans
 - Eliminate subjectivity in analysis

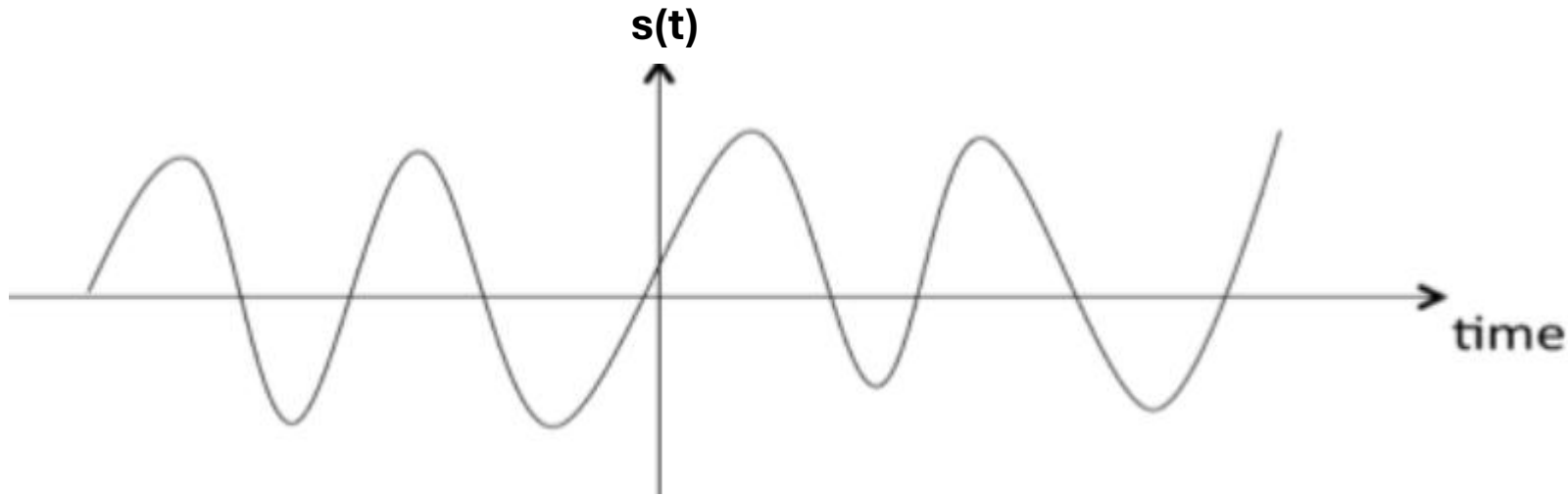
Where is audio processing is used?

Applications across fields:

- Medical (cough detection, heart sounds)
- Music recommendation (Spotify)
- Security (voice recognition)
- Accessibility tools (speech-to-text)
- Telecom, etc

What is a signal?

A signal is a measurable physical quantity which varies as a function of time t . It can be denoted $s(t)$ or $s[n]$



Exemple:

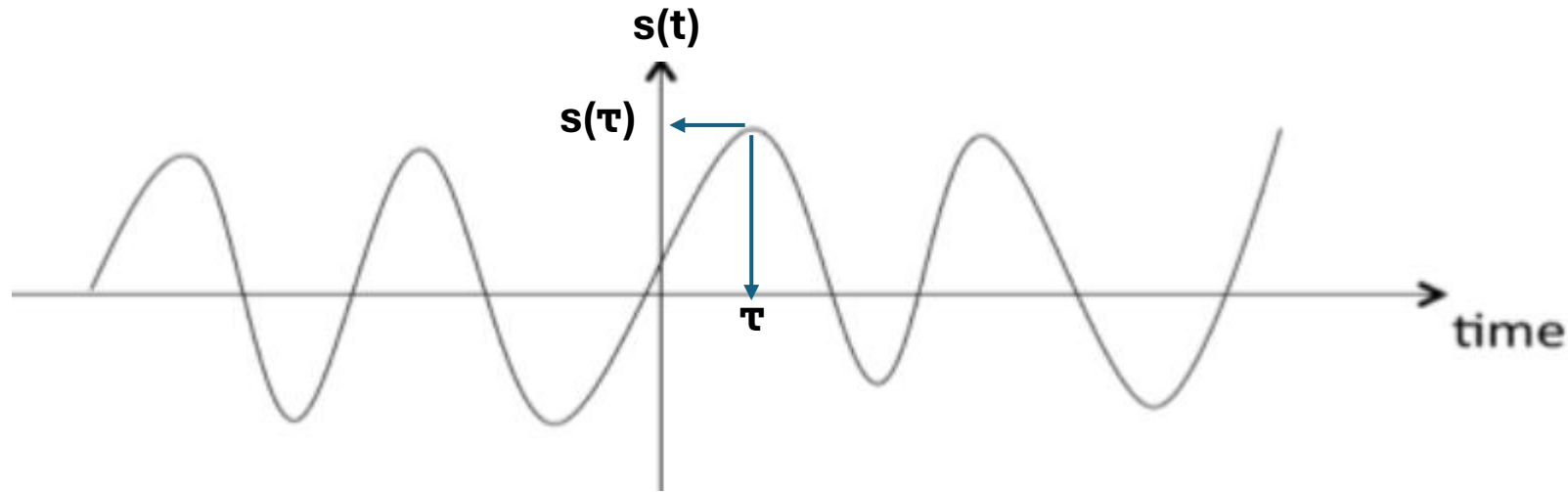
- Temperature
- Stock price
- Image (2D but a signal)
- ECG ElectroCardioGram

Modelling:

- A signal $s(t)$ can be mathematically described by one or more mathematical functions related to each other :
- $\sin(2\pi t)$, $\cos(2\pi t + \pi)$, $\exp(t)$, etc

Amplitude?

The value of the signal at an instant of time τ is called the signal amplitude at time τ .

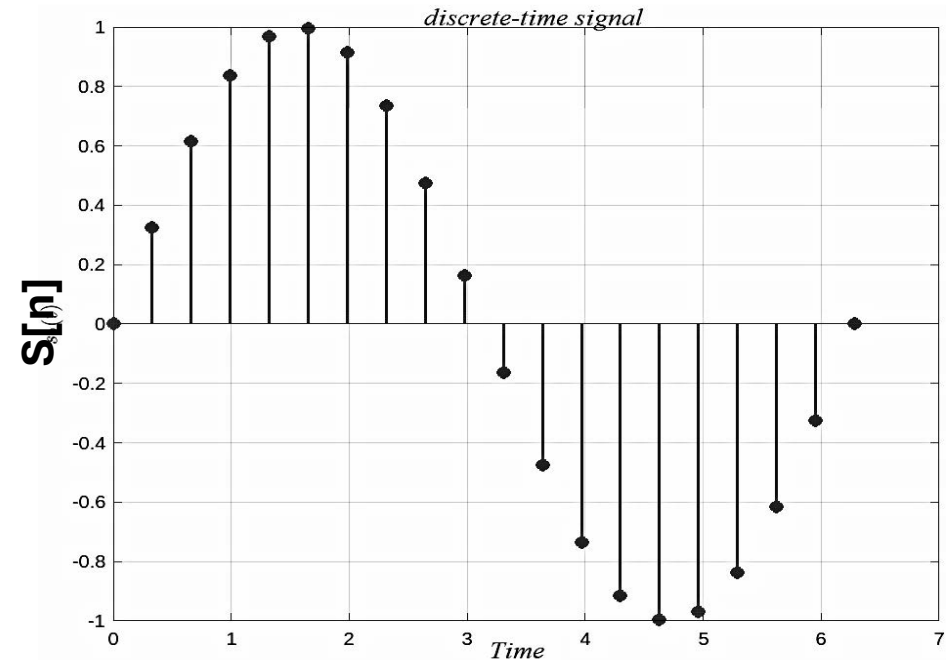
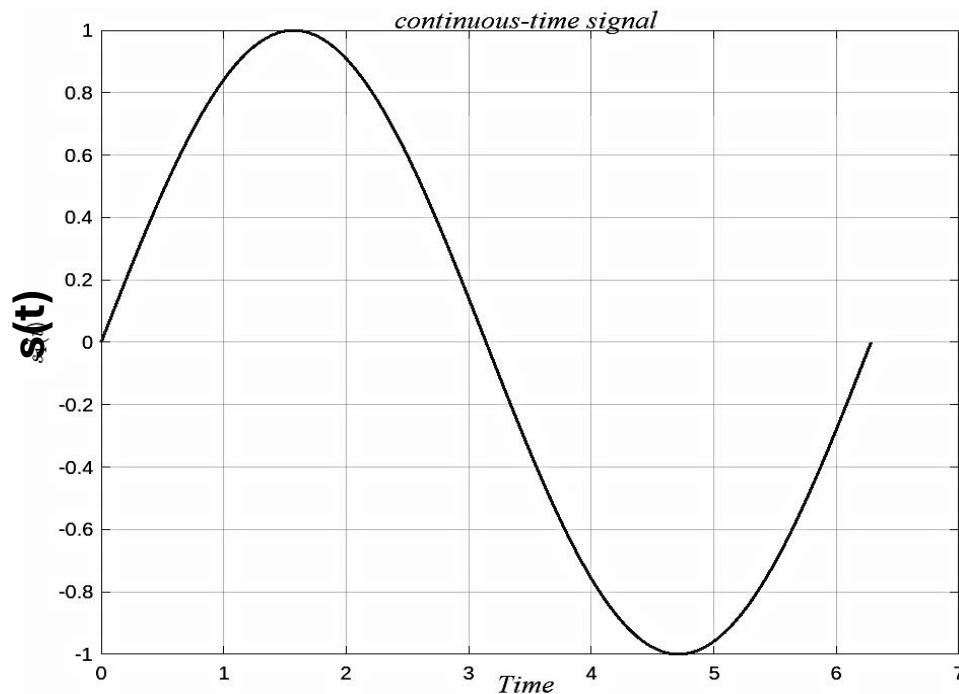


The amplitude of the signal $s(t)$ at time $t = \tau$ is denoted by $s(\tau)$.

Note: If the amplitude does not vary over time, the signal is **constant**

Two type of signals

- **Continuous:** $s(t)$ is a continuous-time signal if it is defined at any time in the interval $[0, t_f]$
- **Discrete:** $s[n]$ is a discrete-time signal if it is defined only at integer time steps n in the interval $[0, t_f]$.

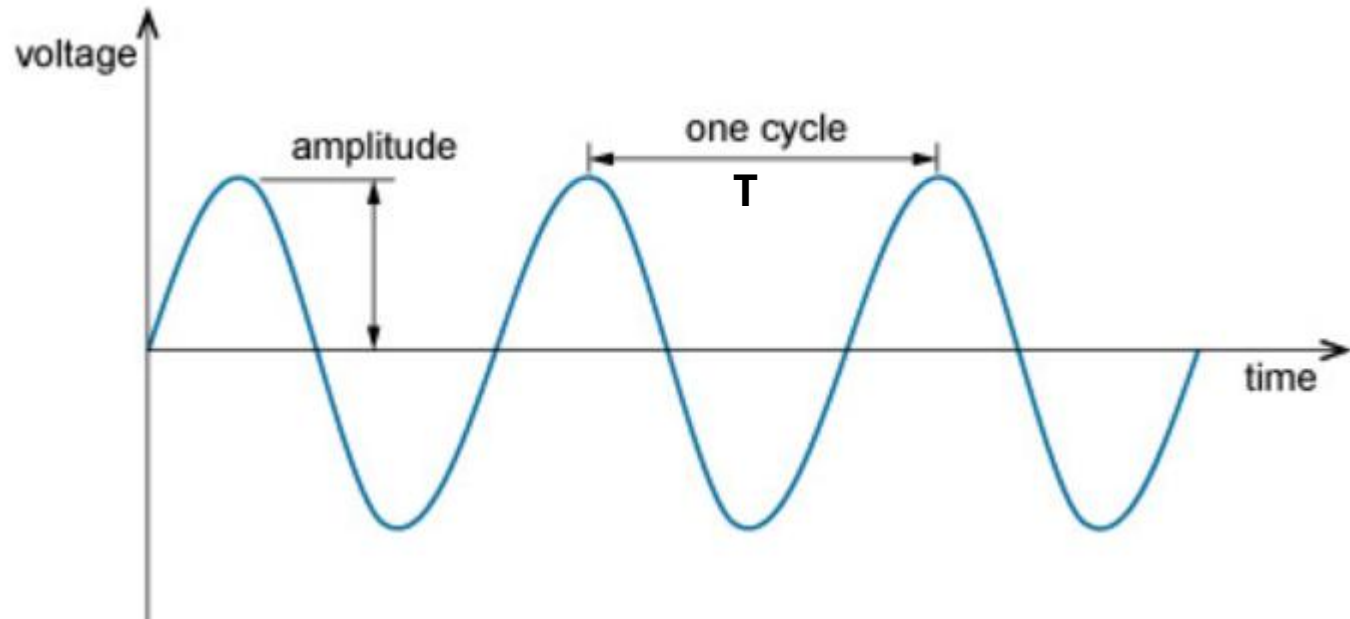


Periodic signal

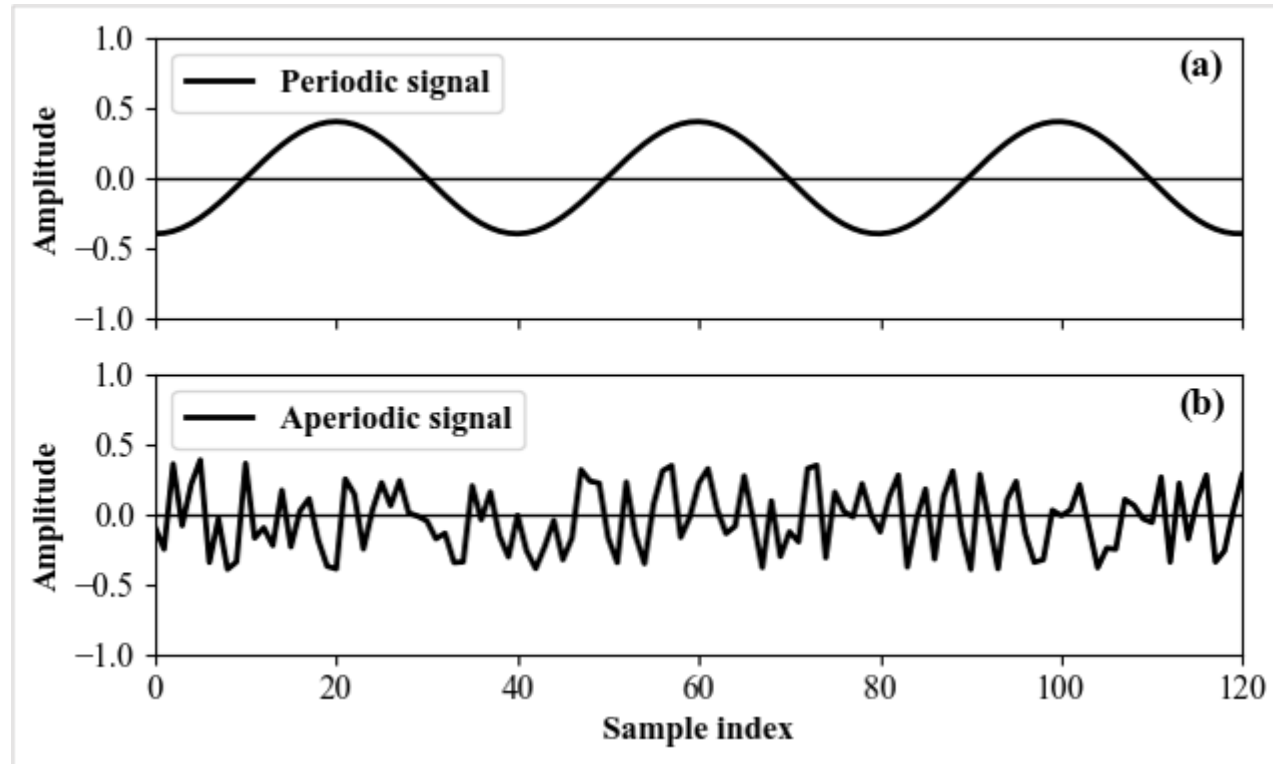
- **Periodic:** if the variations in the amplitude of the signal recur regularly after a constant period T .

$$s(t)=s(t+T)=s(t-T)$$

Frequency: $f=1/T$ (Hz if T in s)



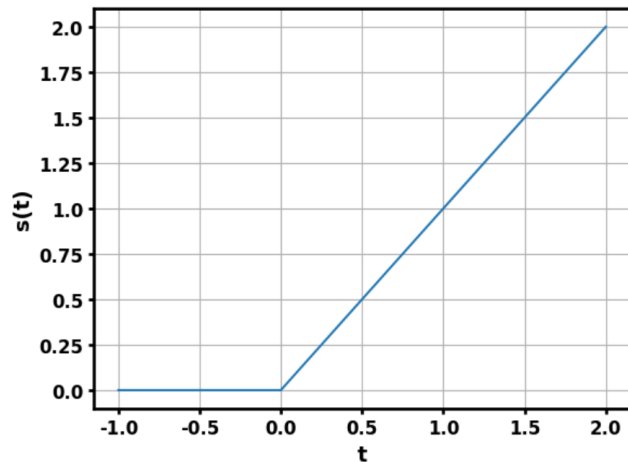
Periodic signal



Signal operation

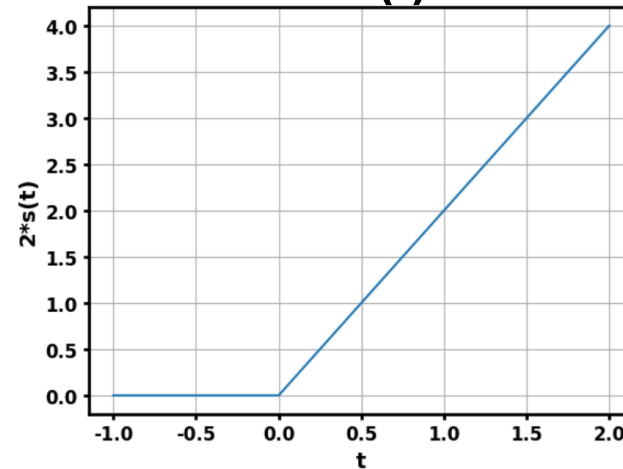
- You can **sum**, **subtract**, **multiply**, or **divide** two signals:

$s(t)$



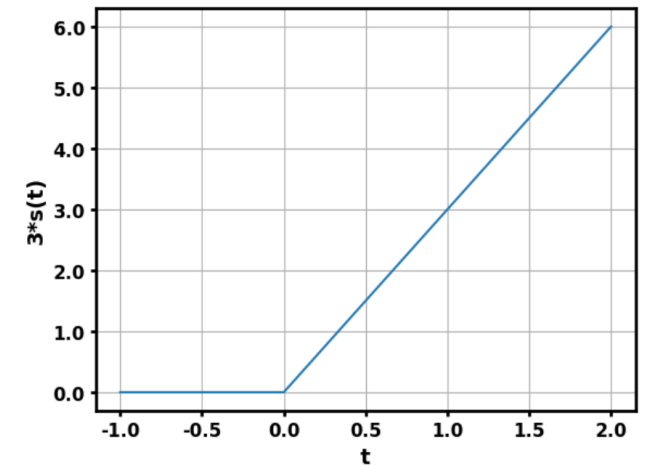
+

$2*s(t)$

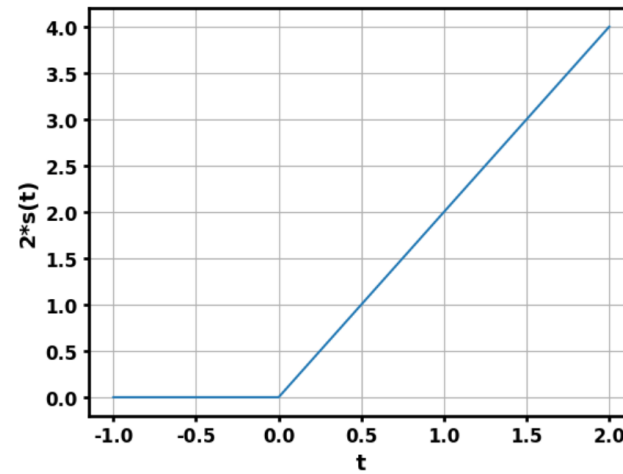


=

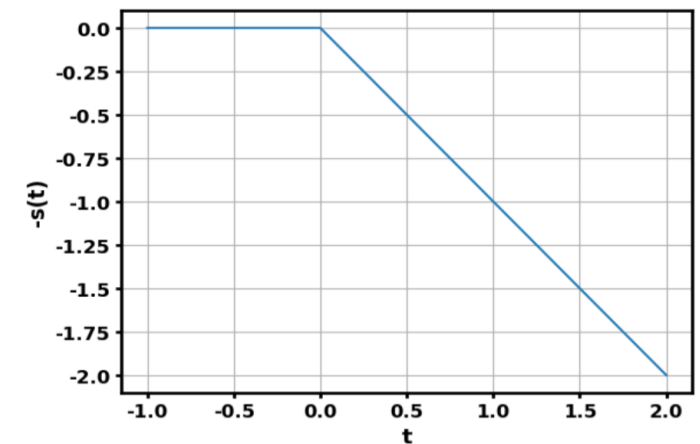
$3*s(t)$



-

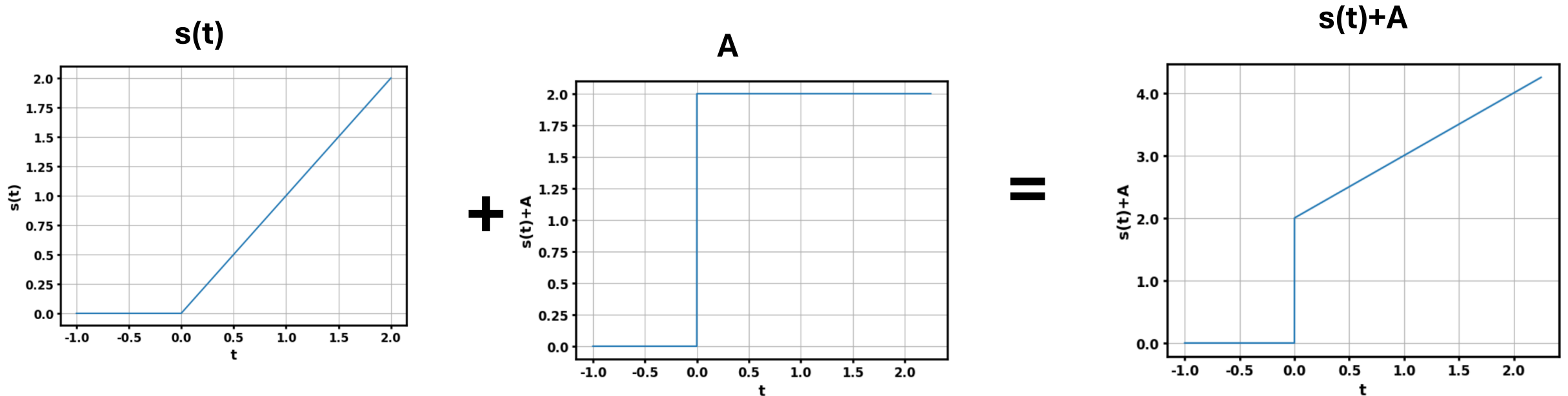


$-s(t)$



Signal operation

- **Offset:** a constant component added to the signal.

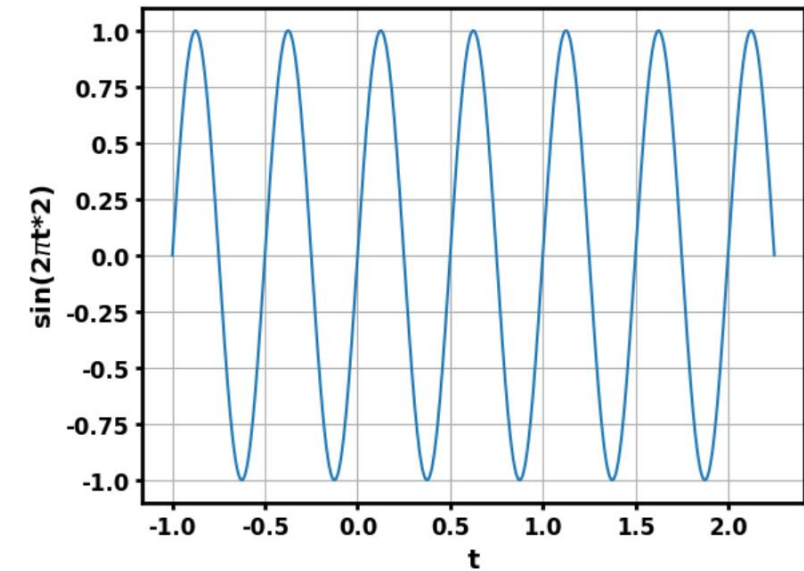
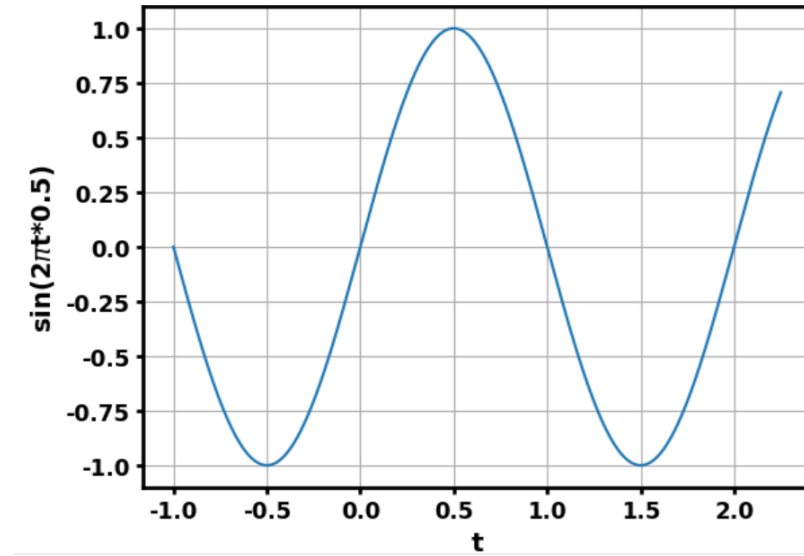
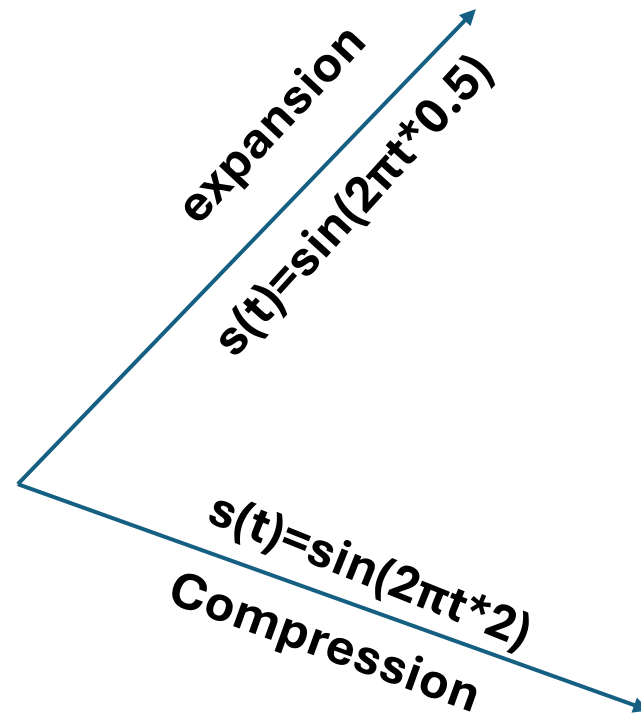
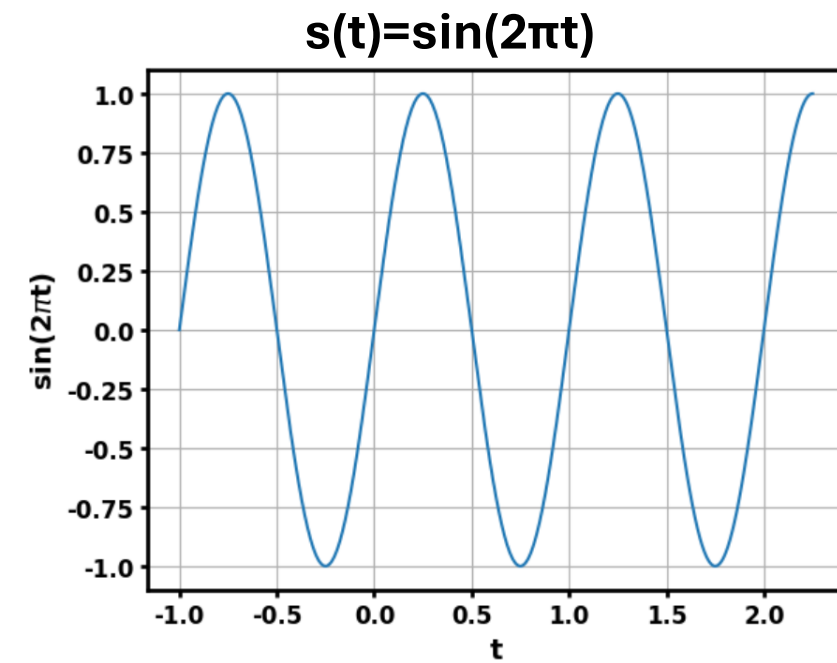


- **Amplification/Attenuation:** Constant to amplify/reduce the amplitude at each instant $s(t)*A$
 - If $A > 1$ = amplification, $0 < A < 1$ = attenuation, if $A < 0$ is a inversion of $s(t)$ with an amplification (if $A < 1$) or an attenuation (if $1 < A < 0$).

Signal operation

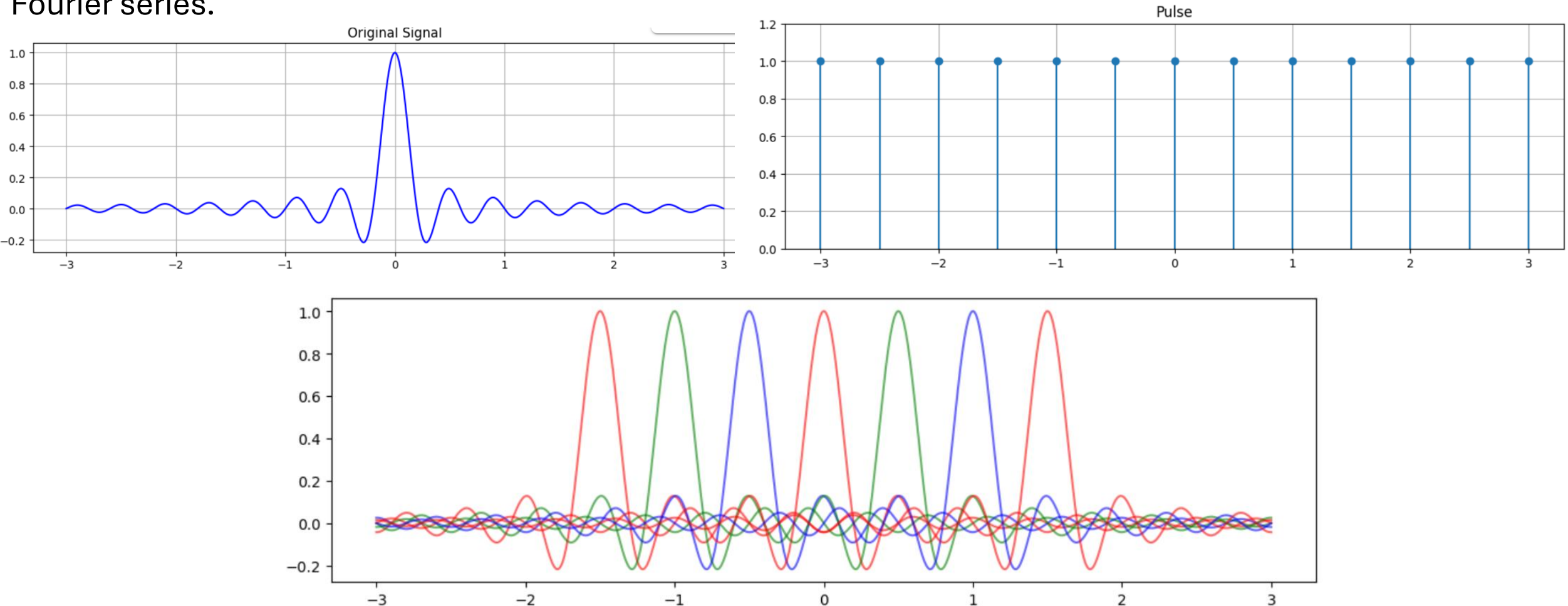
- **Time shift:** use to delay or advance a signal.
 - $s(t)=s_1(t-\alpha)$
 - If $\alpha < 0$ the signal is shifted to left= advanced
 - If $\alpha > 0$ the signal is shifted to right= delayed
- **Compression/expansion:** serves to represent it over a time interval shorter than that of the original signal (compression), or longer (expansion)
 - $s(t)=s_1(t*\alpha)$
 - If $\alpha > 1$ the signal is compressed
 - If $\alpha < 1$ the signal is expanded

Signal operation



Signal operation

Periodization: The periodization of a signal serves to reproduce the same signal every x seconds. This is useful in signal processing, for example when analyzing signals in the frequency domain using Fourier series.



Time versus frequency

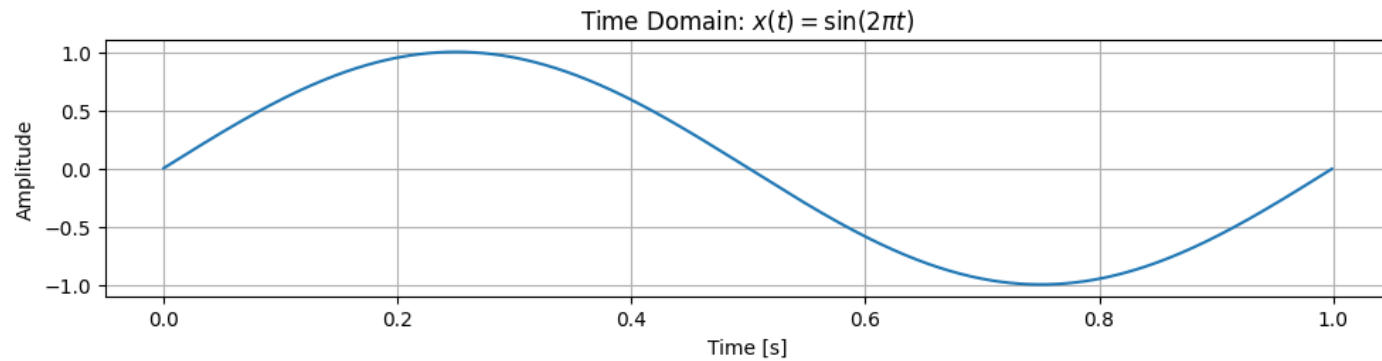
Time Domain:

- Shows amplitude over time
- Useful for waveform inspection

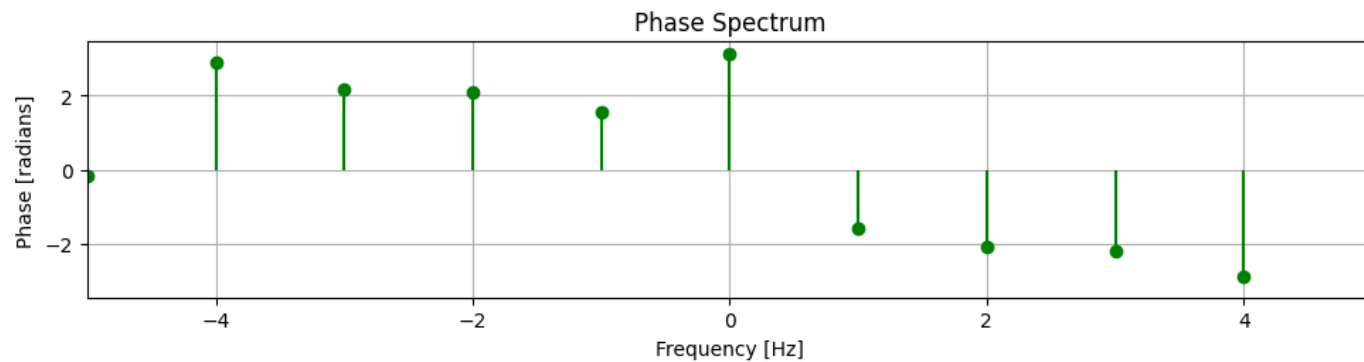
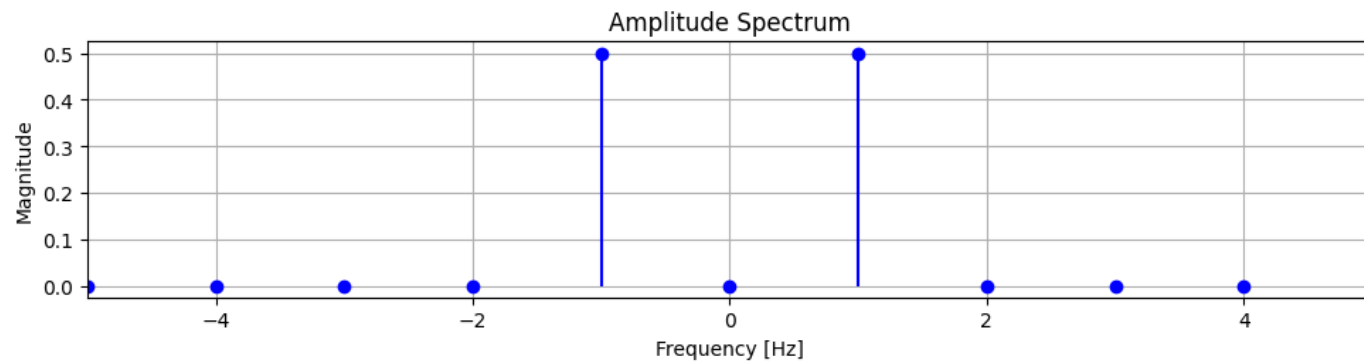
Frequency Domain:

- Graphically represent the frequencies contained in the signal.
- The graphical frequency representation of a signal is called the spectrum of the signal
- The amplitude spectrum is even. It is therefore symmetrical with respect to the ordinate axis.
- The phase spectrum is odd. It is therefore symmetrical with respect to the origin
- Obtained via **Fourier Transform**

Time versus frequency



Period 1s $\rightarrow$ Frequency of 1Hz



Why frequency domain?

- **Highlights underlying patterns in signals (e.g., dominant tones in music or voice)**
- **Enables noise removal (e.g., low-pass, high-pass filters)**
- **Crucial for compression algorithms (e.g., MP3, JPEG)**
- **Helps diagnose issues (e.g., detecting faulty machinery from vibrations)**
- **Facilitates feature extraction for AI/ML (e.g., Mel-spectrograms)**

Filters and Smoothing in Signal Processing

A filter is a system or algorithm that selectively allows certain frequencies to pass through while attenuating others. It is a fundamental tool in signal and audio processing

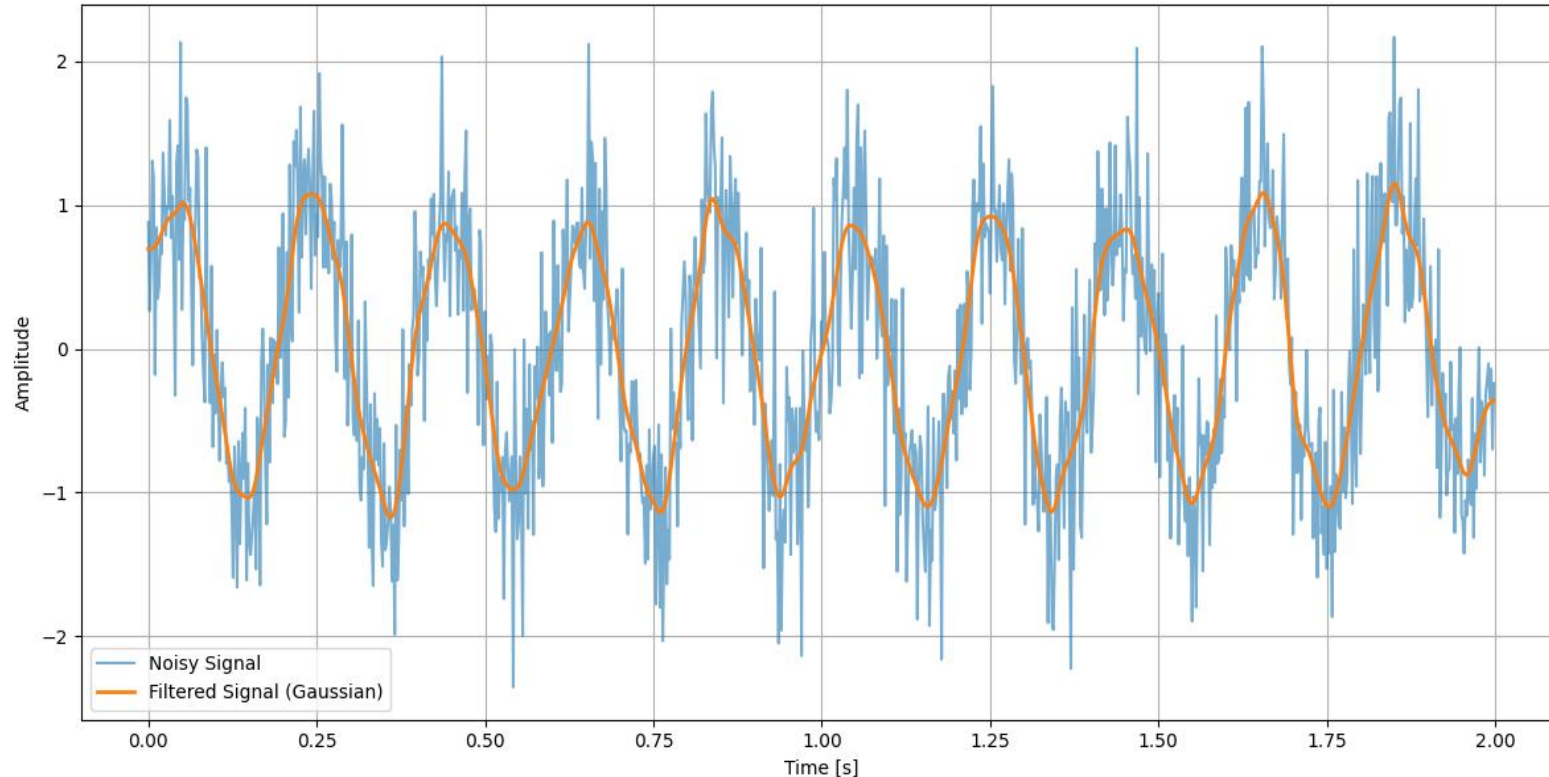
Type	Passes	Blocks	Use Case
Low-pass	Low frequencies	High frequencies	Removing high-frequency noise
High-pass	High frequencies	Low frequencies	Removing drift or DC offset
Band-pass	A specific frequency band	Everything else	Isolating musical notes or channels
Band-stop	Everything except a band	A specific frequency range	Removing hum or specific interference

Example: To remove background hiss from a voice recording, we can apply a **low-pass filter** that cuts off all frequencies above 8 kHz.

Filters and Smoothing in Signal Processing

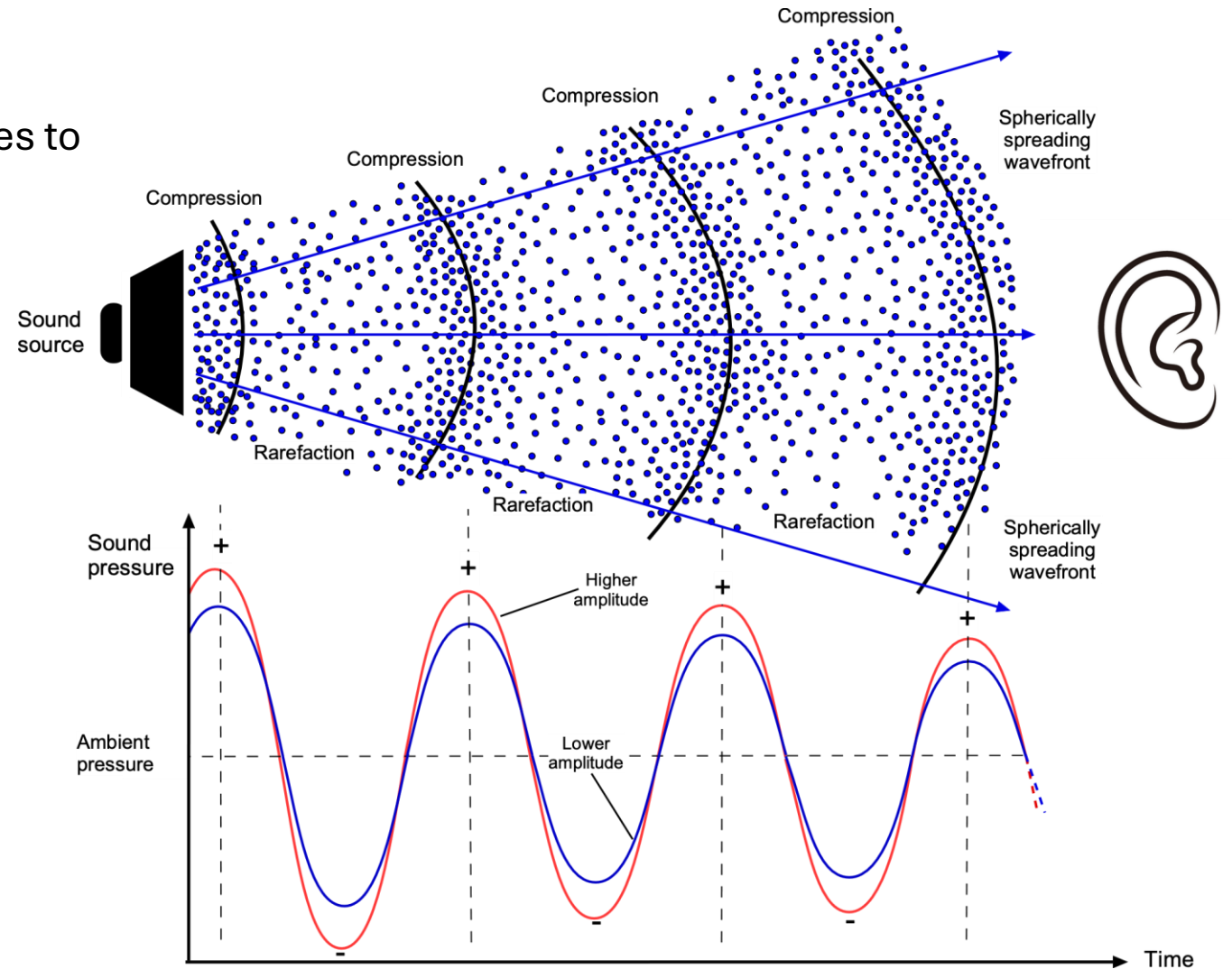
Filters can also operate in the time domain to reduce rapid fluctuations or noise:

Type	Description	Use Case
Moving Average Filter	Averages each point with its neighbors	Smooths short-term fluctuations
Gaussian Filter	Applies a weighted average using a Gaussian kernel	Smooths while preserving transitions

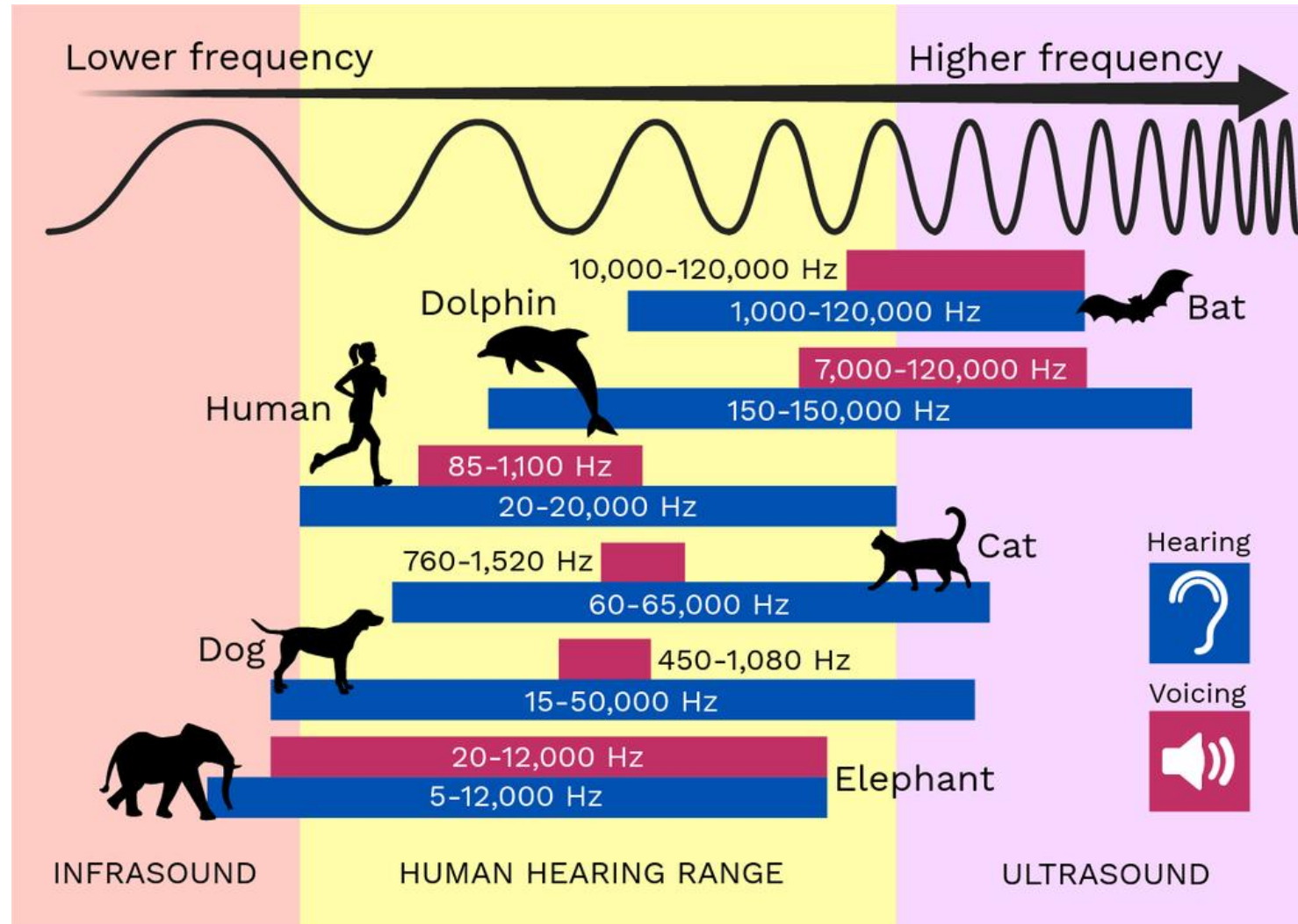


What is the sound?

- Sound is a mechanical wave produced by the vibration of an object which causes air molecules to oscillate.
- It propagates through a medium (like air) as oscillations of pressure.
- These oscillations create compression and rarefaction zones



What is the sound?



What is an audio signal?

Definition:

- An audio signal is a digital or analog representation of sound.
- It contains all the information needed to reproduce the sound.
- Typically stored in the time domain as a waveform.

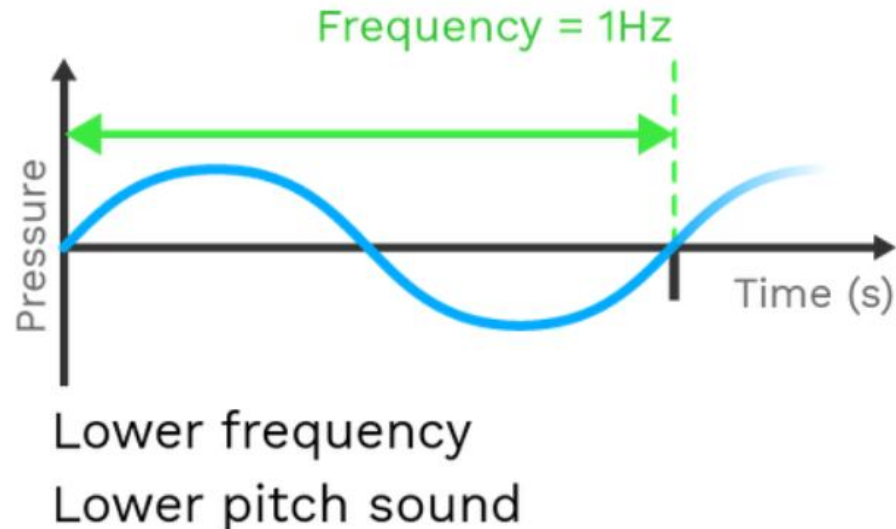
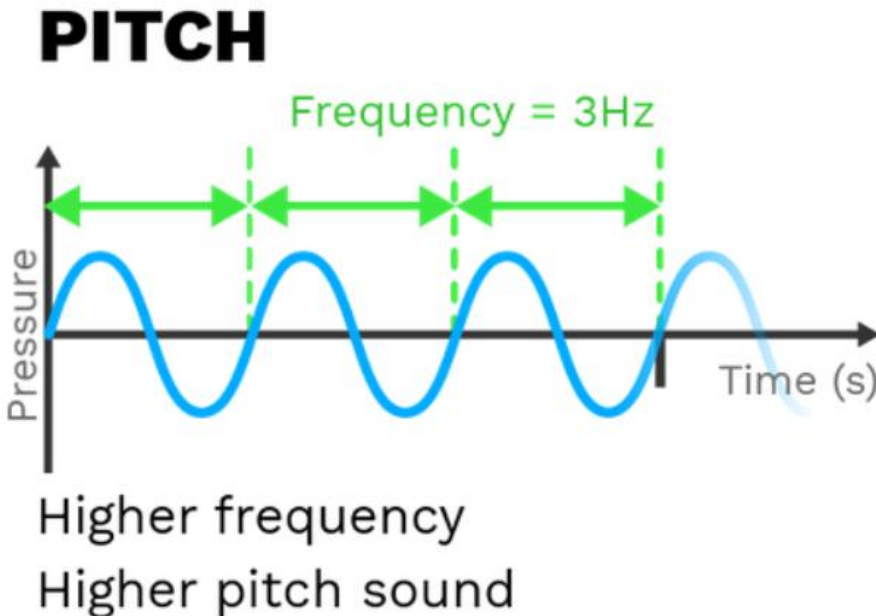
Common properties:

- Frequency
- Amplitude
- Pitch
- Loudness
- Timbre

Pitch

Definition:

- Pitch is the perceptual property of sound that allows us to classify it as "high" or "low".
- Technically, it corresponds to the frequency of the sound wave.
- A higher frequency is perceived as a higher pitch.
- A lower frequency is perceived as a lower pitch.
- Pitch is critical in music and speech processing: Melody, harmony, intonation, and tone depend on pitch



Sound Intensity & Human Perception

Definition:

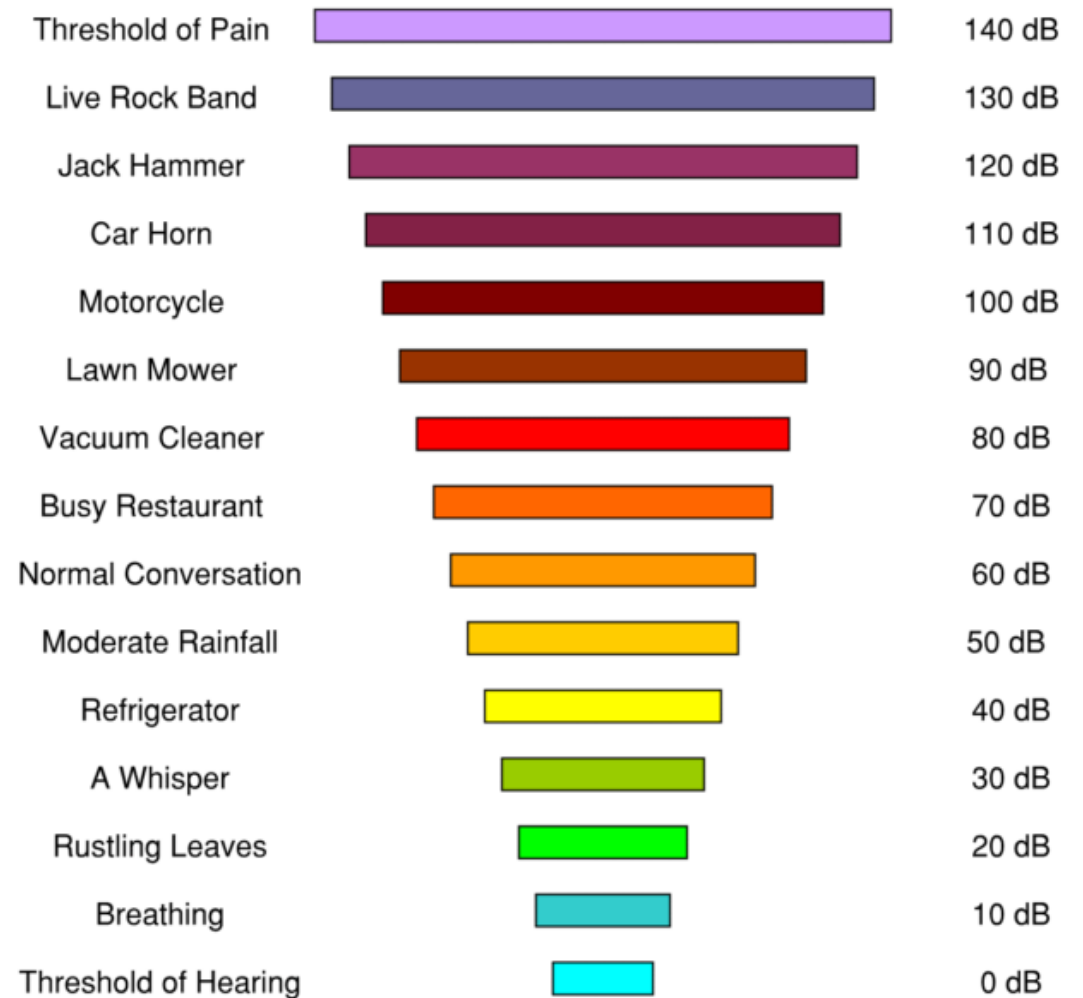
- **Sound Power:** Total sound energy emitted per unit time, measured in watts (W).
- **Sound Intensity:** Sound power per unit area, measured in W/m^2 .
- **Threshold of Hearing (TOH):** The quietest sound a typical human ear can detect. Defined as 10^{-12} W/m^2 .
- **Threshold of Pain (TOP):** The level at which sound becomes painful to hear. Defined as 10 W/m^2 .

Logarithmic Scale & Decibels

- The range of sound intensity humans can perceive spans over 13 orders of magnitude.
- To capture this wide range, we use a logarithmic scale, expressed in decibels (dB).
- A decibel (dB) is a relative unit that expresses the ratio between two values of intensity.
- For sound, we typically compare the intensity to the Threshold of Hearing (TOH).

Formula:

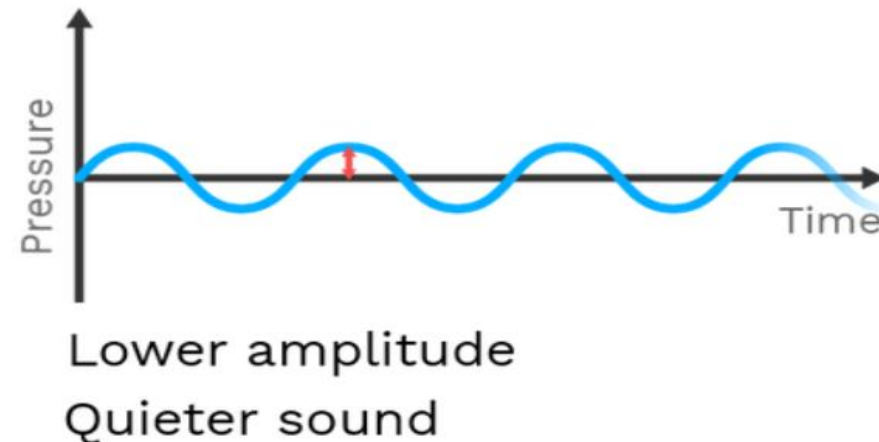
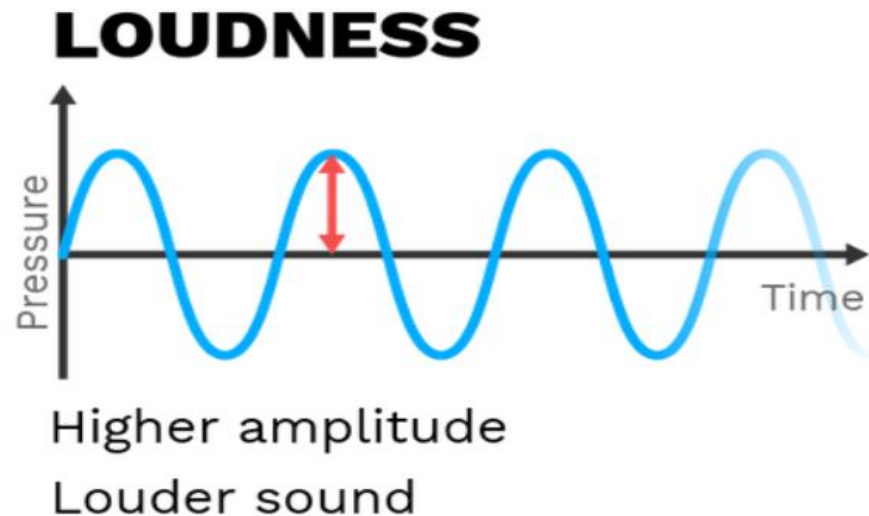
$$\text{dB}(I) = 10 \times \log_{10}(I / I_{\text{toh}})$$



Loudless

Definition:

- Loudness is the **perceived intensity** of sound — how loud or soft a sound feels to the human ear. Technically, it corresponds to the frequency of the sound wave.
- While intensity is a physical property (measured in W/m^2), loudness is a subjective experience.
- It depends not only on the intensity but also on the frequency and the sensitivity of the ear.
- The same intensity can be perceived louder or softer depending on the pitch
- Doubling the perceived loudness $\neq$ doubling the intensity.
- A ~ 10 dB increase is generally perceived as "twice as loud".



Timbre

Definition:

It is the **quality** or **color** of a sound that makes it unique — even when pitch and loudness are the same.

Ex: A **violin** and a **flute** playing the same note at the same volume sound different because of their **timbre**.

Timbre is shaped by:

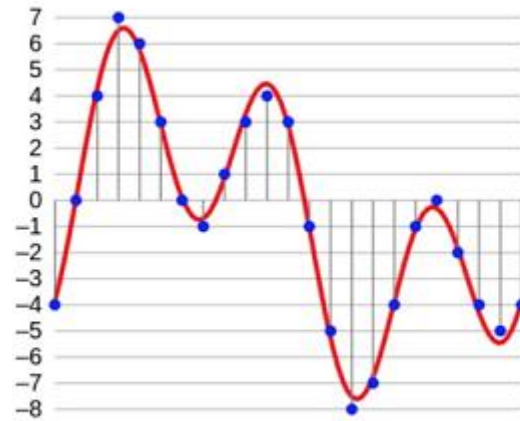
- **Harmonic content:** The number and relative strength of overtones (frequencies above the fundamental). Richer harmonics give a brighter or fuller sound.
- **Envelope (Attack, Decay, Sustain, Release - ADSR):** Describes how the sound evolves over time. A piano and a violin may sustain a note differently.
- **Spectral envelope:** How energy is distributed across the frequency spectrum, affecting the tonal balance and brightness of the sound.

Timbre is critical for:

- Identifying instruments or speakers
- Emotion recognition in speech
- Voice and music classification in ML systems

Digitization

- **Real signals are analog (continuous) in nature**
- **Electronic and computer systems are digital systems**, that is they are able to process information of a digital nature. Need to **convert analog signals to digital signals** in order to be able to process them computationally
- **Digital signals are a sequence of discrete values** and can only take **a finite number of values**.
- This process is called **digitization** or **Analog to Digital Conversion (ADC)** and has main steps:
 - **SAMPLING**
 - **QUANTIZATION**



Sound wave



[-4, 0, 4, 7, 6, 3, 0, -1, 1, 3, 4, 3,
-1, -5, -8, -7, -4, -1, 0, -2, -4, -5, -4]

Array

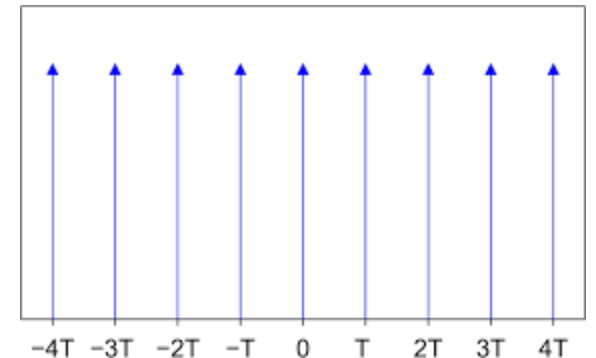
Sampling

Sampling consists of taking at precise times, often regular and equidistant, instantaneous values of the analog signal (audio signal) to be digitized.

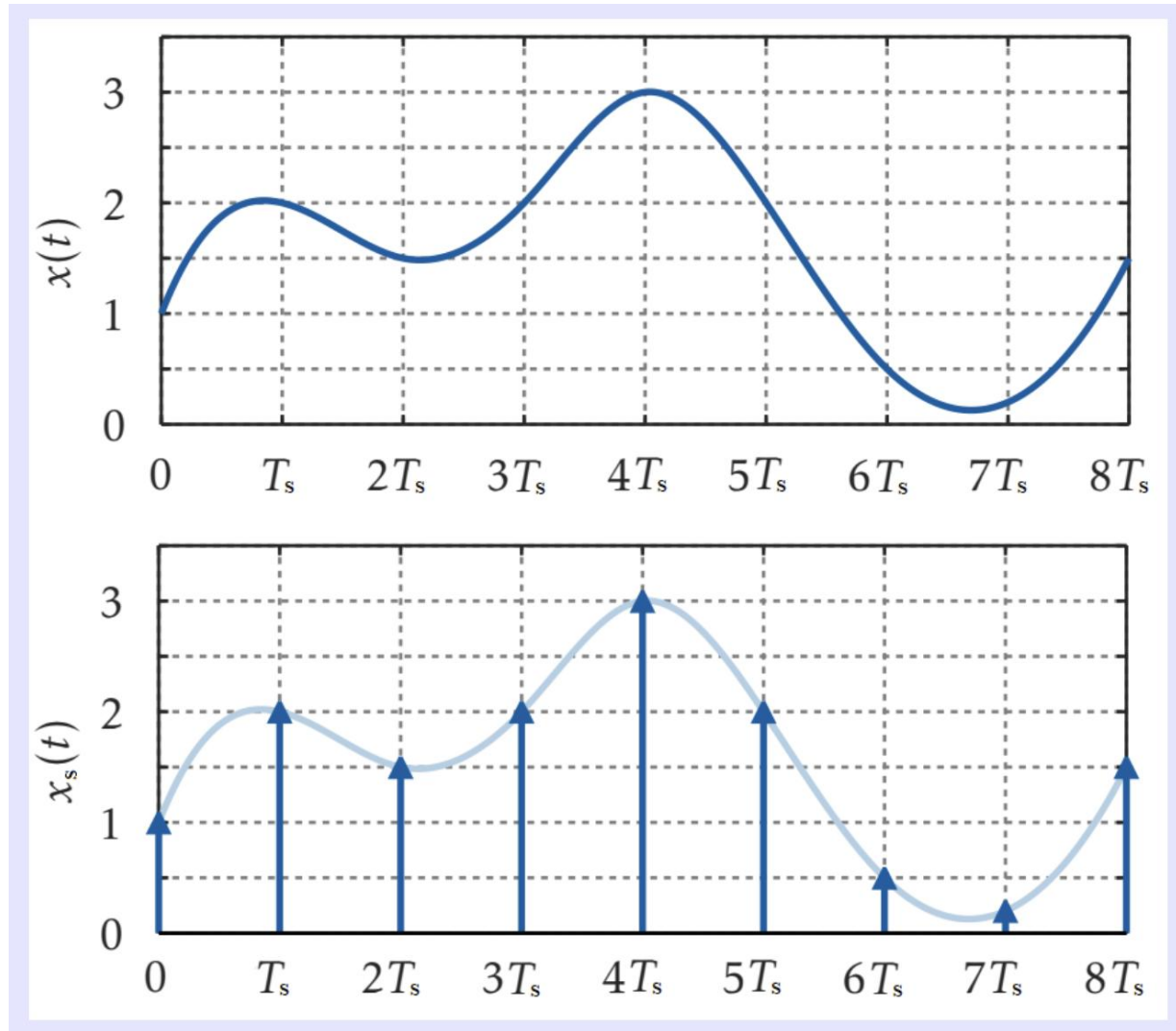
- The time between two samples is called the sampling period, denoted T_s .
- The sampling frequency or sampling rate (F_s) is the number of samples taken per second: $F_s = 1/T_s$
- Smaller T_s (higher F_s) means more accurate signal representation — but also more data
- Sampling inherently introduces sampling error, which depends on the chosen sampling rate.
- Mathematically, ideal sampling is modeled by **multiplying** the continuous signal $x(t)$ by a Dirac comb:

$$x_s(t) = x(t) \cdot \sum_{n=-\infty}^{\infty} \delta(t - nT_s)$$

Attention: Convolution of a signal with a Dirac comb serves to periodize it



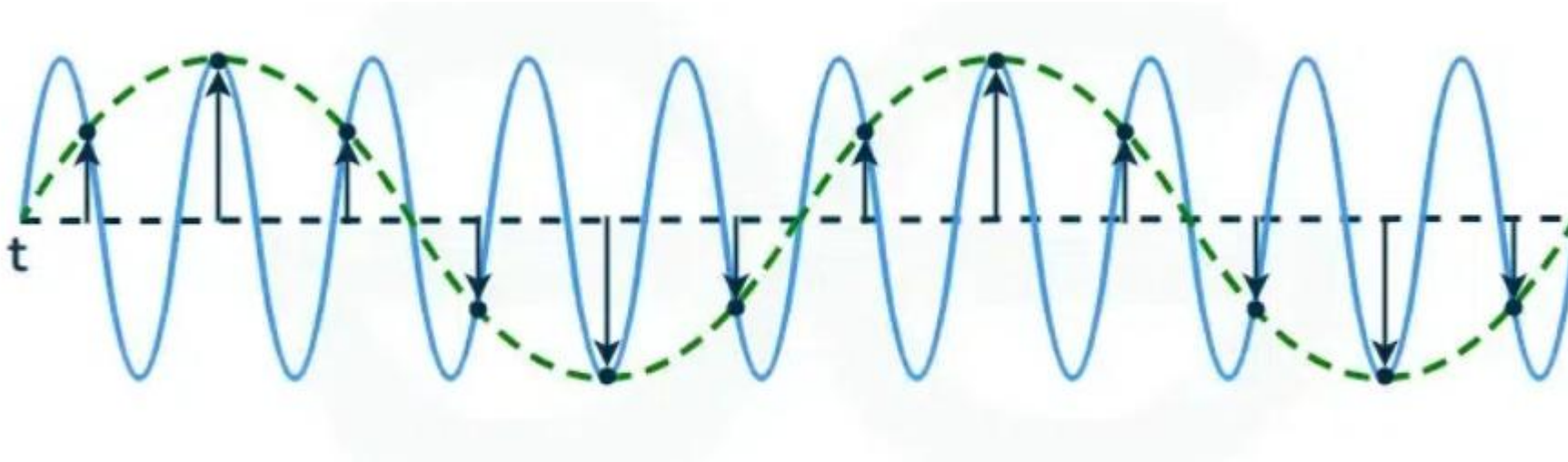
Sampling



Shannon–Nyquist criterion

When we sample a signal, its frequency spectrum becomes periodic (repeats at intervals of the sampling rate).

If the sampling frequency F_s is too low, spectral components overlap : this is called spectral overlap or **aliasing**.



Even though the green dashed curve looks "good", it is not the original signal. It's a false signal introduced by undersampling : this is the alias.

-> **SHANNON-NYQUIST CRITERION**

Shannon–Nyquist criterion

- **Shannon–Nyquist criterion:**

$$F_s \geq 2F_{\max}$$

Where $F_{\max}$ is the highest frequency present in the original signal.

If this condition is not respected, there will be spectral overlap, which will cause the loss of information during the signal restitution.

- **Why does the CD technology use an F_s of 44100Hz?**

Shannon–Nyquist criterion

- **Shannon–Nyquist criterion:**

$$F_s \geq 2F_{\max}$$

Where $F_{\max}$ is the highest frequency present in the original signal.

If this condition is not respected, there will be spectral overlap, which will cause the loss of information during the signal restitution.

- **Why does the CD technology use an F_s of 44100Hz?**

The average human ear can hear frequencies from **20 Hz to 20,000 Hz (20 kHz)**.

So, $2 \times 20\text{kHz} = 40\text{kHz}$therefore, ensures **no aliasing** within the human hearing range.

Sampling

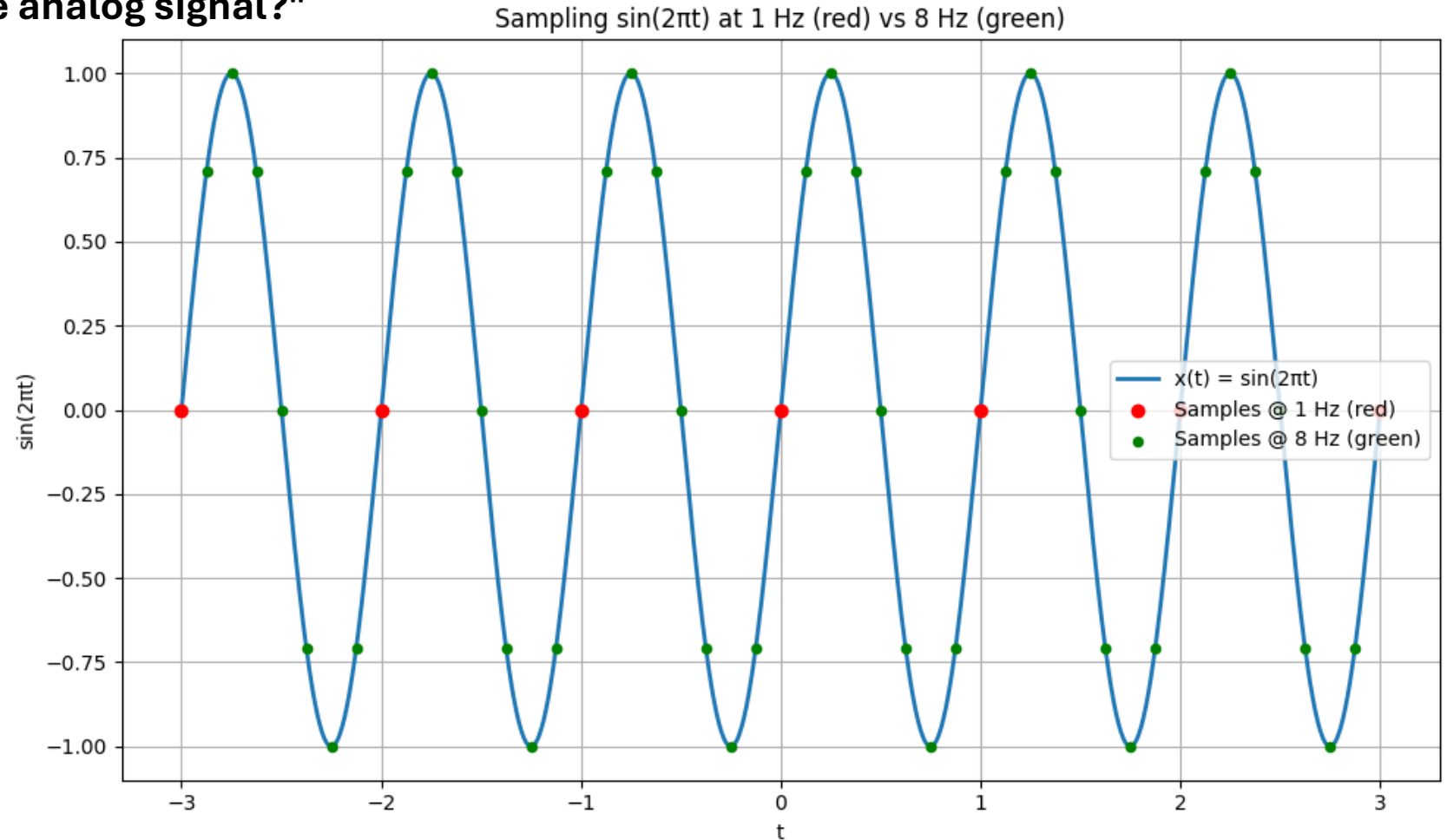
At what time intervals do we read the analog signal?"

Sampling of 1

$F_s = 1\text{Hz} = F_{\text{max}} \rightarrow$ aliasing

Sampling of 8Hz (1pt every 0,125s)

$F_s = 8\text{Hz} \gg F_{\text{max}} \rightarrow$ no aliasing



Quantization

Quantization is the process of mapping each sampled amplitude to the nearest value within a finite set of discrete levels: How precisely do we store each sample's amplitude?

- After sampling (taking measurements at regular time intervals), we still have infinite-precision amplitudes.
- Quantization reduces these to a finite number of values → necessary for digital storage.

How It Works

- Choose a **bit depth** (e.g., 16-bit, 24-bit).
- The number of levels is: $\text{Levels} = 2^{\text{bit depth}}$ (e.g., 16-bit = 65,536 levels)
- Each continuous sample is **rounded** to the nearest level.

Bit Depth	Levels	Notes
8-bit	256	Low quality, used in early games
16-bit	65,536	CD quality audio
24-bit	~16M	Professional audio, better dynamics

Quantization

Sampling times: $t = 0, 0.125, 0.25, 0.375, 0.5, \dots$

Sampled values:

$$x(0) = \sin(0) = 0$$

$$x(0.125) \approx 0.707$$

$$x(0.25) = 1$$

$$x(0.375) \approx 0.707$$

$$x(0.5) = 0$$

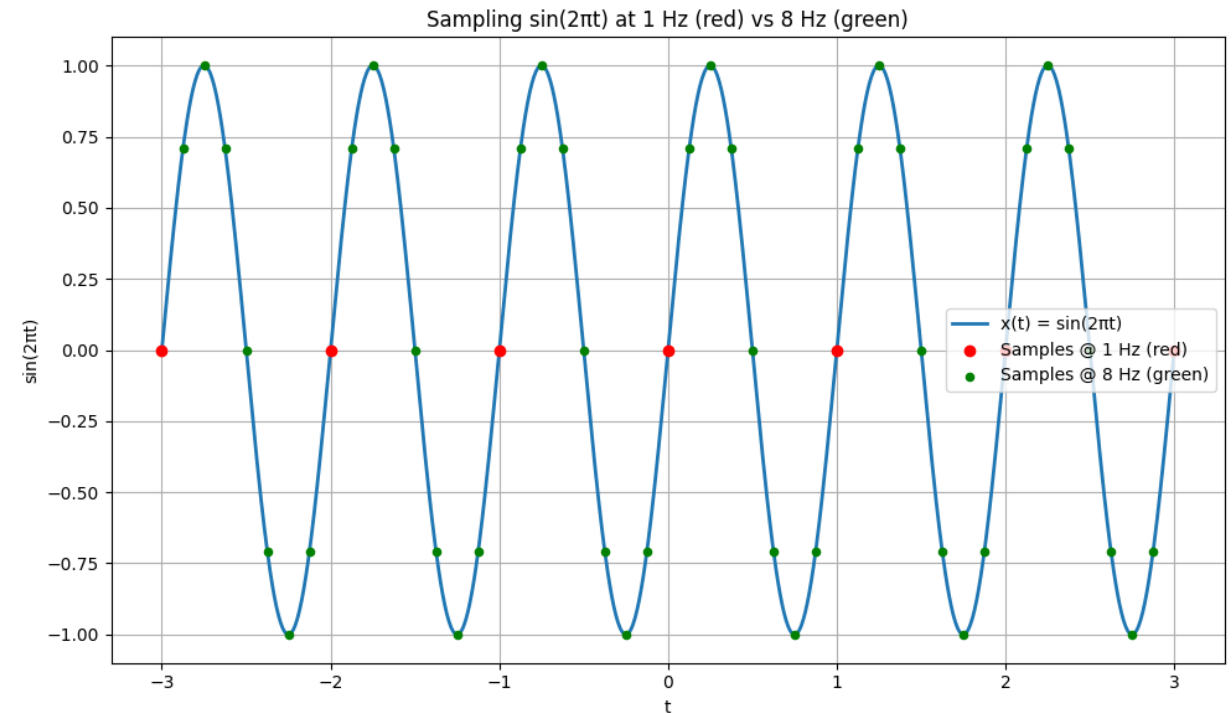
Quantization: Rounding Values to Discrete Levels

Real sampled values: e.g., 0.707, 1.0, 0

Suppose 8-bit quantization $\rightarrow$ 256 levels between -1 and 1

Level spacing ≈ 0.0078

-1.0000, -0.9922, -0.9844, ..., 0.7031, 0.7109, ..., 1.0000



Sampled Value

0.707

1.0

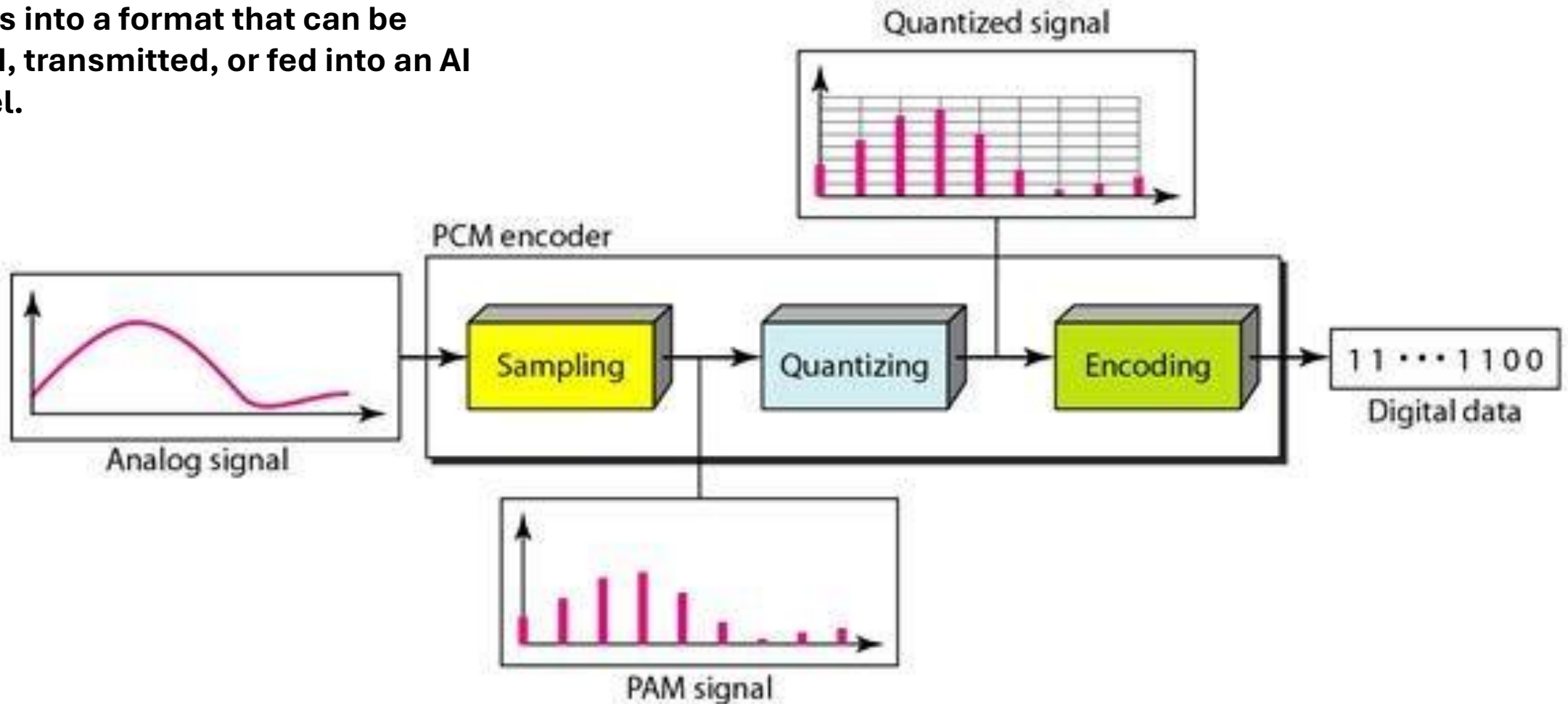
Nearest Quantized Level

~ 0.7031

1.0

Encoding

encoding transforms the quantized values into a format that can be saved, transmitted, or fed into an AI model.



Common Audio File Formats

Audio is typically considered **unstructured data**, but there are standard formats for storing and sharing it. Here are the most common:

Format	Description	Compression	Notes
WAV	Waveform Audio File Format (Microsoft/IBM)	None	High quality, uncompressed; large file size.
AIFF	Audio Interchange File Format (Apple)	None	Similar to WAV; also uncompressed.
MP3	MPEG-1 Audio Layer 3 (Fraunhofer Institute)	Lossy	Widely used; smaller files, reduced sound fidelity.
FLAC	Free Lossless Audio Codec (Xiph.Org Foundation)	Lossless	Compresses audio without quality loss.

Audio Features & Librosa

Session 2

Why Preprocess Audio?

Goal: Prepare raw audio data to be more meaningful and robust for AI models.

Raw audio is often:

- Noisy
- Different durations, loudness, background
- Variable in volume or silence
- Waveforms don't reveal pitch, energy, or phonemes directly

ML models need consistent, compact, and informative representation

Audio Preprocessing Pipeline

A typical audio preprocessing pipeline includes:

- **Resampling** : Make all audio signals use the same sampling rate.
- **Mono Conversion** : Standardize to one channel.
- **Silence Trimming** : Remove uninformative quiet sections.
- **Normalization** : Ensure consistent volume.
- **Framing & Windowing** : Divide into frames for temporal analysis.
- **Feature Extraction** : Extract ZCR, MFCCs (this session).

Resampling

Why Resample Audio?

- Audio files may have different sampling
- ML models require consistent input shape : we resample all audio to a standard rate (e.g., 16,000 or 22,050 Hz).

What is Sampling Rate?

- Number of audio samples per second.
- Nyquist-Shannon theorem: sampling rate must be $\geq 2 \times$ the highest frequency in the signal.

In Python (librosa):

```
y, sr = librosa.load("audio.wav", sr=16000)
```

#librosa.load() handles resampling automatically when sr is specified.

Note:

Choose $sr = 16000$ for voice-only tasks.

Use $sr = 22050$ or $sr = 44100$ for music/audio classification

Mono Conversion

Why convert to mono?

- Stereo files have 2 channels (left/right), which doubles the input shape.
- Most speech/audio tasks don't require spatial information → convert to mono (1 channel).

What Happens?

- Combines left and right channels:

$$y_{\text{mono}} = \frac{1}{2}(y_L + y_R)$$

In Python (librosa):

- `y, sr = librosa.load("audio.wav", mono=True)`

When to keep stereo?

- Music spatialization, localization, binaural audio, etc.

Silence Trimming

Why Remove Silence?

- Long leading/trailing silences are non-informative.
- They skew statistics and waste computation.

Particularly important for:

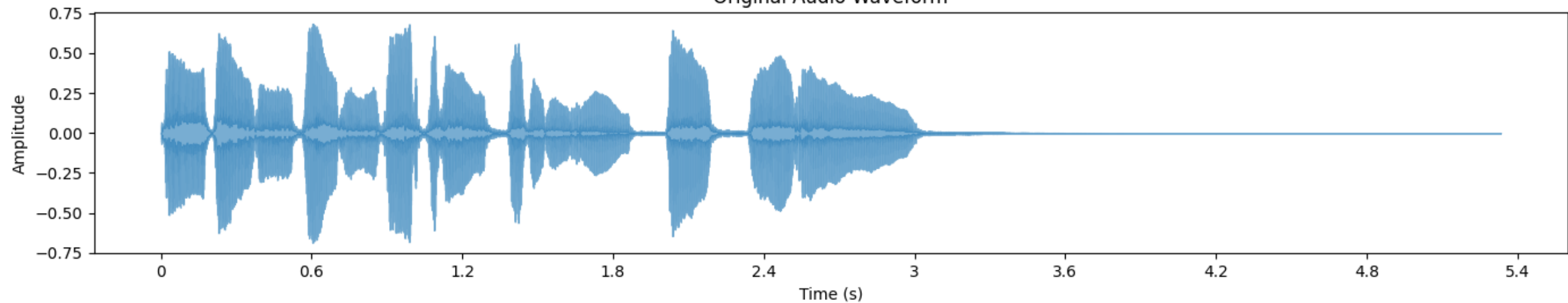
- Speaker ID
- Emotion recognition
- Classification

In Python (librosa):

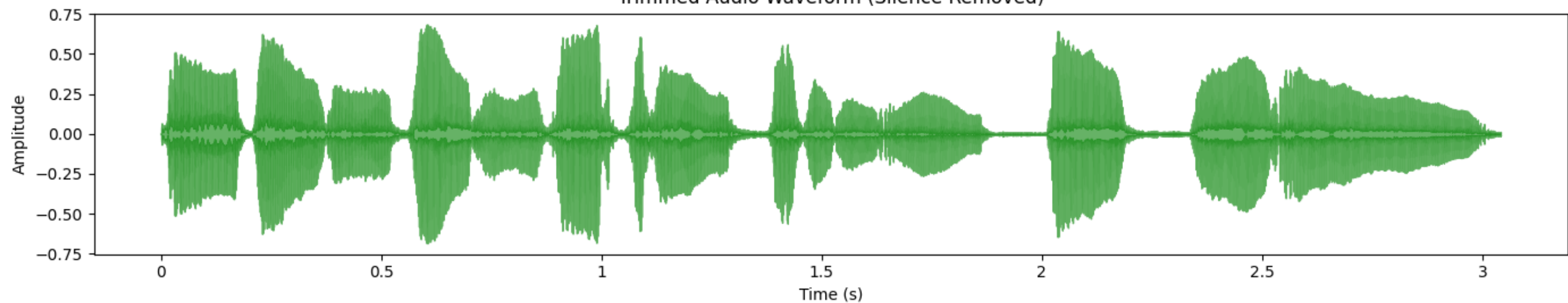
`y_trimmed, index = librosa.effects.trim(y, top_db=20)` # uses an energy-based threshold to detect silence.
top_db: how aggressive the silence removal is (higher = stricter).

Silence Trimming

Original Audio Waveform



Trimmed Audio Waveform (Silence Removed)



Normalization

Rescales signal so max amplitude = 1.0

- Audio files can have different loudness (amplitude).
- ML models may become biased toward louder inputs.
- usually common method: peak normalization $y' = y / \max(|y|)$

Task	Normalize?	Why
Speech Recognition	Yes	Avoid bias from microphone gain
Music Genre Classification	Yes	Ensures dynamics don't dominate
Voice Emotion Recognition	Maybe	Some features use loudness info
Speaker Identification	Maybe	Loudness may be speaker-specific

Framing

Framing is the process of dividing an audio signal into short overlapping segments called *frames*, so we can analyze how the signal changes over time.

A frame is a short, fixed-length segment of an audio signal, typically lasting between 10 to 50 milliseconds.

- Audio is non-stationary, meaning its properties change over time.
- But within a short frame, we can assume the signal is quasi-stationary → we can extract meaningful features (RMS, MFCC, etc.)

Parameter

Value

Frame Size 256 – 2048 samples (power of 2)

Frame Duration 25 ms (speech), 50 ms (music)

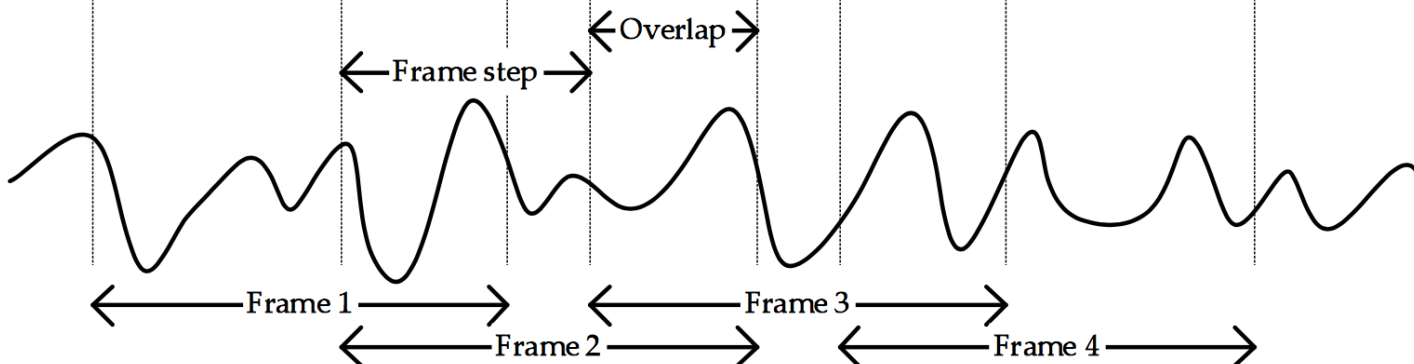
Sampling Rate 16,000 Hz

Hop Length 50% overlap (e.g. 512 hop for 1024 frame)

$$\text{Frame duration (s)} = \frac{\text{Frame size}}{\text{Sampling rate}}$$

=how much we move forward to get the next frame

Smaller hop = more overlap = better temporal resolution



Windowing

Each frame is usually multiplied by a **window function** before analysis to reduce edge effects (spectral leakage).

Common windows:

$$x_w[n] = x[n] \cdot w[n]$$

- Hamming: use for **time-domain energy tracking**, **speech onset detection**, or when using **short frames** (e.g., voice activity)

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right)$$

- Hann: use when you need **good frequency resolution** and **reduced spectral leakage**, such as for computing spectrograms or MFCCs. y detection)

$$w[n] = 0.54 - 0.46 \cos \left(\frac{2\pi n}{N-1} \right)$$

- Blackman: use for **music signal analysis**, **pitch detection**, and **applications where frequency content must be isolated cleanly**, but at the cost of frequency resolution.

$$: w[n] = 0.42 - 0.5 \cos \left(\frac{2\pi n}{N-1} \right) + 0.08 \cos \left(\frac{4\pi n}{N-1} \right)$$

Windowing

Create window functions

```
hann = np.hanning(frame_length)
```

```
hamming = np.hamming(frame_length)
```

```
blackman = np.blackman(frame_length)
```

Apply windowing

```
windowed_hann = frame * hann
```

```
windowed_hamming = frame * hamming
```

```
windowed_blackman = frame * blackman
```

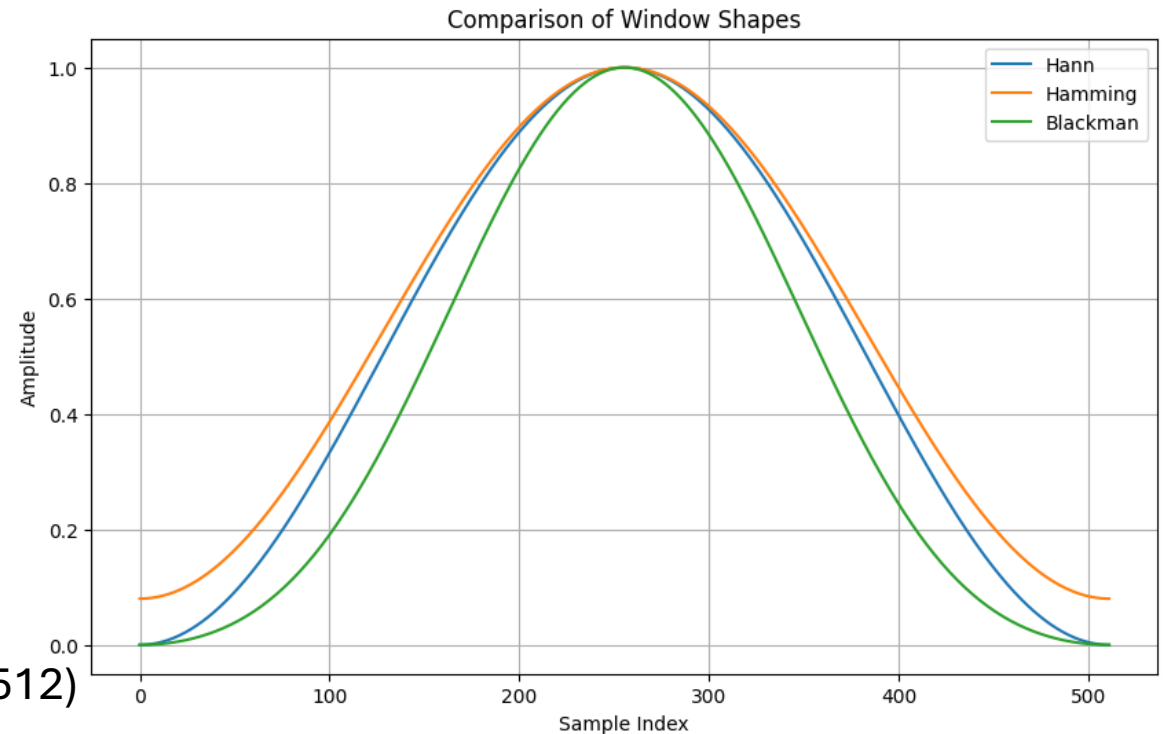
OR

```
window = librosa.filters.get_window("hann", frame_length)
```

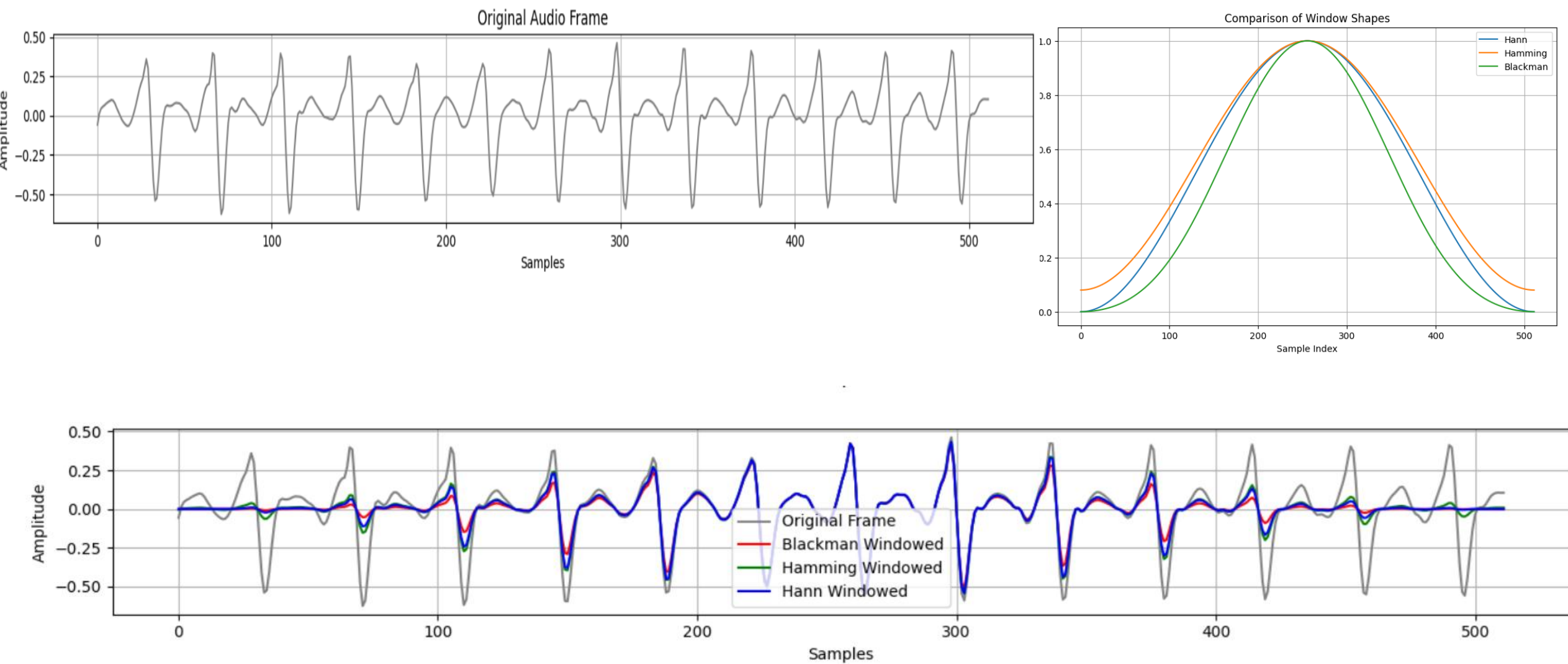
```
frames = librosa.util.frame(y, frame_length=1024, hop_length=512)
```

Apply windowing to each frame

```
windowed_frames = frames * window[:, np.newaxis]
```



Windowing



Harmonic/Percussive Separation

Purpose:

Separate harmonic (tonal, sustained) and percussive (transient, sharp) components of an audio signal. Harmonic: instruments, vocals, stable pitches Percussive: drums, claps, transient sounds

Why do this?

- Enhance audio classification (e.g., bird songs vs. drum beats)
- Improve transcription or source separation
- Reduce noise or isolate rhythmic patterns

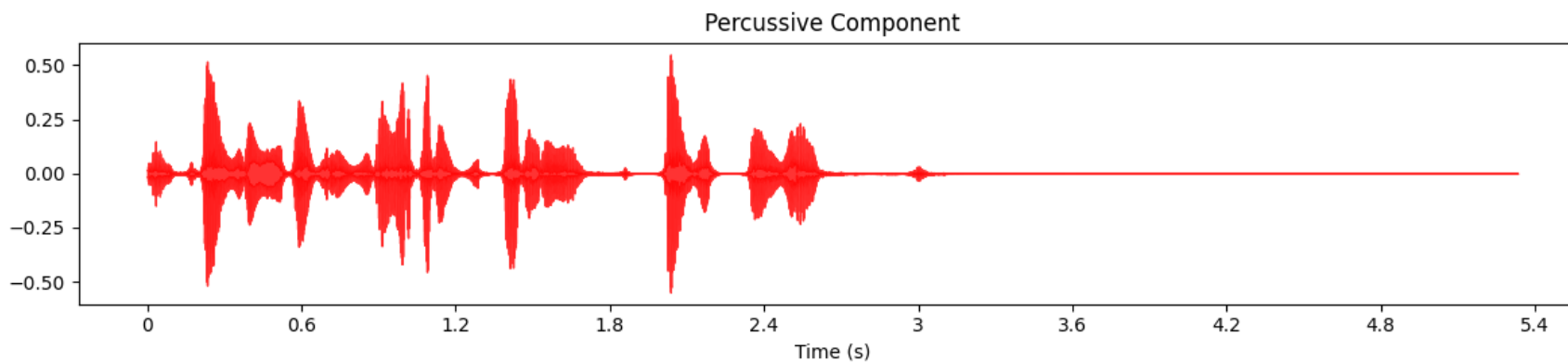
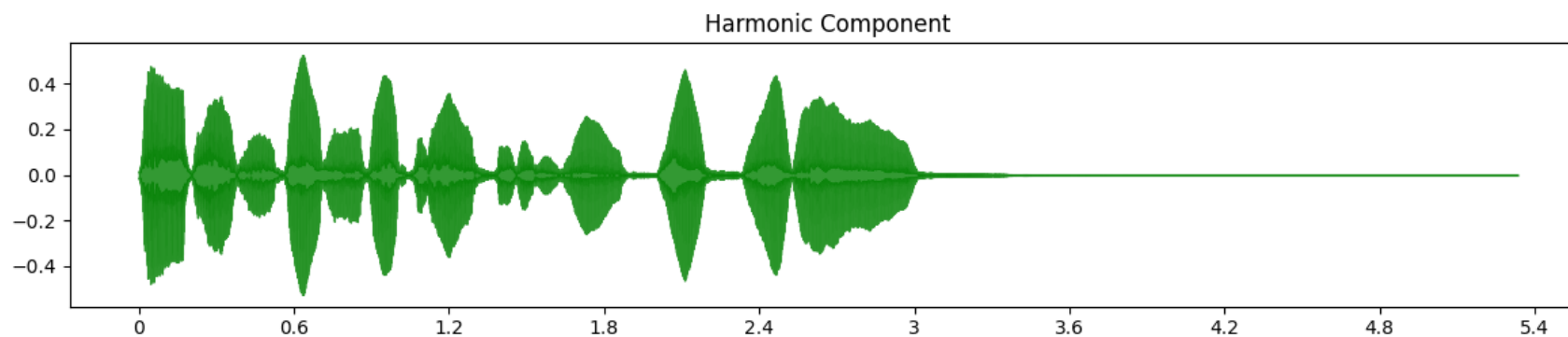
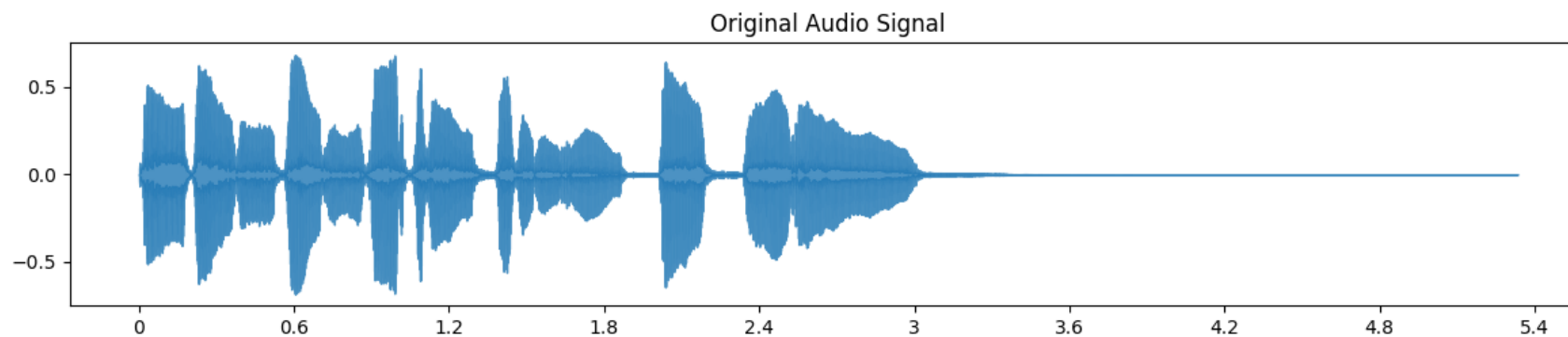
Method:

- Based on differences in spectrogram smoothness:
 - Harmonic: smooth across time (horizontal lines)
 - Percussive: smooth across frequency (vertical lines)

Python (librosa)

- `harmonic, percussive = librosa.effects.hpss(y)`

Harmonic/Percussive Separation



Feature extraction

Once audio is preprocessed, we extract descriptive features to represent the signal. Each feature captures a different signal property — energy, rhythm, pitch, timbre.

- **Time Domain**
 - Amplitude Envelope
 - Root Mean Square Energy (RMS)
 - Zero-Crossing Rate (ZCR)
 - Beat
- **Frequency Domain**
 - Band Energy Ratio
 - Spectral Centroid
 - Spectral Flux
 - Spectral Spread
 - Fourier transformation
- **Time-Frequency Domain**
 - Spectrogram
 - Mel Spectrogram
 - MFCC (Mel-Frequency Cepstral Coefficients)
 - Constant-Q Transform (CQT)

Time Domain: Amplitude Envelope

Tracks the outer shape of the waveform, giving a sense of how loud the signal is at each moment.

- Captures the dynamic range and loudness variation
- Useful for segmenting audio (e.g., separating notes, detecting speech activity)
- Cons: It is sensitive to outliers

How it works?

Divide the signal into short frames, then compute the maximum absolute amplitude for each frame.

$$AE(t) = \max_{n \in \text{frame}} |x[n]|$$

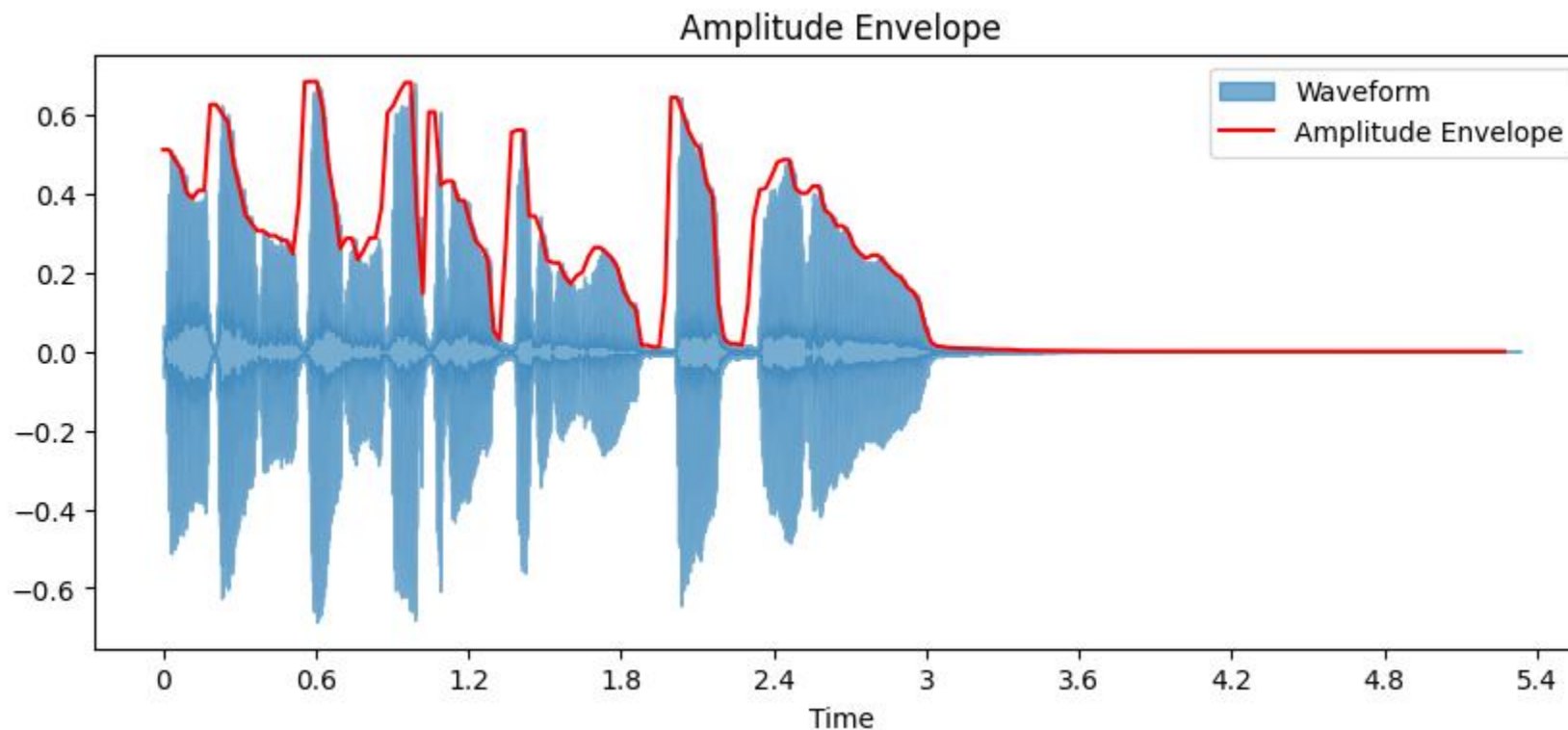
Use Cases:

- Onset detection in music
- Music classification
- Detecting quiet vs. loud segments in a recording

Time Domain: Amplitude Envelope

Python (librosa)

- `frames = librosa.util.frame(y, frame_length=frame_length, hop_length=hop_length)`
- `amplitude_envelope = np.max(np.abs(frames), axis=0)`



Time Domain: RMS

Measures the energy or loudness of the signal over time, accounting for fluctuations across each frame.

- Reflects short-term energy changes
- Less sensitive to outliers

$$\text{RMS}(t) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x[n]^2}$$

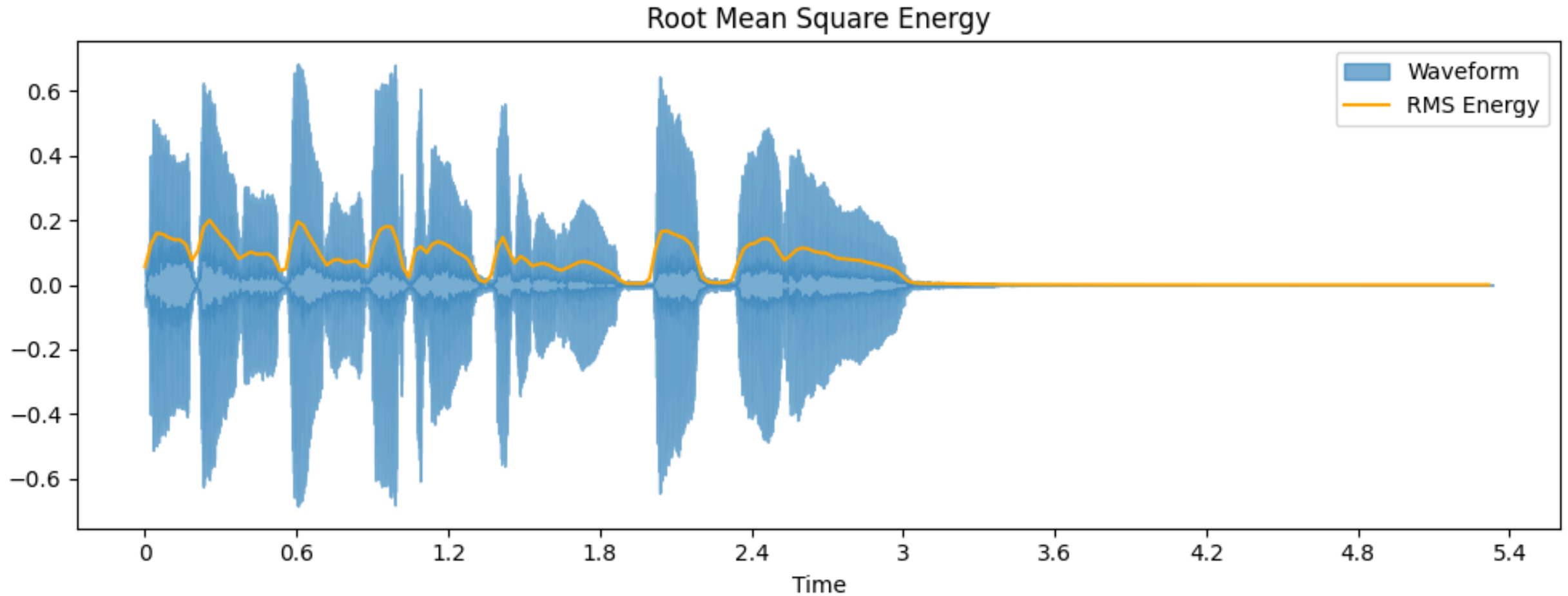
Use Cases:

- Dynamic range analysis
- Silence detection
- Energy-based segmentation (e.g., music/speech detection)

Python (Librosa)

- `signal_rms = librosa.feature.rms(y=signal, frame_length=FRAME_SIZE, hop_length=HOP_LENGTH)[0]`

Time Domain: RMS



Time Domain: Zero Crossing Rate

ZCR measures the rate at which a signal changes sign (i.e., crosses the zero axis). It gives insight into the noisiness or frequency content of a signal

- High ZCR: noisy or percussive sounds
- Low ZCR: harmonic, tonal, or smooth audio

$$ZCR = \frac{1}{T} \sum_{t=1}^T |s(t) - s(t-1)|$$

Where $s(t) = 1$ if s is positive

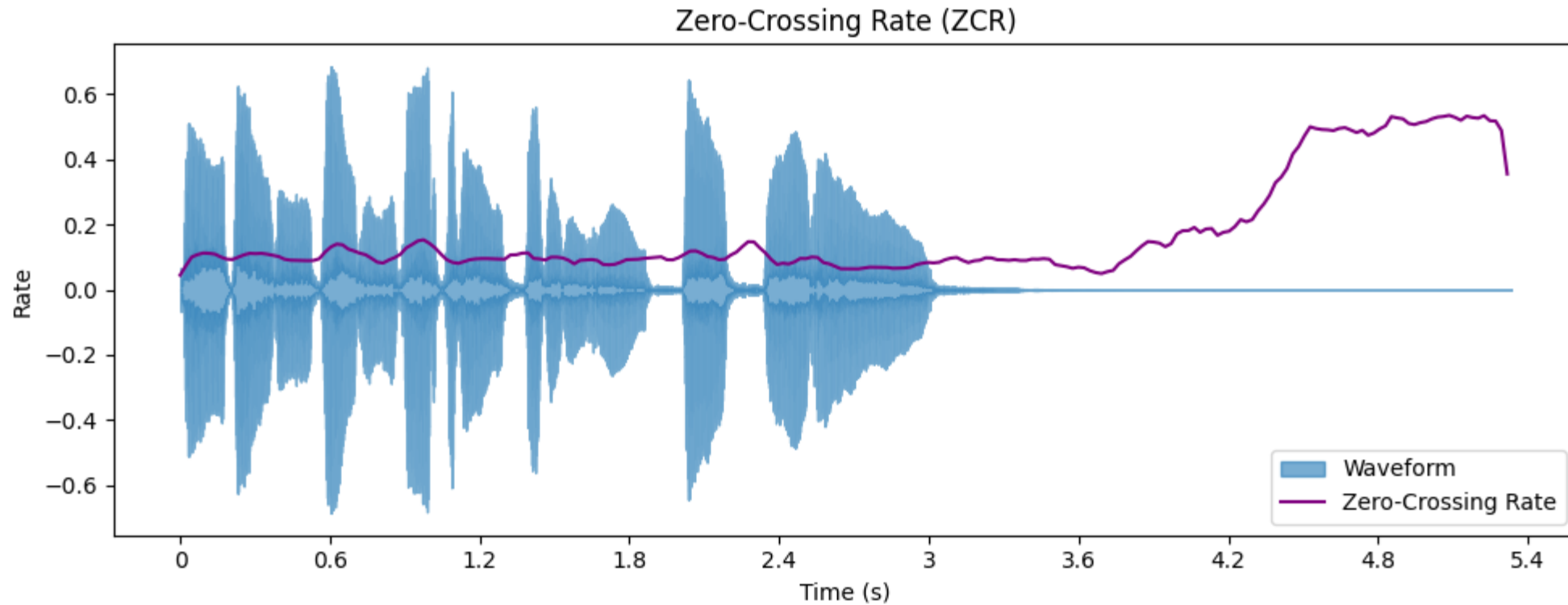
Use Cases:

- speech processing
- music recognition.

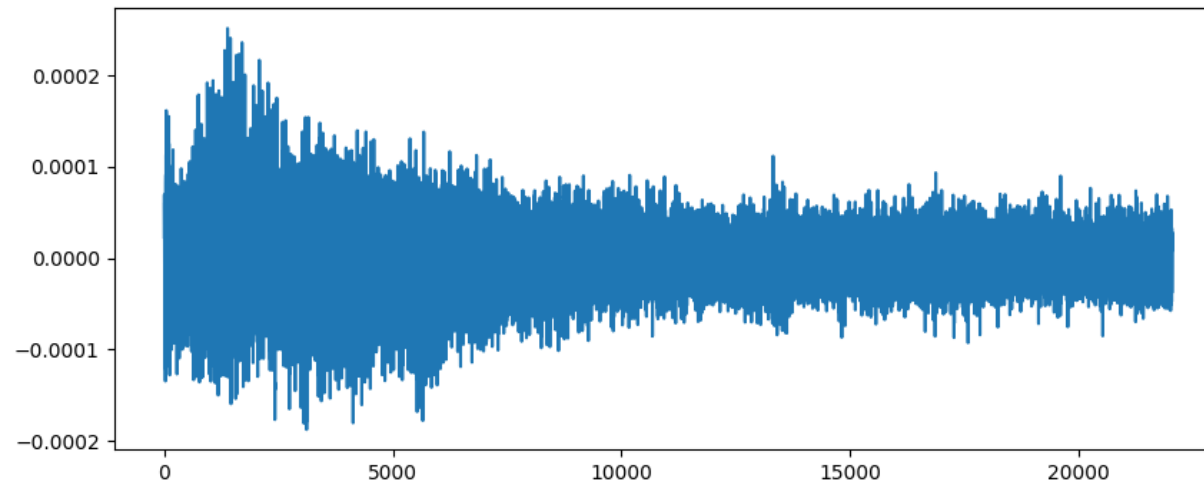
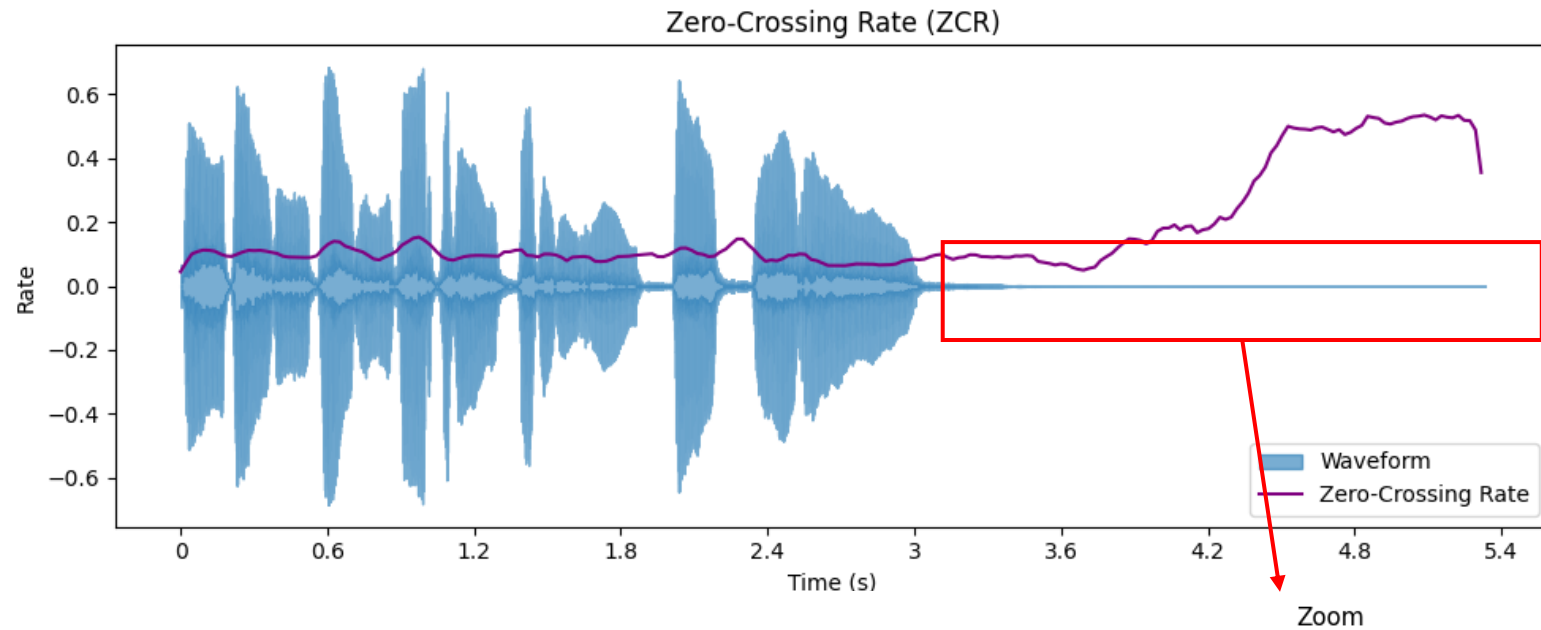
Python (Librosa)

- `zcr = librosa.feature.zero_crossing_rate(y, hop_length=hop_length)[0]`

Time Domain: Zero Crossing Rate



Time Domain: Zero Crossing Rate



Time Domain: Beat tracking

Beat tracking estimates the tempo and detects where beats occur in a piece of music.

How it works:

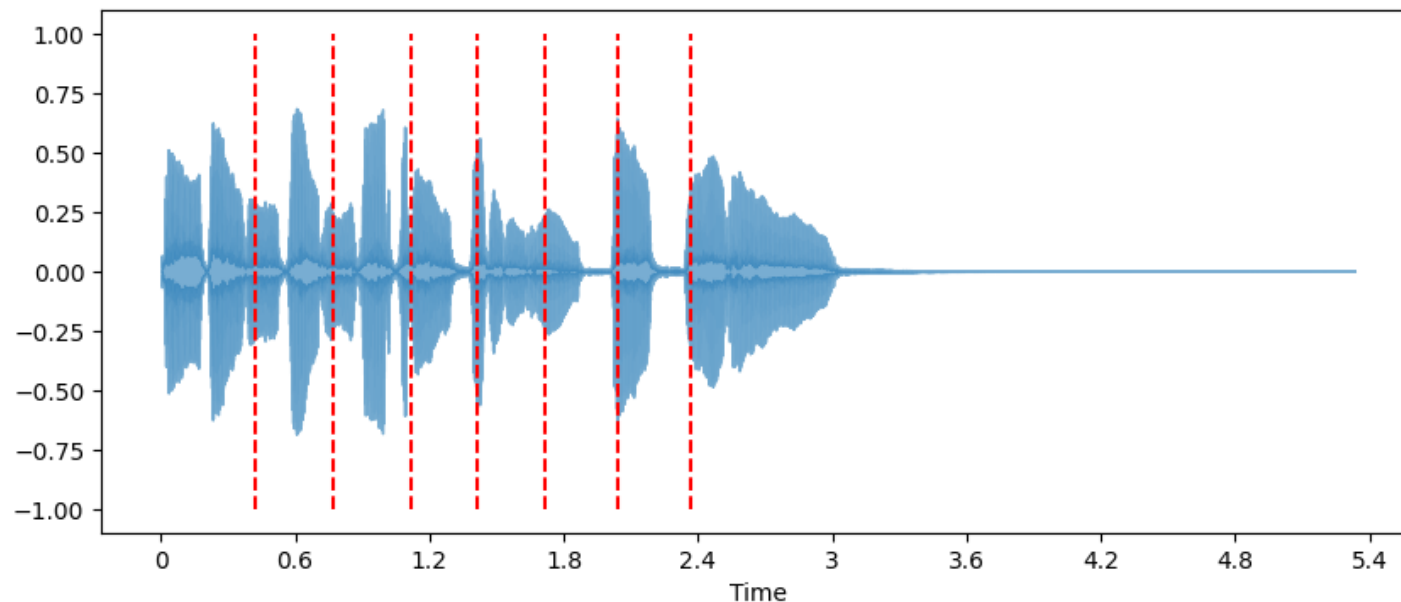
- Uses onset detection (sharp changes in energy)
- Applies dynamic programming or autocorrelation to find periodicities
- Produces a sequence of beat timestamps

Use Cases:

- Tempo estimation for DJ/music apps
- Audio segmentation (chorus, verse)
- Rhythm-aware music generation

Python (librosa)

- `tempo, beats = librosa.beat.beat_track(y=y, sr=sr)`
- `beat_times = librosa.frames_to_time(beats, sr=sr)`



Frequency Domain: FFT

Purpose: To convert a time-domain signal into its frequency-domain representation. The Fourier Transform breaks the signal into a sum of sinusoids. It tells you **how much** of each frequency exists in the signal and **how it is phased**.

Why use it?

- Understand the spectral content
- Detect dominant frequencies or harmonics
- Pre-requisite for most frequency-domain features

$$X(f) = \int_{-\infty}^{\infty} x(t) \cdot e^{-i2\pi ft} dt$$

High-Level

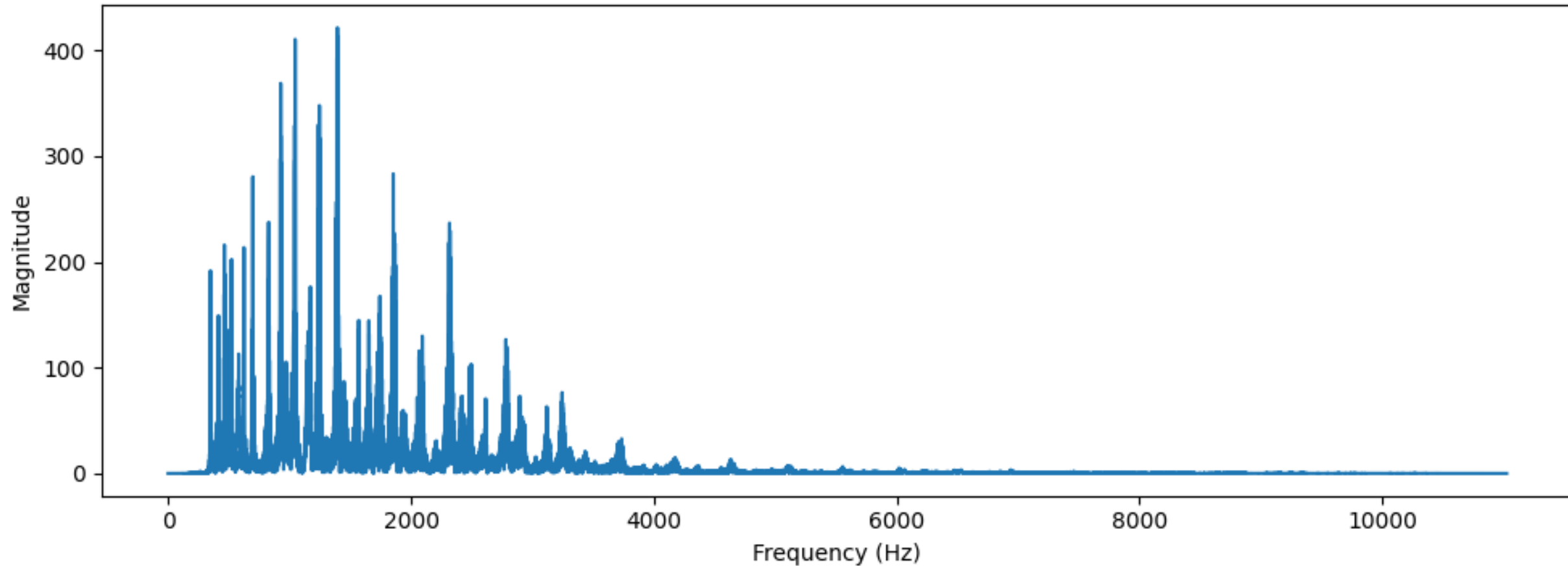
- The signal is compared to many sine waves with different frequencies.
- For each one, we measure **how strong and aligned** the match is — this gives us a **magnitude** and a **phase**.
- After choosing the best frequencies, we fine-tune the **phase** to better match the signal.
- Finally, we look at the **magnitude**:
 - A **high magnitude** means the signal is similar to that sine wave.

Python (numpy)

- `fft_result = np.fft.fft(y)`
- `magnitude = np.abs(fft_result)`
- `freq = np.fft.fftfreq(len(magnitude), 1/sr)`

Frequency Domain: FFT

Magnitude Spectrum (FFT)



Frequency Domain: Band Energy Ratio

Purpose:

Band Energy Ratio measures how the energy of a signal is distributed across low and high frequencies. It helps distinguish tonal vs. noisy or percussive content. Useful for speech music discrimination, music genre classification, mood classification

How it works:

- Apply Short-Time Fourier Transform (splits the signal into overlapping chunks, applies a window function, computes the FFT on each chunk,
- Divide the frequency spectrum into two bands: low and high
- Compute the energy ratio between the two bands for each frame

$$BER(t) = \frac{\sum_{f=0}^{f_c} |X(t, f)|^2}{\sum_{f=f_c+1}^F |X(t, f)|^2}$$

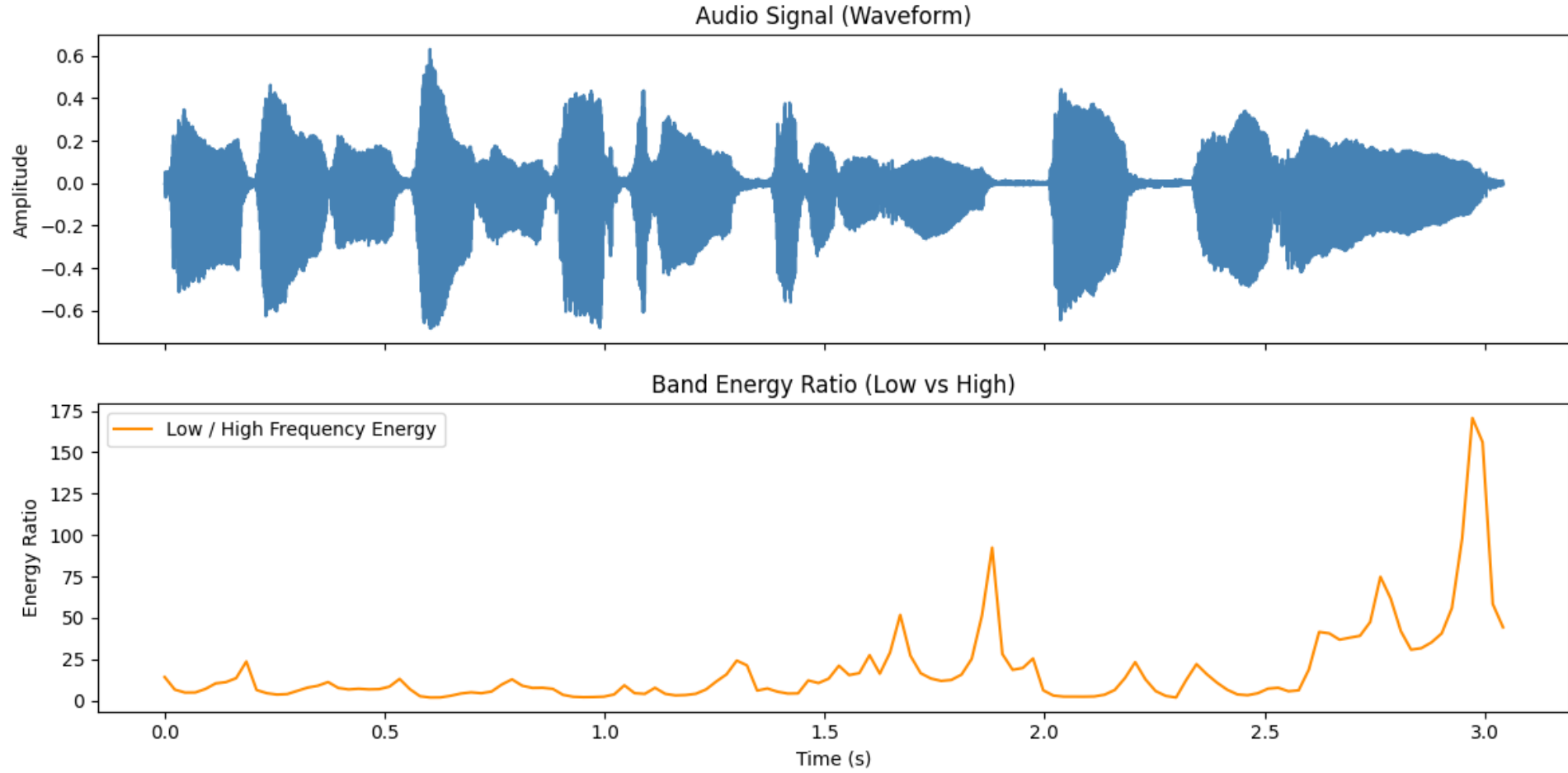
Where:

- $X(t, f)$ is the STFT of the signal
- f_c , is the cutoff frequency separating low and high bands
- F is the maximum frequency bin

Python (librosa):

```
STFTS = np.abs(librosa.stft(y))**2
freqs = librosa.fft_frequencies(sr=sr)
f_c = 2000
low_band = S[freqs <= f_c, :]
high_band = S[freqs > f_c, :]
ber = np.sum(low_band, axis=0) / (np.sum(high_band, axis=0) + 1e-6)
```

Frequency Domain: Band Energy Ratio



Frequency Domain: Spectral Centroid

Purpose:

The spectral centroid indicates where the “center of mass” of the frequency spectrum is, i.e., the frequency band where the most audio signal energy is concentrated (sound’s brightness).

Why it’s useful:

- High spectral centroid: bright or sharp sounds (e.g., cymbal, bird chirp)
- Low spectral centroid: dark or muffled sounds (e.g., bass, drum)

$$\text{centroid} = \frac{\sum_{n=0}^{N-1} f(n)x(n)}{\sum_{n=0}^{N-1} x(n)},$$

where $x(n)$ represents the weighted frequency value, or magnitude, of bin number n , and $f(n)$ represents the center frequency of that bin

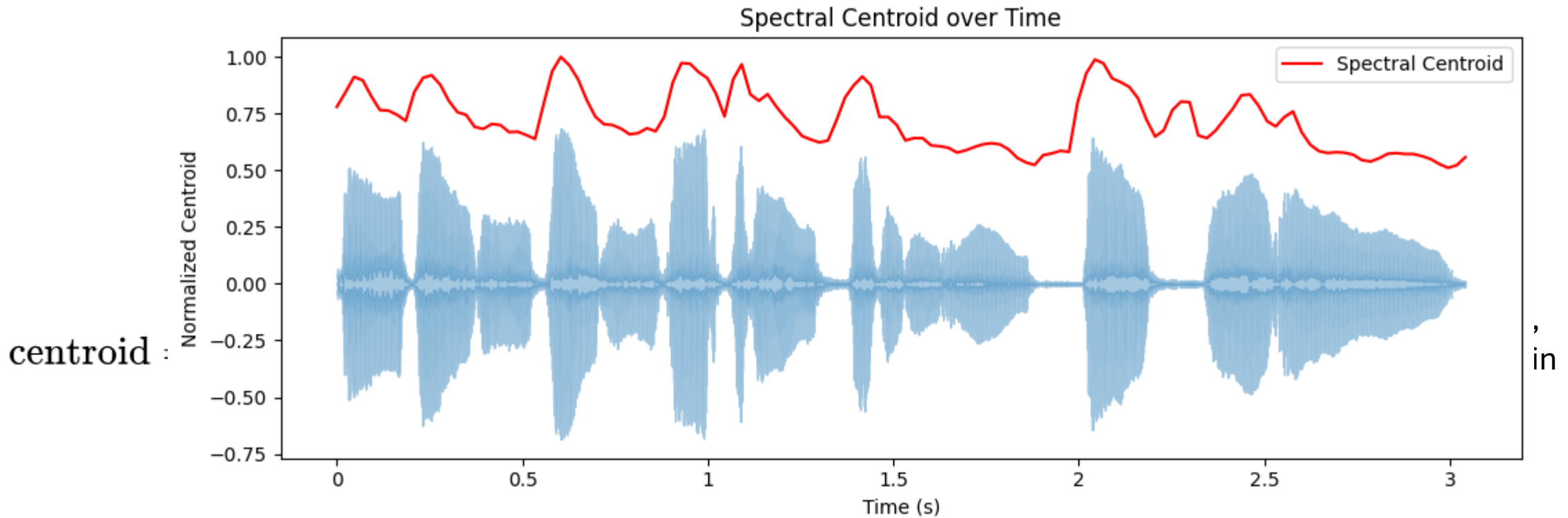
Python (librosa)

- `centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]`

Use Cases:

- Timbre and instrument classification
- Music genre identification
- Audio segmentation

Frequency Domain: Spectral Centroid



- The trumpet's attacks contain more high-frequency energy (sharp, bright)
- The sustains are more low-frequency dominant

Frequency Domain: Spectral Flux

Purpose:

Spectral Flux measures how quickly the power spectrum changes between adjacent frames. It detects onsets and dynamic changes in audio.

Why it's useful:

- High flux = sudden change in spectrum (e.g., onset, beat)
- Low flux = steady or sustained sound

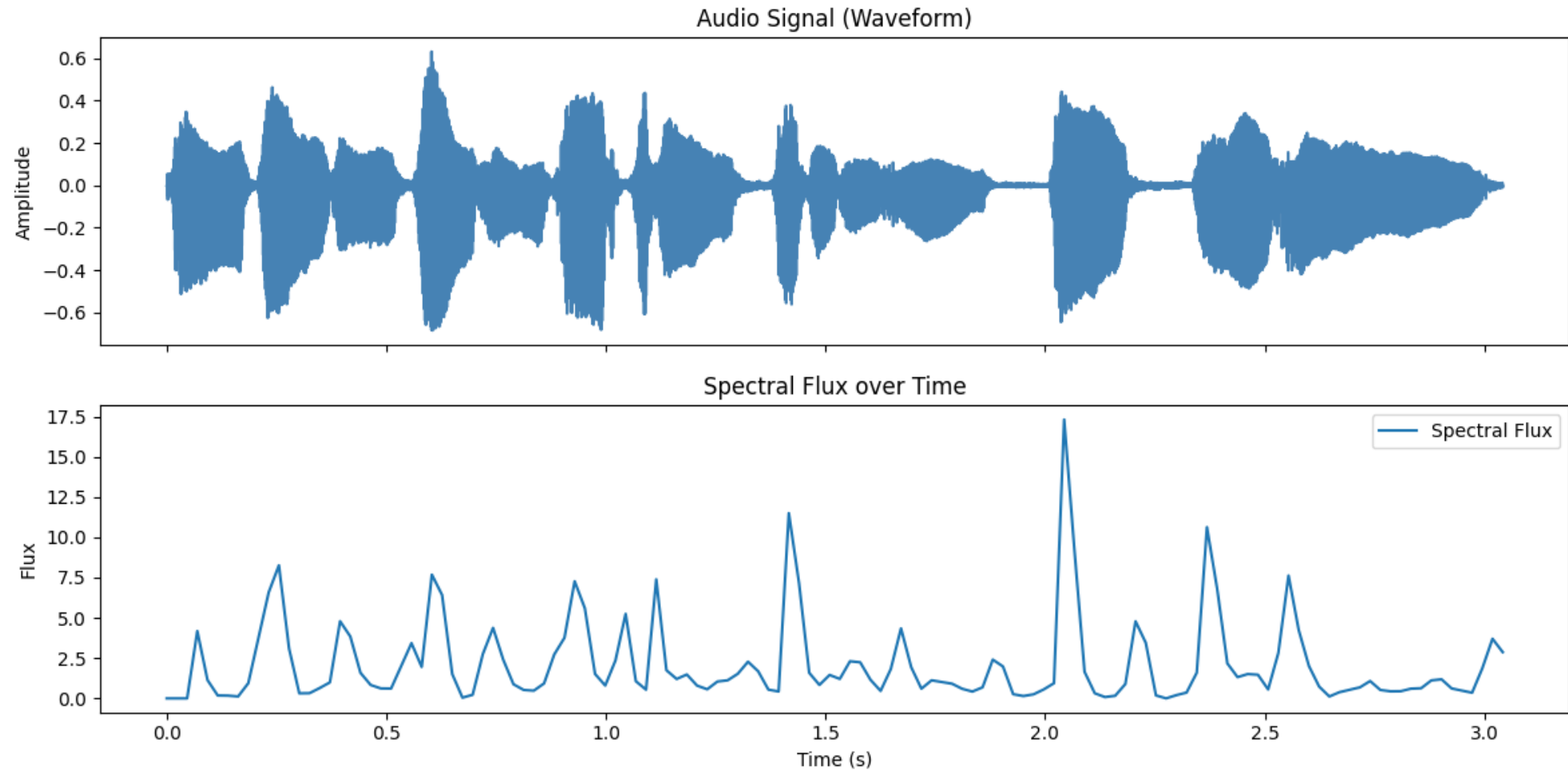
Python (librosa)

```
flux = librosa.onset.onset_strength(y=y, sr=sr)
```

Use Cases:

- Onset detection
- Beat tracking
- Rhythmic analysis

Frequency Domain: Spectral Flux



Frequency Domain: Spectral Spread

Purpose: Spectral Spread measures how spread out the frequency components are around the spectral centroid. It's a descriptor of the bandwidth of a sound.

Why it's useful:

- High spread: noisy, broadband signal (e.g., cymbal)
- Low spread: narrowband or tonal signal (e.g., sine wave)

$$\sqrt{\frac{\sum_{k=b_1}^{b_2} (f_k - \mu_1)^2 s_k}{\sum_{k=b_1}^{b_2} s_k}}$$

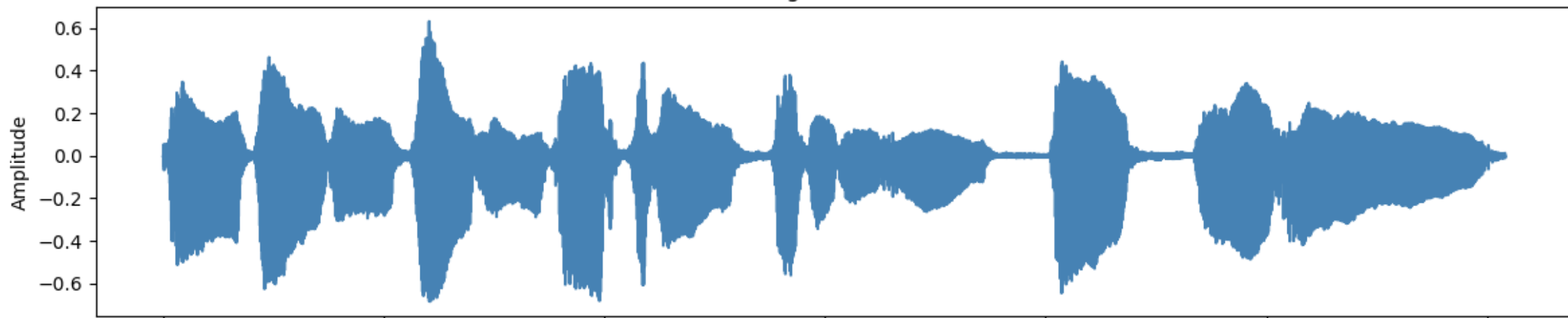
- f_k is the frequency in Hz corresponding to bin k
- s_k is the spectral value at bin k
- b_1 and b_2 are band edges, over which to calculate the spread
- μ_1 is the spectral centroid

Python (librosa)

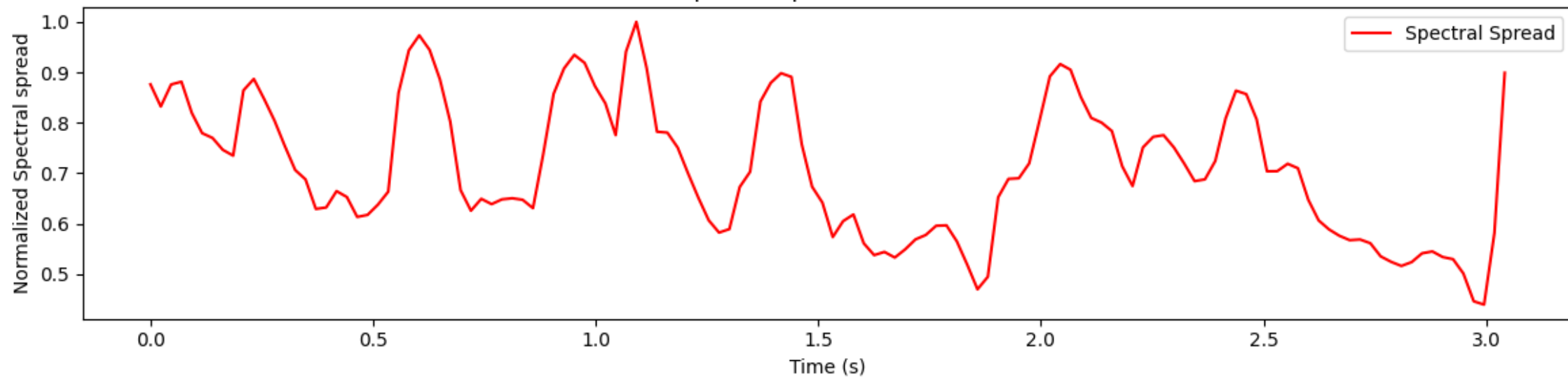
- `spread = librosa.feature.spectral_bandwidth(y=y, sr=sr)[0]`

Frequency Domain: Spectral Spread

Audio Signal (Waveform)



Spectral spread over Time



Time-Frequency Domain: Spectrogram

Purpose:

A spectrogram is a visual representation of the spectrum of frequencies in a sound as they vary with time. For ex, each color represents a musical note. Brighter colors mean the note is played more loudly.

Why it's useful:

- Shows how frequency content evolves over time
- Useful for visual analysis of speech, music, animal sounds, etc.

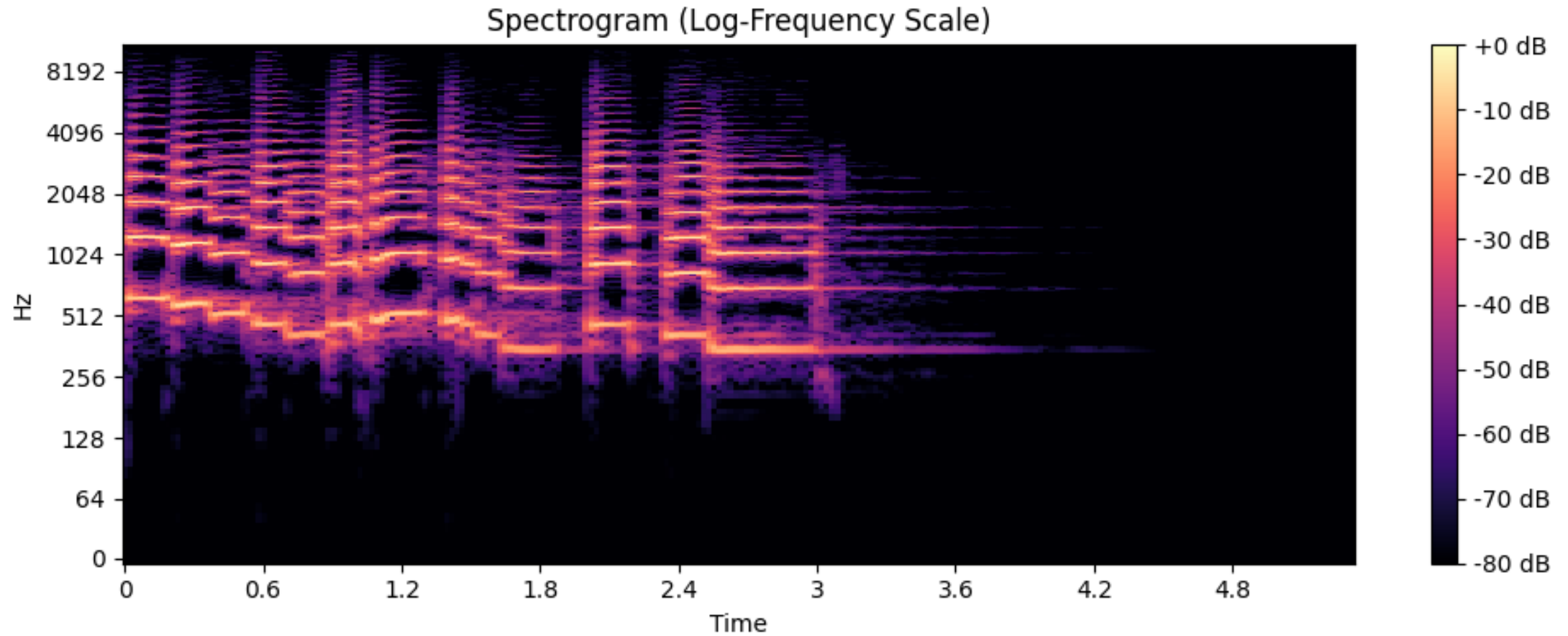
How it's created:

- Perform STFT (Short-Time Fourier Transform) on overlapping windows
- Display magnitude of frequency components over time

Python (librosa):

- Compute the Short-Time Fourier Transform (STFT) with a high number of FFT points for better frequency resolution
`D = librosa.stft(y, n_fft=4096)`
- Convert the complex-valued STFT to magnitude
`S = np.abs(D)`
- Convert to dB
`S_db = librosa.amplitude_to_db(S, ref=np.max)`
- Display Spect
`librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='log')`

Time-Frequency Domain: Log-spectrogram



0 dB = strongest energy in the signal
-80 dB= silent

Time-Frequency Domain: Mel Spectrogram

Purpose: A Mel-Spectrogram maps the frequency axis of a spectrogram to the Mel scale : which better aligns with human auditory perception (Humans perceive frequency better at low frequencies than at higher ones).

Why it's useful:

- Focuses on perceptually relevant frequency bands
- Used in speech, music, emotion and language recognition systems

Mel Scale: Maps frequency to a scale where perceived pitch differences are equal:

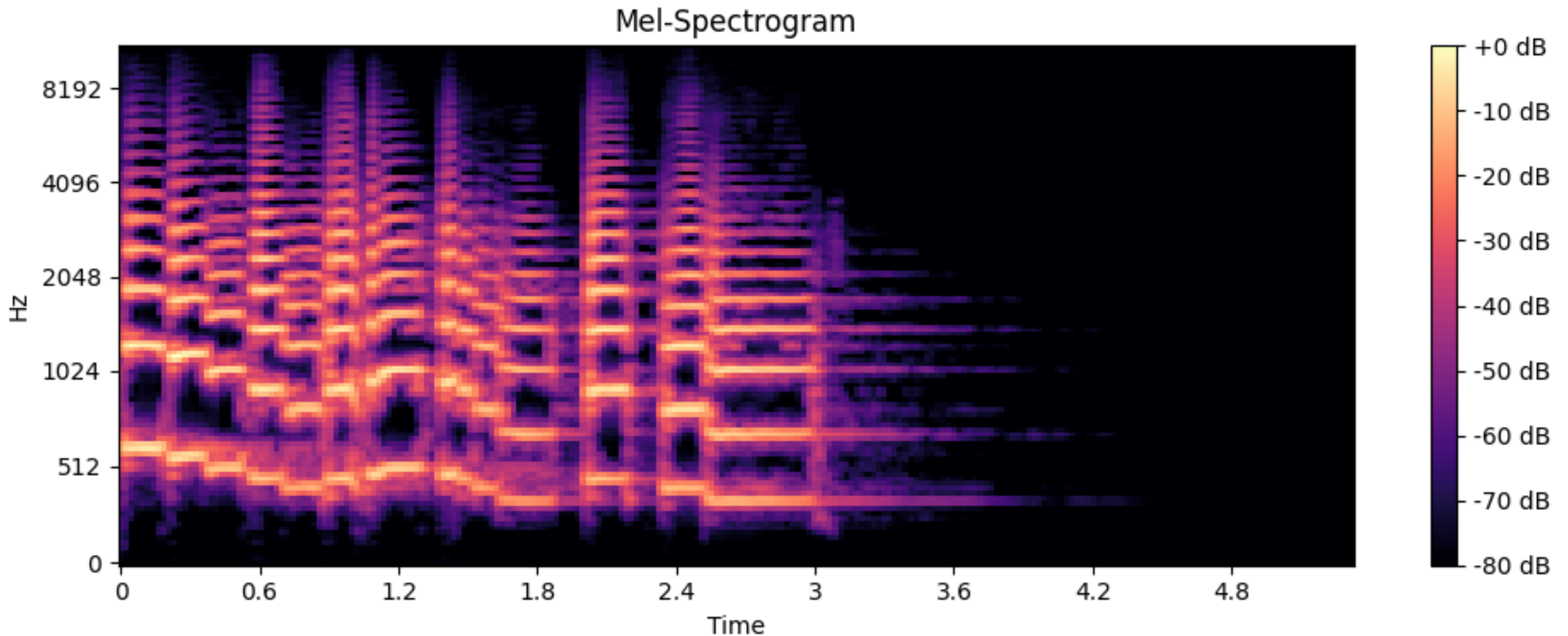
$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

Python (librosa):

```
S = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=2048, hop_length=512, n_mels=128)
```

n_mels: determines the vertical resolution of the Mel spectrogram.

Time-Frequency Domain: Mel Spectrogram



Time-Frequency Domain: MFCC

Purpose:

MFCCs capture the broad shape of the spectral envelope : that is, the general distribution of energy across frequency bands , and compress this shape into a small set of coefficients. This effectively describes the timbre or tonal texture of the sound in a way that aligns with how humans perceive audio

Why it's useful:

- Most widely used feature in speech and audio recognition
- Captures perceptually meaningful spectral shape

How it works:

- Compute Mel-Spectrogram
- Take the log of the Mel energies
- Apply Discrete Cosine Transform (DCT) to decorrelate features (a sum of cosine functions oscillating at different frequencies.)

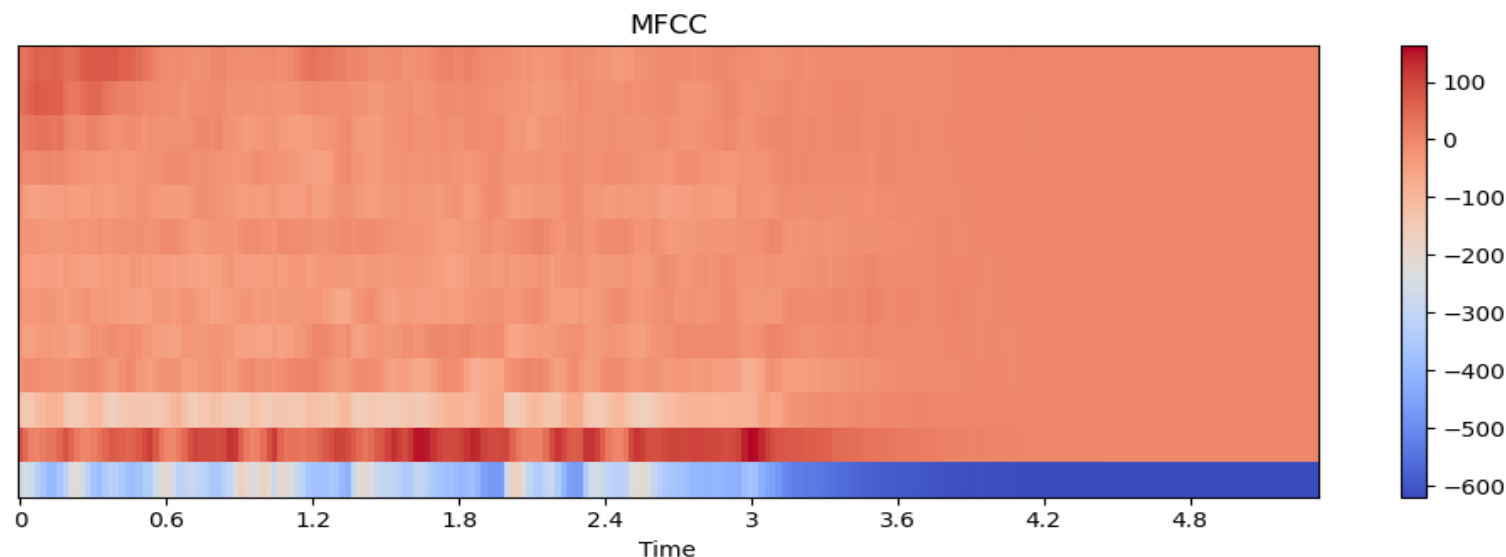
Python (librosa):

```
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
```

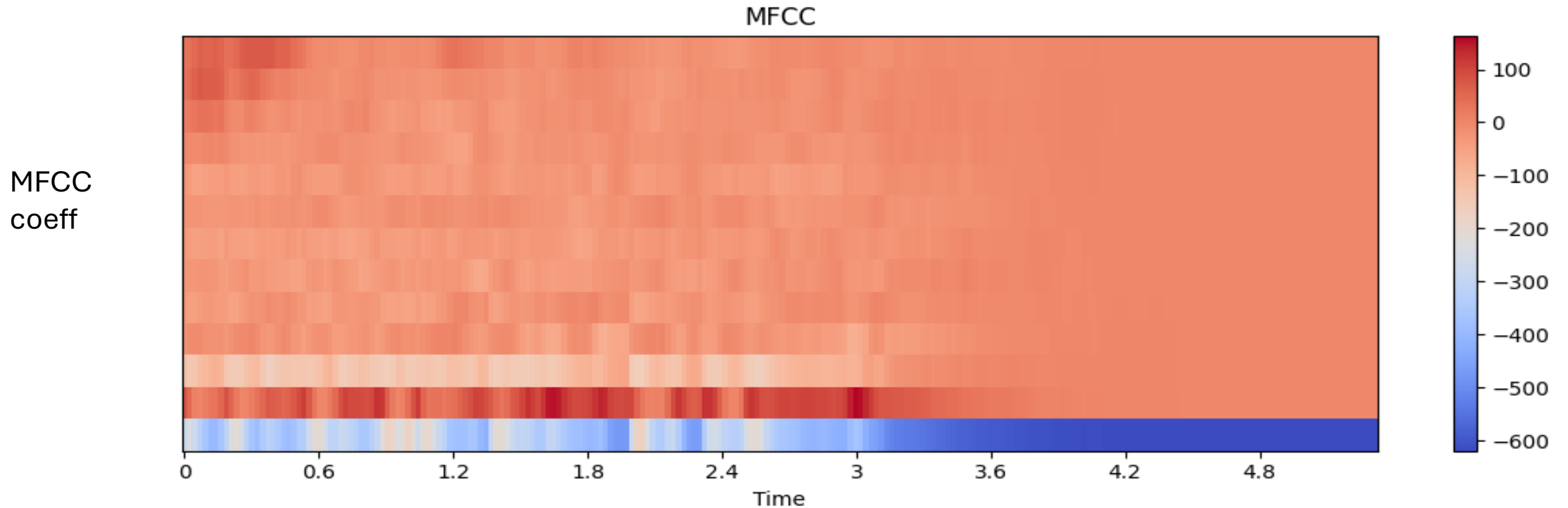
Time-Frequency Domain: MFCC

MFCC signification:

- MFCC 0 (bottom row):→ Represents overall signal energy.
- MFCC 1–2:→ Capture the overall slope of the spectral envelope (e.g., how quickly the spectrum falls off)
- MFCC 3–5:→ Represent formant-like shapes, useful in speech (vowels, resonances).
- MFCC 6+:→ Capture finer spectral detail, like rapid changes or texture — more sensitive to noise and timbre.



Time-Frequency Domain: MFCC



- Before 3s: Higher coefficients have more variation → rich, dynamic signal with changing spectral content.
- After 3s: Most coefficients become uniform and flat → Less structure or detail in the spectrum, noise, silence, or steady low-energy sound.
- Bottom row (MFCC 0) turns dark blue → confirms energy has dropped significantly

Time-Frequency Domain: CQT

Purpose:

CQT transforms a signal into a time–frequency representation like the STFT, but uses logarithmically spaced frequency bins : matching how humans perceive musical pitch.

Why it's useful:

- Better suited for musical signals than STFT
- Preserves pitch-related patterns such as chords, harmonics, or transpositions

Main idea:

- Uses geometrically spaced frequency bins (like piano notes)
- Each bin has a constant ratio between center frequency and bandwidth (Q factor)

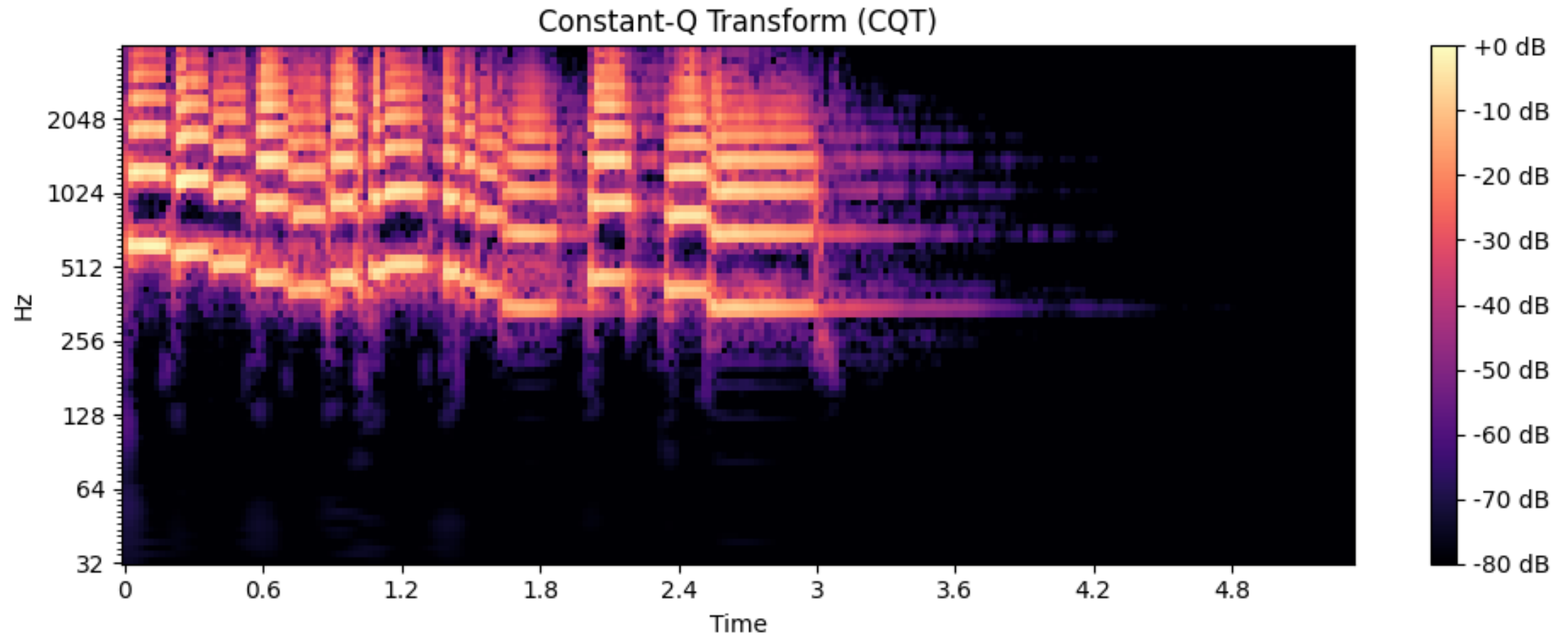
Python Code:

```
C = np.abs(librosa.cqt(y, sr=sr))
```

Use Cases:

- Music transcription
- Pitch detection in polyphonic recordings
- Chord recognition

Time-Frequency Domain: CQT



Audio Processing with Torchaudio

Session 3

Audio Processing with Torchaudio

Session 3

Objectives:

- Understand what Torchaudio is and how it integrates with PyTorch.
- Load, visualize, and transform audio data using Torchaudio.
- Compare Torchaudio with Librosa for audio processing.
- Build a preprocessing pipeline suitable for ML tasks.
- Prepare audio features for classification.

What is Torchaudio?

Torchaudio is a PyTorch domain library that provides audio processing tools for deep learning applications.

Why it matters:

- Seamless integration with the PyTorch ecosystem
- GPU acceleration through Torch tensors
- Optimized for batch processing with Dataloaders
- Includes pre-trained models, datasets, transforms, and I/O utilities

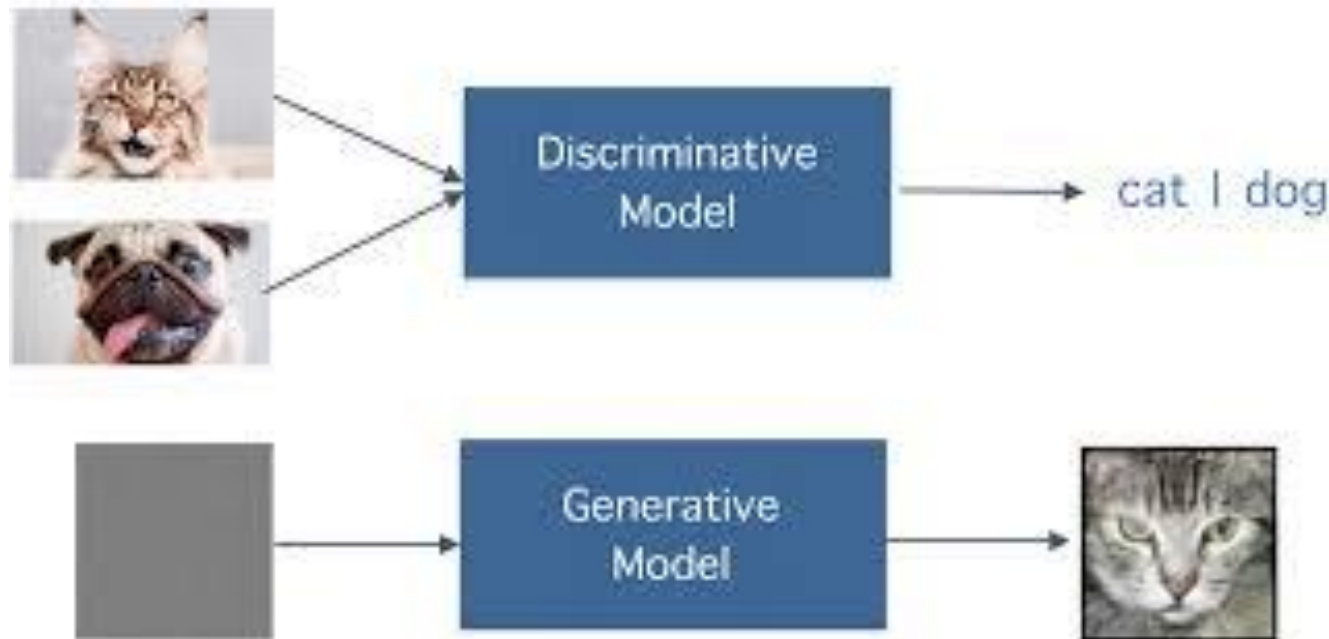
Installation:

```
pip install torchaudio
```

Machine learning basics

Discriminative models classify data. They estimate the probability that an input with certain features belongs to a specific class: identifying objects in an image or the genre of a song. They are widely used in face recognition, autonomous driving, medical diagnosis, and more.

Generative models can also classify, but they go further: they can *create new data*. While discriminative models predict a label from features, generative models estimate the features given a label. So if asked to produce “a picture of a cat,” a generative model creates an image based on learned cat-like characteristics.



Machine learning basics

An **artificial neural network** is a machine learning model inspired by how neurons and synapses work in the brain. It is built from layers of interconnected nodes, each passing its output to the next layer.

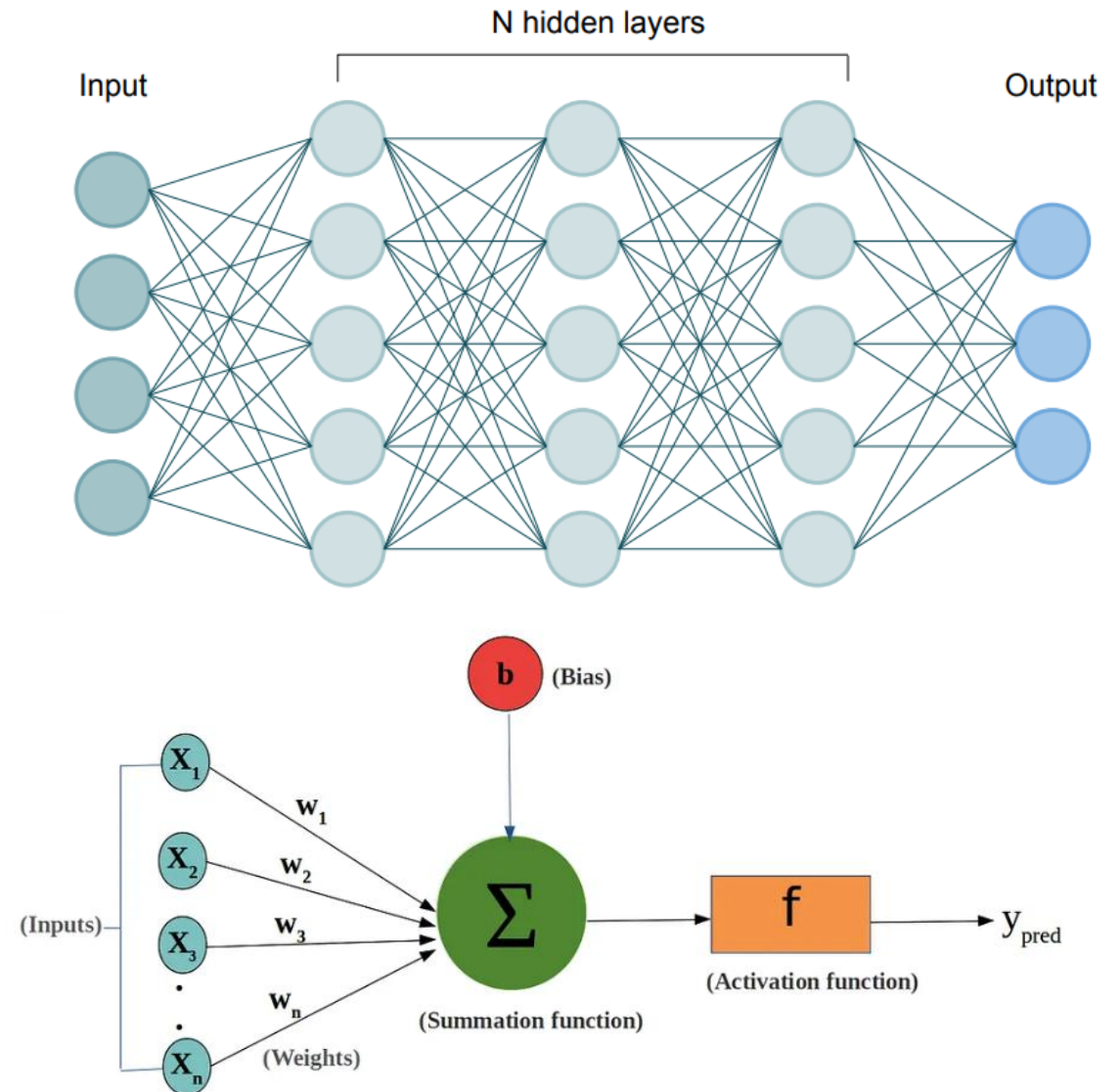
Each node applies an activation function: it multiplies inputs by weights, adds them together with a bias, and produces an output. During training, these weights are updated so the network “learns.”

Training can be **supervised** or **unsupervised**.

Feed-forward Neural Network: The simplest type of ANN. It does not form any cycles, which means that the information is sent in only one direction in the graph.

Recurrent Neural Network - A recurrent neural network the outputs from the nodes are sent back as input to the node or into previous node layers. An advantage of RNNs is that they can predict sequential data, which the standard feed-forward neural network cannot

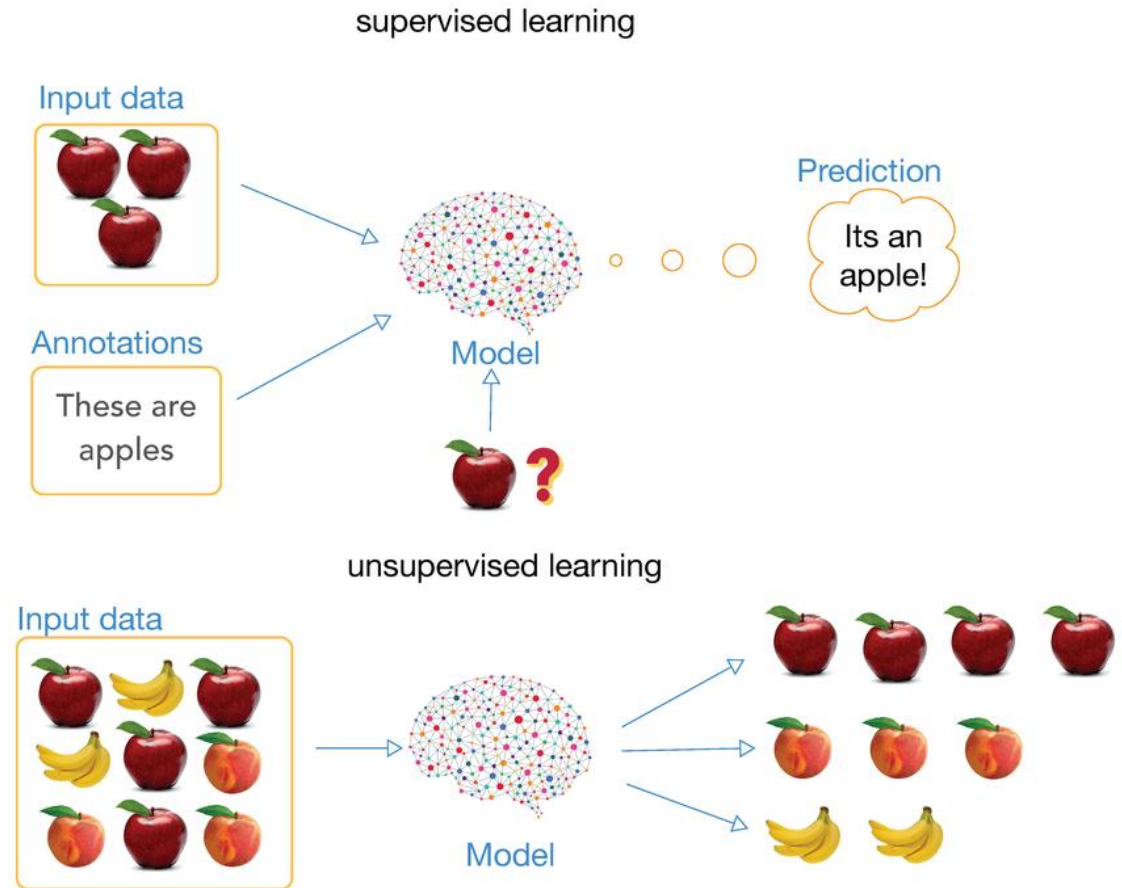
...



Machine learning basics

In **supervised learning**, the data has labels. The network's predictions are compared with the correct answers, and the weights are adjusted based on the error.

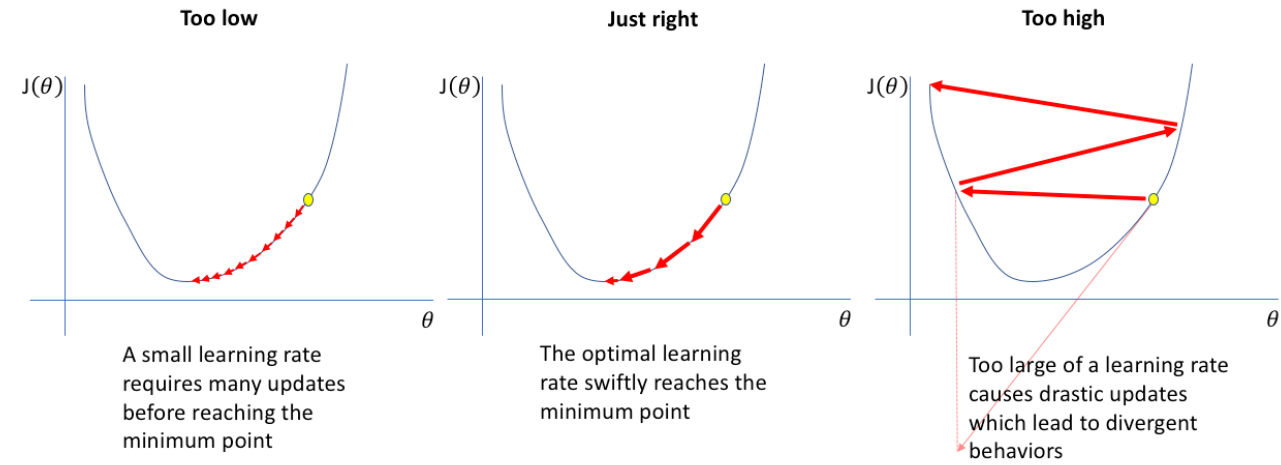
In **unsupervised learning**, the data has no labels. The network discovers patterns on its own: grouping images of cats and dogs without knowing which is which. An unsupervised generative model can produce new images resembling its data, but it cannot generate a specific labeled object on command, because it has never learned the meaning of those labels.



Machine learning basics: optimization

Optimization:

- **Learning rate** controls how quickly a neural network updates its weights during training. A very small learning rate makes training slow, while a very large one can cause the model to converge too fast and produce poor results.
- **Loss** measures the difference between the network's output and the desired output. Training aims to minimize this value. For sequential data, a common choice is the cross-entropy loss function.
- **Batch size** is the number of samples processed before the model updates its weights. Very large batches can reduce the model's ability to generalize to new data. The best batch size depends on the model and the dataset.



Torchaudio vs librosa

Torchaudio for model pipelines, Librosa for feature engineering

- **Torchaudio:**

- Tensor-based, GPU-ready
- Ideal for ML and DL pipelines
- Modular and differentiable transformations
- Works well with PyTorch's DataLoader, Dataset, and autograd

- **Librosa:**

- NumPy-based, CPU-only
- Rich feature extraction, ideal for analysis and prototyping
- Excellent visualization and educational tool

Loading Audio with Torchaudio

Torchaudio loads audio as PyTorch tensors

Python:

```
import torchaudio  
waveform, sample_rate = torchaudio.load("audio.wav")
```

Where:

- waveform: Tensor of shape [channels, time]
- sample_rate: sampling rate of the audio

Visualizing Waveform and Spectrogram

Torchaudio loads audio as PyTorch tensors

Python:

```
import torchaudio.transforms as T
import matplotlib.pyplot as plt
```

Waveform

```
plt.figure(figsize=(10, 2))
plt.plot(waveform.t().numpy())
plt.title("Waveform")
plt.show()
```

Spectrogram

```
spec = T.Spectrogram()(waveform)
plt.imshow(spec.log2()[0,:,:].numpy(), cmap='viridis', aspect='auto')
plt.title("Spectrogram")
plt.colorbar()
plt.show()
```

Basic Transforms in Torchaudio

- **Resample – Change sampling rate**

```
resampler = T.Resample(orig_freq=48000, new_freq=16000)  
resampled_waveform = resampler(waveform)
```

- **Spectrogram – Linear-frequency spectrogram**

```
spec = T.Spectrogram(n_fft=400, hop_length=160)(waveform)
```

- **MelSpectrogram – Perceptual frequency scale**

```
mel_spec = T.MelSpectrogram(sample_rate=16000, n_fft=1024, hop_length=512)(waveform)
```

- **MFCC – Mel Frequency Cepstral Coefficients**

```
mfcc = T.MFCC(sample_rate=16000, n_mfcc=13)(waveform)
```

- **AmplitudeToDB – Convert to decibel scale**

```
db_spec = T.AmplitudeToDB()(mel_spec)
```

- **Padding – Ensure the same length**

```
lengths = [waveform.shape[1] for waveform in dataset] #choose percentile  
target_len = int(np.percentile(lengths, 95))  
pad_amt = target_len - waveform.shape[1]  
padded = F.pad(waveform, (0, pad_amt))
```

Data augmentation: adding noise

Adding noise is a simple but effective data augmentation technique to simulate real-world conditions:

Why add noise?

- Increases generalization
- Prevents overfitting
- Simulates noisy environments (street sounds, wind, crowds)

Example:

- **White Noise Addition:**

```
def add_white_noise(waveform, noise_level=0.005):  
    noise = torch.randn_like(waveform) * noise_level  
    return waveform + noise
```

Data augmentation: time/frequency masking

Improve model generalization by hiding parts of the spectrogram during training.

Masking Type	Description	Purpose
Time Masking	Randomly masks a range of time frames (horizontal bands)	Simulates dropouts or silent periods
Freq Masking	Randomly masks frequency bins (vertical bands)	Simulates channel loss or occlusion

```
T.TimeMasking(time_mask_param=30),  
T.FrequencyMasking(freq_mask_param=15)
```

Where:

- `time_mask_param`: maximum width (in time frames) to mask
- `freq_mask_param`: maximum width (in frequency bins) to mask

Data augmentation: add effect

Torchaudio.sox_effects.apply_effects_tensor lets you apply SOX (Sound eXchange, the Swiss Army knife of audio manipulation) audio effects programmatically to a waveform. This is ideal for adding realism or creating synthetic variations in data.

```
torchaudio.sox_effects.apply_effects_tensor(waveform, sample_rate, effects_list)
```

Example:

```
effects = [  
    ["lowpass", "-1", "300"],    # Apply single-pole lowpass filter  
    ["speed", "0.8"],           # Reduce playback speed  
    ["rate", f"{sample_rate}"], # Adjust sample rate back to original  
    ["reverb", "-w"]            # Add wet reverb  
]
```

This is useful for:

- Simulating different microphones/environments
- Building data augmentation pipelines
- Testing robustness of models

Data augmentation: add effect

Effect	Description	Key Parameters	Example Usage
Reverb	Adds reverberation (echo-like effect). Can simulate small to large spaces.	-w (wet-only), reverberance, HF-damping, room-scale, stereo-depth, pre-delay, wet-gain	["reverb", "-w", "50", "50", "80"] → Wet-only, room size 80%
		Default values: 50 50 100 100 0 0 (if not specified)	["reverb"] → basic reverb with default dry/wet mix
speed	Changes the speed and pitch together.	Factor (e.g., 1.2 = +20%, 0.8 = -20%)	["speed", "0.8"] → slower and lower pitch
tempo	Changes the speed without changing pitch.	Factor (>1 = faster, <1 = slower)	["tempo", "1.1"] → 10% faster without pitch shift
pitch	Shifts pitch without changing speed.	Value in cents (100 = 1 semitone)	["pitch", "200"] → 2 semitones higher
lowpass	Applies a low-pass filter (keeps frequencies < cutoff)	Cutoff frequency in Hz	["lowpass", "300"] → suppress high frequencies
highpass	Applies a high-pass filter (keeps frequencies > cutoff)	Cutoff frequency in Hz	["highpass", "1000"] → suppress low frequencies
equalizer	Adjust specific frequency bands	Frequency, width (Q), gain	["equalizer", "1000", "2.0q", "-10"]
norm	Normalize audio to target amplitude level	Target gain in dB or -n for RMS normalization	["norm", "-3"] → normalize to -3 dB
compond	Apply compression/expansion to dynamic range	Attack/decay, transfer function	["compond", "0.3,1", "-60,-20,-10", "-5", "-90", "0.2"]
vol	Adjust volume	Gain in dB or linear scale	["vol", "0.5"] → half the volume
rate	Resample audio to a new sample rate	New sample rate	["rate", "16000"] → resample to 16kHz

Data augmentation: time shifting

What is Time Shifting?

You roll (shift) the audio waveform left or right along the time axis, without changing its content.

Why use it?

- Increases variability of training data
- Simulates timing variations in speech or sound events
- Helps models become invariant to when a sound occurs

```
def time_shift(waveform, shift_limit):  
    shift = int(np.random.uniform(-shift_limit, shift_limit))  
    return torch.roll(waveform, shifts=shift, dims=1)
```

Audio Preprocessing Pipeline

Create a full audio pipeline using `torch.nn.Sequential`:

```
audio_transform = torch.nn.Sequential(  
    T.Resample(orig_freq=48000, new_freq=16000), # Resample to 16kHz  
    T.MelSpectrogram(sample_rate=16000,          # Compute Mel spectrogram  
                      n_fft=1024,  
                      hop_length=512,  
                      n_mels=64),  
    T.AmplitudeToDB(),                          # Convert to decibels  
    T.TimeMasking(time_mask_param=30),          # Apply time masking  
    T.FrequencyMasking(freq_mask_param=13)      # Apply frequency masking  
)
```

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):
    def __init__(self, file_list):
        self.files = file_list
        self.transform = audio_transform # use Sequential transform defined previous slide

    def __getitem__(self, idx):
        waveform, sr = torchaudio.load(self.files[idx])
        return self.transform(waveform), self.files[idx]

    def __len__(self):
        return len(self.files)

from torch.utils.data import DataLoader
loader = DataLoader(AudioDataset(file_list), batch_size=16, shuffle=True)
```

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):
```

```
    def __init__(self, file_list):
```

```
        self.files = file_list
```

```
        self.transform = audio_transform # use Sequential transform defined previous slide
```

The `__init__` function is run once when instantiating the Dataset object. We initialize the directory containing the images, the annotations file, and both transforms

If it is for classification, label should be added there

```
self.aud_labels = pd.read_csv(annotations_file)
```

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):  
    def __init__(self, file_list):  
        self.files = file_list  
        self.transform = audio_transform # use Sequential transform defined previous slide  
  
    def __getitem__(self, idx):  
        waveform, sr = torchaudio.load(self.files[idx])  
        return self.transform(waveform), self.files[idx] # optional: return filename
```

The `__getitem__` function loads and returns a sample from the dataset at the given index `idx`. Based on the index, it identifies the image's location on disk, calls the transform functions on them (if applicable), and returns the tensor image and corresponding label in a tuple.

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):  
    def __init__(self, file_list):  
        self.files = file_list  
        self.transform = audio_transform # use Sequential transform defined previous slide  
  
    def __getitem__(self, idx):  
        waveform, sr = torchaudio.load(self.files[idx])  
        return self.transform(waveform), self.files[idx] # optional: return filename  
  
    def __len__(self):  
        return len(self.files)
```

The `__len__` function returns the number of samples in our dataset.

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
from torch.utils.data import DataLoader
```

```
loader = DataLoader(AudioDataset(file_list), batch_size=16, shuffle=True)
```

DataLoader creates shuffled, batched data ready for training.

Sound Event Detection & Classification

Session 4

Sound Event Detection & Classification

Session 4

Objectives:

- Understand the core challenges of Sound Event Detection (SED)
- Learn tools, datasets, and feature extraction methods
- Discover architectures like CNNs for audio classification
- Practice model training, evaluation, and improvement techniques

Applications

- Smart assistants (e.g., Alexa, Google Home)
- Urban surveillance (e.g., gunshot detection)
- Wildlife monitoring (animal calls)
- Health (e.g., cough or apnea detection)
- Sound-based content tagging in videos

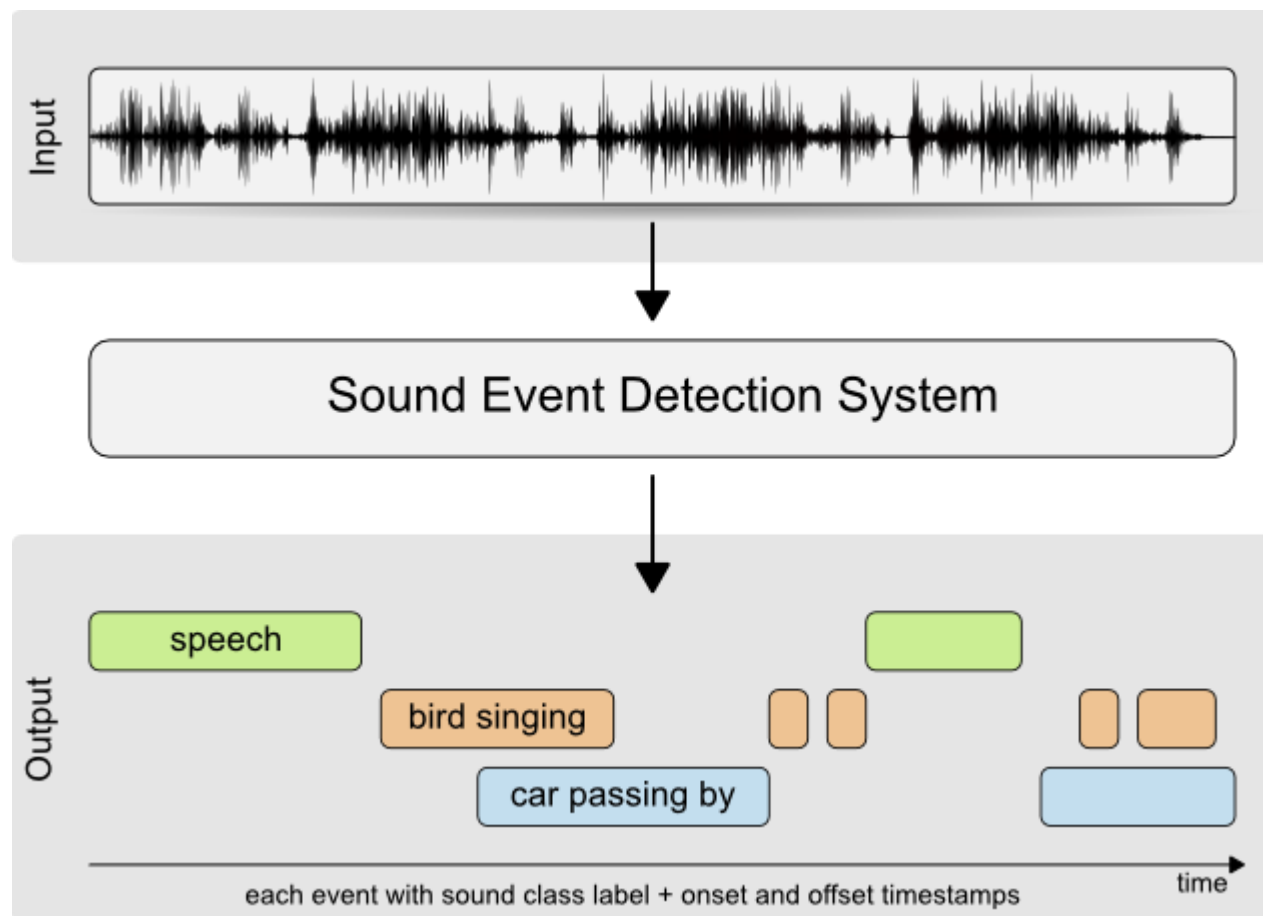
What is Sound Event Detection (SED)?

SED aims to **identify and locate sound events** in an audio stream

May be:

- **Classification:** What sound is present?
- **Detection:** When does it happen?
- **Multi-label:** More than one event at a time

It's the **audio equivalent** of object detection in images.



Tools and libraries

- torchaudio, librosa : feature extraction
- torch, tensorflow : deep learning
- pyannote-audio, openl3, panns : pretrained SED models

CNNs for Classification

Convolutional Neural Networks (CNNs) are powerful architectures designed specifically for **image and spatial data**. Unlike standard neural networks that treat all inputs independently, CNNs process images by scanning small regions using **convolutional filters**. This allows them to **capture local patterns** like edges, shapes, and textures effectively. **For audio, features are treated as images!!**

A typical CNN is composed of the following layers:

- **Input Layer:**

Receives the features/images.

- **Convolutional Layer:**

Applies filters to extract features.

- **Activation Function:**

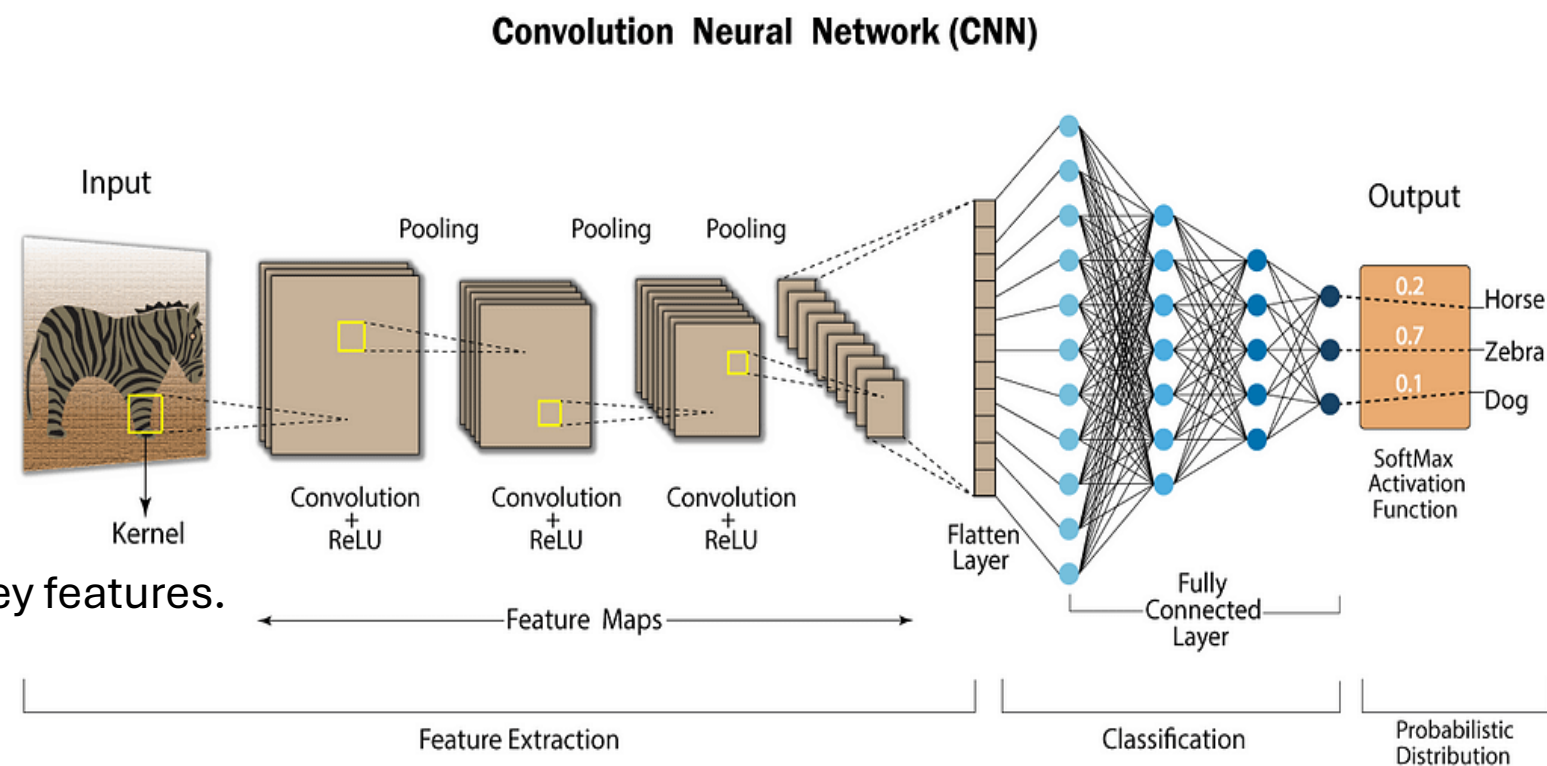
Introduces non-linearity (commonly ReLU).

- **Pooling Layer:**

Reduces dimensionality and emphasizes key features.

- **Fully Connected Layer:**

Performs the final classification.



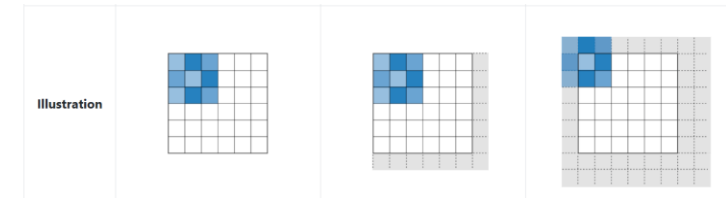
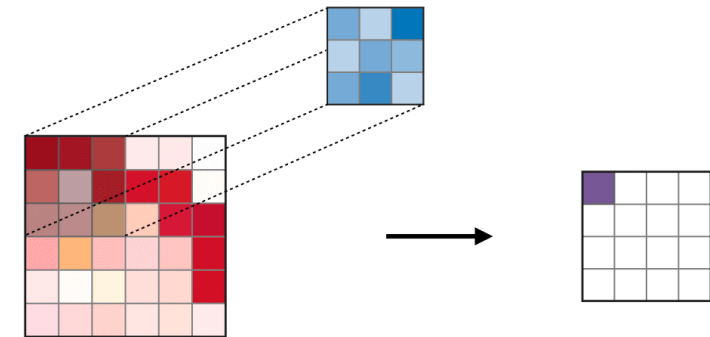
CNN: Convolutional Layer

This layer is responsible for extracting important features from the input image using a set of learnable filters/kernels (kernel are learned parameters!!)

- Applies a small learnable filter across the input image.
- Detects features like edges, corners, and textures.
- Uses the same weights at each location (weight sharing).

Its hyperparameters include:

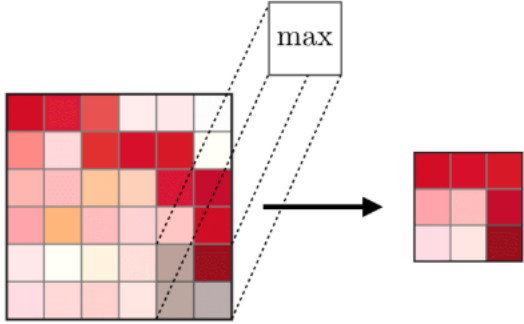
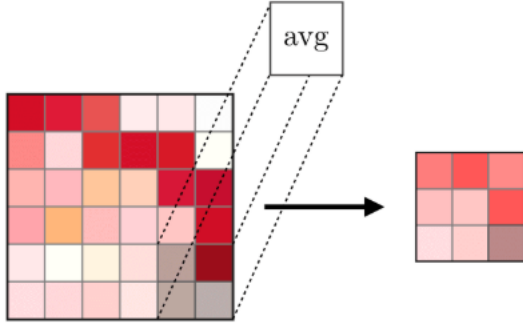
- Padding (P): adds a border of zeros around the input image. This helps control the spatial size of the output and preserves edge information when applying filters.
- Filter size (F): determines the spatial area each filter covers (e.g., 3×3)
- Stride (S): controls how far the filter moves across the image (e.g. 1 pixel) , larger strides reduce output size



Adding an activation function to the activation layers add nonlinearity to the network

CNN: Pooling

Pooling layers follow convolution layers and are used to reduce the spatial dimensions of feature maps while retaining important information.

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

CNN

Benefits:

- Feature extraction: Automatically learns features from raw image data, reducing manual preprocessing.
- Spatial invariance: Can recognize patterns regardless of their location, orientation, or size.
- Robust to noise: Performs well even when input images are cluttered or noisy.
- Transfer learning: Pre-trained CNNs can be adapted to new tasks with less data and compute.
- High performance: Achieves state-of-the-art results in tasks like classification, detection, and segmentation.

Limitations:

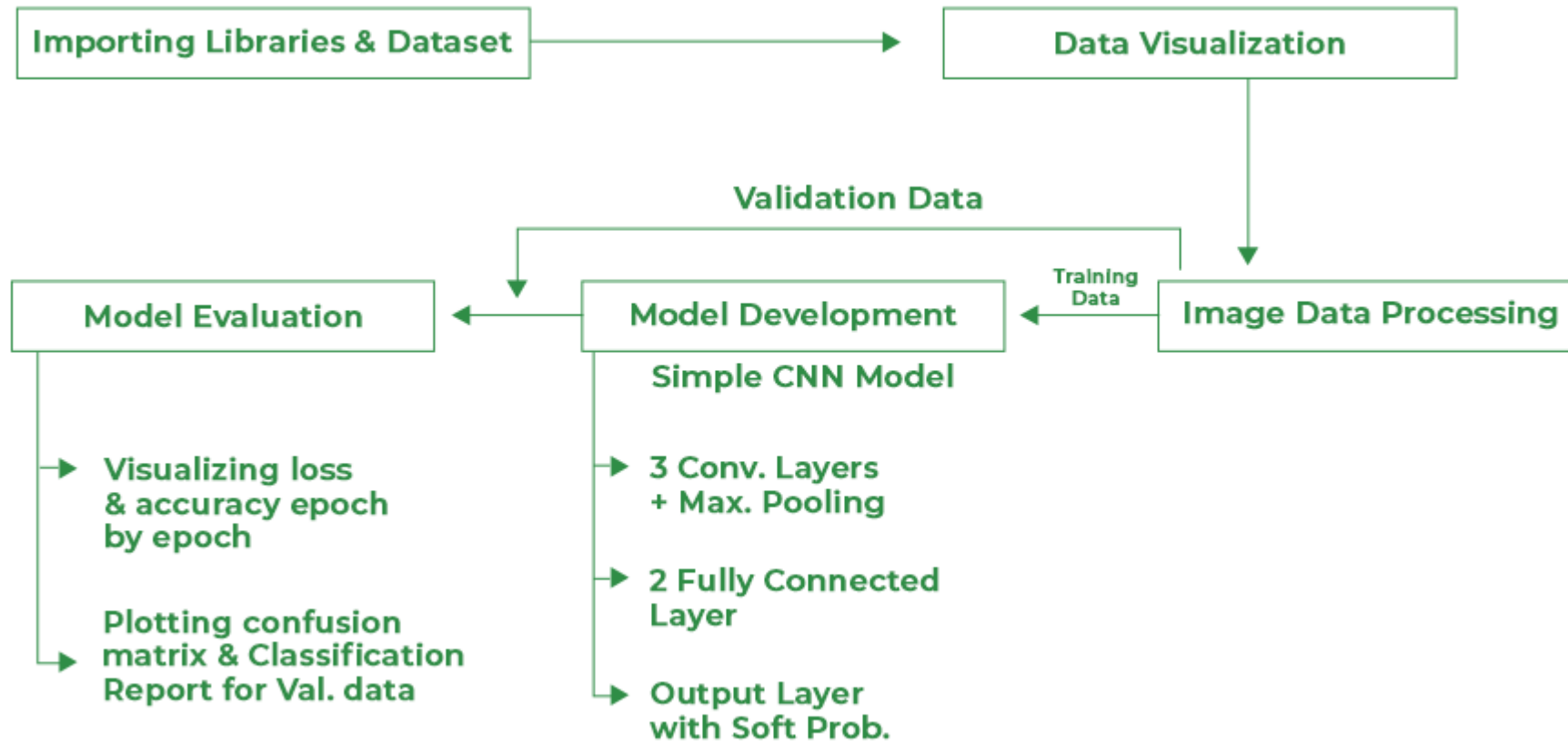
- Computational cost: Training deep CNNs can require a lot of resources and time.
- Overfitting: Prone to memorizing training data when datasets are small.
- Lack of interpretability: It's difficult to explain how or why a CNN made a specific decision.
- Structured input required: Works only with grid-like data (e.g., images), not irregular formats

Training a CNN

We will do the following steps in order:

- Load data
- Clean your data: normalization, augmentation, resize, etc
- Extract features log-mel spectro
- Define a Convolutional Neural Network (number of con2D, pooling)
- Define a loss function to compare predictions to true labels
- Train the network on the training data
 - Compute the gradients of the loss with respect to the model parameters (weights and biases) using backpropagation.
 - Adjust the weights and biases using an optimizer (e.g., SGD, Adam) to minimize the loss.
 - Loop over all batches for multiple epochs until the model performance stabilizes.
- Test the network on the test data

Training a CNN



Overfitting

When a model learns the training data too well, including noise and details that don't generalize.

- Results in good training accuracy but poor performance on unseen (test) data.

How to prevent or reduce overfitting?

- More training data: Exposes the model to more variations.
- Data augmentation: Randomly change training audio (noise, effect, ...).
- Regularization: Add penalty terms to the loss (e.g., L2 weight decay).
- Dropout: Randomly disable some neurons during training.
- Early stopping: Stop training when validation loss starts to rise.
- Batch normalization: Helps by regularizing and stabilizing training.
- Simpler model: Use fewer layers or neurons if the model is too complex for the data.

Batch normalization

Applied after convolutional or fully connected layers:

Normalizes intermediate activations to zero mean and unit variance

Why?

- **Stabilizes training:** reduces fluctuations in layer outputs, helping the model learn more consistently.
- **Reduces internal covariate shift:** Keeps the input distribution of each layer more stable during training.
- **Allows higher learning rates:** Makes training more robust to large updates, speeding up convergence.
- **Acts as a form of regularization:** Adds slight noise from mini-batch stats, which helps prevent overfitting.

Recommended in most CNNs for audio/image tasks

Pre trained model

Classification Models:

- VGGish (Google): Audio tagging
- YAMNet: EfficientNet-based, pretrained on AudioSet
- Wav2Vec2: Self-supervised model, usable for audio embeddings

SED-Specific Models:

- PANNs: CNN-based, weakly-supervised
- CRNN (Convolutional Recurrent): Temporal modeling
- HTS-AT: Transformer-based audio tagging
- AST (Audio Spectrogram Transformer): Vision Transformer adapted to audio

PANNs

What are PANNs?

Pretrained CNN models for audio tagging & SED, trained on AudioSet (2M+ clips)

Model Types:

- CNN6, CNN10, CNN14 — progressively deeper CNNs
- Wavegram-CNN, Cnn14-ResNet, MobileNet-based variants available

Input Features:

- Log-Mel spectrograms (usually 64 Mel bins)

Outputs:

- Clip-level: Tagging (is "dog bark" present?)
- Frame-level: When did "dog bark" happen?

Create a Custom Dataset?

Why Do We Need `MyCustomDataset` ? When working with audio datasets like UrbanSound8K:

Raw Data:

- - `.csv` file with labels and filenames
- - Folders of `.wav` audio clips

CNN need:

- Tensors as input, not file paths
- Uniform shape and format (e.g., [batch, channels, freq, time])

`MyCustomDataset` Solves These Challenges:

- Inherits from `torch.utils.data.Dataset`
- Dynamically loads each `.wav` file
- Applies **audio transformations** (e.g., log-Mel spectrograms)
- Pads/truncates all signals to same duration
- Returns `(spectrogram_tensor, class_label)` ready for training

Benefits:

- Memory-efficient (on-the-fly loading)
- Customizable (sampling rate, transforms)
- Integrates with `DataLoader` and GPU/CPU handling

Audio Preprocessing Pipeline

Create a full audio pipeline using `torch.nn.Sequential`:

```
audio_transform = torch.nn.Sequential(  
    T.Resample(orig_freq=48000, new_freq=16000), # Resample to 16kHz  
    T.MelSpectrogram(sample_rate=16000,          # Compute Mel spectrogram  
                      n_fft=1024,  
                      hop_length=512,  
                      n_mels=64),  
    T.AmplitudeToDB(),                          # Convert to decibels  
    T.TimeMasking(time_mask_param=30),           # Apply time masking  
    T.FrequencyMasking(freq_mask_param=13)       # Apply frequency masking  
)
```

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):  (or from torch.utils.data import Dataset)
```

```
    def __init__(self, file_list):
```

```
        self.files = file_list
```

```
        self.transform = audio_transform # use Sequential transform defined previous slide
```

```
    def __getitem__(self, idx):
```

```
        waveform, sr = torchaudio.load(self.files[idx])
```

```
        return self.transform(waveform), self.files[idx]
```

```
    def __len__(self):
```

```
        return len(self.files)
```

```
from torch.utils.data import DataLoader
```

```
loader = DataLoader(AudioDataset(file_list), batch_size=16, shuffle=True)
```

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):
```

```
    def __init__(self, file_list):
```

```
        self.files = file_list
```

```
        self.transform = audio_transform # use Sequential transform defined previous slide
```

The `__init__` function is run once when instantiating the Dataset object. We initialize the directory containing the images, the annotations file, and both transforms

If it is for classification, label should be added there

```
self.aud_labels = pd.read_csv(annotations_file)
```

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):  
    def __init__(self, file_list):  
        self.files = file_list  
        self.transform = audio_transform # use Sequential transform defined previous slide  
  
    def __getitem__(self, idx):  
        waveform, sr = torchaudio.load(self.files[idx])  
        return self.transform(waveform), self.files[idx] # optional: return filename
```

The `__getitem__` function loads and returns a sample from the dataset at the given index `idx`. Based on the index, it identifies the image's location on disk, calls the transform functions on them (if applicable), and returns the tensor image and corresponding label in a tuple.

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
class AudioDataset(torch.utils.data.Dataset):
    def __init__(self, file_list):
        self.files = file_list
        self.transform = audio_transform # use Sequential transform defined previous slide

    def __getitem__(self, idx):
        waveform, sr = torchaudio.load(self.files[idx])
        return self.transform(waveform), self.files[idx] # optional: return filename

    def __len__(self):
        return len(self.files)
```

The `__len__` function returns the number of samples in our dataset.

Batch processing with Pytorch

Torchaudio supports batched input and works directly with PyTorch's Dataset and DataLoader.

A custom Dataset class must implement three functions: `__init__`, `__len__`, and `__getitem__`.

Ex:

```
from torch.utils.data import DataLoader
```

```
loader = DataLoader(AudioDataset(file_list), batch_size=16, shuffle=True)
```

DataLoader creates shuffled, batched data ready for training.

Sequence Modeling with RNNs and LSTMs

Session 5

Sequence Modeling with RNNs and LSTMs

Session 5

Objectives :

- Understand why sequence modeling is important for audio
- Learn the basics of Recurrent Neural Networks (RNNs)
- Identify the problems of RNNs (vanishing/exploding gradients)
- Understand how LSTMs and GRUs solve these problems
- Practice with PyTorch LSTM

Why Sequences Matter in Audio?

- **Audio is Inherently Sequential**

Audio signals have strong temporal dependencies. If you break an audio into small pieces and shuffle them, the result would no longer make sense.

- Speech: words depend on previous words for context
- Music: notes create melodies, rhythms have patterns

- **Example:**

The phrase "I saw a bat" - meaning depends on what comes next

- "I saw a bat flying" (animal)
- "I saw a bat and ball" (sports equipment)

Standard neural nets (CNN) can't "remember" the past.

How do we build models that understand temporal context?

Recap - CNNs for Audio

- **What We've Done So Far : CNNs**

Input Spectrogram → Conv2D → Pool → Conv2D → Pool → Flatten → Dense → Output

- Extract local features (frequency patterns)
- Work well for classification tasks
- Use pooling to reduce dimensionality

- **Limitations:**

- CNNs can only look at a fixed window of time.
- Cannot remember information from far in the past.
- Max/average pooling loses temporal order information.
- Loss of sequence order. Same output irrespective of the order of features.
- Inability to capture long-term dependencies.

- **We need models that can:**

- Process sequences of variable length
- Maintain memory of past inputs
- Model temporal dependencies explicitly

Recap - CNNs for Audio

- **Imagine you want to predict the stock market's data**

A simple machine learning model or an Artificial Neural Network may learn to predict the stock prices based on a number of features: the volume of the stock, the opening value etc. While the price of the stock depends on these features, it is also largely dependent on the stock values in the previous days

In the conventional feed-forward neural networks, all test cases are considered to be independent. That is when fitting the model for a particular day, there is no consideration for the stock prices on the previous days.

Let use another model which can remember past inputs

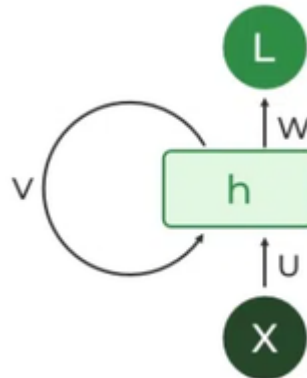
Recurrent Neural Networks (RNNs)

A recurrent neural network or RNN is a deep neural network trained on sequential or time series data to create a machine learning (ML) model that can make sequential predictions or conclusions based on sequential inputs

- While standard neural networks pass information in one direction i.e from input to output, RNNs feed information back into the network at each step.

Key Concept: At each time step, the RNN:

- Takes current input
- Combines it with previous hidden state
- Produces output and new hidden state



Recurrent Neural Networks (RNNs)

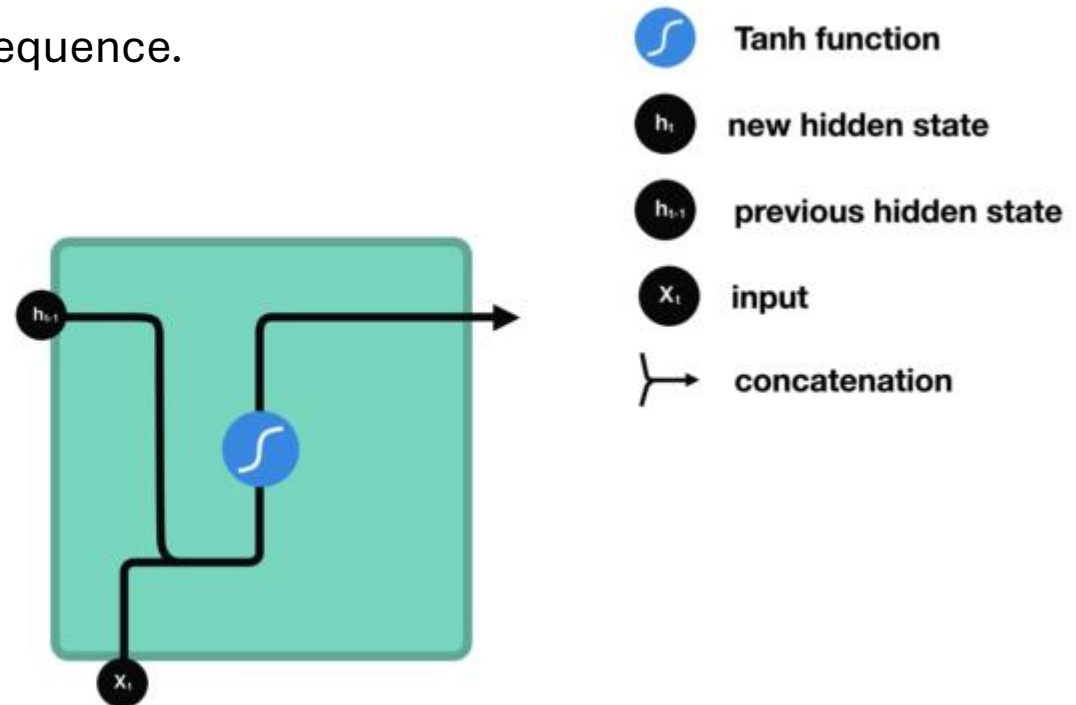
The basic structure includes:

Input Layer: Receives the sequence data.

Hidden Layer: Processes input and maintains information from earlier time steps through recurrent connections.

Output Layer: Generates predictions based on the current hidden state

At each timestep, the recurrent cell takes two inputs:
its previous hidden state and the next step of the input sequence.
These two inputs inform the next hidden state,
which creates the network's output for that step.



Recurrent Neural Networks (RNNs)

The best way to visualize this process is by ‘unrolling’ the recurrent neural network into a series of conventional feedforward neural.

These parameters are updated using backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as **backpropagation through time**

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

x_t : input at time t

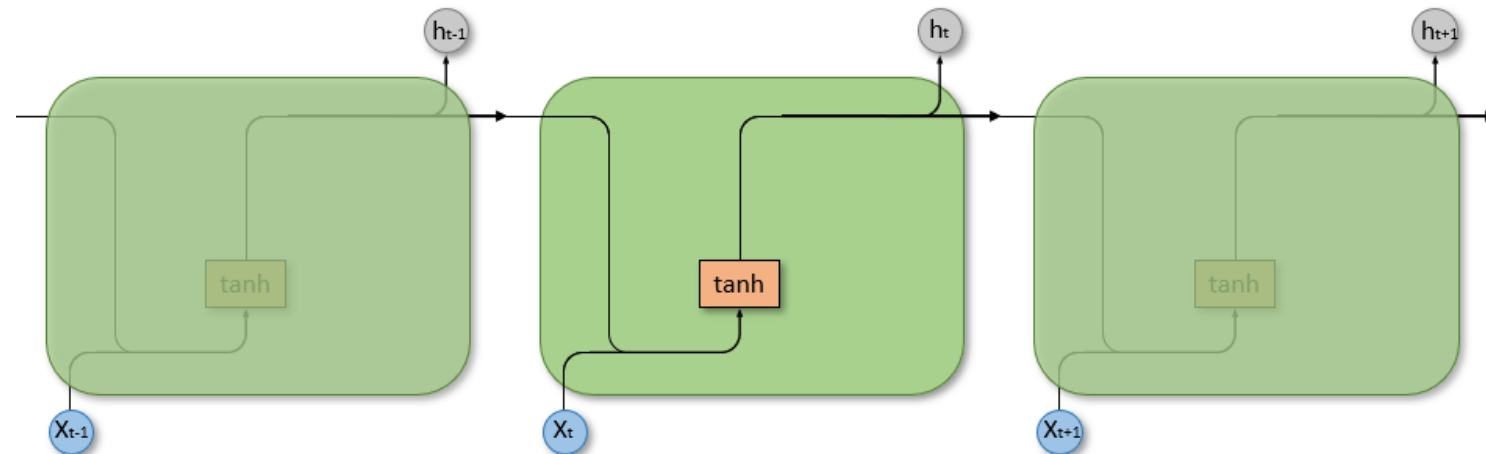
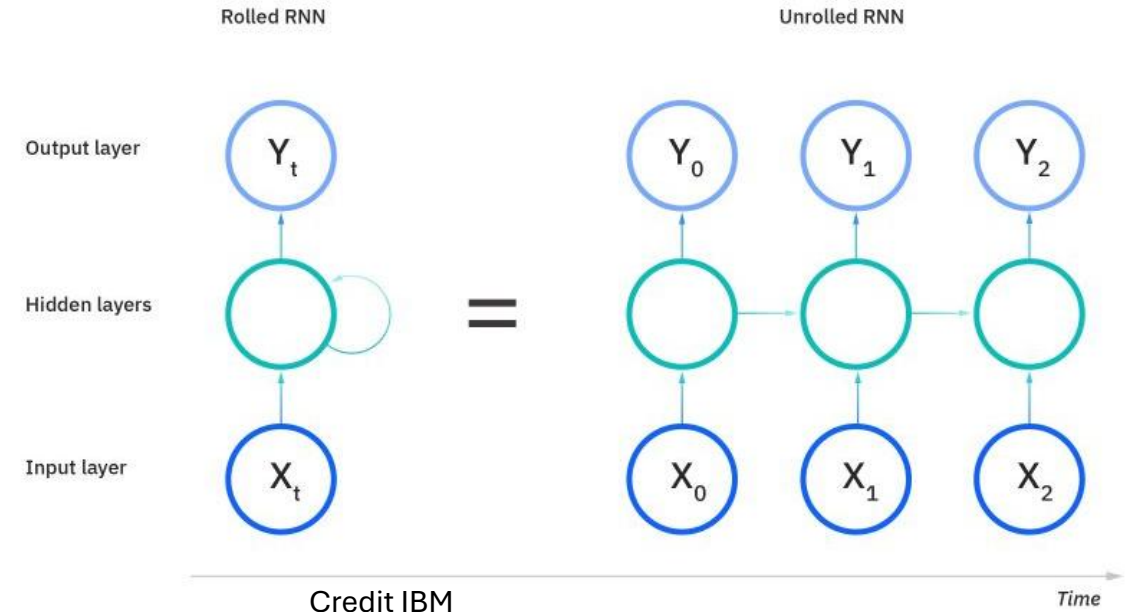
h_t : hidden state at time t

y_t : output at time t

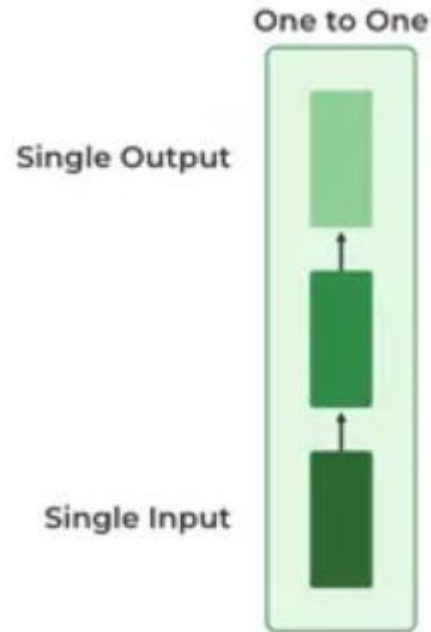
W_{hh} : hidden-to-hidden weights

W_{xh} : input-to-hidden weights

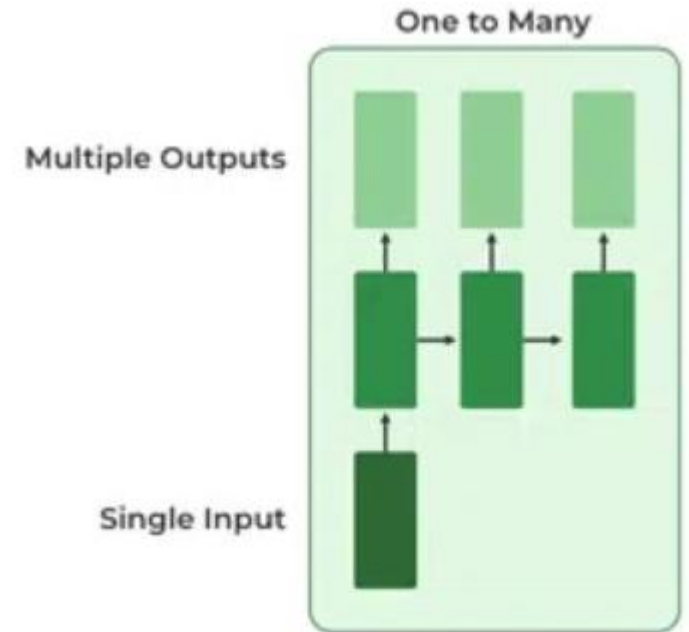
W_{hy} : hidden-to-output weight



Types Of Recurrent Neural Networks

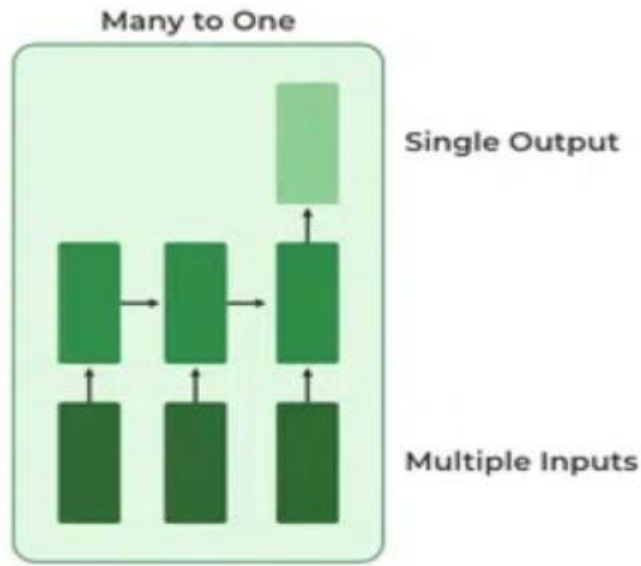


This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.

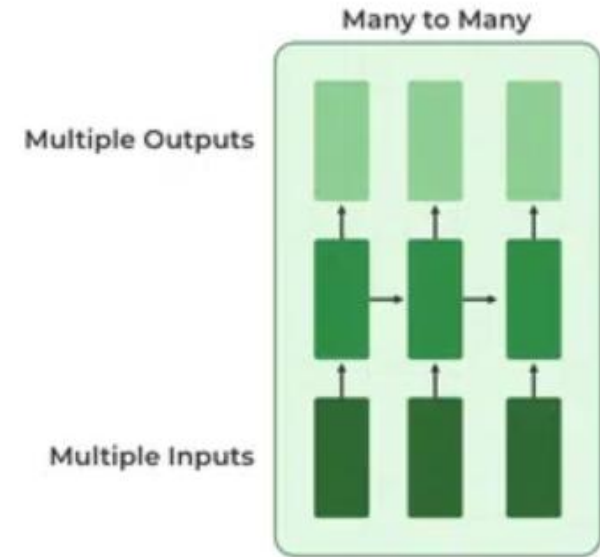


In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs). For example in image captioning a single image can be used as input to generate a sequence of words as a caption.

Types Of Recurrent Neural Networks



The **Many-to-One RNN** receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



The **Many-to-Many RNN** type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output

When can we use RNN

- Speech recognition
- Music generation
- Sentiment classification
- DNA sequence analysis
- Machine translation
- Video activity recognition
- Name entity recognition
- All tasks with sequential data structures
- Image captioning
- Time series prediction (e.g. stock market trend prediction)
- Handwriting generation
- ...

Limitations : RNN

- **Vanishing Gradient Problem:** When backpropagating over many time steps, gradients tend to shrink exponentially, making early time steps contribute very little to weight updates. This causes the network to “forget” long-term dependencies.

Ex: I was born in France, but I have been working in South Africa working for ... (another 200 words)
... Therefore my mother tongue is:

- **Exploding Gradient Problem:** Gradients may also grow uncontrollably large, causing unstable updates and making training difficult.

SOLUTION: Two specialised RNNs were created that make use of an internal mechanism called “gates” that can regulate the flow of information. Gating is a technique that helps the network learn which data in a sequence is important to keep (by adding it to the hidden state) and which should be thrown away

- Long Short-Term Memory (LSTM)
- Gated Recurrent Units (GRU)

LSTM

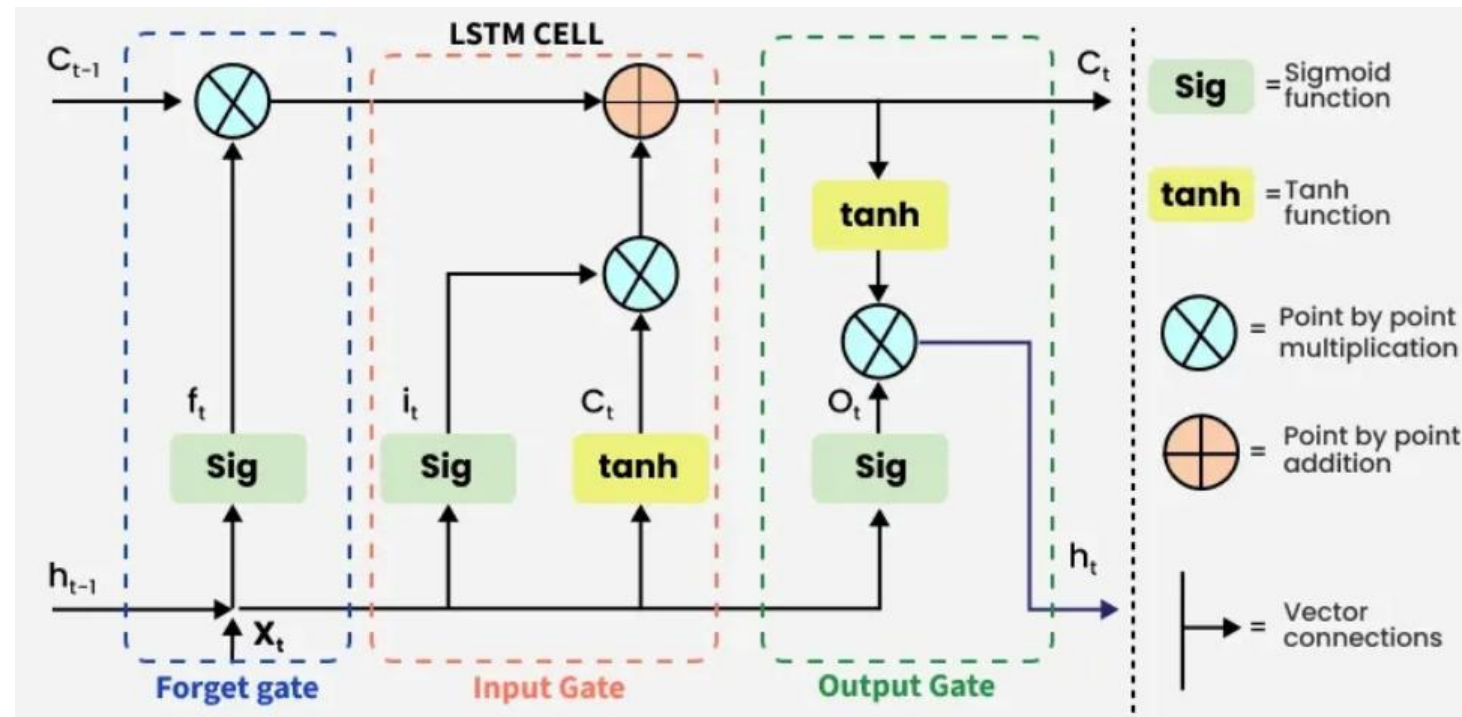
Long Short-Term Memory (LSTM) is an enhanced version of the Recurrent Neural Network (RNN) designed by Hochreiter and Schmidhuber. LSTMs can capture long-term dependencies in sequential data making them ideal for tasks like language translation, speech recognition and time series forecasting.

Unlike traditional RNNs which use a single hidden state passed through time LSTMs introduce a **memory cell** that holds information over extended periods addressing the challenge of learning long-term dependencies.

LSTM : Architecture

An LSTM has:

- **Cell state (C_t):** The "memory" highway
- **Hidden state (h_t):** The output/short-term memory
- **Three gates:**
 - **Input Gate:** Controls how much new information should be added to the cell state.
 - **Forget Gate:** Decides what past information should be discarded.
 - **Output Gate:** Regulates what information should be output at the current step. This selective memory enables LSTMs to handle long-term dependencies, making them ideal for tasks where earlier context is critical

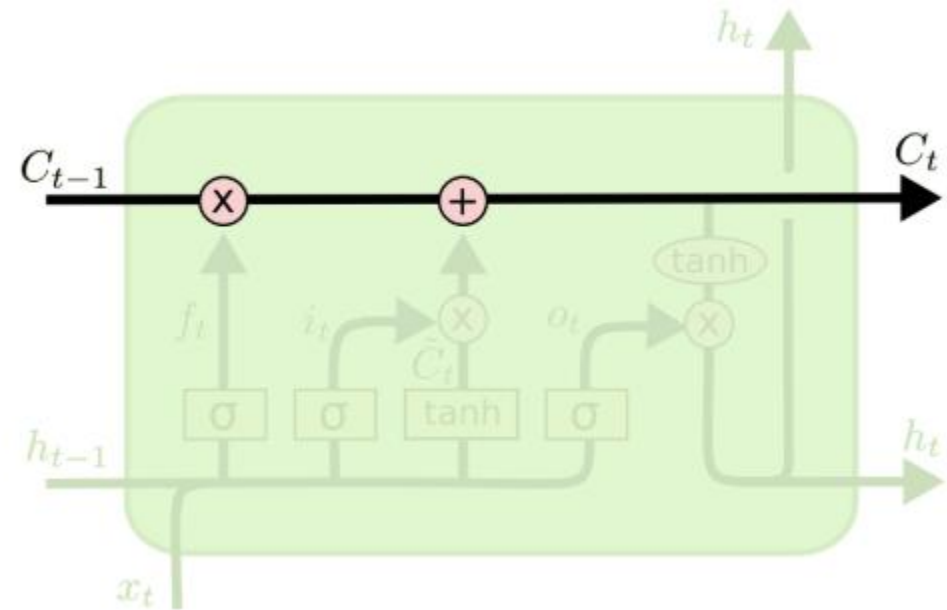


Step-by-Step LSTM Walk Through

The key to LSTMs is the cell state, the horizontal line running through the top of the diagram.

The cell state is kind of like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.

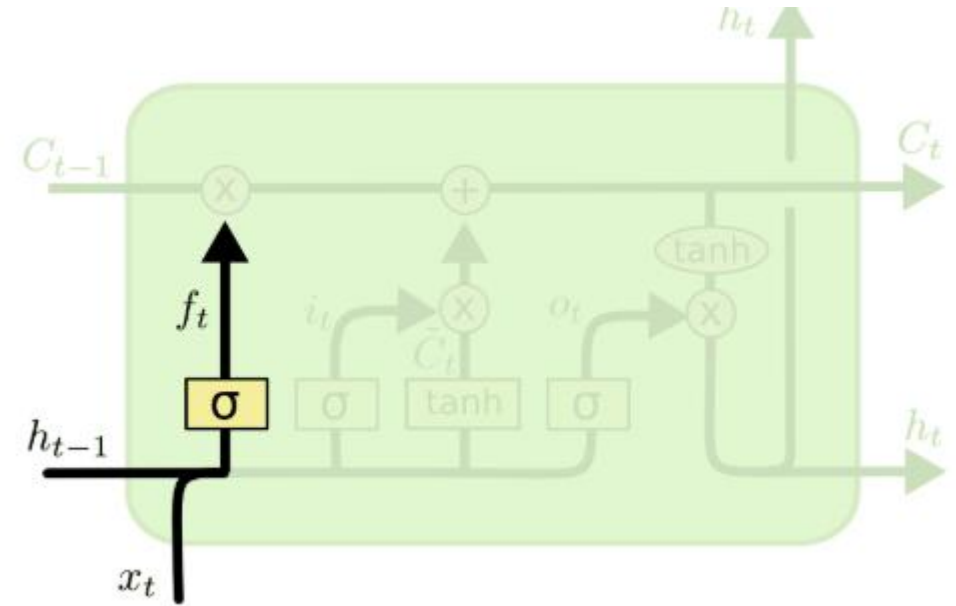
The LSTM does have the ability to remove or add information to the cell state, carefully regulated by structures called gates.



Step-by-Step LSTM Walk Through

The first step in our LSTM is to decide what information we're going to throw away from the cell state. This decision is made by a sigmoid layer called the “forget gate layer.”

It looks at h_{t-1} and x_t , and outputs a number between 0 and 1 for each number in the cell state C_{t-1} . A 1 represents “completely keep this” while a 0 represents “completely get rid of this.”

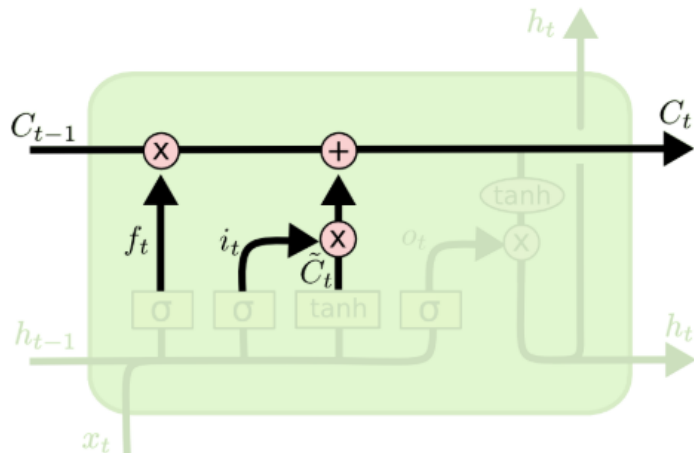


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

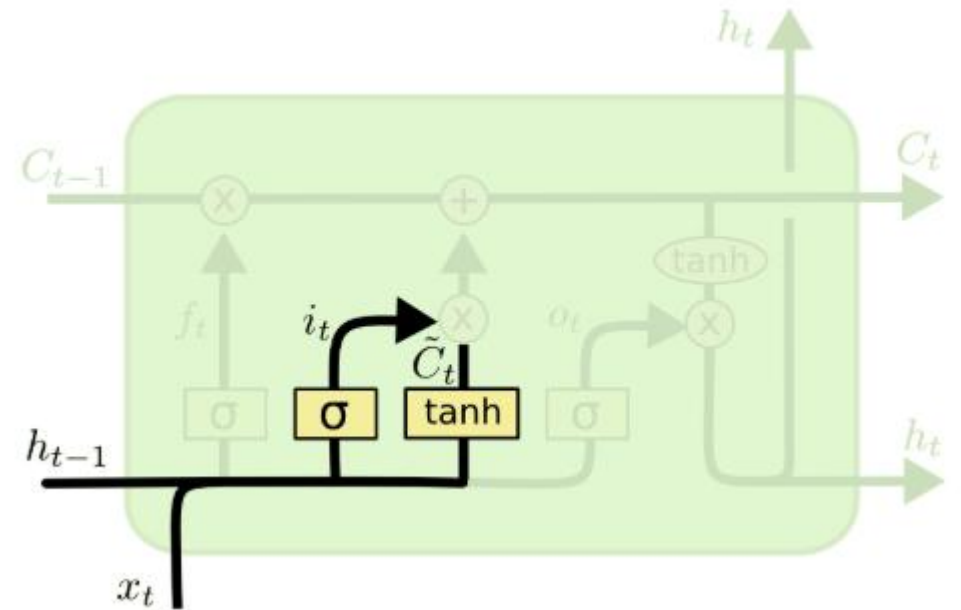
Step-by-Step LSTM Walk Through

The next step is to decide what new information we're going to store in the cell state. This has two parts. First, a sigmoid layer called the “input gate layer” decides which values we'll update. Next, a tanh layer creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state.

Finally, Update the old cell state C_{t-1} into the new cell state C_t :



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

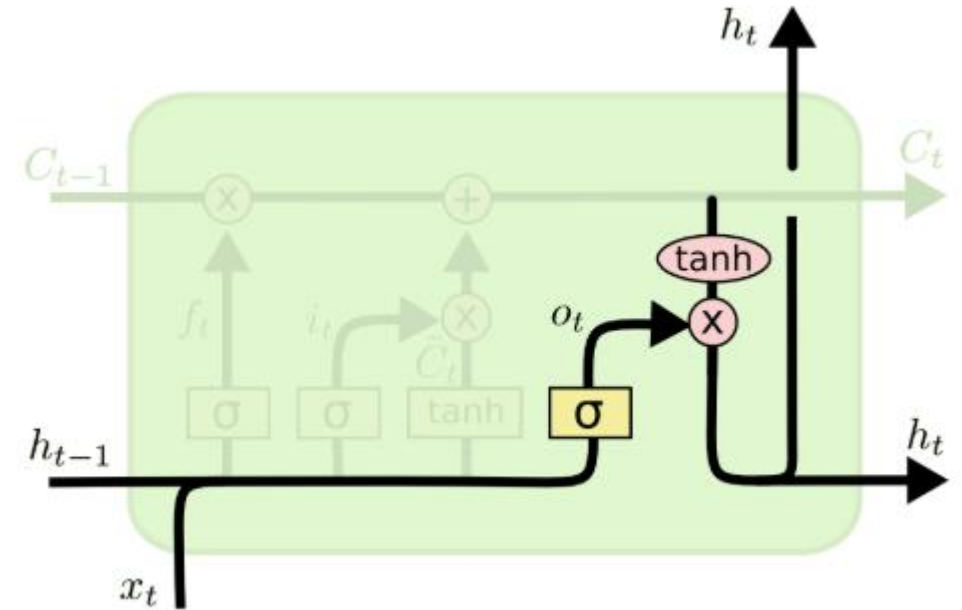


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Step-by-Step LSTM Walk Through

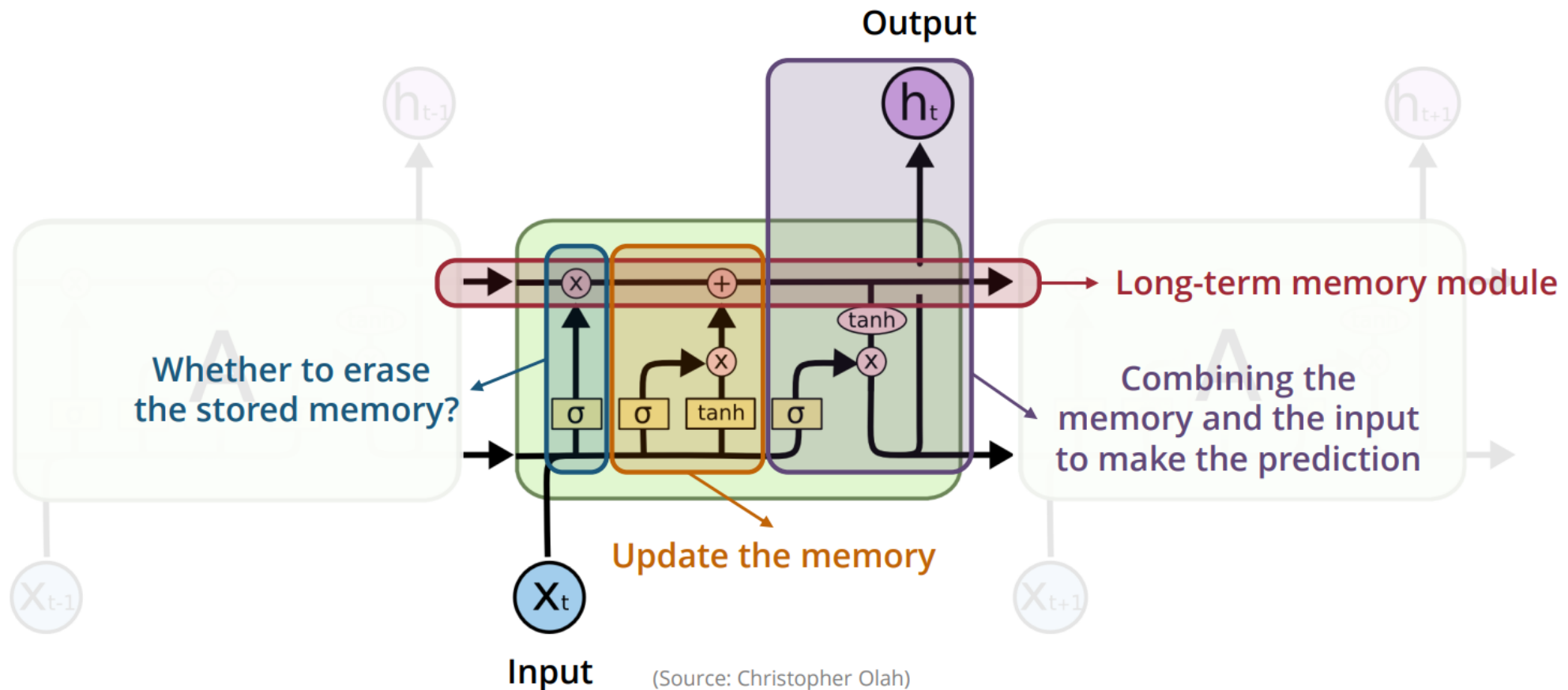
Finally, we need to decide what we're going to output. This output will be based on our cell state, but will be a filtered version. First, we run a sigmoid layer which decides what parts of the cell state we're going to output. Then, we put the cell state through tanh and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Step-by-Step LSTM Walk Through



LSTM: Applications

Language Modeling: Used in tasks like language modeling, machine translation and text summarization. These networks learn the dependencies between words in a sentence to generate coherent and grammatically correct sentences.

Speech Recognition: Used in transcribing speech to text and recognizing spoken commands. By learning speech patterns they can match spoken words to corresponding text.

Time Series Forecasting: Used for predicting stock prices, weather and energy consumption. They learn patterns in time series data to predict future events.

Anomaly Detection: Used for detecting fraud or network intrusions. These networks can identify patterns in data that deviate drastically and flag them as potential anomalies.

Recommender Systems: In recommendation tasks like suggesting movies, music and books. They learn user behavior patterns to provide personalized suggestions.

Video Analysis: Applied in tasks such as object detection, activity recognition and action classification. When combined with Convolutional Neural Networks (CNNs) they help analyze video data and extract useful information.

LSTM: Applications

```
import torch.nn as nn
```

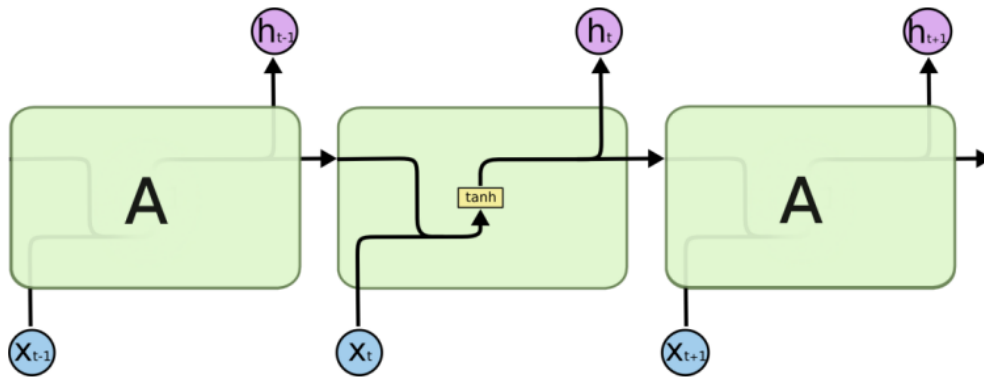
```
lstm = nn.LSTM(input_size=40, hidden_size=128, num_layers=2, batch_first=True)
```

- `input_size`: features per time step
- `hidden_size`: neurons in each LSTM layer
- `num_layers`: stacked LSTM layers

Summary

Vanilla RNN

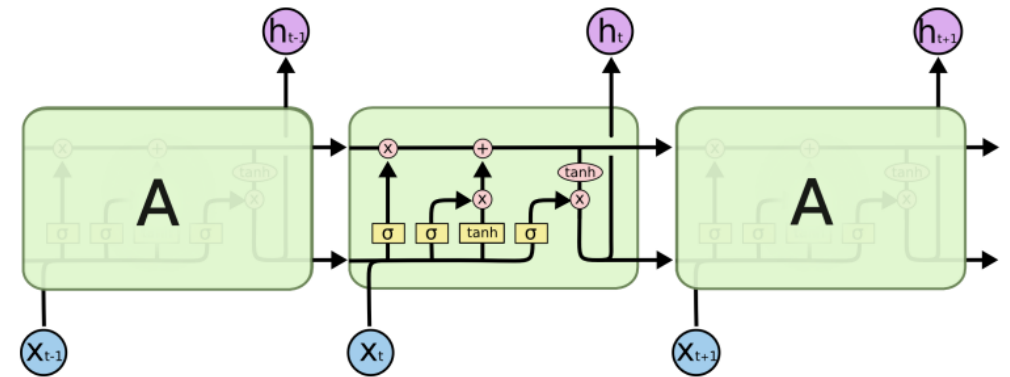
- Simplest form of RNNs
- Limited long-term memory



(Source: Christopher Olah)

LSTM

- Improved memory module
- Better long-term memory



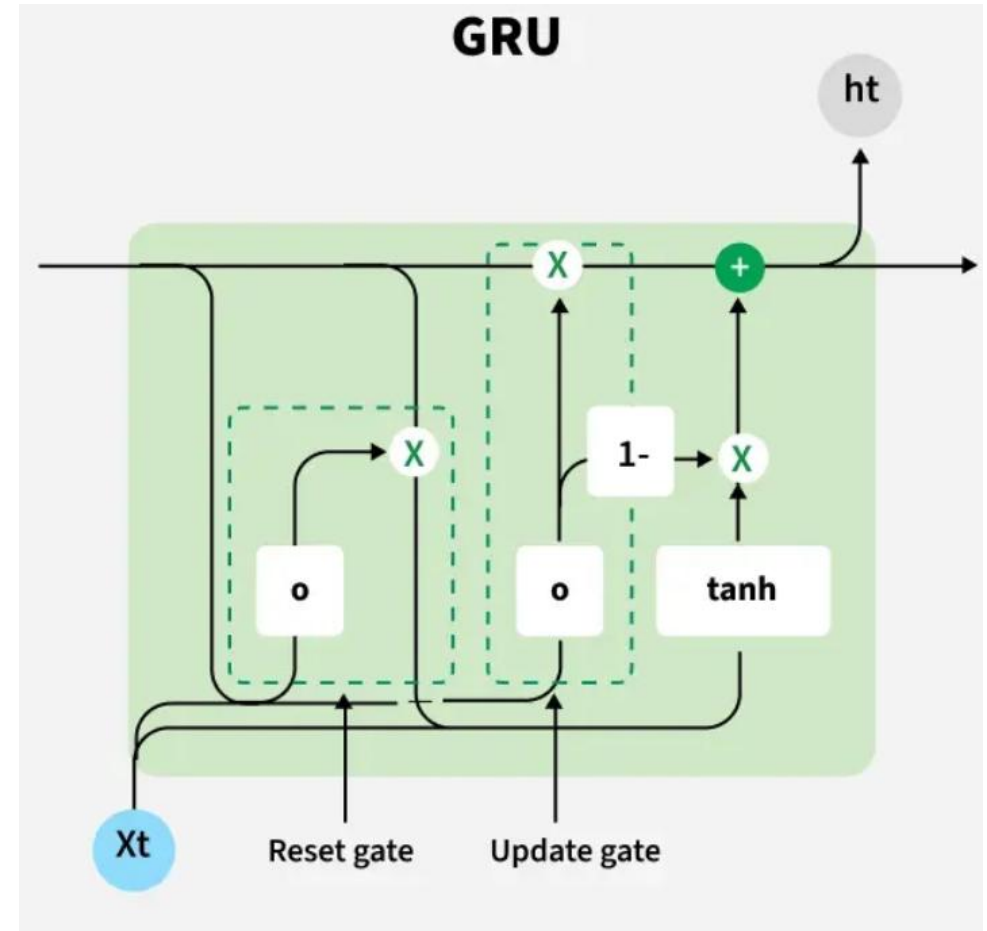
(Source: Christopher Olah)

Gated Recurrent Units (GRU)

GRUs are a simplified version of LSTMs, they combine the input and forget gates into a single update gate helps in reducing the number of parameters and making the model less computationally demanding. Here's how it works:

Update Gate: This gate decides how much of the information from the past should be kept for future steps. It helps the GRU remember important details.

Reset Gate: This gate decides how much past information should be forgotten. If something is no longer relevant it forget that part.



Automatic Speech Recognition (ASR)

Session 6

Automatic Speech Recognition (ASR)

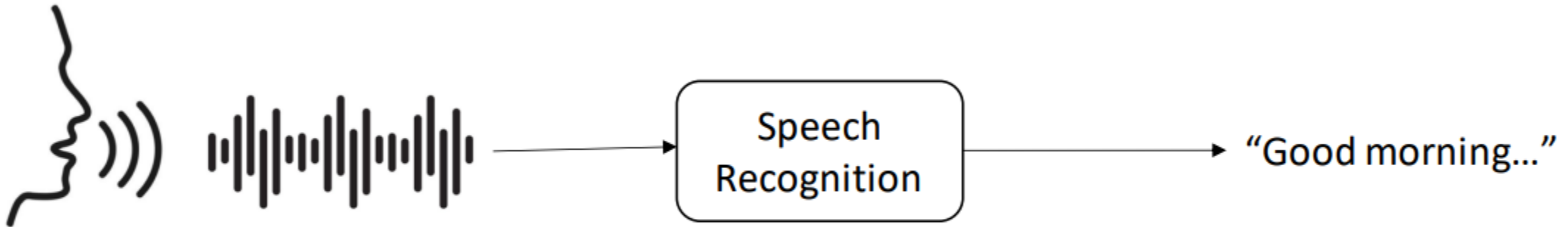
Session 6

Objectives:

- Explain how attention enables context modeling in sequences.
- Describe the Transformer architecture.
- Understand the traditional ASR pipeline and its evolution.
- Explain the role of CTC loss in aligning speech and text.
- Compare CTC-based vs attention-based end-to-end ASR systems.
- Describe modern self-supervised ASR models (Wav2Vec, Whisper).

WHAT IS ASR?

- Automatic Speech Recognition (ASR) is the technology that allows us to convert human speech into digital text.



- Also called: Speech-to-Text (STT)
- ASR allows human to talk to machine in the most natural way (Google home, Siri, Alexia..)
- State-of-the-art models: **Wav2Vec 2.0** and **Whisper**.

Applications

- Voice Assistants & Smart Devices

Examples: Siri (Apple), Alexa (Amazon), Google Assistant, Samsung Bixby

- Transcription & Captioning

Examples: Otter.ai, Zoom Live Transcription, Google Meet Captions, Microsoft Teams

Converts speech to text in meetings, classrooms, news broadcasts, and YouTube videos

Example use: Journalists transcribing interviews automatically

- Automotive & Navigation

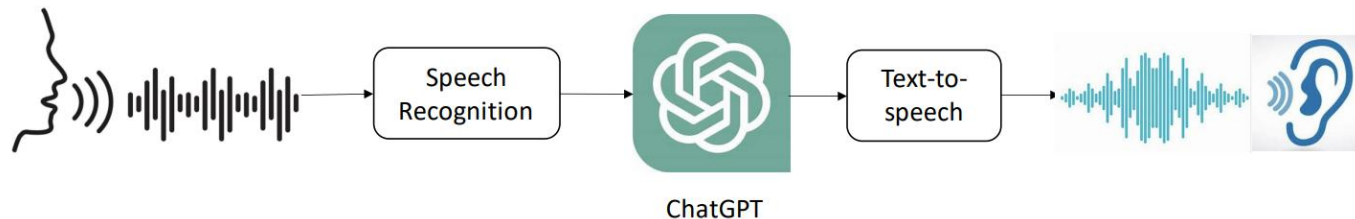
Examples: Tesla Voice Commands, Apple CarPlay, Android Auto, BMW Intelligent Personal Assistant

Enables hands-free navigation, music control, and phone calls while driving

- Language Learning & Education

Duolingo, ELSA Speak, Rosetta Stone, Google Read Along

Provides pronunciation feedback, listening practice, and oral assessments



Why is it hard? Several sources of variability

- **Style:**

Continuously spoken or isolated? Planned monologue or spontaneous conversation?

Why is it hard? Several sources of variability

- **Style:**

Continuously spoken or isolated? Planned monologue or spontaneous conversation?

- **Environment:**

Noise, competing speakers, channel conditions (microphone, phone line, room acoustics)

Why is it hard? Several sources of variability

- **Style:**
Continuously spoken or isolated? Planned monologue or spontaneous conversation?
- **Environment:**
Noise, competing speakers, channel conditions (microphone, phone line, room acoustics)
- **Speaker:**
Tuned for a particular speaker, or speaker-independent? Adaptation to speaker characteristics (rate of speech, accent)

Why is it hard? Several sources of variability

- **Style:**
Continuously spoken or isolated? Planned monologue or spontaneous conversation?
- **Environment:**
Noise, competing speakers, channel conditions (microphone, phone line, room acoustics)
- **Speaker:**
Tuned for a particular speaker, or speaker-independent? Adaptation to speaker characteristics (rate of speech, accent)
- **Vocabulary:**
Machine-directed commands, scientific language, colloquial expressions

Why is it hard? Several sources of variability

- **Style:**
Continuously spoken or isolated? Planned monologue or spontaneous conversation?
- **Environment:**
Noise, competing speakers, channel conditions (microphone, phone line, room acoustics)
- **Speaker:**
Tuned for a particular speaker, or speaker-independent? Adaptation to speaker characteristics (rate of speech, accent)
- **Vocabulary:**
Machine-directed commands, scientific language, colloquial expressions
- **Accent/dialect:**
Recognize the speech of all speakers who speak a particular language

Why is it hard? Several sources of variability

- **Style:**
Continuously spoken or isolated? Planned monologue or spontaneous conversation?
- **Environment:**
Noise, competing speakers, channel conditions (microphone, phone line, room acoustics)
- **Speaker:**
Tuned for a particular speaker, or speaker-independent? Adaptation to speaker characteristics (rate of speech, accent)
- **Vocabulary:**
Machine-directed commands, scientific language, colloquial expressions
- **Accent/dialect:**
Recognize the speech of all speakers who speak a particular language
- **Other paralinguistics**
Emotional state, social class, . . .

Why is it hard? Several sources of variability

- **Style:**
Continuously spoken or isolated? Planned monologue or spontaneous conversation?
- **Environment:**
Noise, competing speakers, channel conditions (microphone, phone line, room acoustics)
- **Speaker:**
Tuned for a particular speaker, or speaker-independent? Adaptation to speaker characteristics (rate of speech, accent)
- **Vocabulary:**
Machine-directed commands, scientific language, colloquial expressions
- **Accent/dialect:**
Recognize the speech of all speakers who speak a particular language
- **Other paralinguistics**
Emotional state, social class, . . .
- **Language:** spoken Estimated 7,000 languages, most with limited training resources; code-switching; language change

Why is it hard? Machine point of view

- **As a classification problem:** very high-dimensional output space.
- **As a sequence-to-sequence problem:** very long input sequences, with limited reordering between acoustic and word sequences.
- **Data issues:** speech is often noisy, with many nuisance factors of variation (speaker, environment, microphone).
- **Limited training data:** far fewer transcribed words than in text-based NLP; manual transcription is costly (~10× real time).
- **Linguistic complexity:** the hierarchical and compositional nature of speech production and comprehension is difficult to model with a single approach.

Early recognition

1920's Radio Rex

- Celluloid dog with iron base held within house by electromagnet against force of spring
- Current to magnet flowed through bridge which was sensitive to energy at 500 Hz.
- 500 Hz energy caused bridge to vibrate, interrupting current, making dog spring forward
- The sound “e” (ARPAbet [eh]) in Rex has 500 Hz component



Digit recognition

1962 Shoebox: The First Digital Speech Recognizer

- Developed by IBM engineer William C. Dersch in 1962.
- Recognized 16 spoken words and digits (0–9).
- Input: microphone → analog filter bank → logic circuits → electrical counters.

The IBM Shoebox used a microphone and a bank of analog filters to detect energy in specific frequency bands of speech. Each word produced a unique pattern across those filters, which simple logic circuits compared to stored templates allowing it to recognize digits and commands

- Basis for later HMM-based recognizers.



ASR Evaluation

- **Word Error Rate:** is the ‘de facto’ metric for speech recognition. It calculates substitutions, insertions and deletions on the *word level*. This means errors are annotated on a word-by-word basis.

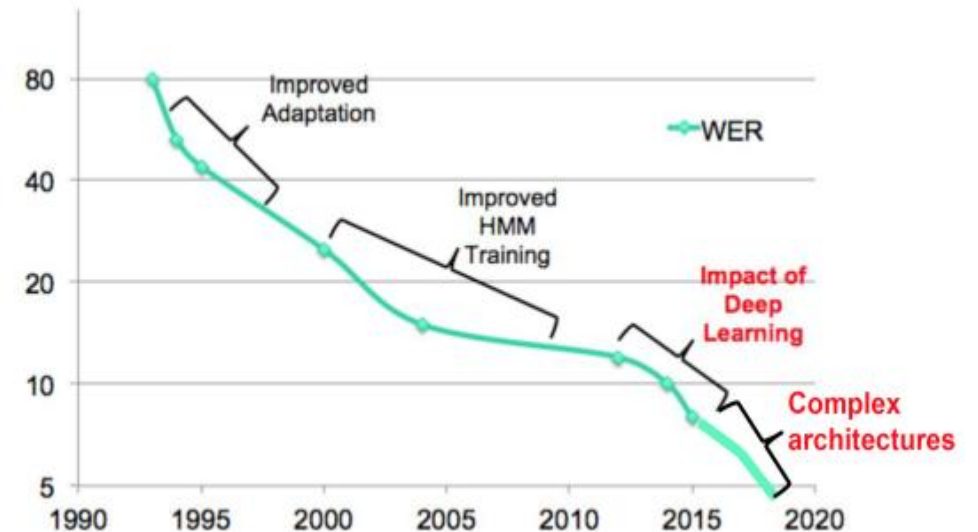
$$WER = \frac{S + I + D}{N}$$

Reference:	the	cat	sat	on	the	mat
Prediction:	the	cat	sit	on	the	
Label:	✓	✓	S	✓	✓	D

The word “sit” only has one character that is wrong, but the entire word is marked incorrect. This is a defining feature of the WER: spelling errors are penalised heavily, no matter how minor they are.

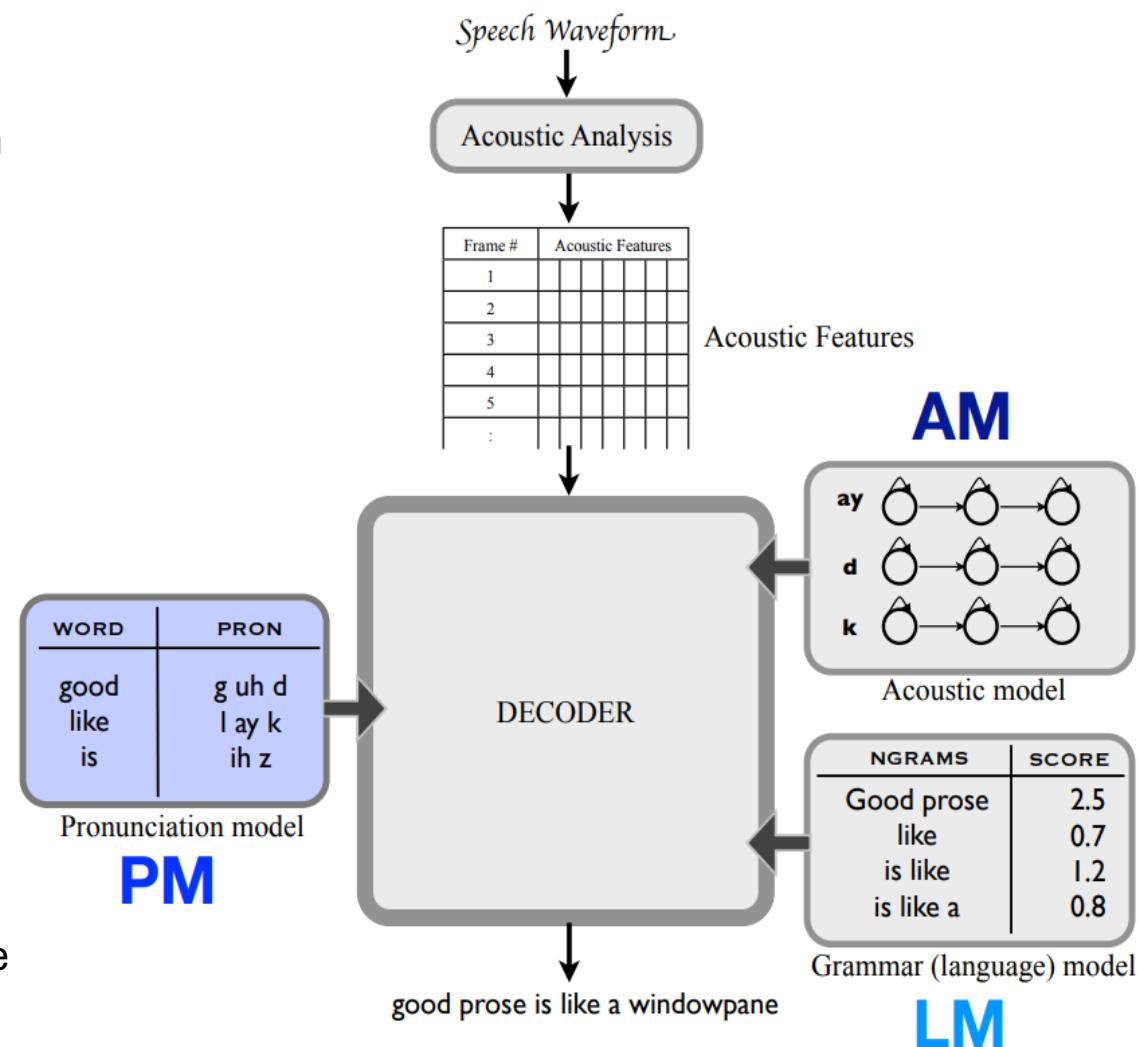
Character Error Rate: the same but on characters

Reference:	t	h	e		c	a	t		s	a	t		o	n		t	h	e		m	a	t	
Prediction:	t	h	e		c	a	t		s	i	t		o	n		t	h	e					
Label:	✓	✓	✓	✓	✓	✓	✓	✓	✓	S	✓	✓	✓	✓	✓	✓	✓	✓		D	D	D	D



ASR architecture: the classical way

- **Speech Waveform** : The raw audio signal input containing spoken words.
- **Acoustic Analysis** : Extracts numerical features (e.g., MFCCs) from the speech waveform over short time frames.
- **Pronunciation Model (PM)** : Maps each word to its phonetic transcription (how it sounds).
- **Acoustic Model (AM)** : Models the statistical relationship between phonetic units (like phonemes) and the acoustic features.
- **Language Model (LM)** : Assigns probabilities to word sequences based on grammar and linguistic context.
- **Decoder** : Combines the AM, LM, and PM to find the most likely word sequence given the observed speech.
- **Recognized Text Output** : The final transcribed text produced by the ASR system.



Fundamental Equation of Statistical Speech Recognition

- If $\mathbf{X}$ is the sequence of acoustic feature vectors (observations) and $\mathbf{W}$ denotes a word sequence, the most likely word sequence $\mathbf{W}^*$ is given by:

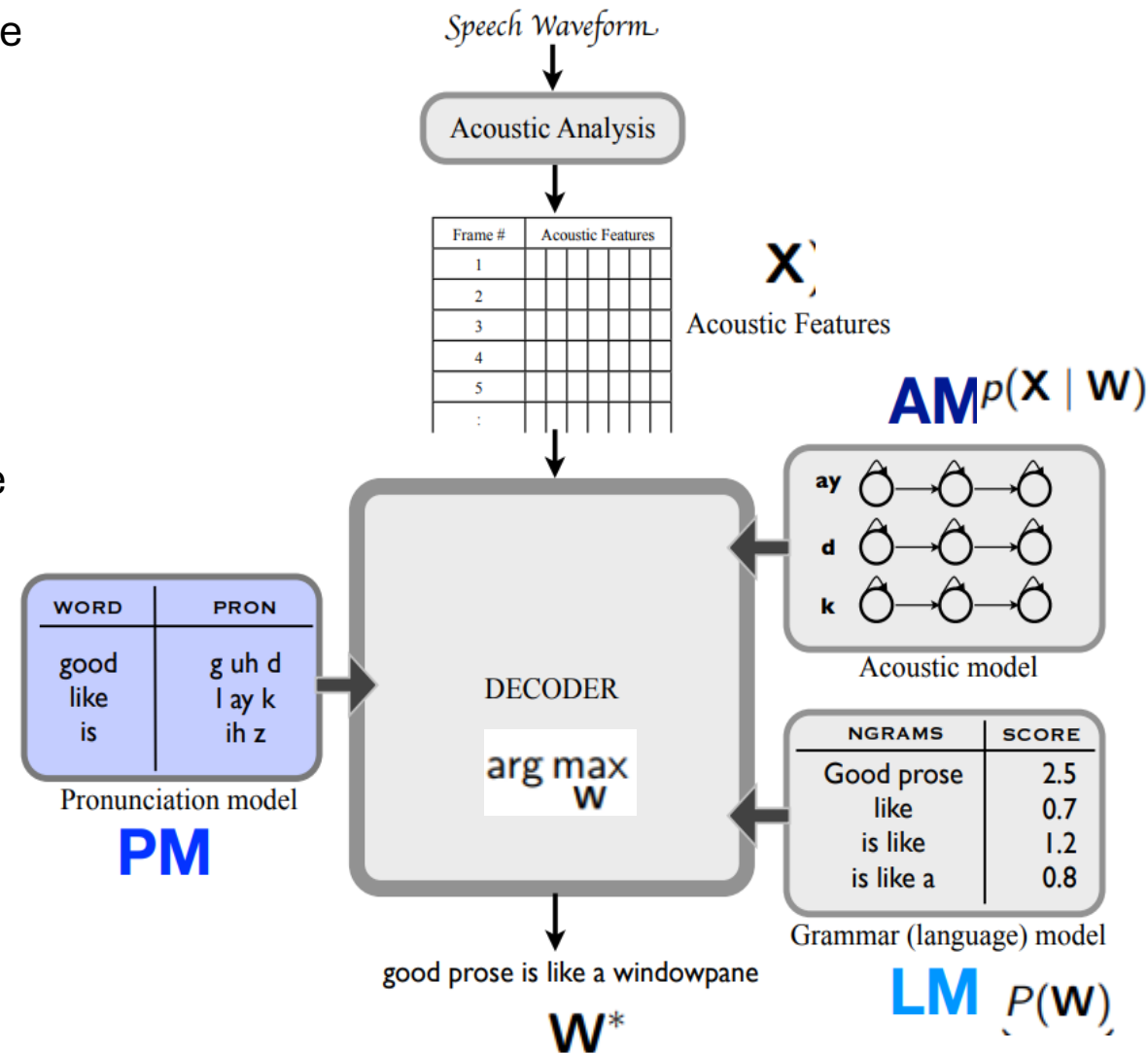
$$\mathbf{W}^* = \arg \max_{\mathbf{W}} P(\mathbf{W} | \mathbf{X})$$

- Find the word sequence $\mathbf{W}^*$ that has the **highest probability** given the observed acoustic features $\mathbf{X}$.

The **posterior probability** that the word sequence $\mathbf{W}$ produced the acoustic signal $\mathbf{X}$ is $P(\mathbf{W}|\mathbf{X})$

Applying Bayes theorem: $P(\mathbf{W} | \mathbf{X}) = \frac{p(\mathbf{X} | \mathbf{W})P(\mathbf{W})}{p(\mathbf{X})}$

$$\mathbf{W}^* = \arg \max_{\mathbf{W}} \underbrace{p(\mathbf{X} | \mathbf{W})}_{\text{Acoustic model}} \underbrace{P(\mathbf{W})}_{\text{Language model}}$$



ASR architecture: the classical way

- **Speech Waveform** : The raw audio signal input containing spoken words.

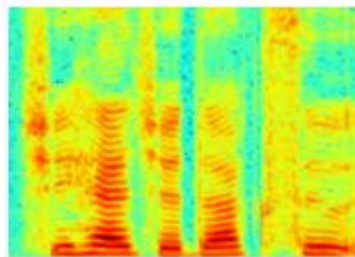
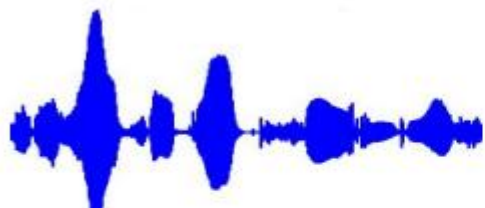


Speech Waveform
↓

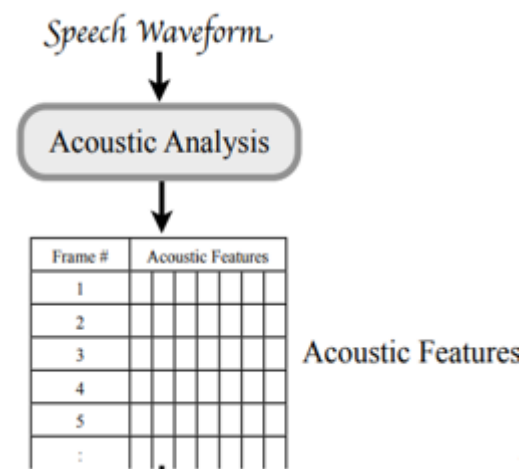
- Pre processing the data:
 - All the data should be in the same dimension (mono or stereo)
 - Standardize and convert all audio to the same sampling rate
 - Resize all the audio samples to have the same length by either extending its duration by padding it with silence, or by truncating it

ASR architecture: the classical way

- **Acoustic Analysis** : Extracts numerical features (e.g., MFCCs) from the speech waveform over short time frames.



$$\mathbf{X} = \mathbf{x}_1 \mathbf{x}_2 \dots \mathbf{x}_T$$

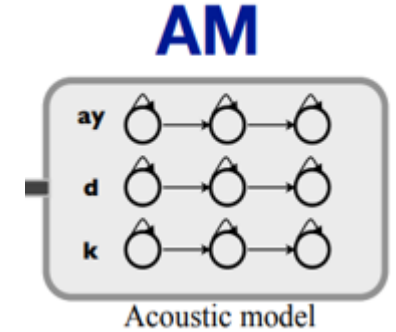


Why small frames?

- The vocal tract configuration changes slowly → speech can be treated as *stationary* for 20–30 ms.
- Too large: mix multiple phonemes → lose detail.
- Too small: poor frequency resolution

ASR architecture: the classical way

- **Acoustic Model (AM)** : Models the statistical relationship between phonetic units (like phonemes) and the acoustic features.



Speech definition

- **Phoneme:** A discrete & distinctive unit of a language that can be used to differentiate between words. Most of the language have 20-60 phonemes.

VOWELS	monophthongs				diphthongs		Phonemic Chart voiced unvoiced	
	i:	ɪ	ʊ	u:	ɪə	eɪ		
	sheep	ship	good	shoot	here	wait		
	e	ə	ɜ:	ɔ:	ʊə	ɔɪ		
CONSONANTS	bed	teacher	bird	door	tourist	boy		
	æ	ʌ	ɑ:	ɒ	eə	aɪ		
	cat	up	far	on	hair	my		
	p	b	t	d	tʃ	dʒ	k	g
	pea	boat	tea	dog	cheese	June	car	go
	f	v	θ	ð	s	z	ʃ	ʒ
	fly	video	think	this	see	zoo	shall	television
	m	n	ŋ	h	l	r	w	j
	man	now	sing	hat	love	red	wet	yes

[a]  Chat	[e]  Éléphant	[ɛ]  Chèvre	[ə]  Grenouille	[i]  Tigre	[o]  Dauphin	[ɔ]  Crocodile
[y]  Autruche	[u]  Souris	[ø]  Paresseux	[œ]  Écureuil	[ã]  Serpent	[ẽ]  Lapin	[õ]  Cochon
[j]  Papillon	[w]  Poisson	[ɥ]  Huître	[b]  Baleine	[k]  Caméléon	[d]  Dragon	[f]  Phoque
[g]  Gorille	[ʒ]  Girafe	[l]  Lion	[m]  Mouton	[n]  Rhinocéros	[p]  Panda	[R]  Renard
[s]  Singe	[z]  Zèbre	[t]  Tortue	[v]  Vache	[ʃ]  Chien	[ʒ]  Araignée	 www.francoisavec pierre.com

Why do we need to use basic unit

Why do we need phonemes from a modelling point of view?

Why do we need to use basic unit

Why do we need phonemes from a modelling point of view?

Imagine that you trained your model with only samples of 1, 4, and 5



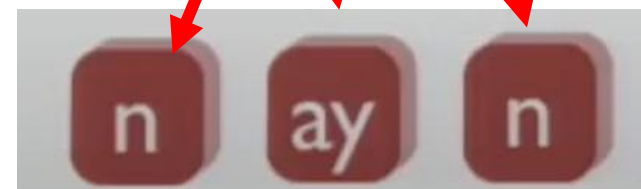
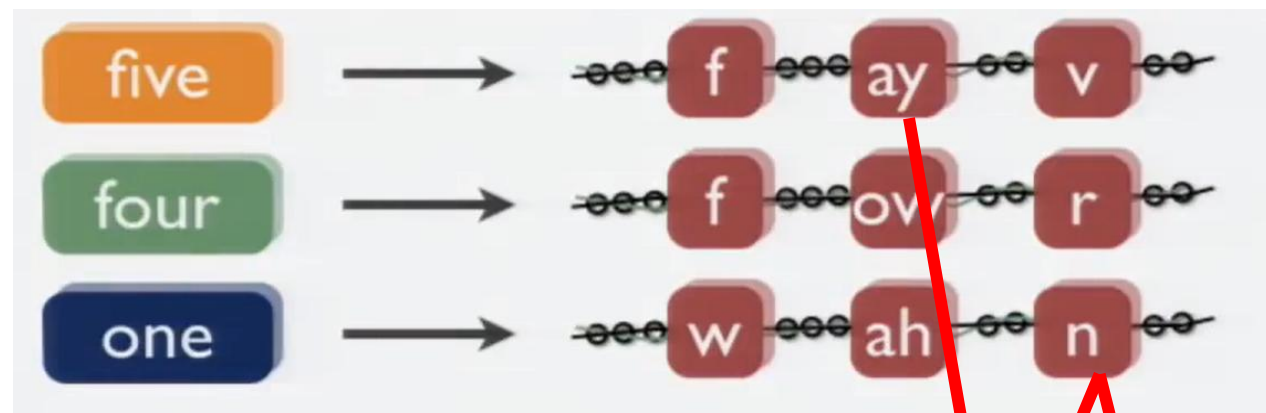
From training!!



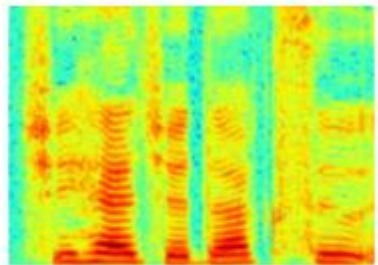
A test speech wavefront

**How will you do with a new word never
seen during training???**

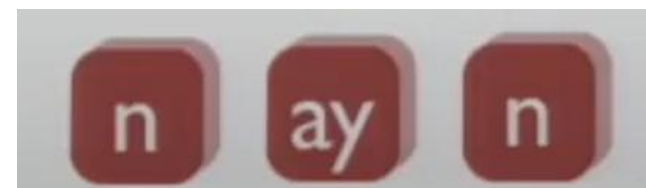
Why do we need to use basic unit



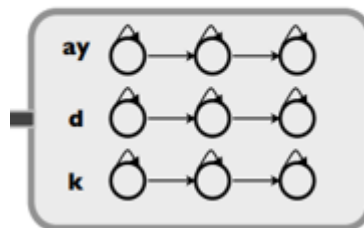
Features X to phonemes?



$$\mathbf{X} = x_1 x_2 \dots x_T$$



AM

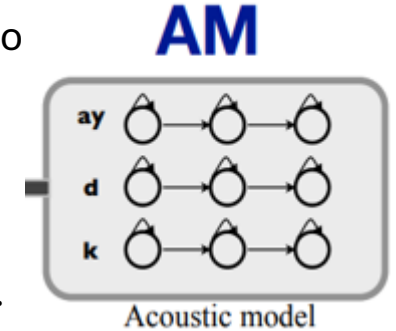


Acoustic model

Acoustic Model: HMM+GMM

Hidden Markov Model: A simple but powerful model for mapping a sequence of continuous observations to a sequence of discrete outputs. The system has a set of discrete states and it evolves over time through **hidden states**, which emit **observable outputs**.

Why hidden? In an HMM, the states like phonemes or sub-phonemes are not directly observed. What we observe are features, such as MFCC vectors, that depend probabilistically on those hidden states. That's why it's called *hidden*.

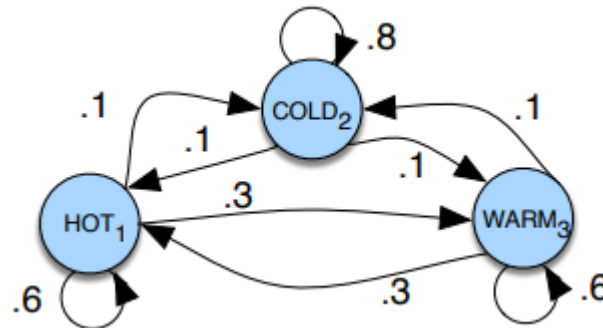


Composition

- **Hidden States:** The actual states of the model, which are not directly observable.
- **Observations:** The visible outputs, which are influenced by the hidden states.
- **Transition Probabilities:** As in Markov Chains, these describe the chance of transitioning from one hidden state to another.
- **Emission Probabilities:** These probabilities model the likelihood of observing a particular visible state given a hidden state.
- **Initial State Probabilities:** These are the probabilities of starting in a particular state.

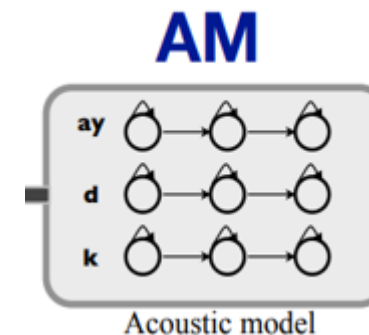
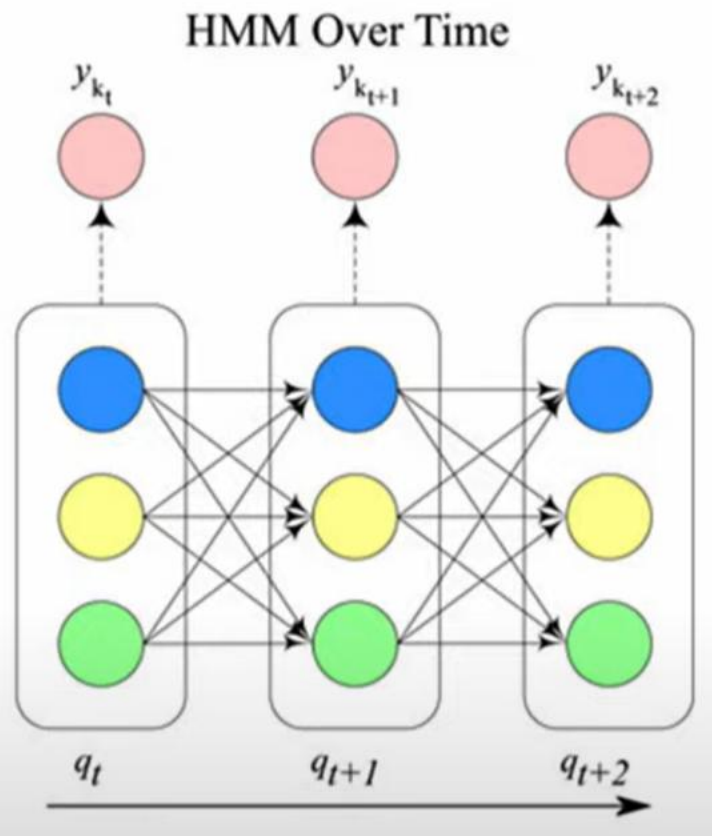
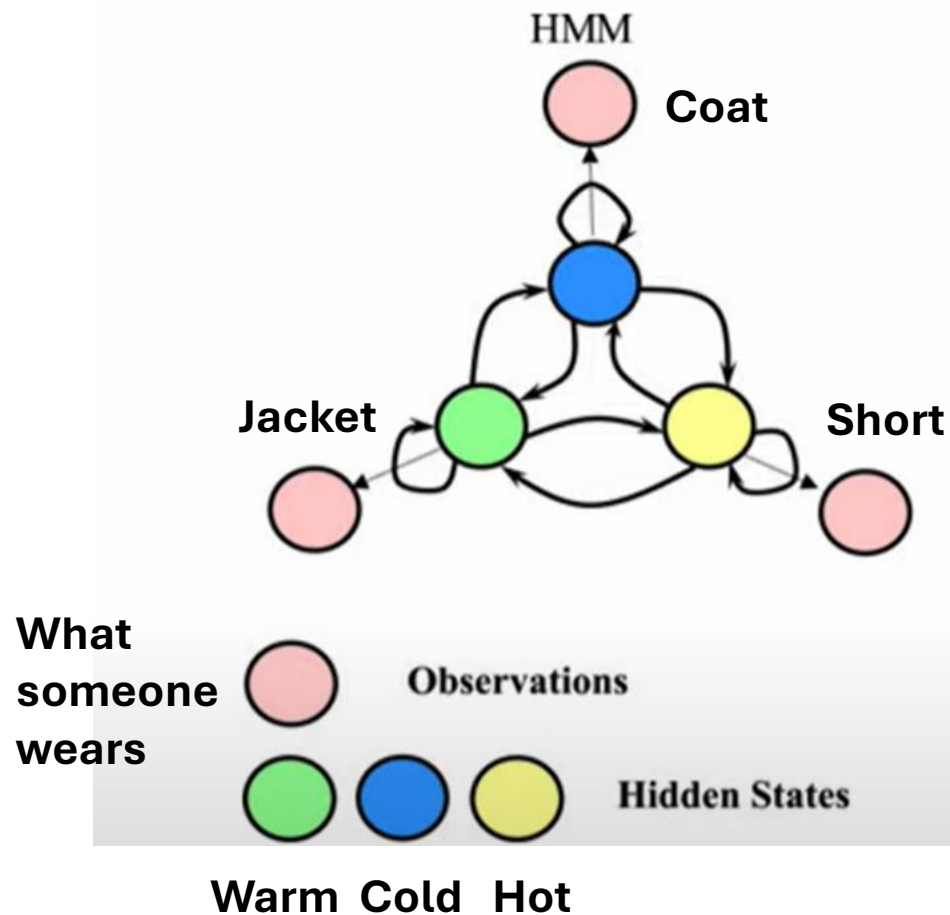
Markov chain?

- **A Markov chain** is a model that tells us something about the probabilities of sequences of random variables, states, each of which can take on values from some set. Therefore, it represents a system that evolves over time with probabilistic transitions between states.
- A Markov chain makes a very strong assumption that if we want **to predict the future in the sequence**, all **that matters is the current state**
- Transition from state to state according to transition probabilities
 - Ex: What will be the weather tomorrow. Only look at today to predict and not yesterday



A start distribution π is required; setting $\pi = [0.1, 0.7, 0.2]$ for hot, cold, warm

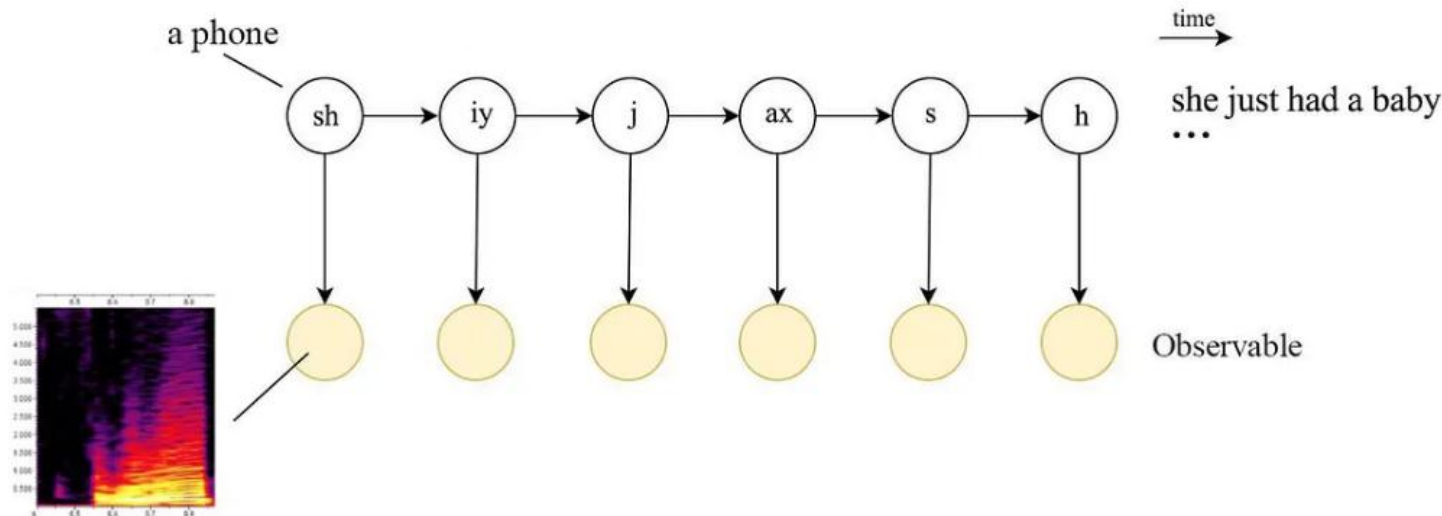
Acoustic Model: HMM+GMM



You don't see the weather directly

Acoustic Model: HMM+GMM

Ex: Nodes below represent the phones and the bottom nodes represent the corresponding observables (the audio features). The horizontal arrows demonstrate the transition in the phone sequence

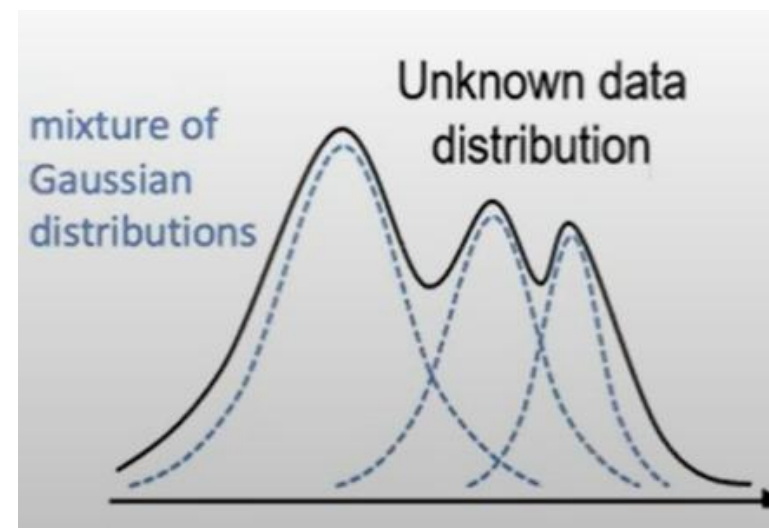
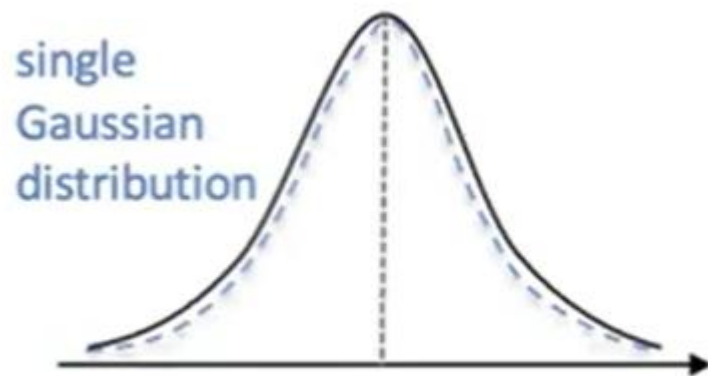


- Traditional HMMs for ASR tend to model each phone with three distinct states. Then, the phone model topologies can be concatenated to form a HMM for the whole word.
- This model naturally generates an alignment between states and observations (and hence words/phones).

Acoustic Model: HMM+GMM

GMM: Gaussian Mixture Model. It is used to model state distribution in HMM and capture the variability in the feature vectors associated with the same phoneme. A single Gaussian can't capture this complex, multi-shaped distribution.

- Speaker differences
- Microphone conditions
- Pronunciation style
- Noise



Acoustic Model: HMM+GMM

How HMM and GMM work in practice?

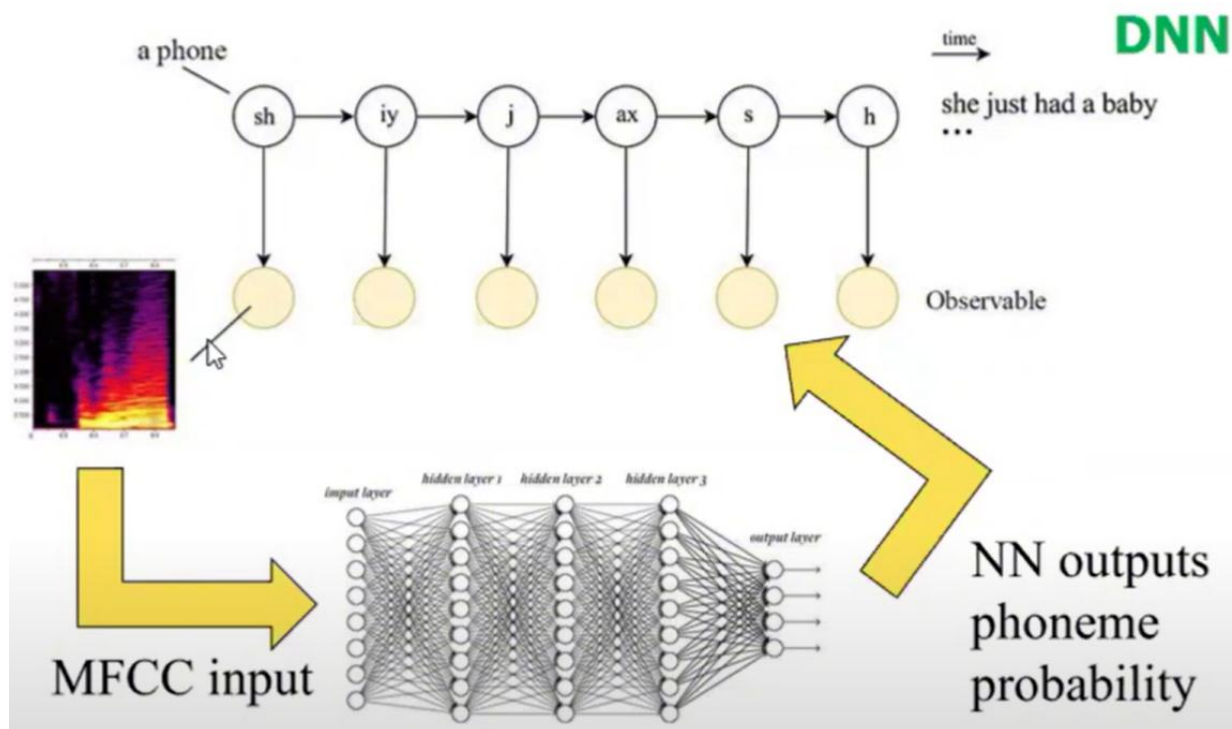
Let's trace how a GMM-HMM would process the audio for the word "cat" (/k/ /æ/ /t/). The system uses an HMM where states correspond to these phonemes.

- State /k/: The first few frames of audio (corresponding to the "k" sound) are fed into the system. The HMM is in the /k/ state. The GMM specifically trained for the /k/ phoneme evaluates these frames. It calculates a high probability, confirming that these audio features are a good match for the /k/ sound. GMMs for other phonemes, like /æ/ or /s/, would return a very low probability for these same frames.
- Transition to /æ/: The HMM knows from its training data that a transition from /k/ to /æ/ is common in English. It moves to the /æ/ state.
- State /æ/: The next set of audio frames (for the "a" sound) are now evaluated by the GMM associated with the /æ/ state. This GMM finds a strong match and outputs a high emission probability.
- Transition and State /t/: The process repeats. The HMM transitions to the /t/ state, and the GMM for /t/ successfully validates the final audio frames of the word.

The total probability of the path /k/ -> /æ/ -> /t/ is calculated by multiplying the transition probabilities and emission probabilities along the way. The decoder is responsible for finding the sequence of states that has the highest overall probability

Acoustic Model: more modern

Combine with Deep Neural Network!!: Instead of to use the probability from the GMM use DNN



Aspect	GMM	DNN	Why DNN Wins
Data modeling	Assumes Gaussian shape	Learns arbitrary nonlinear mapping	Handles complex acoustic patterns
Feature correlations	Assumes MFCC dimensions are independent	Captures inter-feature dependencies	Speech features are correlated
Context modeling	Uses one frame at a time	Can use multiple frames (temporal context)	Captures coarticulation
Parameter efficiency	Needs many Gaussians	Shared weights & layers	Fewer parameters, more expressive

Language model

An acoustic model can determine the probability of phonemes given a piece of audio, but this alone is insufficient for accurate transcription. For example, the phrases "recognize speech" and "wreck a nice beach" can sound very similar. An acoustic model might assign high probabilities to both interpretations. To resolve this ambiguity, the system needs to understand which sequence of words is more likely in the given language.

NGRAMS	SCORE
Good prose	2.5
like	0.7
is like	1.2
is like a	0.8

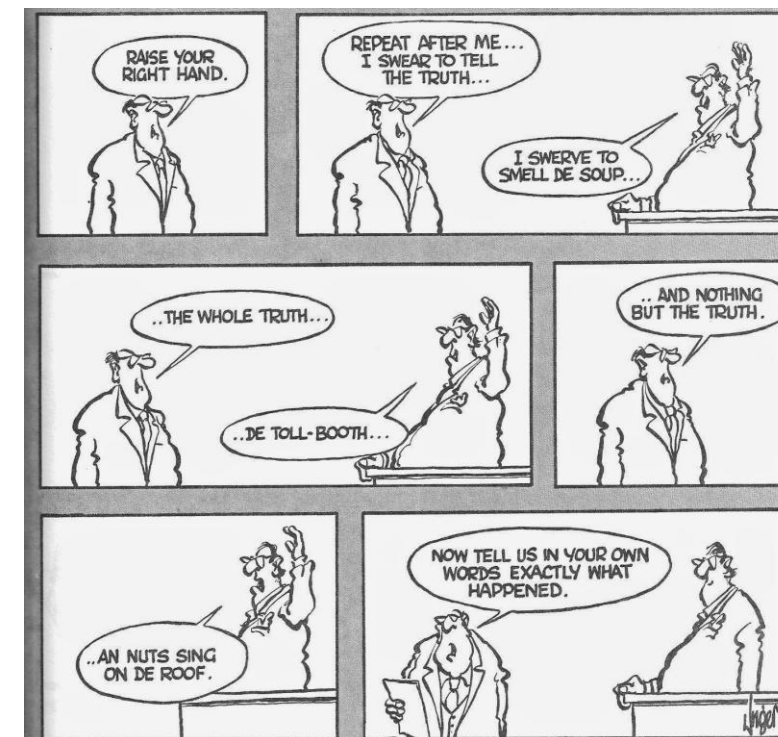
Grammar (language) model

LM

Language Model (LM) : Assigns probabilities to word sequences based on grammar and linguistic context.

Why do you need a language model?

- Some sequences of words are more likely to be a good English sentence than others
- Phoneme order from the acoustic model isn't enough. Phoneme sequences often correspond to multiple words (two, too, to).



Language model

A language model computes the probability of a word sequence W , denoted as $P(W)$. A higher probability means the sequence is more common or grammatically sound.

$$P(\text{"recognize speech"}) \gg P(\text{"wreck a nice beach"})$$

NGRAMS	SCORE
Good prose	2.5
like	0.7
is like	1.2
is like a	0.8

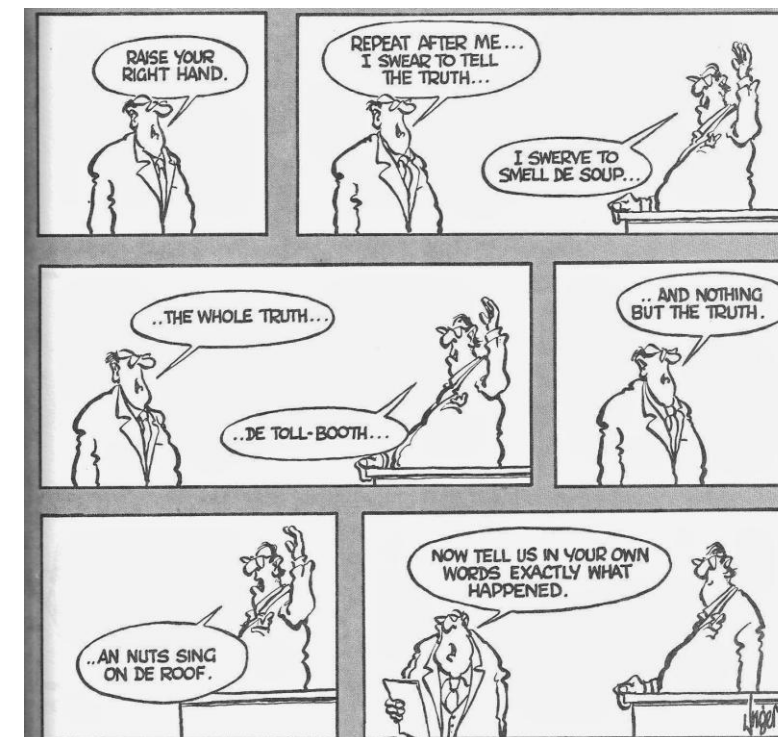
Grammar (language) model

LM

Where Does Language Knowledge Come From?

Language models learn these probabilities by being trained on enormous datasets of text:

- Which words are most common.
- Which words tend to follow other words (e.g., "speech" is more likely to follow "recognize" than "wreck").
- Common grammatical structures and phrasing.



Sentence probability

Ex: “Nothing, but , the, truth”

$P(\text{“Nothing”, “but”, “the”, “truth”}) = ?$

Chain rule with A, B, C, D:

$$P(A, B, C, D) = P(A) P(B \mid A) P(C \mid A, B) P(D \mid A, B, C).$$

$$\begin{aligned} P(\text{“Nothing”, “but”, “the”, “truth”}) &= P(\text{“Nothing”}) \\ &\quad \times P(\text{“but”} \mid \text{“Nothing”}) \\ &\quad \times P(\text{“the”} \mid \text{“Nothing”, “but”}) \\ &\quad \times P(\text{“truth”} \mid \text{“Nothing”, “but”, “the”}). \end{aligned}$$

N-gram Language Models

YOU WILL NEVER HAVE ENOUGH DATA TO COMPUTE THE PROBABILITY FOR LONG SENTENCES.

N-gram models: predict conditional probability $p(W|h)$ of a word W given some history h .

The probability of a word depends only on the $n-1$ previous words

Using a ****bigram**** model: $P(\text{one} \mid \text{give, me, the, right}) \approx P(\text{one} \mid \text{right})$

Using a ****trigram**** model: $P(\text{one} \mid \text{give, me, the, right}) \approx P(\text{one} \mid \text{the, right})$

N-gram models are built by **counting word sequences** in large text corpora

$$P(w_i | w_{i-1}, \dots, w_{i-N+1}) = \frac{\text{Count}(w_{i-N+1}, \dots, w_i)}{\text{Count}(w_{i-N+1}, \dots, w_{i-1})}$$

But unseen sequences cause zero probabilities!

To fix that, we use **smoothing, i.e., use a combination of low order n-grams or add one**

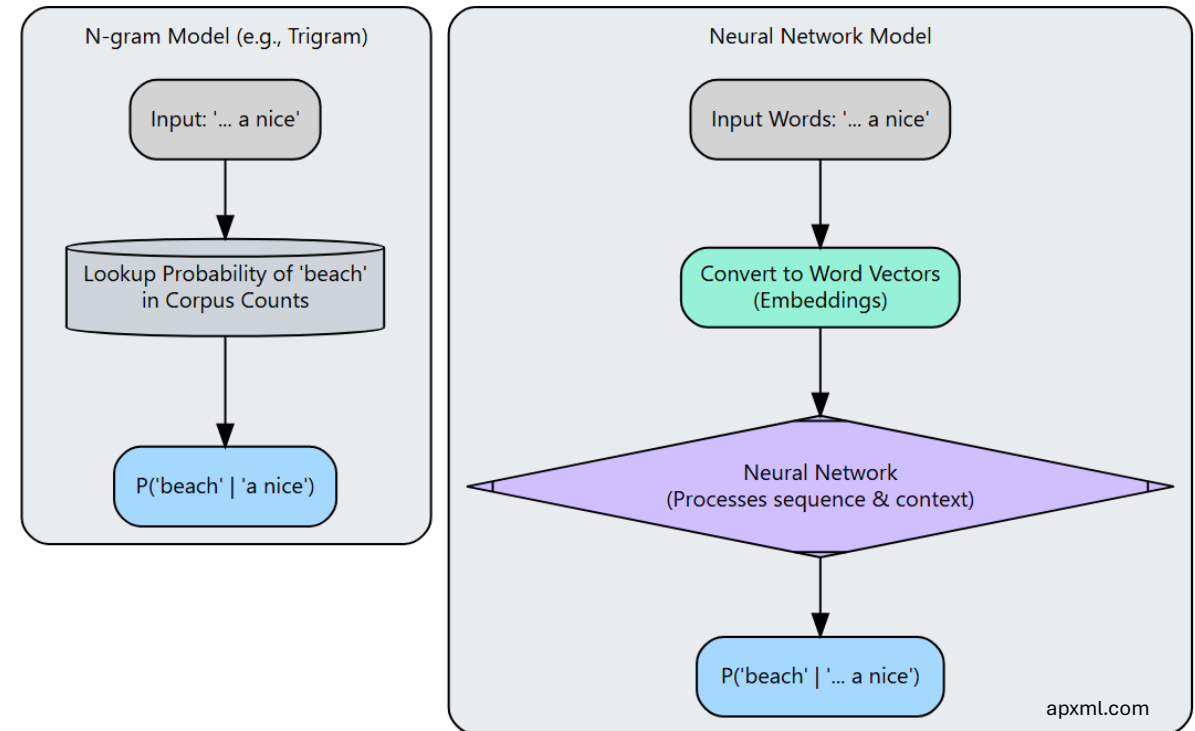
$$P(w_n | w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n) + 1}{\text{Count}(w_{n-1}) + V} \quad V \text{ is the size of the } \mathbf{vocabulary},$$

More modern!! Neural network language

The first step in a neural language model is to move away from treating words as simple, distinct text labels. Instead, each word is mapped to a dense list of numbers called a **word embedding** or a **word vector**.

The important property of these vectors is that words with similar meanings or that are used in similar contexts will have similar vectors. For example, the vectors for "nice", "good", and "lovely" would be mathematically close to each other in the vector space.

If the model has learned the phrase "a good day", it can infer that "a nice day" is also a likely phrase because the vectors for "good" and "nice" are similar.



Pronunciation model

Pronunciation Model (PM) : Also called Lexicon, it maps each word to its phonetic transcription (how it sounds).

This ASR module, is the only one not learned by model.

WORD	PRON
good	g uh d
like	l ay k
is	ih z

Pronunciation model

PM

Cat: /kæt/

Sits: /sɪts/

On: /ɒ:n

A: /ə/

Mat: /mæt/

Decoder

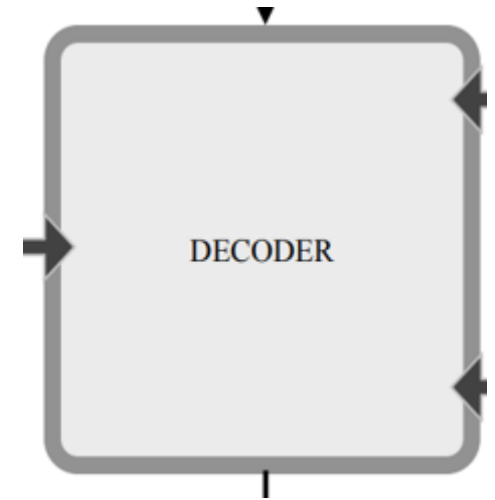
Decoder : Combines the AM, LM, and PM to find the most likely word sequence given the observed speech.

Remember, the core task of speech recognition is to find the word sequence (W) that is most probable given the observed audio features (X). This is often expressed as finding the sequence that maximizes the product of two probabilities

$$W^* = \arg \max_W \underbrace{p(\mathbf{X} | \mathbf{W})}_{\text{Acoustic model}} \underbrace{P(\mathbf{W})}_{\text{Language model}}$$

Therefore, it is “only” a search problem

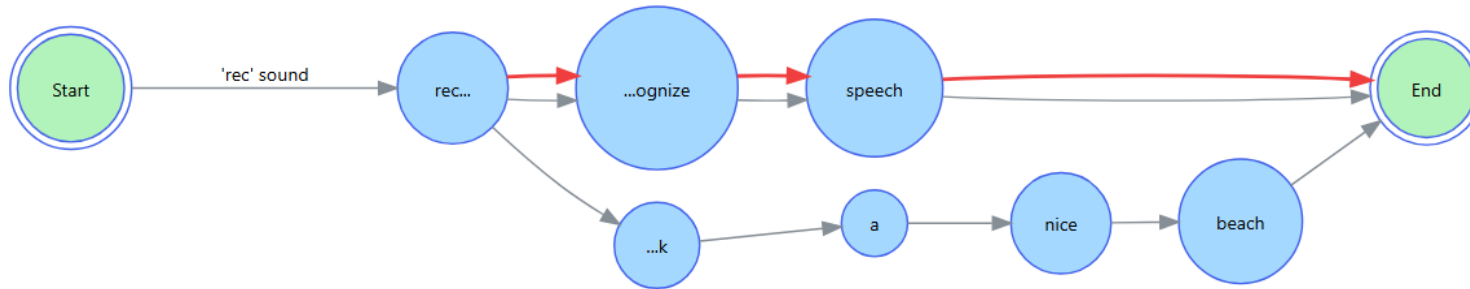
It takes the acoustic score for a potential sentence and multiplies it by the language model score for that same sentence. It does this for every plausible hypothesis and chooses the one with the highest combined score.



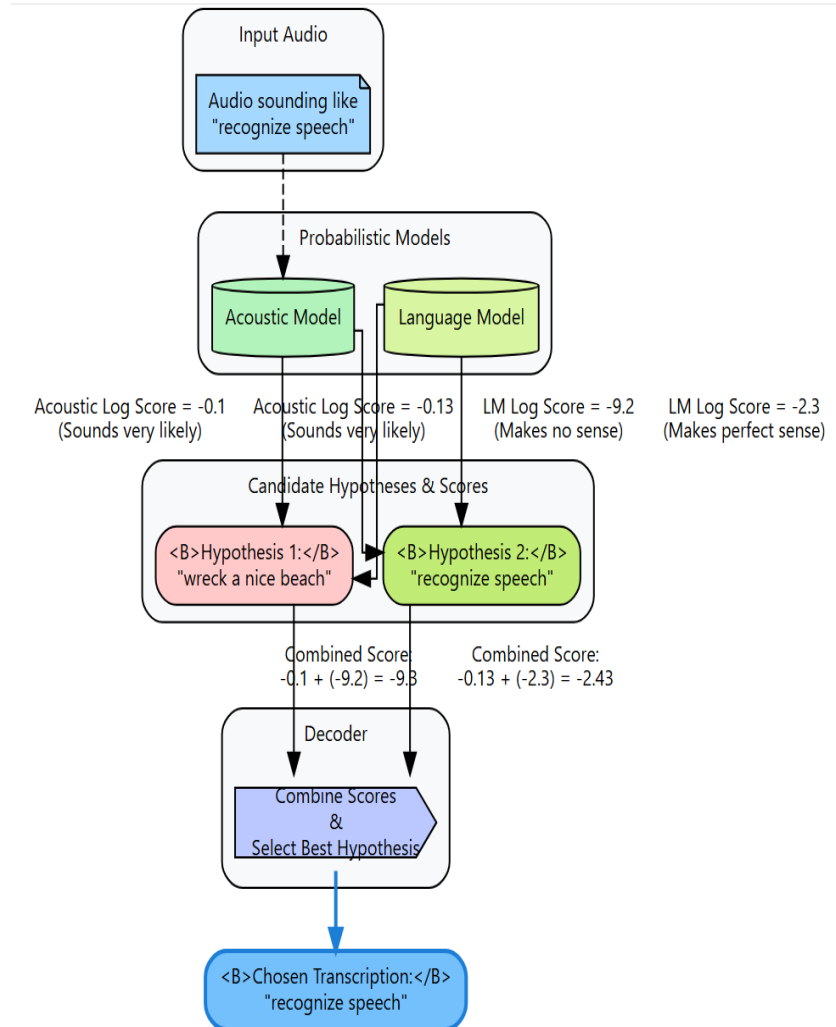
Decoder

Checking every single possibility is computationally impossible (20,000 words, the number of possible 5-word sentences would be $20,000^5$)

A better way to approach this is to reframe it as a pathfinding problem. Imagine a giant graph where each point represents a possible word that could follow the previous one.



Pruning: As the decoder builds up sentence hypotheses word by word, it keeps track of their scores. If one path becomes significantly less likely than the best-known path so far, the algorithm simply abandons it.



Decoder: Viterbi

The Viterbi algorithm was first proposed by A. Viterbi in 1967

It is a dynamic programming algorithm use to find the **most likely sequence of hidden states** that could have produced the observed features. Unlike the beam search we keep only 1 sequence per state. Viterbi **does look at all possible transitions** from the previous step...but **only one step back**, *not all paths in history*.

That's the key difference between **exploring everything** and **dynamic programming**.

Ex: You hear a **sequence of activities**, and you want to guess the **weather**: (sunny or rainy)

You have : transition probabilities $P(R \rightarrow R) = 0.7$ $P(R \rightarrow S) = 0.3$ Emission prob: $P(\text{Walk}|\text{Sunny})=2/3$ $P(\text{Shop}|\text{Sunny})=1/3$
 $P(S \rightarrow R) = 0.4$ $P(S \rightarrow S) = 0.6$ $P(\text{Walk}|\text{Rainy})=0.2$ $P(\text{Shop}|\text{Rainy})=0.8$

Start: $\pi(\text{Sunny})=0.4$, $\pi(\text{Rainy})=0.6$

We observe **$t_1=\text{Walk}$** , **$t_2=\text{Shop}$**

Viterbi (2 steps):

$t_1 = \text{Walk}$

$\delta_1(S) = 0.4 \times 2/3 = 0.2667$

$\delta_1(R) = 0.6 \times 0.200 = 0.1200$

Sunny -> Rainy

For Sunny at t_2 :

from S: $0.2667 \times 0.6 = 0.1600$

from R: $0.1200 \times 0.3 = 0.0360$

→ pick **0.1600**, then $\times P(\text{Shop}|S)=0.333 \rightarrow \delta_2(S)=0.0533$

For Rainy at t_2 :

from S: $0.2667 \times 0.4 = 0.1067$

from R: $0.1200 \times 0.7 = 0.0840$

→ pick **0.1067**, then $\times P(\text{Shop}|R)=0.800 \rightarrow \delta_2(R)=0.0853$

Summary

1. Audio Input & Feature Extraction

- Raw waveform → short frames (~25 ms) → feature vectors (e.g., MFCCs).
- Produces a compact, informative representation of the sound.

2. Acoustic Model (AM) – “What did it sound like?”

- Computes $P(O|W)$: likelihood of the observed features given phonemes/words.
- Outputs probabilities for each phoneme at every frame.

3. Language Model (LM) – “Does it make sense?”

- Computes $P(W)$: probability of a word sequence based on linguistic context.
- Prefers plausible sentences (“recognize speech” >> “wreck a nice beach”).

4. Decoder – “Combine and decide.”

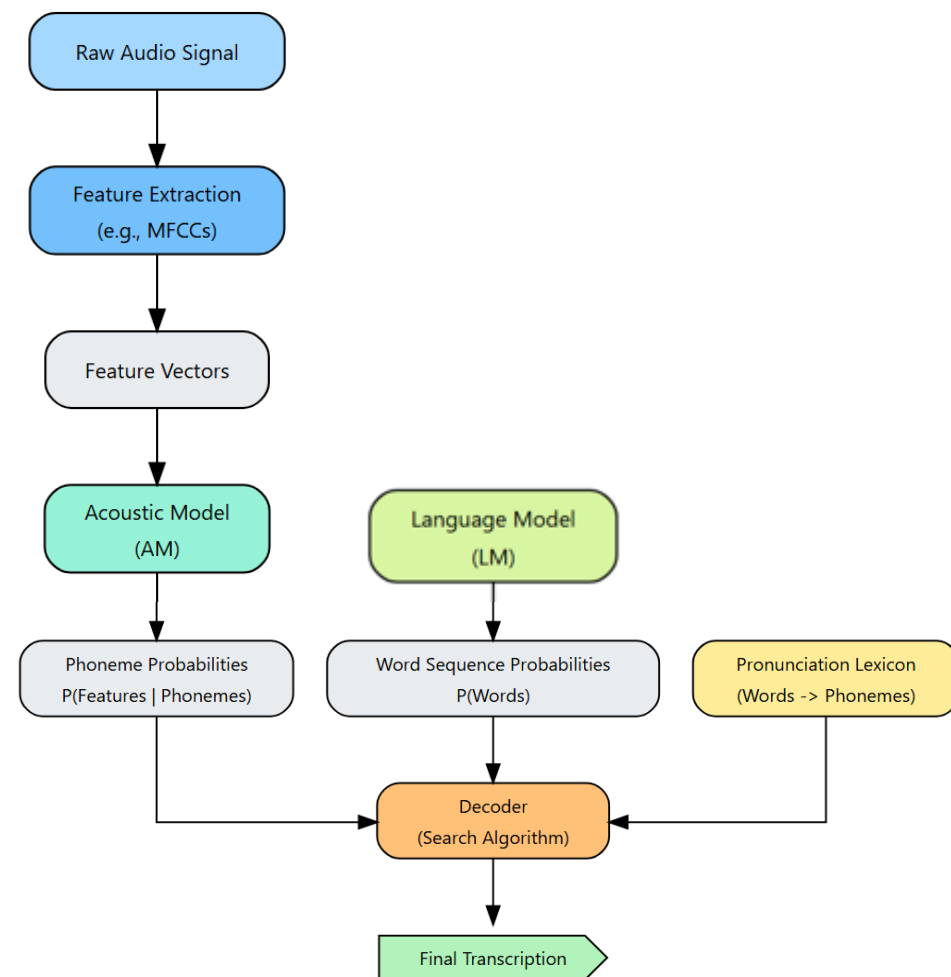
- Integrates AM + LM + Pronunciation Lexicon.
- Finds

$$\hat{W} = \arg \max_W P(O|W) P(W)$$

using search algorithms (e.g., Viterbi).

5. Final Output

- The most probable word sequence — the **recognized transcription**.



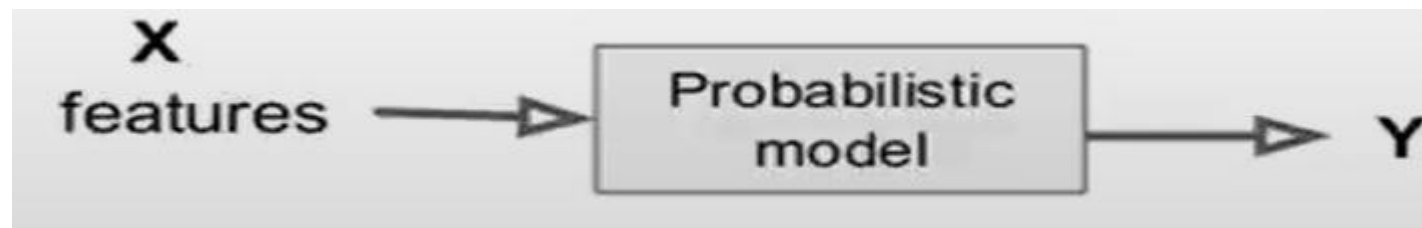
End-to-end ASR

Each component is trained independently, with a different objective:

- Errors in one component may not behave well with errors in another component
- Instead, let's train models that encompass all of these components together

Two approaches for end-to-end ASR:

- **Connectionist Temporal Classification (CTC):** *encoder-only* models with a linear classification head on top
- **Sequence-to-sequence (Seq2Seq):** *encoder-decoder* models, with a cross-attention mechanism between the encoder and decoder



Prior to 2022, CTC was the more popular of the two architectures, with encoder-only models such as Wav2Vec2, HuBERT and XLSR

Connectionist Temporal Classification (CTC)

Why CTC Was Needed?

In ASR, input and output lengths differ:

- Audio: hundreds of time steps (frames)
- Transcript: only a few words or characters

Ex: Audio frames $\rightarrow [x_1, x_2, x_3, \dots, x_{100}]$

Transcript $\rightarrow$ "HELLO"

The problem: we don't know which part of the audio corresponds to each character.

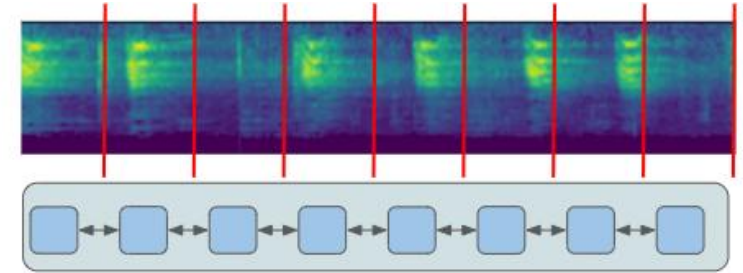
Traditional systems needed manual alignment or HMM state modeling.

Connectionist Temporal Classification (CTC)

- Introduced by Graves et al. (2006).
- It's **not a model architecture**, it's a **mathematical method** used to train models that output sequences (like speech → text) when the alignment between input and output is *unknown*.
- CTC is used to align the input and output sequences when the input is continuous and the output is discrete, and there are no clear element boundaries that can be used to map the input to the elements of the output sequence.
- Introduces a special blank token ($\emptyset$) that represents “no output”. Each time step predicts one of: {A–Z, space, blank}

“Slice” the audio into a sequence of frames

Feed that sequence to the RNN



[source](#)

The job of the CTC algorithm is to take these character probabilities and derive the correct sequence of characters

Connectionist Temporal Classification (CTC)

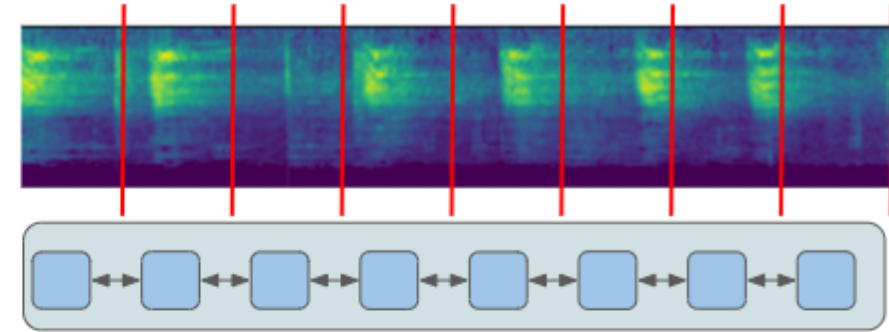
- Merge any characters that are repeated, and not separated by a blank. For instance, we can merge the "oo" into a single "o", but we cannot merge the "o-oo".

This is how the CTC is able to distinguish that there are two separate "o"s and produce words spelled with repeated characters. eg. "-G-o-od"

- Finally, since the blanks have served their purpose, it removes all blank characters. eg. "Good".

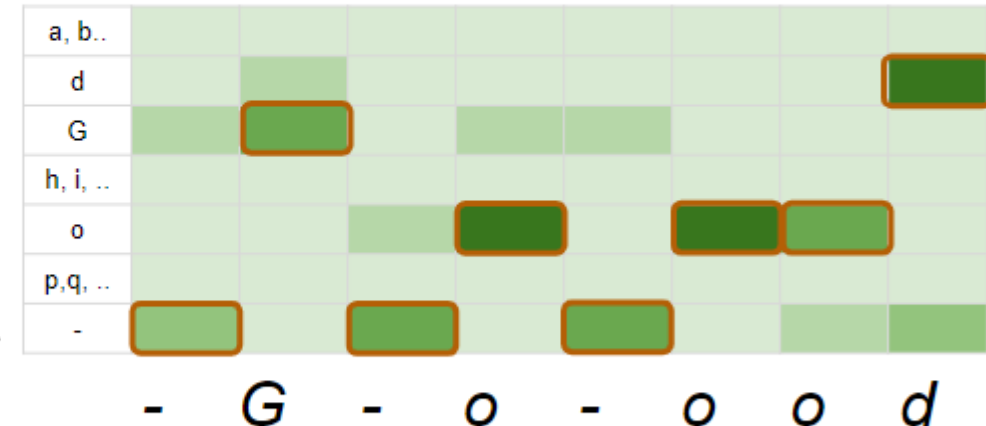
"Slice" the audio into a sequence of frames

Feed that sequence to the RNN



RNN outputs character probabilities

Blank character



Pick best probabilities

Merge repeated characters

Remove blanks

-G-o-od
↓
Good

Transformers

A transformer is a type of artificial intelligence model that learns to understand and generate human-like text by analyzing patterns in large amounts of text/audio data.

Unlike the encoder-decoder architecture relies mainly on Recurrent Neural Networks (RNNs) and process input sequentially, **Transformers** rely entirely on self-attention mechanisms to compute representations of their input and output, allowing for significantly more parallel computation and capturing long-range dependencies more effectively.

Transformers

The Transformer: “Attention Is All You Need” (Vaswani et al., 2017)

A paper that revolutionized deep learning!!!

Most cited paper in deep learning

Important things are presented in this paper:

- Attention mechanism, of course!!!
- Transformer architecture

Has impact on many domains:

- NLP
- Image processing
- Audio (Generative, ASR, etc)
- ...

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

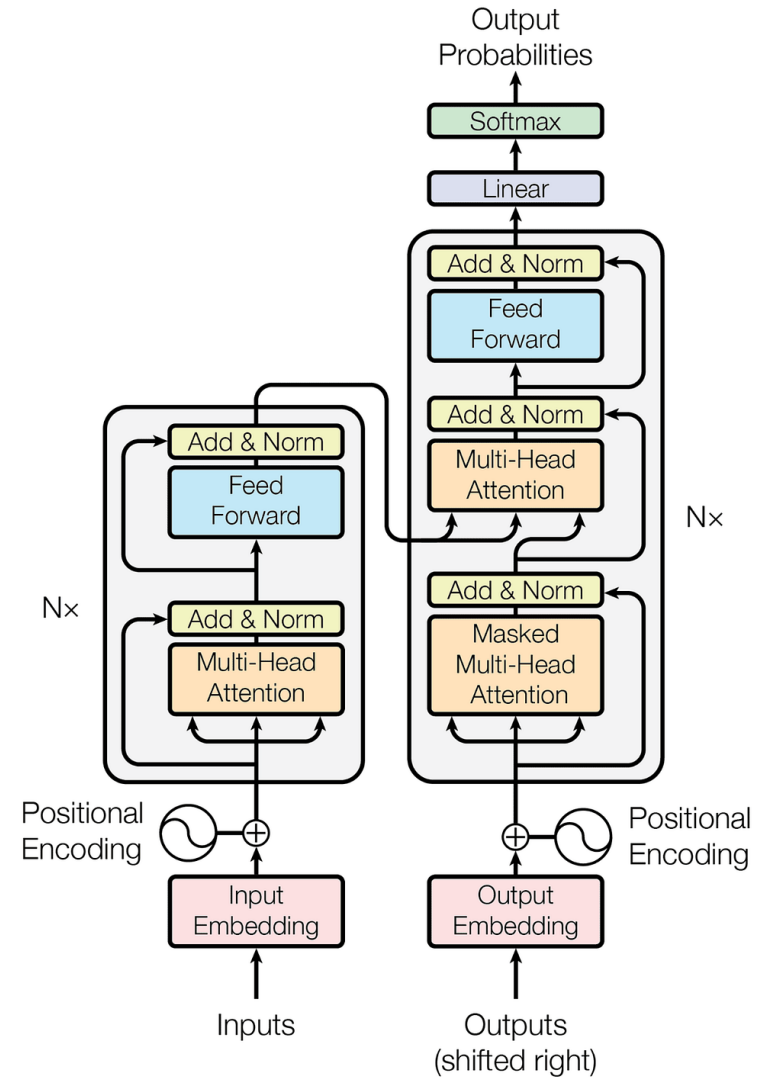
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Transformers: architecture

- Deal with sequential data
- Capture long-term dependencies
- Get rid of recurrence and use self-attention mechanism

Do not be scared by the overall architecture of a transformer, even if it looks complex



Transformers

We can replace this complexity by of two main components:

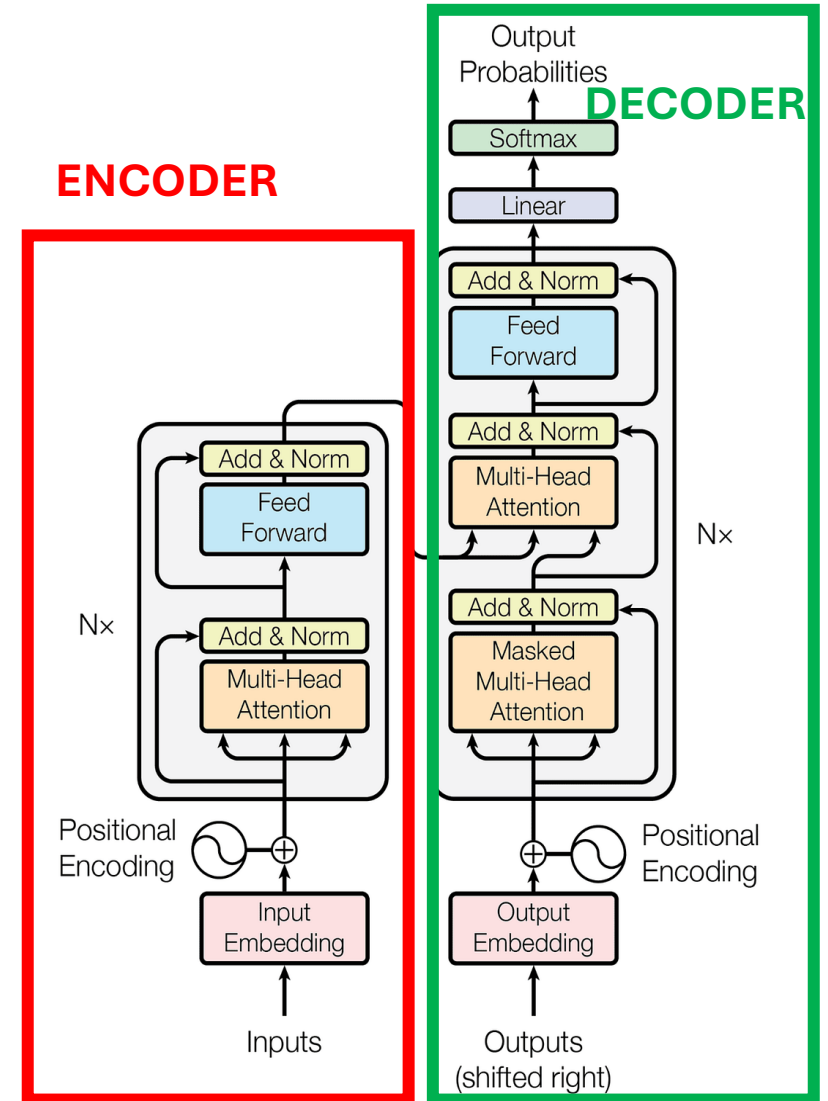
- **Encoder** (left): processes input sequence.
- **Decoder** (right): generates output sequence.

The core architecture of a transformer it is an ENCODER and a DECODER

How does intuitively it work?

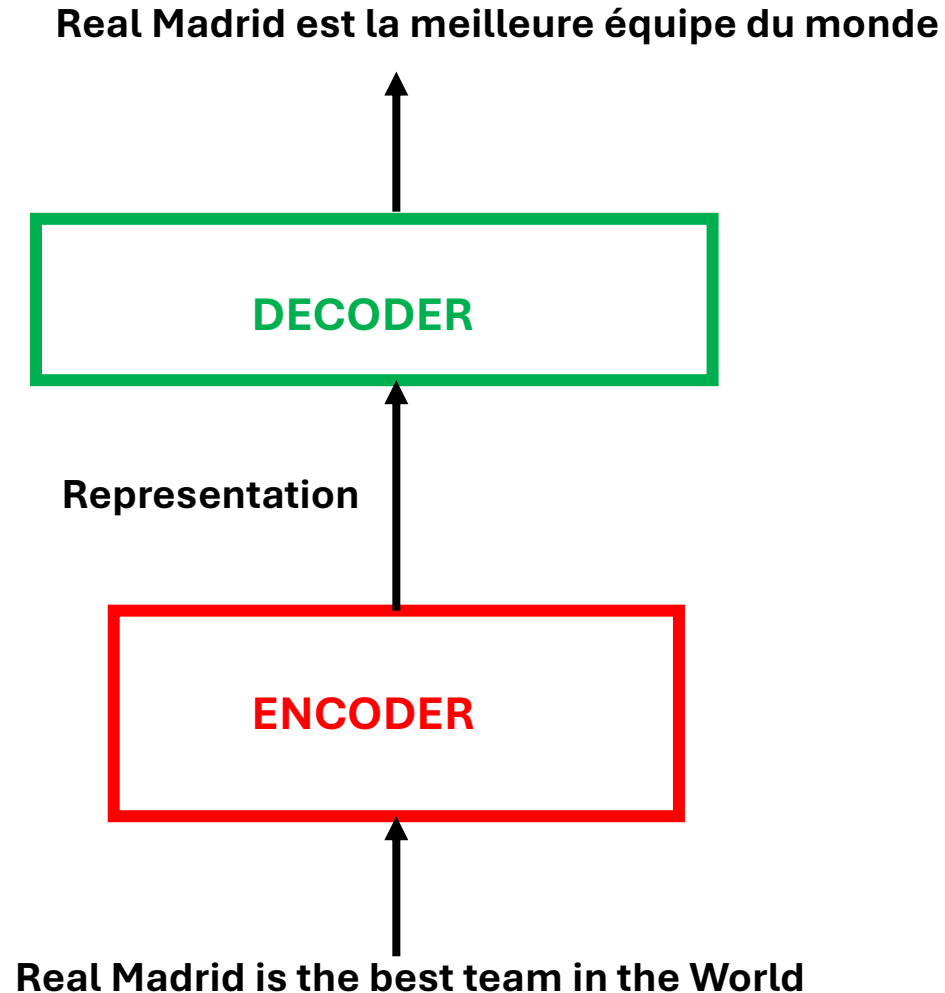
Using the example of language translation as in the paper.

- 1) Feed a sentence to encoder (the sentence you want to translate)
- 2) Encoder outputs representation of the sentence (embedding, technique that maps complex, high-dimensional data into a lower-dimensional vector space)
- 3) Representation is fed to decoder
- 4) Decoder generates translation



Transformers

The encoder transforms the raw input sequence into a high-level representation, a set of “features” capturing temporal information.



Transformers: encoder

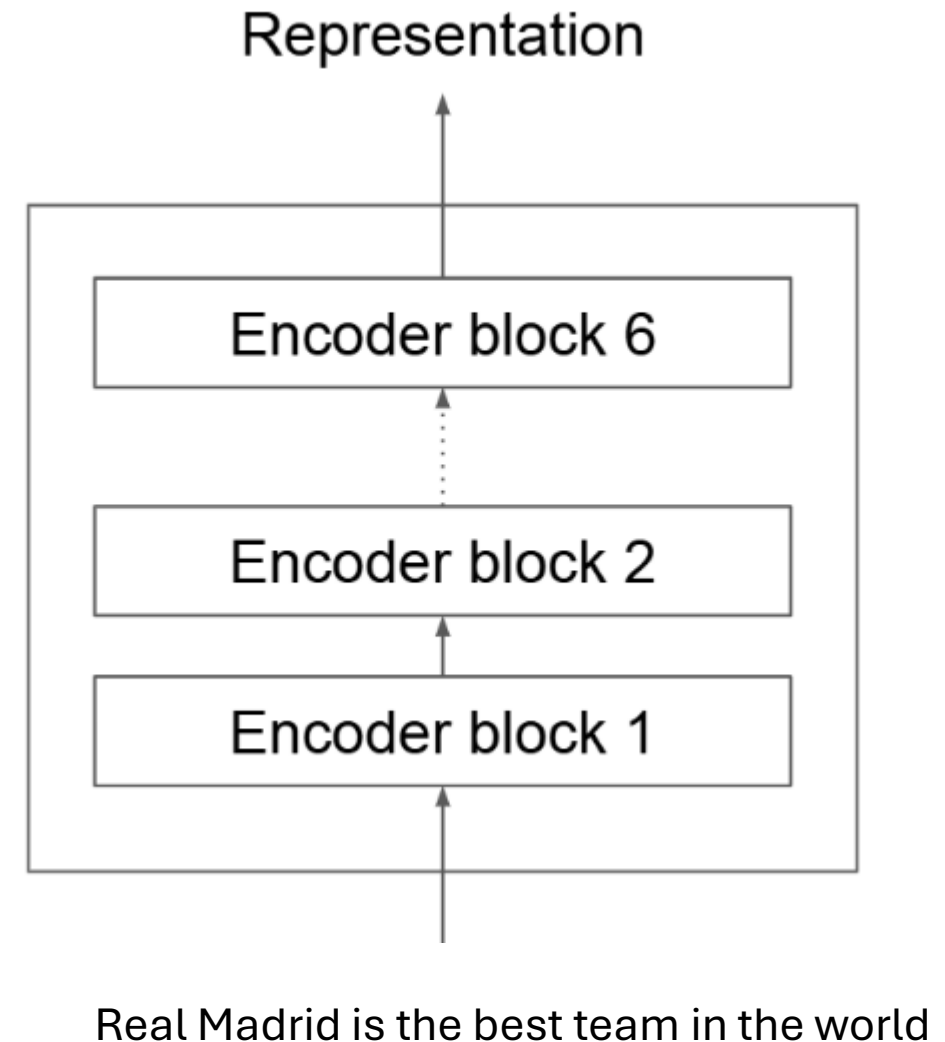
NOT ONLY ONE ENCODER

→ STACK OF N ENCODERS

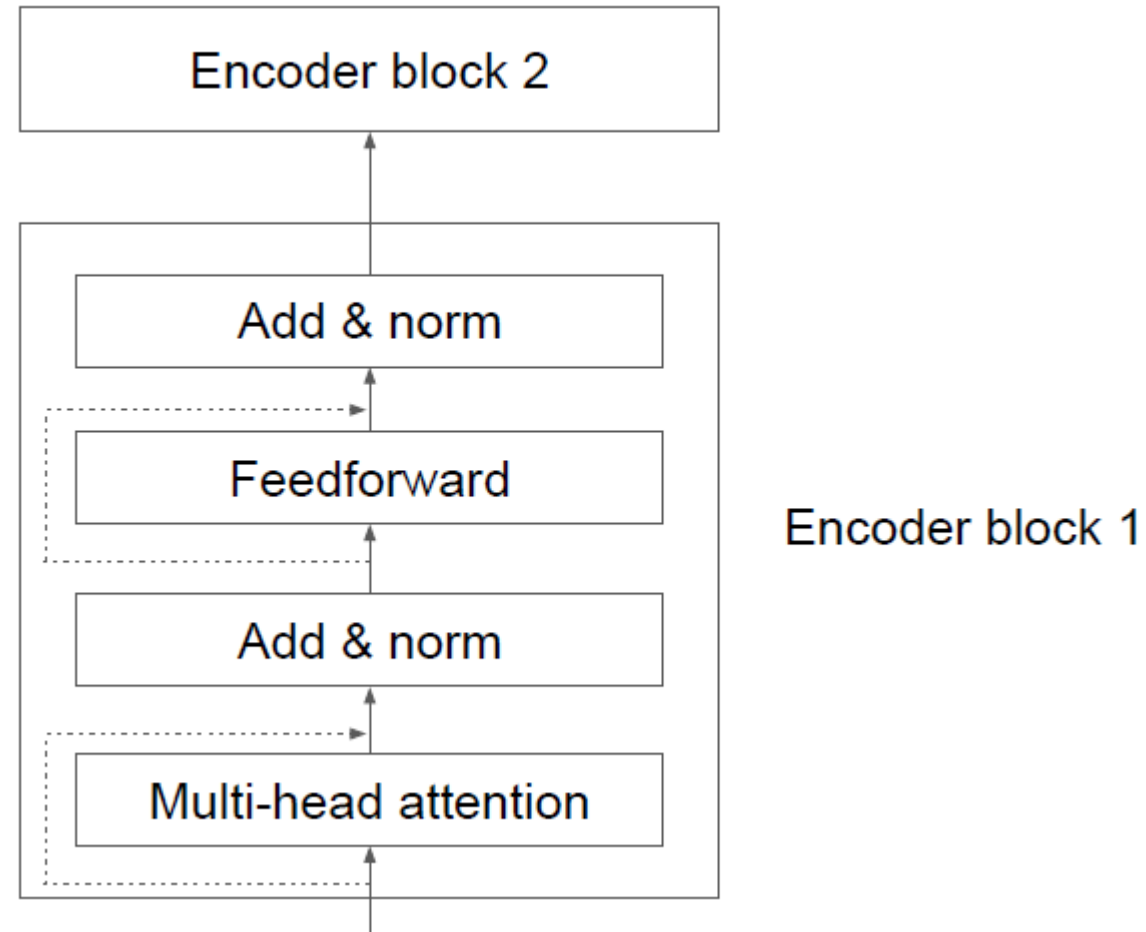
- Generally, you have 6 encoder blocks
- Each encoder block does an encoding and passes it over the next encoder block
- Last encoder block returns representation

Why stacking encoder blocks?

- Complex representation (similar with CNN and number of layers)
- Learn different aspect at each layer



Transformers: encoder



Attention



Attention

Most of your **ATTENTION** is on the Girl with a blue dress

We shouldn't treat all past information equally; instead, we should focus computational effort on the most relevant context for what we're trying to predict.



Attention

Most of your **ATTENTION** is on the Girl with a blue dress

We shouldn't treat all past information equally; instead, we should focus computational effort on the most relevant context for what we're trying to predict.

For example, if we are interested in the word "ferry" we should focus on TICKET



Attention

To understand the concept of attention, we need to introduce a context and the problem we had with previous models.

My friends like tomatoes because they are tasty

Ambiguity in this sentence: **they** is related to my friends or tomatoes.

For humans, it is pretty clear they they refers to tomates and not to my friends...but for a model is not easy. This is why we need to take into account long term relationship!! -> **ATTENTION**

Solution:

We need to compute an embedding that not only has information about each word but each word in relationship with all the other words. **Like this you can understand the overall context!!**

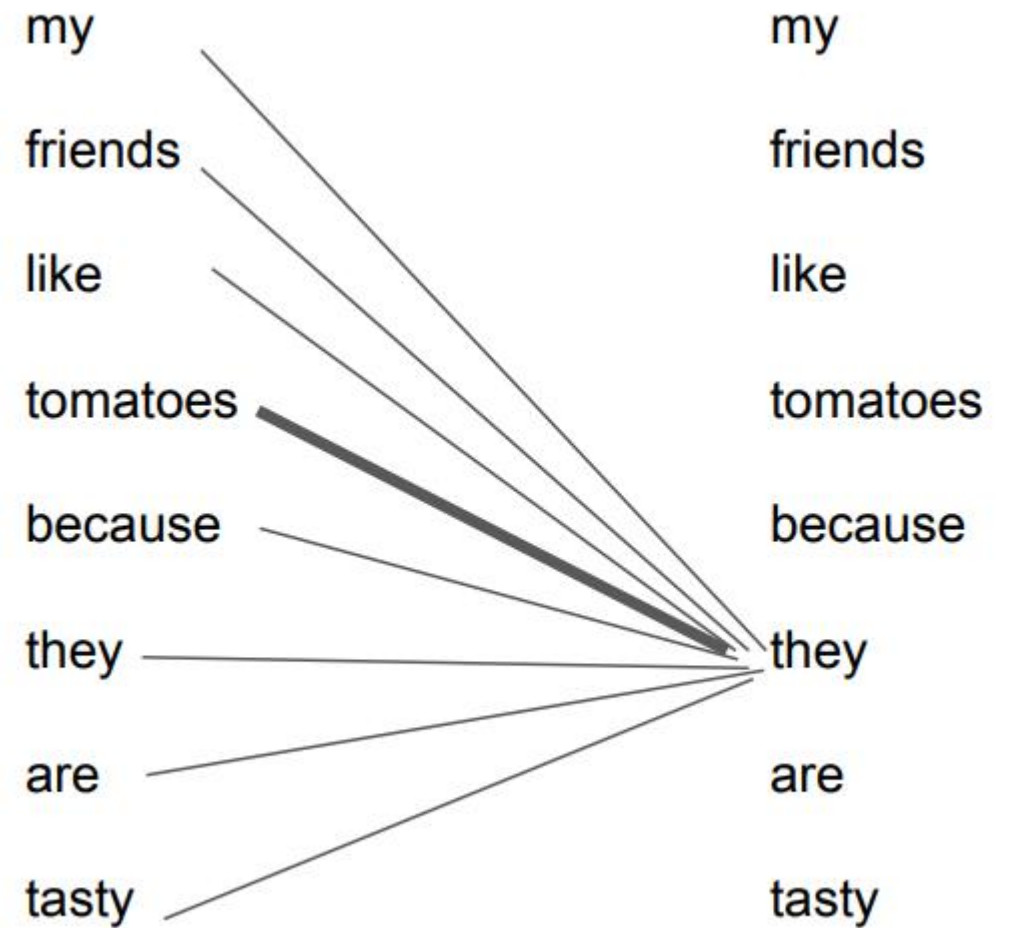
Attention

We create a connection between they and all the other words.

They is related to tomatoes (thickness of the line)

THIS IS ATTENTION

We want to understand the relation for each word with all the words.



Attention

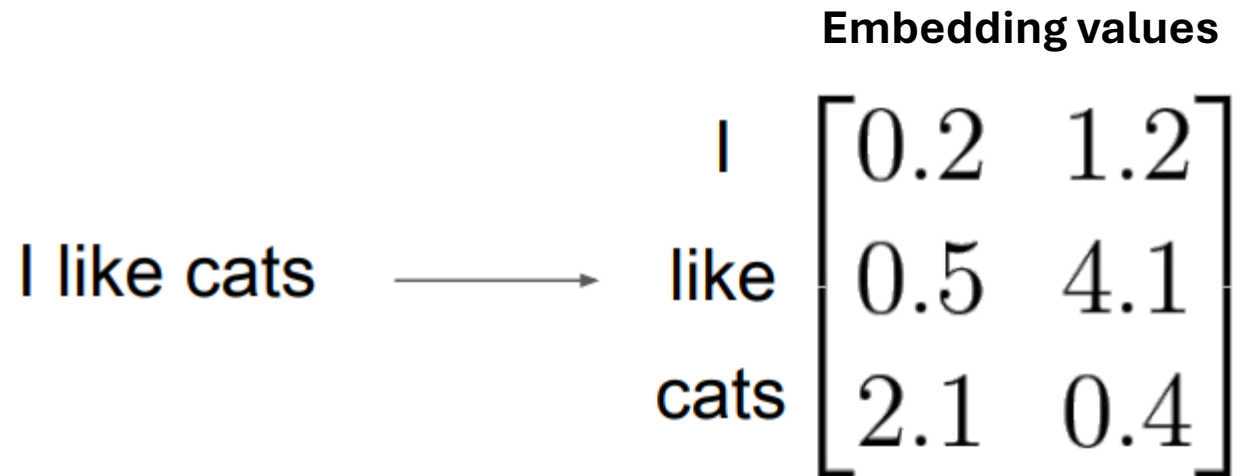
Now, that we have an idea of what is attention, we will look at all the math behind!!

For attention you have mainly 4 protagonists:

- Input embedding matrix (I)
- Query matrix (Q)
- Key matrix (K)
- Value matrix (V)

Attention: input embedding matrix

- Dimension dépend on the number of words and the embedding dimension (64,128, etc... as you want)
- Each row is a word vector
- Exemple, we select 2D embedding for simplicity



Attention: QKV

- QKV matrices have same dimension than matrix I

You obtain QKV by multiplying I with 3 weight matrices

W are obtained with the training

Relation of a word with all other words is computed using those 3 matrix (QKV)

The attention matrix Z is :

$$\begin{aligned}IW_Q &= Q \\IW_K &= K \\IW_V &= V\end{aligned}$$

$$Z(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Attention: QK^T

Step 1:

$$QK^T = \begin{matrix} & \begin{matrix} I \\ like \\ cats \end{matrix} & \begin{bmatrix} 1.3 & 0.8 \\ 0.7 & 3.5 \\ 1.9 & 0.1 \end{bmatrix} & \begin{matrix} q_1 \\ q_2 \\ q_3 \end{matrix} & \begin{matrix} I \\ like \\ cats \end{matrix} & \begin{bmatrix} 0.6 & 0.8 & 2.5 \\ 2.4 & 1.7 & 0.3 \end{bmatrix} & \begin{matrix} k_1 \\ k_2 \\ k_3 \end{matrix} \end{matrix} = \begin{bmatrix} q_1 k_1 & q_1 k_2 & q_1 k_3 \\ q_2 k_1 & q_2 k_2 & q_2 k_3 \\ q_3 k_1 & q_3 k_2 & q_3 k_3 \end{bmatrix} = \begin{matrix} & \begin{matrix} I \\ like \\ cats \end{matrix} & \begin{bmatrix} 2.7 & 2.4 & 3.49 \\ 8.82 & 6.51 & 2.8 \\ 1.38 & 1.69 & 4.78 \end{bmatrix} \end{matrix}$$

Similarity score
between *cats* and *like*

Similarity score between all words

Step 2: Divide by square root of embedding dimension of K

In this case: 2.

$$\left(\frac{QK^T}{\sqrt{d_k}} \right)$$

Why? By scaling the values, we obtain more stable gradients, therefore faster convergence.

Attention: QK^T

Step 3: We apply the softmax function

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

Why?

We have a scale score matrix with values all over the place (not normalized),
So, in this step we just normalize our scaled score matrix.

What softmax does?

It take each word vector (row) adds up 1 (probability)

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) = \begin{matrix} & \text{I} & \text{like} & \text{cats} \\ \text{I} & 0.7 & 0.2 & 0.1 \\ \text{like} & 0.2 & 0.6 & 0.2 \\ \text{cats} & 0.4 & 0.1 & 0.5 \end{matrix}$$

Attention score, it gives the relevance of different parts of the sequence to each other. How important are the connection between words

Attention: V

Step 4: We multiply the attention score and the Value matrix (V)

V, carries **the content representation the model wants to share** during that layer

In a noisy party, **Q** is what you're interested in, while **K** is the signal each person gives off about what they can talk about.

Your brain compares Q with all Ks using **QK^T**, then uses **softmax** to decide who is most relevant to pay attention to in the noisy room.

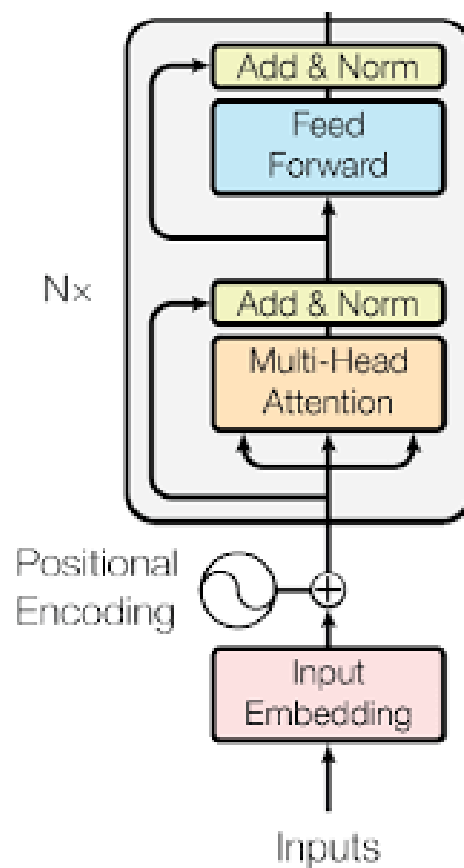
Only after choosing whom to focus on do you actually listen to what they're saying, that's **V**, the real content of their conversation.

The final attention output is a blend of the conversations you care about most, using the attention weights to combine the V vectors.

$$\vec{z}_{they} = 0.0\vec{v}_1 + \boxed{0.0}\vec{v}_2 + 0.0\vec{v}_3 + \boxed{0.9}\vec{v}_4 + 0.0\vec{v}_5 + 0.1\vec{v}_6 + 0.0\vec{v}_7 + 0.0\vec{v}_8$$

my friends like tomatoes because they are tasty

Transformer: encoder



Transformers: Input Embeddings

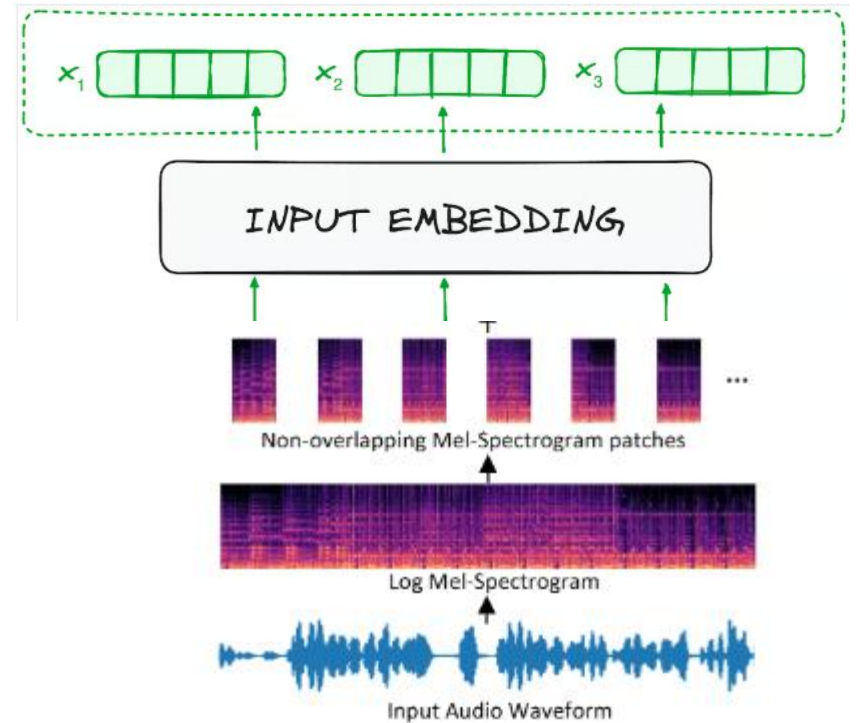
STEP 1 - Input Embeddings

The **embedding** step happens only in the **bottom-most encoder**.

In ASR systems, the encoder begins by converting **acoustic features** (log-Mel spectrograms or MFCCs) into numerical **feature vectors** using a small neural layer (often a linear or convolutional projection).

These embeddings represent the short-time spectral characteristics of the audio and provide a compact numerical representation suitable for the Transformer.

All the encoders receive a list of vectors, each of size 512 (fixed-sized). In the bottom encoder, that would be the word embeddings, but in other encoders, it would be the output of the encoder that's directly below them.



Transformers : encoder

STEP 2 - Positional Encoding

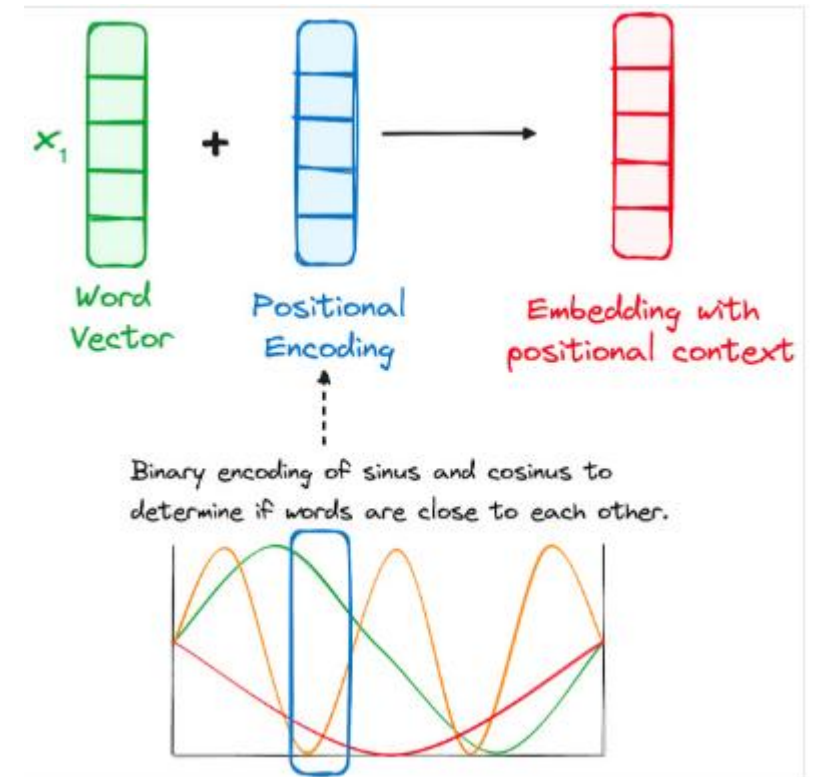
To ensure our model takes into account the **temporal order of the audio features**, we use something called **positional encoding (matrix P)**.

Matrix P has the same dimension than I

Since the Transformer has no inherent sense of sequence, this additional vector is added to each **acoustic embedding** to represent its **position in time**.

This way, the model has access to both the **acoustic content** (from the embedding) and the **temporal structure** (from the positional encoding), allowing it to understand how sounds evolve over time and form meaningful speech patterns.

To do so, the researchers proposed using a combination of **sine and cosine functions** of different frequencies to generate these positional vectors. Each dimension of the vector corresponds to a sine or cosine wave with a unique **frequency and phase offset**, producing values that range from **-1 to 1**. This design enables the positional encoding to generalize naturally to **audio sequences of any length**, while preserving a smooth and continuous representation of **time**



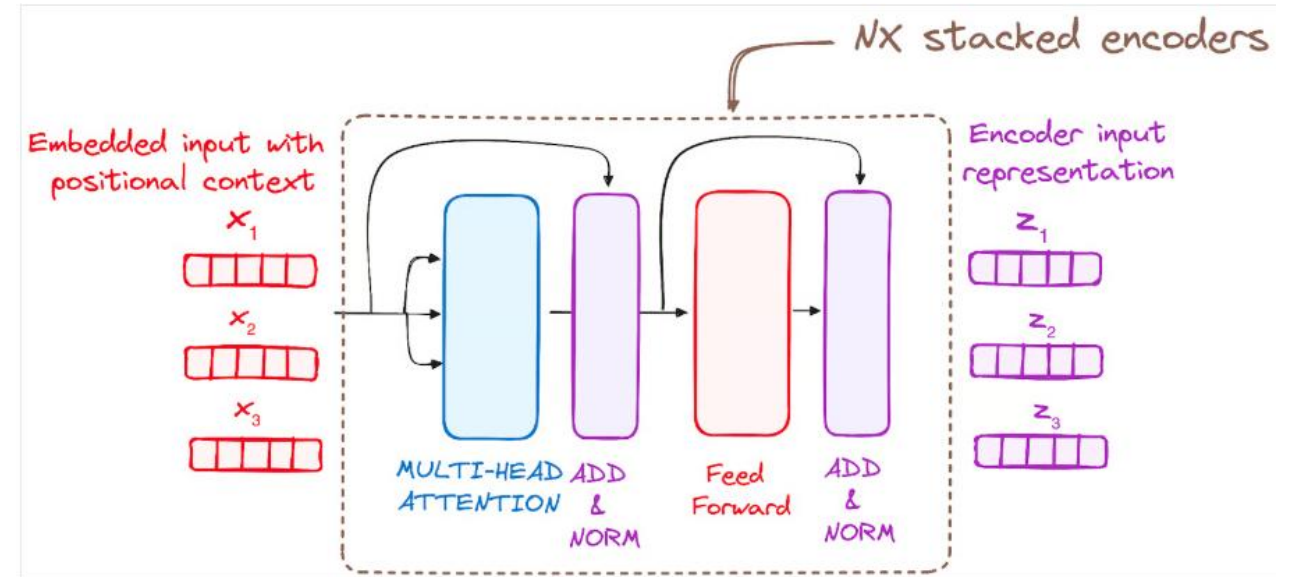
Transformers : encoder

Step 3: Stack of Encoder Layers

The Transformer encoder consists of a stack of identical layers (6 in the original Transformer model). The encoder layer serves to transform all input sequences into a continuous, abstract representation that encapsulates the learned information from the entire sequence.

This layer comprises two sub-modules:

- A multi-headed attention mechanism.
- A fully connected network.
- Additionally, it incorporates residual connections around each sublayer, which are then followed by layer normalization



Transformers : encoder

STEP 3.1 Multi-Headed Self-Attention

Mechanism:

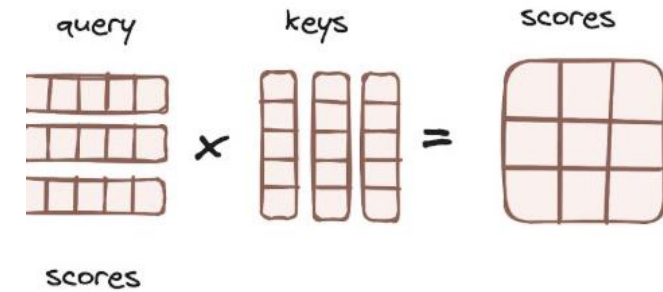
Run multiple instances of the self-attention mechanism in parallel

The encoder needs to understand how these frames relate to each other. For example, frames corresponding to the beginning and end of a phoneme, or syllable, may need to be connected to correctly predict the output

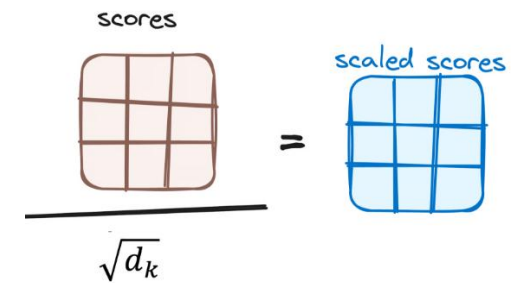
This is where **self-attention** comes in. We compute three vectors for each frame:

- **Query (Q):** Represents the frame we're currently focusing on. Think of it as 'which frames do I care about?'
- **Key (K):** Represents every frame in the sequence for comparison. It's like a reference to see which frames match or are relevant.
- **Value (V):** Contains the information that will actually be used to update the current frame's representation.

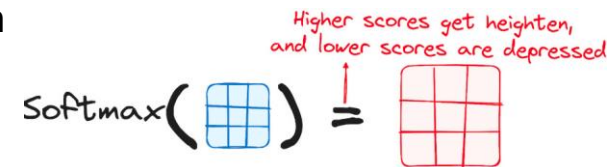
- **Dot Product / Score Matrix:** Multiply each Query by all Keys. This gives us a score for each frame pair, telling us how much each frame should pay attention to the others. Higher scores $\rightarrow$ stronger connection.



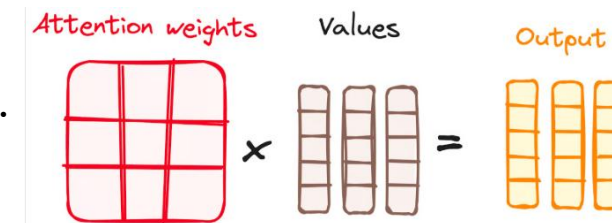
- **Scaling:** We divide the scores by the square root of the vector dimension. This prevents the dot product values from becoming too large, which could destabilize learning.



- **Softmax:** We apply the softmax function to convert the scores into probabilities between 0 and 1. This tells us the weight of influence each frame has on the current frame.



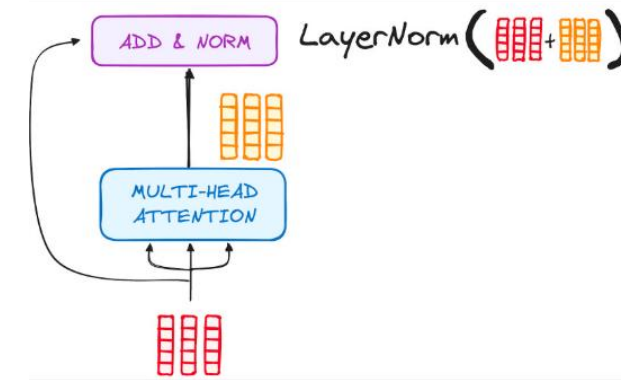
- **Weighted Sum:** Finally, we multiply these attention weights by the Values to get an updated representation for the current frame. Frames with higher weights contribute more to the output.



Transformers : encoder

STEP 3.2 Normalization and Residual Connections

Each sub-layer in an encoder layer is followed by a normalization step. Also, each sub-layer output is added to its input (residual connection) to help mitigate the vanishing gradient problem, allowing deeper models. This process will be repeated after the Feed-Forward Neural Network too.

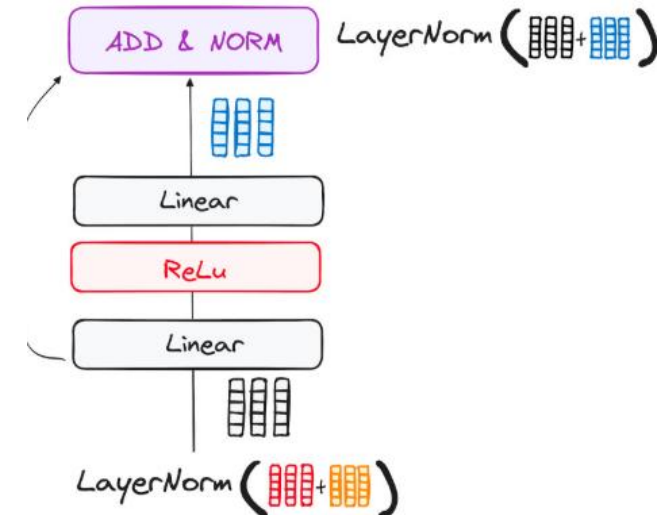


STEP 3.3 Feed-Forward Neural Network

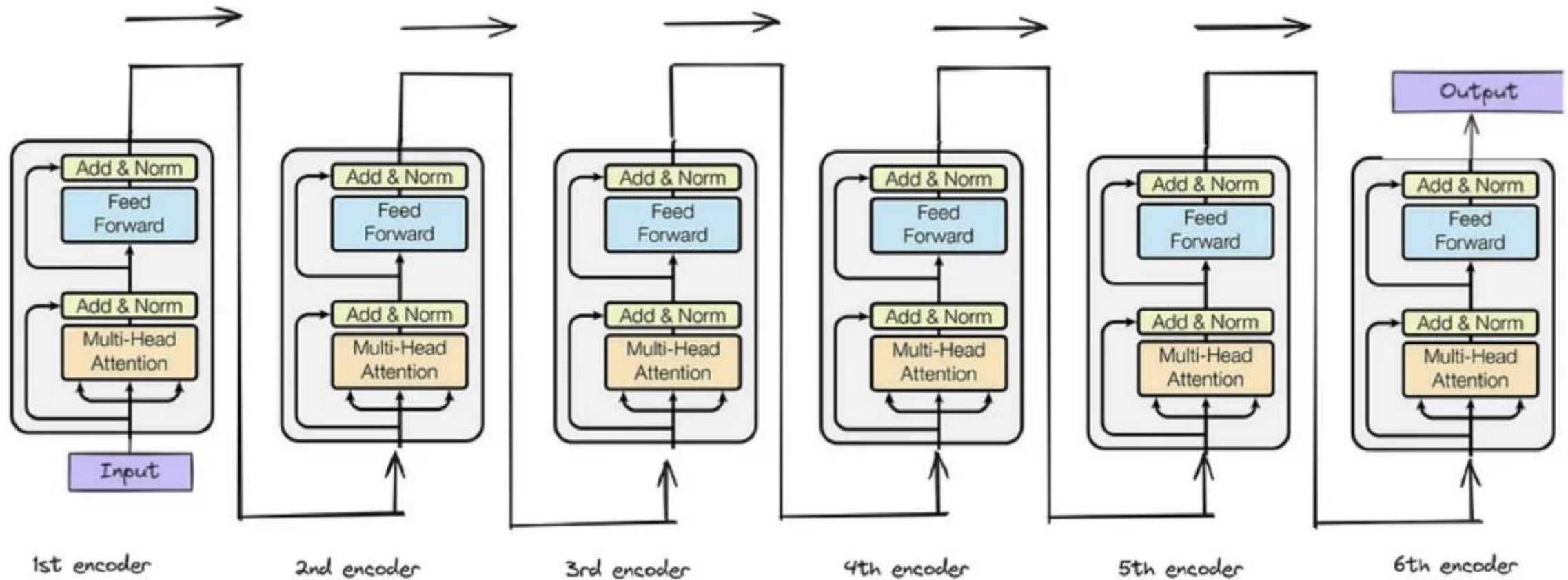
After self-attention has enriched each acoustic frame with context from the entire sequence, the resulting output still needs further refinement.

This is where the **pointwise feed-forward network**, or FFN, comes in.

Think of the FFN as a simple but powerful **duo of linear layers with a ReLU activation** in between. The ReLU introduces non-linearity, allowing the network to model more complex transformations of the acoustic features.

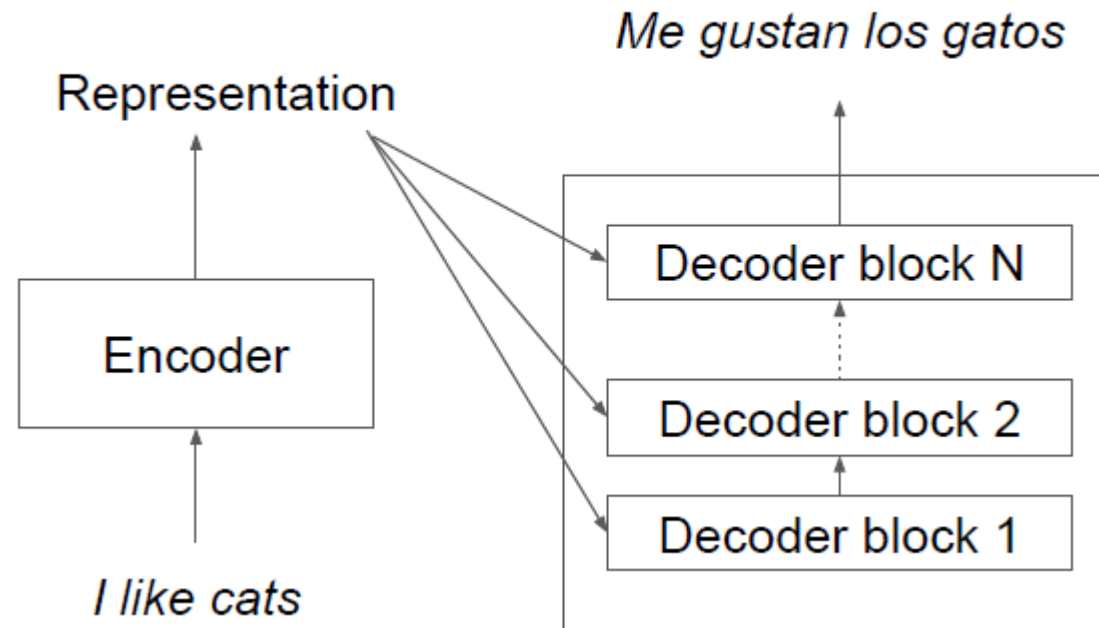


Transformers



Transformers : decoder

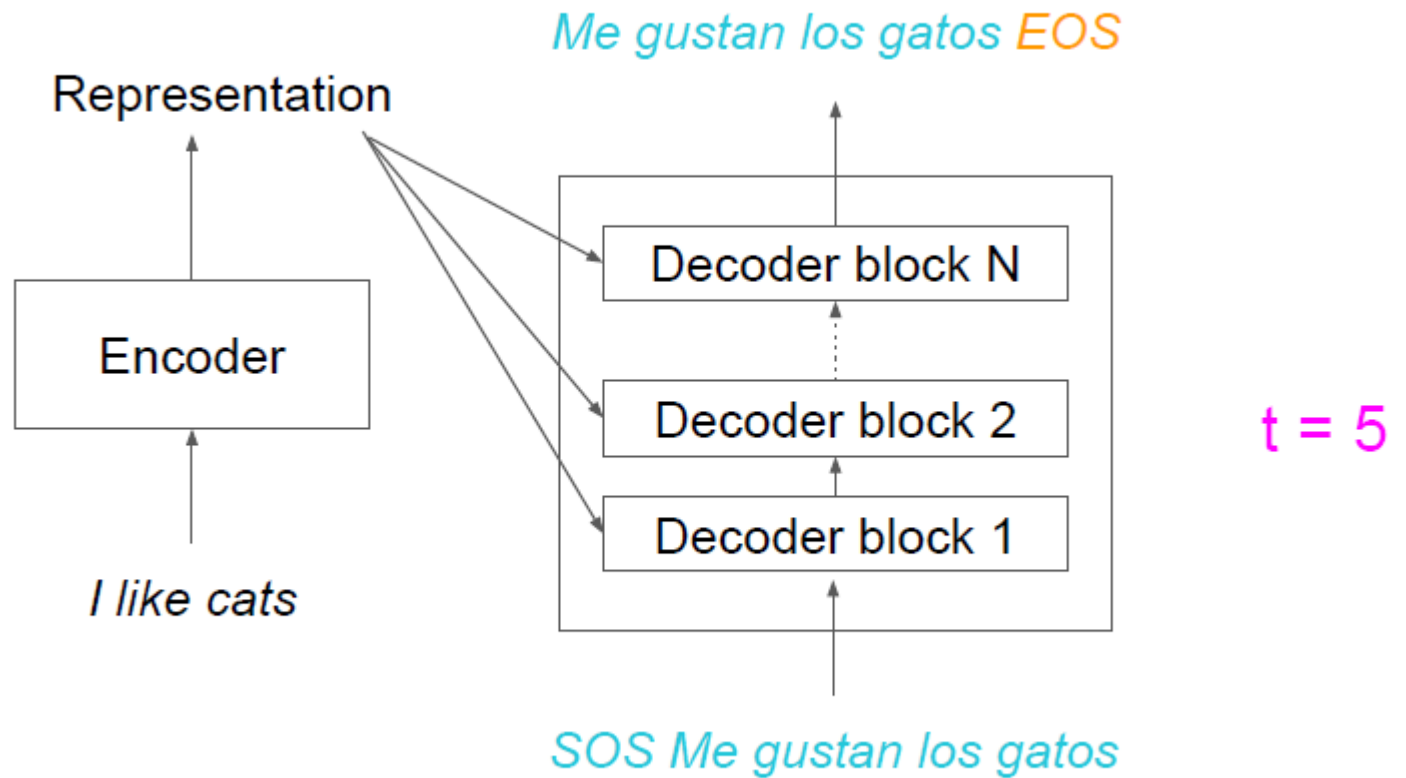
- N stack decoder block
- Each decoder block has **2 inputs**:
 - Encoder representatio
 - Output from previous decoder
- Word by word



Transformers : decoder

- SOS: Start Of Sentence
- t1 Me
- t2 gustan
- t3 los
- t4 gatos
- t5 EOS (End)

SOS & EOS are markers



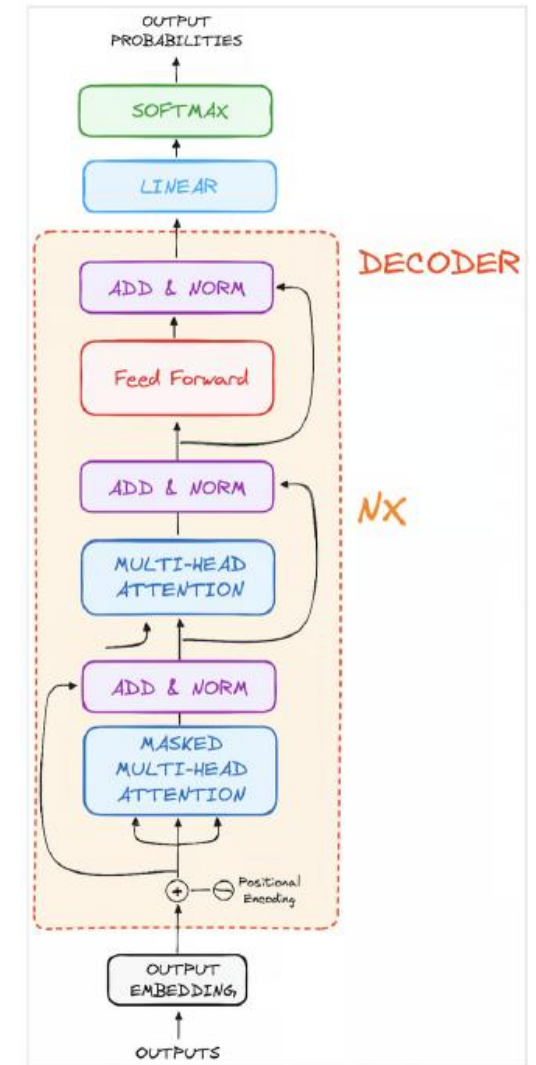
Transformers : decoder

In automatic speech recognition, the decoder's primary role is to generate the output sequence (phonemes, characters, or words) based on the encoded acoustic frame representations from the encoder.

- It has two multi-headed attention layers and a pointwise feed-forward layer.
- It also incorporates both residual connections and layer normalization after each sub-layer.

Decoding steps:

- Masked Self-Attention Mechanism
- Encoder-Decoder Multi-Head
- Attention or Cross Attention
- Feed-Forward Neural Network
- Linear Classifier and Softmax for Generating Output Probabilities



Transformers : decoder

In the decoder, the first attention layer is called **masked self-attention**. The key idea here is **causality**: when predicting the next token in a sequence, the model **cannot see future tokens**. This is critical in ASR because we generate the transcription **one token at a time**, based on the frames we've already processed and the tokens we've already predicted.

The diagram illustrates the process of applying a look-ahead mask to scaled scores. It consists of three 3x3 grids. The first grid, labeled 'scaled scores' in blue, contains the values: $\begin{bmatrix} 0.5 & 0.2 & 0.1 \\ 0.1 & 0.6 & 0.2 \\ 0.1 & 0.2 & 0.3 \end{bmatrix}$. A plus sign follows. The second grid, labeled 'Look-ahead mask' in brown, contains: $\begin{bmatrix} 0 & -\text{inf} & -\text{inf} \\ 0 & 0 & -\text{inf} \\ 0 & 0 & 0 \end{bmatrix}$. An equals sign follows. The third grid, labeled 'Masked Scores' in orange, contains: $\begin{bmatrix} 0.5 & -\text{inf} & -\text{inf} \\ 0.1 & 0.6 & -\text{inf} \\ 0.1 & 0.2 & 0.3 \end{bmatrix}$.

- Compute attention scores:

For each token in the decoder input, we calculate how much attention it should pay to all tokens in the sequence. Normally, in self-attention, a token can attend to every other token, including those that come later.

- Apply the mask:

The mask blocks future tokens by setting their attention scores to negative infinity (or a very large negative number) before applying the softmax.

After softmax, these masked positions get a probability of zero, meaning the current token will ignore future tokens completely.

This masking ensures that the predictions for a particular position can only depend on known outputs at positions before it.

Transformers : decoder

After the decoder has applied masked self-attention to its own previous outputs, it moves on to the second multi-headed attention layer: **encoder-decoder attention**, also called **cross-attention**.

The key idea here is that the decoder now needs to **look at the encoder's output**, which contains all the contextualized acoustic frame representations. This allows the decoder to know **which parts of the input speech are most relevant** when predicting the next output token.

- Queries (Q):

From the decoder's previous layer (masked self-attention output).
Each token asks: "Which parts of the encoder output are relevant?"

- Keys (K) & Values (V):

From the encoder output (acoustic frame representations).

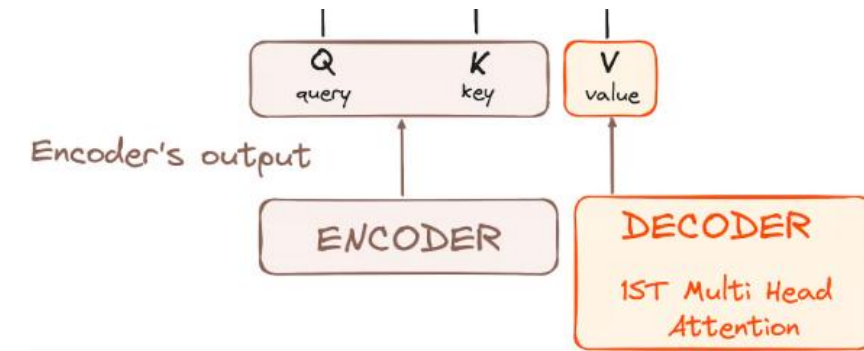
Keys compute attention scores; values provide the information to update the decoder token.

- Attention Computation:

Compute scores between queries and keys.

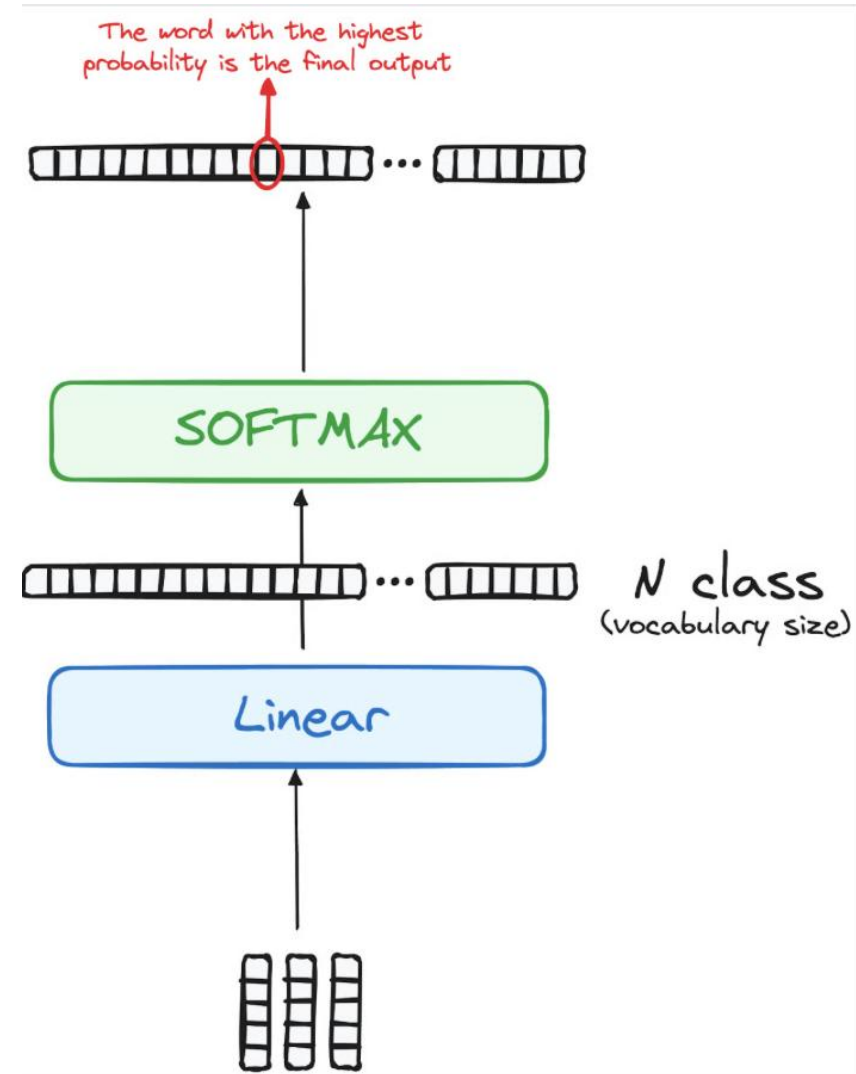
Apply softmax → attention weights (probabilities).

Multiply weights by values → contextualized decoder representation.



Transformers : decoder

- Similar to the encoder, each decoder layer includes a fully connected feed-forward network, applied to each position separately and identically.
- At the output of the decoder, there is a final linear layer which functions as a classifier.
- The size of the classifier corresponds to the vocabulary size of the model.
- The output then passes through a softmax layer, which gives a probability score between 0 and 1. The index of the highest probability corresponding to the most likely word the model predicts as the next word.
- The decoder then adds the output at the current instant to the growing inputs at the next instant.
- The cycle continues until the decoder predicts a special token signaling completion.



Important models

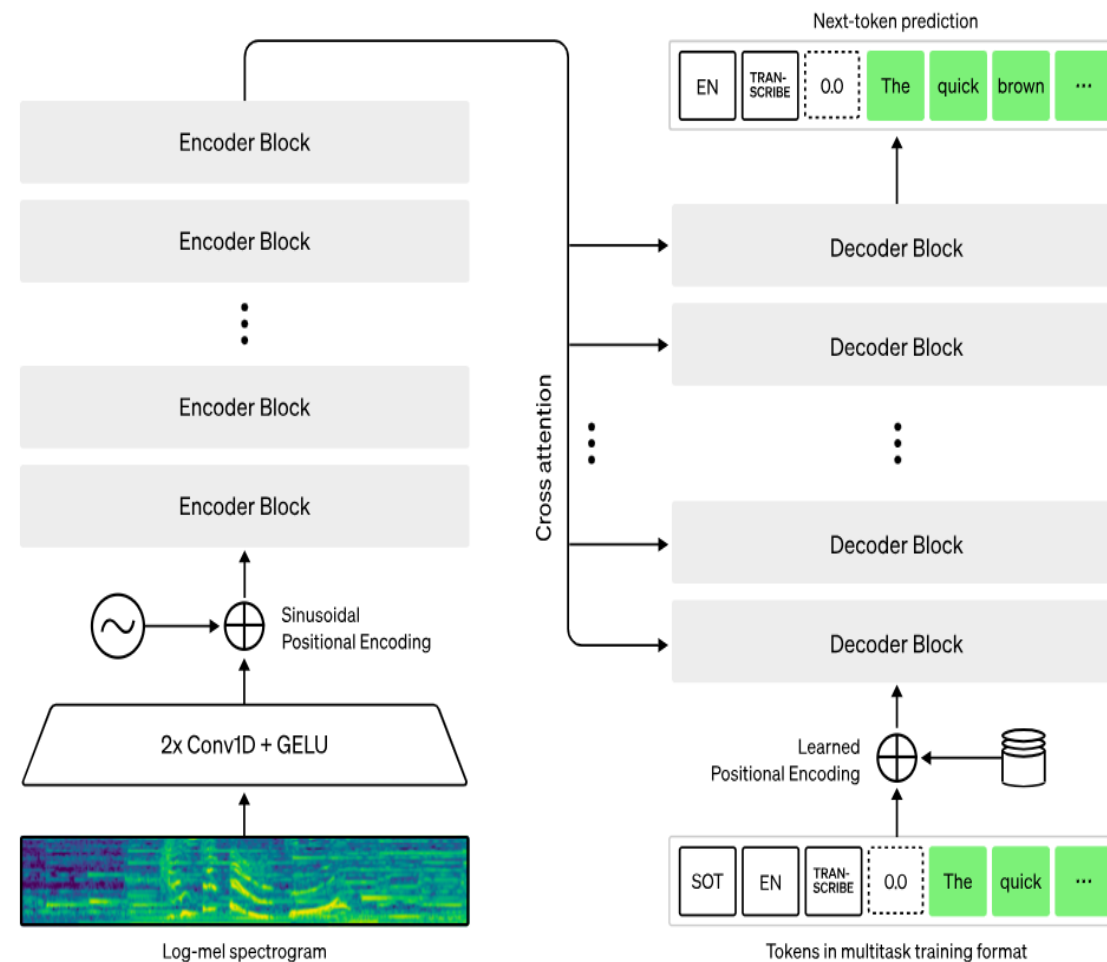
Model Name	Release Year	Number of Parameters	Tasks			Multimodal
			Speech Synthesis (TTS)	Speech Translation (ST)	Automatic Speech Recognition (ASR)	
Tacotron [53]	2017	13 million [56]	✓	×	×	×
Tacotron 2 [54]	2017	28.2 million [57]	✓	×	×	×
Transformer-TTS [55]	2018	30.7 million [58]	✓	×	×	×
vq-wav2vec [44]	2019	34 million [59]	×	×	✓	×
Mockingjay [60]	2019	85 million [59]	×	×	✓	×
FastSpeech [58]	2019	23 million [61]	✓	×	×	×
wav2vec [62]	2019	16 million [63]	×	×	✓	×
wav2vec 2.0 [45]	2020	317 million [59]	×	×	✓	×
FastSpeech 2 [61]	2020	27 million [61]	✓	×	×	×
FastPitch [64]	2020	26.8 million	✓	×	×	×
Conformer [65]	2020	1 billion [59]	×	✓	✓	×
DeCoAR 2.0 [66]	2020	317 million [59]	×	×	✓	×
w2v-Conformer	2021	1 billion [59]	×	×	✓	×
w2v-BERT [67]	2021	1 billion [59]	×	×	✓	×
HuBERT [68]	2021	317 million [59]	×	×	✓	×
XLS-R [48]	2021	2 billion [59]	×	✓	✓	×
UniSpeech [69]	2021	317 million [59]	×	×	✓	×
UniSpeech-SAT [70]	2021	317 million [59]	×	×	✓	×
BigSSL [71]	2021	8 billion [59]	×	×	✓	×
WavLM [72]	2021	317 million [59]	×	×	✓	×
DeltaLM [73]	2021	360 million [74]	✓	✓	×	✓
SpeechT5 [75]	2021	11 billion [75]	✓	✓	✓	✓
data2vec [49]	2022	*	×	×	✓	✓
data2vec 2.0 [76]	2022	*	×	×	✓	✓
SpeechFormer [63]	2022	3.5 million [63]	×	×	✓	✓
Whisper [52]	2022	1.6 billion [77]	×	✓	✓	×
VALL-E [78]	2023	*	✓	×	×	×
VALL-E X [79]	2023	*	✓	×	×	×

Important models: Whisper

The Whisper architecture is a simple end-to-end approach, implemented as an encoder-decoder Transformer.

Based on 680,000 hours of multilingual and multitasking data collected on the Internet.

Input audio is split into 30-second chunks, converted into a log-Mel spectrogram, and then passed into an encoder.

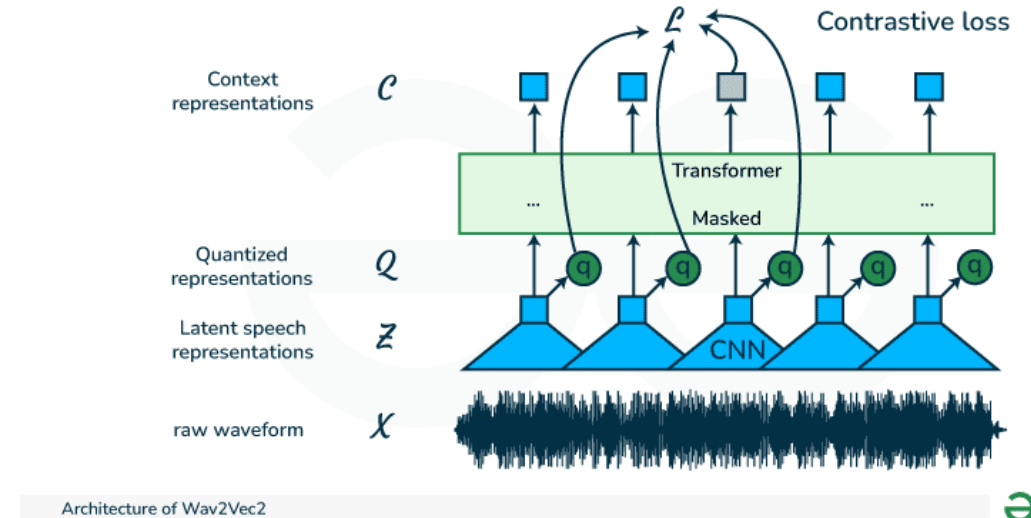


Important models: Wave2vec

Pre-Training: Here the model is trained to develop broad general representations by learning from extensive datasets that are unlabeled. This step is believed to enhance performance on a downstream task with constrained data. (60k hours). The model learns contextualized speech representations by randomly masking feature vectors before passing them to a transformer network

Fine Tuning : The pre trained model is then fine tuned using a small labelled dataset (10 min of data).

Without any labeled data at all, it can achieve 5.9% Word Error Rate



LAB

Generative IA audio

Session 7

Intelligent vs creative tasks

Intelligent task: object detection, speech recognition

Pretty easy to undersand, I have a picture and I want to find some objects and name them:
Bike, car, traffic light, etc

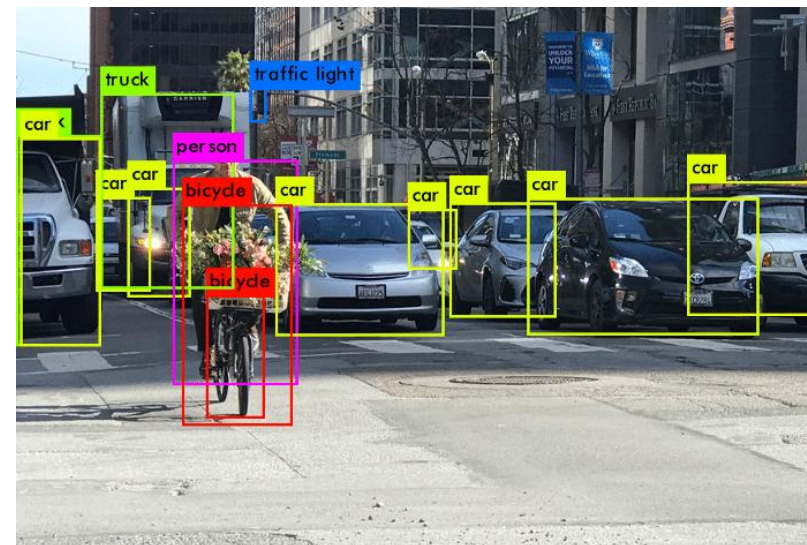
I speak to Siri and I want her to understand what I am asking

Intelligent task have clear objective success metric

You can use this metric to evaluate your model. For example, look how objects have been identified.

Compare the number of word correctly identified.

Intelligent task have wel- defined problem description.



Intelligent vs creative tasks

Creative task do not have clear objective success metric

How can you evaluate music in an objective way? It is impossible. It is like painting, some painting are very famous now but not when they were created!!

Creative task do not have wel- defined problem description.

Can we formalize music? NO!!, it is too complex, a lot of factors, variables (physics of sounds, perception), it is subjective, too ambiguous

They are very difficult to reproduce with a machine because you do not have a metric!!!



What is generative IA music?

The **art and science** of developing computer algorithms that create music with a varying degree of autonomy.

Varying degree of autonomy because or you can use prompt and generate a music or you can work together with your machine to create something. Therefore, you can or not give autonomy to the machine.



What are the challenges?

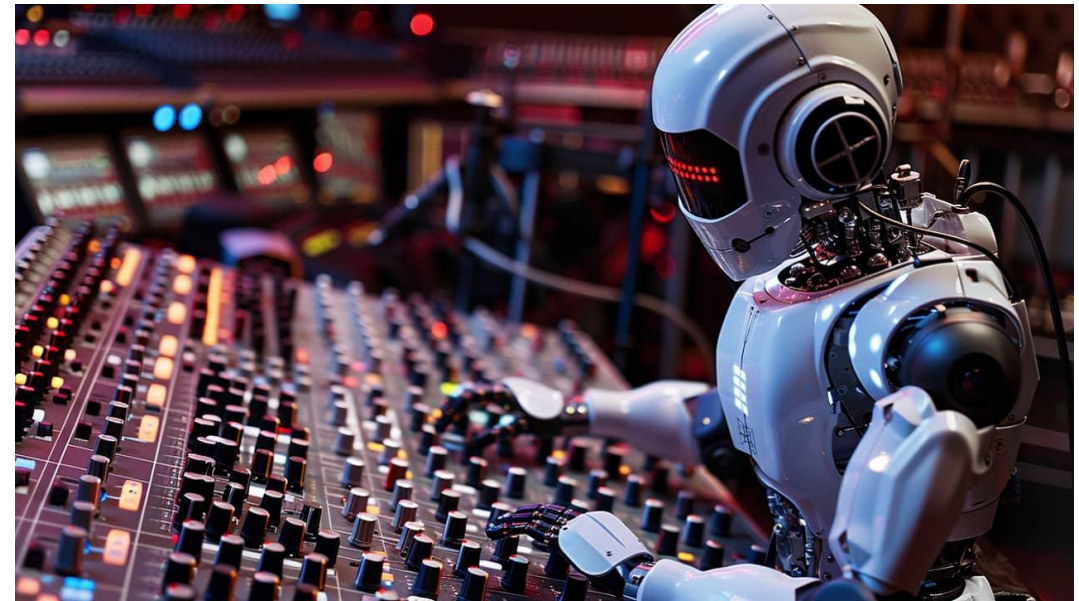
- How can we represent music?
Symbolic, statistical based, audio, spectrogram, etc...

We still do not have the perfect music representation.

- How can we evaluate the output?

No objective metric so we need to define specific tools for each task
- Who should evaluate the output?

Normal people? Musicians?



History of the GenIA music

History of the GenIA music

Mozart dice game (1787)

Created in the late 18th century as a form of musical entertainment.

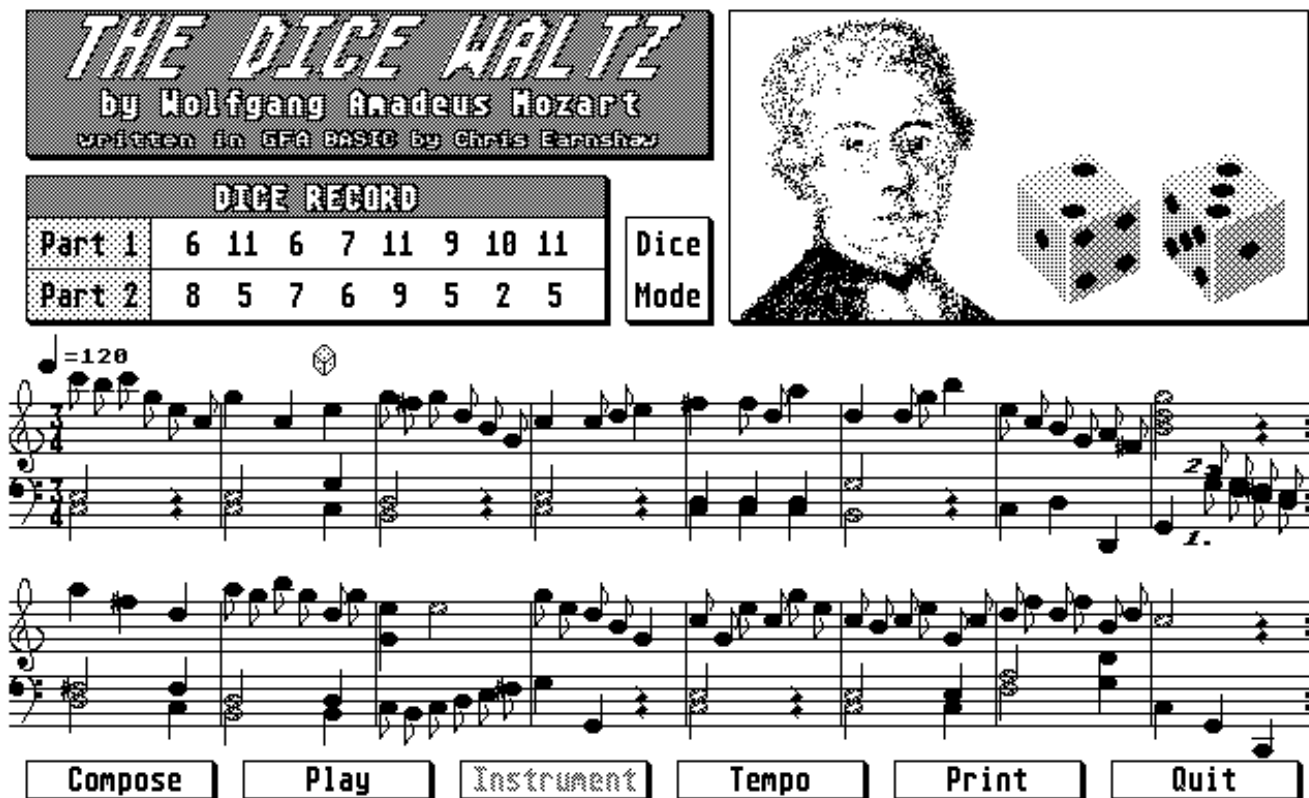
Uses a grid of numbered musical measures; each dice roll selects which measure to use.

The game has 16 measures.

For each of those measures, there are 11 possible musical options.

$16 \times 11 = 176$ total musical fragments

Even though the selection is random, all combinations are designed to sound harmonious.



The screenshot displays the 'THE DICE WALTZ' game interface. At the top, the title 'THE DICE WALTZ' is prominently displayed in a stylized font, followed by 'by Wolfgang Amadeus Mozart' and 'written in GFA BASIC by Chris Earnshaw'. Below this is a 'DICE RECORD' table with two rows: 'Part 1' and 'Part 2'. The 'Dice Mode' button is located to the right of the table. To the right of the table is a portrait of Wolfgang Amadeus Mozart and two dice. The main area of the screen shows two staves of musical notation. The top staff has a tempo marking of '=120' and a small dice icon above it. The bottom staff has a '2.' marking. At the bottom of the screen, there are five buttons: 'Compose', 'Play', 'Instrument', 'Tempo', and 'Quit'.

DICE RECORD								
Part 1	6	11	6	7	11	9	10	11
Part 2	8	5	7	6	9	5	2	5

Dice Mode

Compose

Play

Instrument

Tempo

Print

Quit

History of the GenIA music

Academic era (1957 - 2009)

Research activity

- Lots of experimentation
- Incremental advancements
- Scattered community

Musical output

- Full-piece generation is rare
- No focus on audio production quality
- Score generation (symbolic)
- Classical music



History of the GenIA music

Big tech experiments (2016 - 2022)

- Deep learning focus
 - Massive datasets
 - Massive computational power
 - No commercial end goal
-
- AWS DeepComposer (Amazon, 2019)
 - Jukebox (OpenAI, 2020)



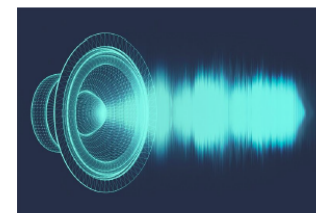
Now

- MusicLM (Google 2023)

MusicLM

A fusion of reggaeton and electronic dance music, with a spacey, otherworldly sound.

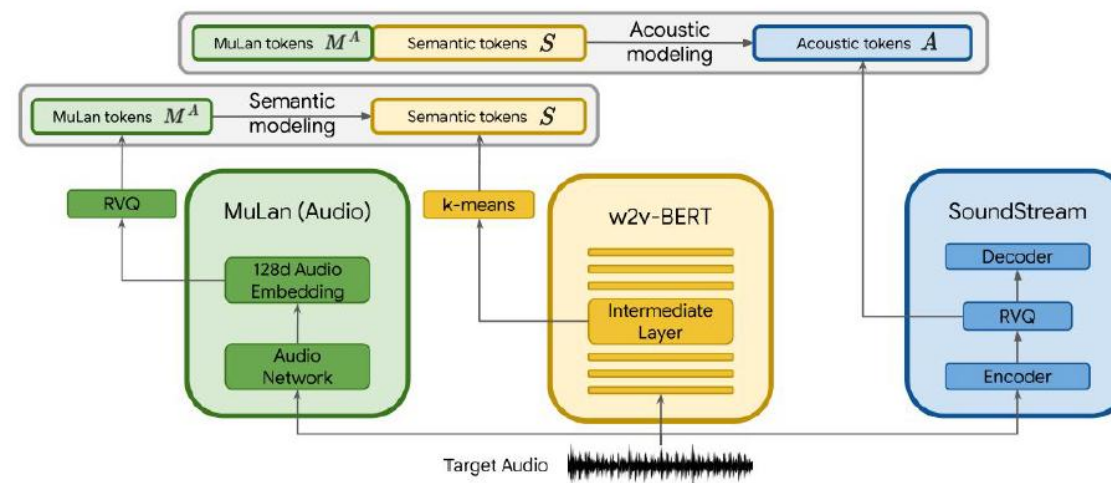
Music model



- MusicGen (Meta 2023)

MusicGen: Simple and Controllable Music Generation

- Mousai
- AudioLDM
- SingSong
- RAVE 2
- ...



Ethics

- Is it fair to use artists' music to train GM models?
- Is artists' consent necessary to use their music?
- What constitutes copyright infringement?
- How can we compensate artists?
- Is it OK to use AI to generate full compositions? (remove musicians)
- Is it OK to use AI to reproduce the likeness / voice of musicians?
- Training large GM models takes a lot of energy

Symbolic versus audio representation

This is what musicians use to speak about music.

This is all the information that someone should pass to a performer to reproduce/generate the music.

Then the musician transforms something on paper to an audio representation!!

*À son Altesse Sérénissime
Monseigneur le Prince régnant de Lobkowitz
Duc de Raudnitz
et à son Excellence Monsieur le Comte de Rasumoffsky*

Symphonie Nr. 5
c-moll
op. 67

Ludwig van Beethoven

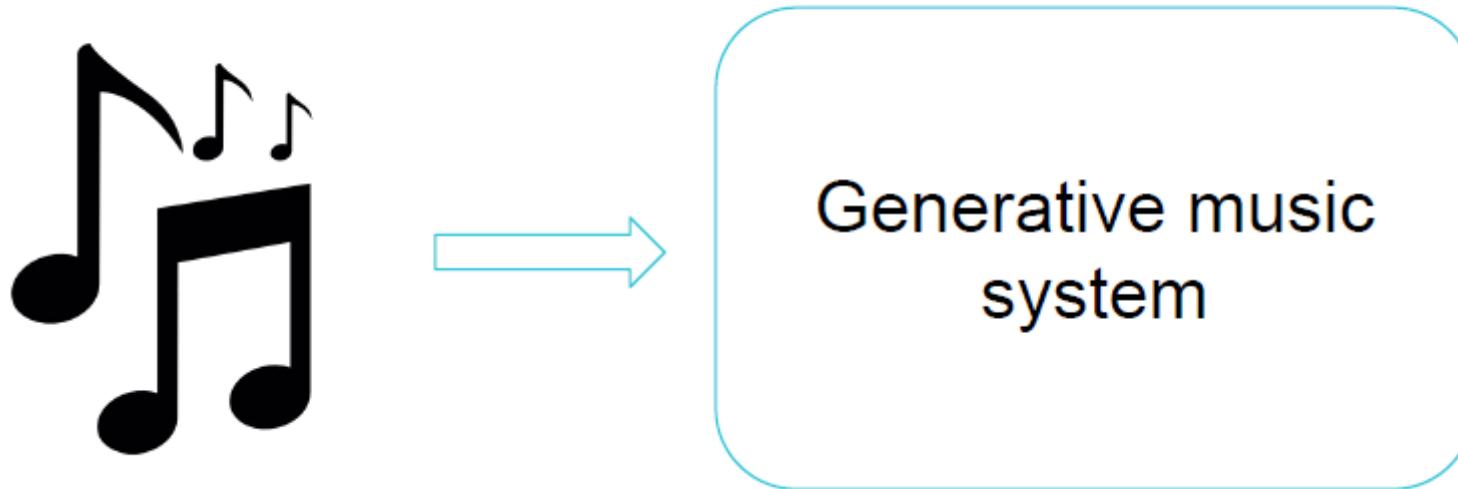
Allegro con brio *)

*) Beethoven's metronome marking of 1817 / Beethovens Metronombezeichnung von 1817: ♩ = 108

© 1999 by Bärenreiter-Verlag, Kassel

Symbolic versus audio representation

We need to encode the music in a format that can be easily fed to a model to generate music.



Symbolic versus audio representation

We need to encode the music in a format that can be easily fed to a model to generate music.



Generative music
system

Symbolic

- **Symbols (e.g., notes, instruments)**
- **Very symilar to a music score**
- **Type of symbolic représentations:**
 - MIDI (Musical Instrument Digital Interface)
timestamp, midi note=pitch, velocity

Symbolic

- **Symbols (e.g., notes, instruments)**
- **Very similar to a music score**
- **Type of symbolic representations:**
 - MIDI (timestamp, midi note=pitch, velocity)
 - MusicXML

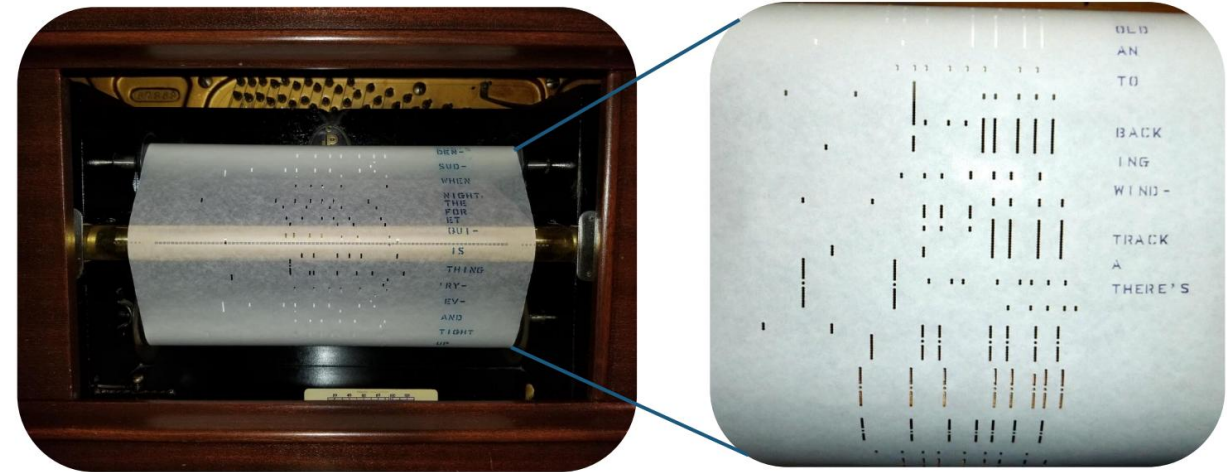
```
<note>  
  <pitch>  
    <step>E</step>  
    <alter>-1</alter>  
    <octave>4</octave>  
  </pitch>  
  <duration>2</duration>  
  <type>half</type>  
</note>
```



Figure 1.15 from [Müller, FMP, Springer 2015]

Symbolic

- Symbols (e.g., notes, instruments)
- Very similar to a music score
- Type of symbolic representations:
 - MIDI (timestamp, midi note=pitch, velocity)
 - MusicXML
 - Piano roll (pitch, tstart, tend)



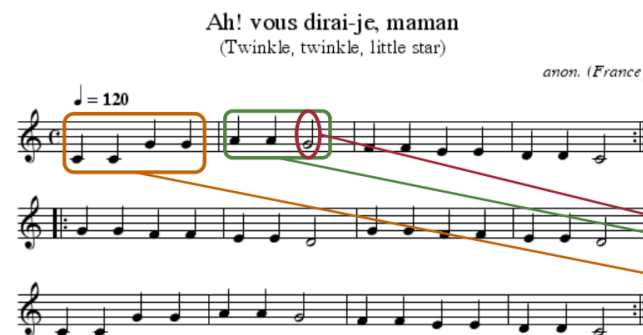
(Source: Draconichiaro)

(Source: Tangerineduel)



Symbolic

- Symbols (e.g., notes, instruments)
- Very similar to a music score
- Type of symbolic representations:
 - MIDI (timestamp, midi note=pitch, velocity)
 - MusicXML
 - Piano roll (pitch, tstart, tend)
 - ABC notation (here key: C major)

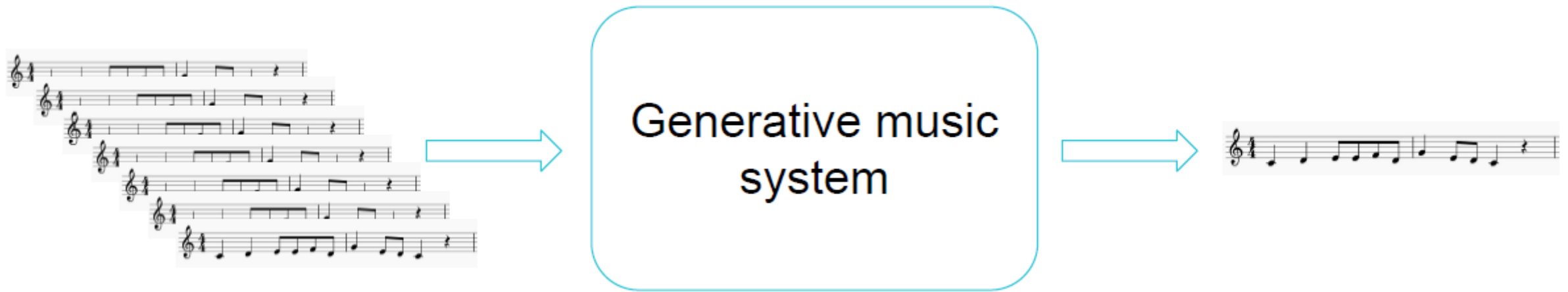


Metadata

```
X:571
T:Ah! vous dirai-je, maman
T:(Twinkle, twinkle, little star)
C:anon.
O:France
R:Nursery song
M:C Meter
L:1/4 Unit note length (temporal resolution)
Q:120 Tempo
K:C Key
CCGG|AAG2|FFEE|DDC2:|
|:GGFF|EED2|GGFF|EED2|
CCGG|AAG2|FFEE|DDC2:|
```


Symbolic

- MuseNet (OpenAI, 2019)
 - GPT2 architecture
 - Trained on MIDI files
 - Predict next token



Symbolic



- Compact
- Easy to manipulate
- Clear and precise
- Lots of compositional info
- Capture long-term dependencies
- Small models

When to use

Structure + composition is focus

Notated Western music (e.g., classical, jazz)



- Oversimplified
- Musical limitations
- Limited performance info
- No production info
- Output isn't audio

When not?

Performance + production is focus

Electro music

Audio representation :uses waveform or a transform

Type of representation:

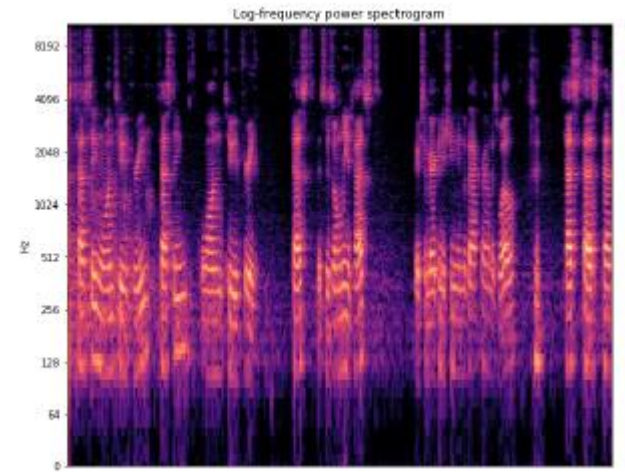
- Raw audio (waveform)



Audio representation

Type of representation:

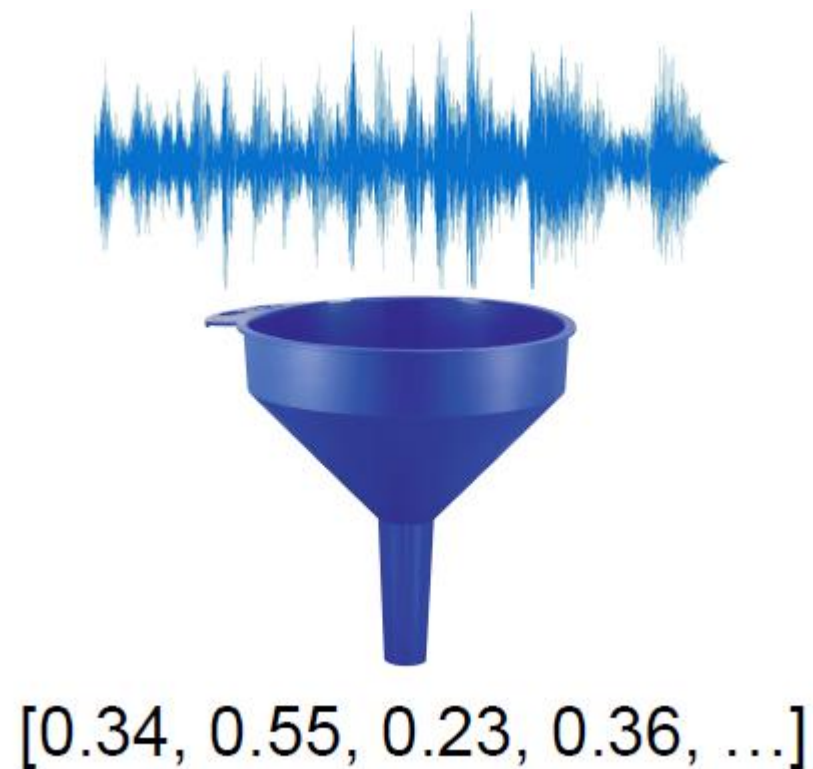
- Raw audio (waveform)
- Spectrogram



Audio representation

Type of representation:

- Raw audio (waveform)
- Spectrogram
- Audio embedding



Audio representation

Sample audio-based models:

- Jukebox (OpenAI)
- MusicLM (Google)
- MusicGen (Meta)
- RAVE (Ircam)

Audio representation



- Lots of performance info
- Lots of production info
- Complex and rich
- Audio output

When ?

Performance + production is focus

Electro music



- Large dimensionality / size
- Difficult to manipulate
- No compositional info
- Model size
- Difficult to capture long-term dependencies

When not?

Structure + composition is focus

Notated Western music (e.g., classical, jazz)

Audio representation



Symbolic music generation

Text-based

```
Program_change_0,  
Note_on_60, Time_shift_2, Note_off_60,  
Note_on_60, Time_shift_2, Note_off_60,  
Note_on_76, Time_shift_2, Note_off_67,  
Note_on_67, Time_shift_2, Note_off_67,  
...
```

MIDI

Image-based



Piano roll



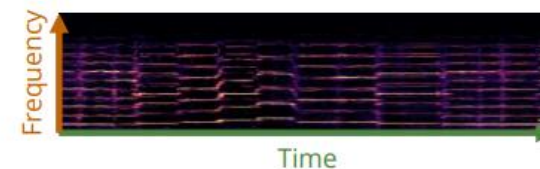
Audio-domain music generation

Time series-based



Waveform

Image-based



Spectrogram

Independent Component Analysis

Cocktail Party Problem :

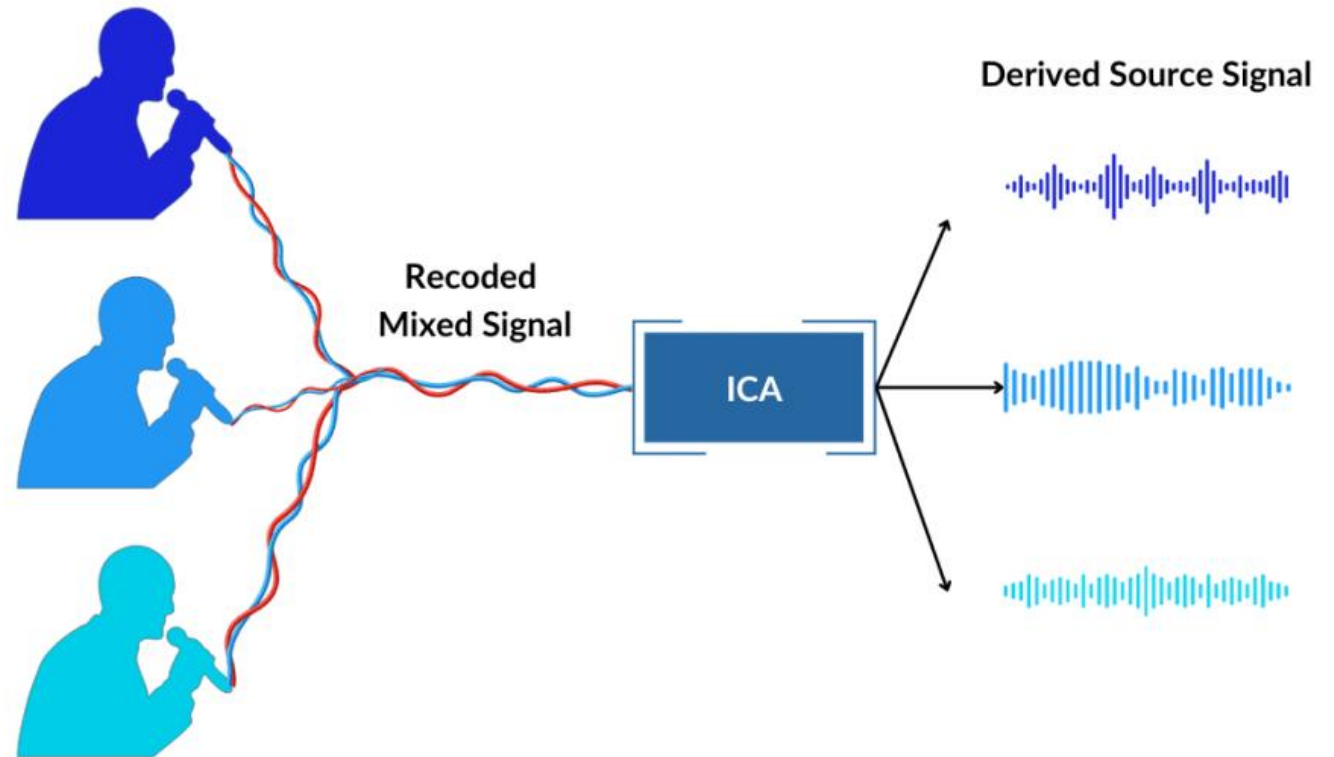
At a cocktail party, you have multiple speakers, therefore what microphone record is a combination of the speaker's voice



Independent Component Analysis

Cocktail Party Problem :

At a cocktail party, you have multiple speakers, therefore what microphone record is a combination of the speaker's voice



Independent Component Analysis

Mathematical intuition behind linear mixture of signals is defined as follows,

$$\mathbf{x} = \mathbf{A}\mathbf{s} \quad (\text{Linear Mixture})$$

Where,

- $x_1(t), x_2(t), x_3(t), \dots, x_n(t)$ where t is the index of the sample(time)
- $\mathbf{A}$ is an unknown invertible, square matrix that mixes the components of the sources
- $\mathbf{s} = s_1(t), s_2(t), s_3(t), \dots, s_n(t)$ are the independent sources

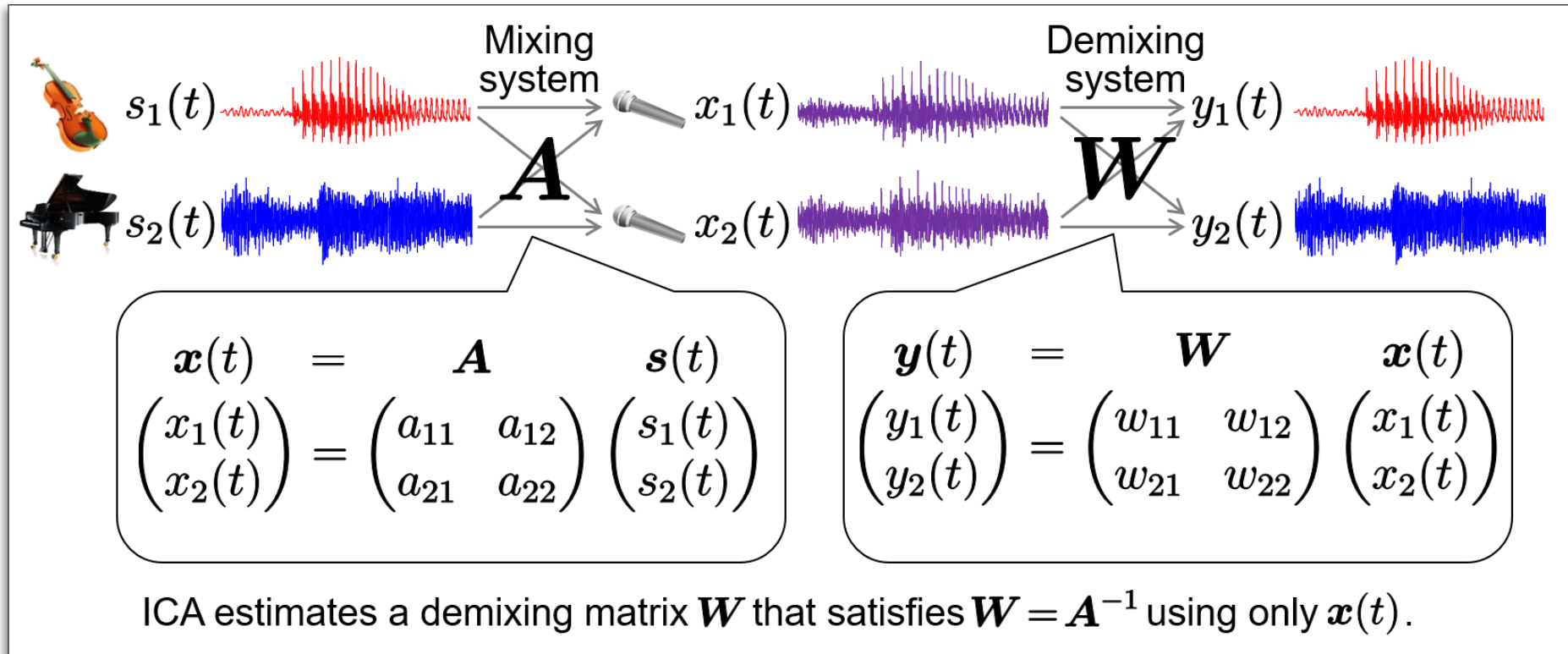
Our goal is to solve for the matrix $\mathbf{A}$, rather $\mathbf{A}^{-1}$, i.e. from *eqn.1*

$$\mathbf{s} = \mathbf{A}^{-1}\mathbf{x}$$

Since this is an approximation problem, we shall rewrite the equation as follows.

$$\hat{\mathbf{s}} = \mathbf{W}\mathbf{x}$$

Independent Component Analysis



Independent Component Analysis

Assumptions: the independent components are...

- **Statistically independent:** $p(x,y)=p(x)*p(y)$ meaning the joint probability distribution is the product of the marginal probability distribution function of each random variable

The independence assumption means that the sound sources operate without influencing one another

- **Non-gaussian**



Non-Gaussianity is crucial because it implies a lack of linearity and allows ICA to exploit statistical independence.

Independent Component Analysis

What happens if they are Gaussian?

If you assume s Gaussian, therefore x also. Let take the example of 2 signal x_1, x_2 . Their joint density is:

$$p(x_1, x_2) = \frac{1}{2\pi} \exp\left(-\frac{x_1^2 + x_2^2}{2}\right)$$

The Figure shows that the density is completely symmetric, therefore it does not contain any information on the directions of the columns of the mixing matrix A, this is why A cannot be estimated.

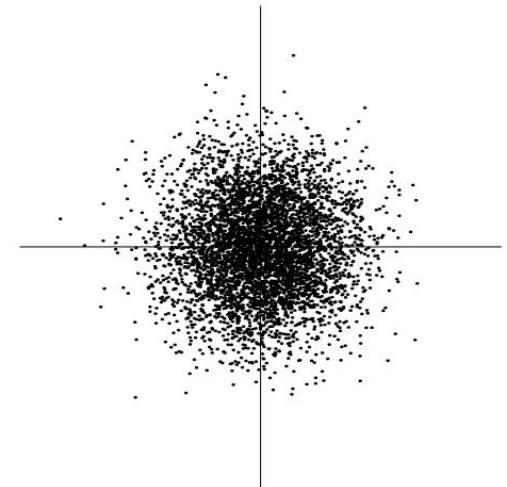


Figure 7: The multivariate distribution of two independent gaussian variables.

Independent Component Analysis

Before applying an ICA algorithm on the data it is usually very useful to do some preprocessing

- **Centering:** Remove the mean from the data because ICA assumes zero-mean sources
- **Whitening (sphering):** Transform the data so its covariance is the identity matrix. Note that after whitening, the mixing matrix becomes **orthogonal**.

ICA: Algorithms and Techniques

- **FastICA** : FastICA is a popular and efficient ICA algorithm that aims to find independent components by maximizing non-Gaussianity. I

It is based on a fixed-point iteration scheme and can work with linear and nonlinear mixtures.

- FastICA maximizes negentropy, a measure of non-Gaussianity, to separate independent components.
- It employs a deflation algorithm to extract components one at a time.
- FastICA is computationally efficient and widely used in various applications, including image and speech processing.

ICA: Algorithms and Techniques

- **Infomax** : Infomax is an ICA algorithm inspired by information theory. It seeks to maximize the mutual information between the estimated independent components. It is often used in applications like blind source separation and neural network training.
 - Infomax maximizes the mutual information, which measures the statistical independence of the components.
 - It relies on neural network architectures and learning rules to iteratively adjust the mixing matrix and estimate independent components.

ICA: Algorithms and Techniques

- **JADE (Joint Approximate Diagonalization of Eigenmatrices)** : JADE is an ICA algorithm designed to work with non-Gaussian sources. It operates by jointly diagonalizing the fourth-order cumulant matrices of the observed data. This approach is convenient in scenarios where sources exhibit super-Gaussian behaviour.
 - JADE is well-suited for separating sources with higher-order statistical properties.
 - It provides an alternative to kurtosis-based approaches and is robust to non-Gaussianity.

Project presentation

Session 8